ETL GUARD
Complete Technical & Operations Manual

Architecture | Configuration | API | MySQL 8 | SCD Type 2 | QA | Deployment | Incident Response

Document control	Value
Product	ETL Guard / etl-validation-toolkit
Release	v0.1.3
Package basis	ETLGuard_v0.1.3_Production_Readiness_20261111.zip
Prepared	12 November 2026
Audience	Developers, ETL operators, DBAs, QA, security, data owners and release approvers
Validation baseline	Fresh local v0.1.3 tests on Python 3.13.16 / MySQL 8.4.13 as reported in the supplied release evidence
Production disposition	NO-GO - real-target pilot and human owner approvals still required
Scope	Complete engineering and operational reference; appendices include full relevant source and test listings

> Documentation describes the delivered v0.1.3 code and accompanying evidence. A successful test or a complete manual is not authorization for production database changes.

Explore the manual

Eight connected sections. One technical reference.

SELECT A SECTION

A FOUNDATIONS

System architecture, principles and environment setup

STRUCTURE / DESIGN OPEN PART >

B CONFIGURATION

Sources, validation, cleaning and quality rules

DATA / QUALITY OPEN PART >

C DATABASE & HISTORY

MySQL transactions, replay safety and SCD Type 2

MYSQL / SCD2 OPEN PART >

D DEVELOPER EXPERIENCE

Command line, Python interface and browser demo

CLI / PYTHON / UI OPEN PART >

E PRODUCTION OPERATIONS

Release control, observability and recovery

OPS / RECOVERY OPEN PART >

F TESTING & SECURITY

Evidence, QA, performance and auditability

VERIFICATION OPEN PART >

G REFERENCE CATALOGS

Configuration fields, code registry and tests

REFERENCE OPEN PART >

H CANONICAL APPENDICES

Full source listings, runbooks and CI evidence

SOURCE / EVIDENCE OPEN PART >

READING KEY KEYWORD FUNCTION STRING COMMENT NUMBER

Use the colored side tabs on any page to jump between sections.

Document control, evidence policy and navigation

This manual documents the exact ETL Guard v0.1.3 release provided in the attached production-readiness packet. Statements about executed tests, database restore exercises, performance and browser behavior are attributed to the packet evidence and are not presented as independent certification of a future production environment.

The source code, tests, configuration examples, operator scripts and documentation are authoritative for implementation behavior. The release status documents are authoritative for what the supplied local engineering exercise claims to have run. Production readiness remains a separate authorization process.

- Implementation behavior: source/etl-validation-toolkit/src/etlguard/ and pyproject.toml.
- Developer usage: source/etl-validation-toolkit/README.md and docs/configuration.md.
- Local execution evidence: evidence/qa/, evidence/staging/, evidence/operations/, evidence/browser/ and evidence/package/.
- Operational disposition: FINAL_STATUS.md, GO_NO_GO_DECISION.md, DEPLOYMENT_CHECKLIST.md, ROLLBACK_PLAN.md and MONITORING_PROCEDURE.md.
- Historical materials: historical/ and v0.1.1-v0.1.2 docs remain informative only; they do not supersede v0.1.3 results.

Manual organization

Part	Coverage
A	Executive orientation, traceability and architecture
B	Configuration, data contracts and validation
C	Database loading, SCD2 and atomicity
D	Developer, CLI and browser interfaces
E	Deployment, observability, recovery and release
F	Test evidence, performance and production controls
G	Exhaustive field, API, test and artifact catalogs
H	Source-code and runbook appendices

Reading and using this document

Read Parts A-F for the supported design and operating procedure. Part G is a reference index of exact declared settings, code entrypoints, tests and archive evidence. Part H reproduces implementation, test and operational-script source with line numbers so reviewers can trace behavior without switching documents. Use the Word Navigation Pane or PDF bookmarks for heading navigation; file paths are relative to the extracted release packet unless otherwise stated.

KEYWORDS	FUNCTIONS	STRINGS	COMMENTS
Violet	Cyan	Amber	Green
NUMBERS	REFERENCES	WARNINGS	NAVIGATION
Rose	Cobalt	Red	Teal

SECTION MAP

Contents and chapter navigation

Select a title below to jump to that chapter. Word and PDF heading navigation also expose all subheadings and source-module sections.

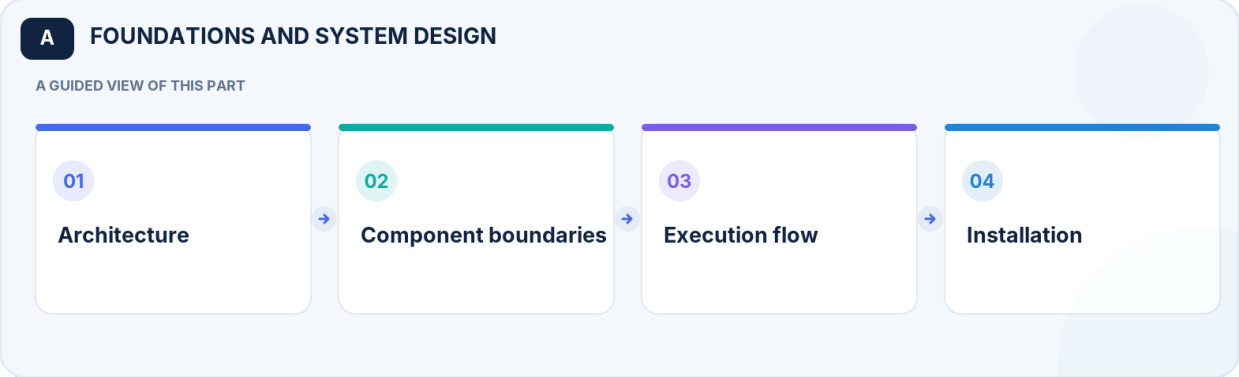
Chapter	Open section
1	1. Executive overview and bounded release status
2	2. Architecture and component boundaries
3	3. End-to-end execution sequence and state transitions
4	4. Installation footprints and dependency boundaries
5	5. Strict YAML configuration system
6	6. CSV and Excel ingestion
7	7. REST API ingestion and retry/pagination controls
8	8. Deterministic cleaning and normalization
9	9. Pydantic row validation and errors
10	10. Dataset checks, rejection thresholds and counting invariants
11	11. Reject reports and run-summary schema
12	12. MySQL 8 schema and table contracts
13	13. Append, transactional replacement, upsert and SCD2
14	14. Atomic audit, commit ledger, idempotent replay
15	15. MySQL isolation, target locks and v0.1.3 fix
16	16. SCD Type 2 time intervals and concurrency
17	17. Count reconciliation and transaction failure modes
18	18. Command-line interface guide
19	19. Public Python integration surface
20	20. Browser demo: shared Python core, local data handling
21	21. Developer extension points and versioning
22	22. Developer workstation and isolated MySQL staging
23	23. Controlled shadow pilot procedure

Chapter	Open section
24	24. Operational monitoring and incident triage
25	25. Failure, uncertain commit, rollback and compensation
26	26. Backup, restore, retention and recovery objectives
27	27. Performance envelopes and capacity limits
28	28. Release process and promotion decision
29	29. Verification provenance and exact tested environment
30	30. Testing strategy and regression taxonomy
31	31. Security, privacy and supply-chain controls
32	32. Troubleshooting playbooks
33	33. Exact-target production change packet template
34	34. Field-by-field configuration reference
35	35. Module and callable reference - runtime
36	36. Exact test-case register
37	37. Operator script, staging helper and CLI safety reference
38	38. Evidence inventory and interpretation guide
39	39. Full release file register
40	40. Appendix A - Complete v0.1.3 runtime Python source
41	41. Appendix B - Complete developer and staging scripts
42	42. Appendix C - Complete verification test source
43	43. Appendix D - Operator automation implementation
44	44. Appendix E - Configuration, CI and browser assets
45	45. Appendix F - Current operator runbooks and release decisions
46	46. Appendix G - Existing project design notes and historical documentation
47	47. Appendix H - Glossary, engineering invariants and final checklist
48	48. Appendix I - Structured operational and release evidence

Chapter	Open section
49	49. Appendix J - Release patch and changelog
50	50. Appendix K - Example input walkthrough and quality case studies
51	51. Appendix L - Worked SCD Type 2 transitions and DBA queries
52	52. Appendix M - Detailed supervised-pilot evidence checklist

PART A

FOUNDATIONS AND SYSTEM DESIGN



Technical reference and operational procedures. Use the heading navigation pane to locate topics.

1. Executive overview and bounded release status

ETL Guard is a Python 3.11+ configuration-driven validation and loading framework. It accepts CSV, Excel and REST API records; applies deterministic cleaning; validates rows using **Pydantic v2**; performs selected dataset checks; writes detailed rejection reports; and, when explicitly configured, loads accepted rows into **MySQL 8** using append, transactional replacement, upsert or slowly changing dimension Type 2 (**SCD2**) semantics.

The central correctness property is that a valid source batch either commits its target changes together with a successful audit and durable batch ledger, or leaves those transaction-scoped changes uncommitted. A stable canonical batch UUID and request hash distinguish safe identical replay from a conflicting reuse of the same ID. This guarantee assumes correctly configured InnoDB storage and coordinated writers, not arbitrary external database modifications.

Release summary

Area	Observed/reported result	Important boundary
Runtime	v0.1.3 wheel, sdist and source bundle	No unattended production authorization
Quality	137 pytest passed; 39 safety checks; Ruff and mypy passed	Claimed by included fresh local v0.1.3 evidence
Coverage	91.19% branch-aware coverage	Measured scope excludes certain example/browser modules
MySQL	8.4.13 on disposable loopback instance	No fresh 8.0 or Python 3.11 run claimed for v0.1.3
Browser	7 exercised cases; formula-safe reject CSV download	Startup downloads Pyodide/Pydantic/SQLite; browser demo is validation-only
Wheel CLI	9 installed-wheel acceptance checks	Examples are synthetic and local
Restore	160,510 rows, 24 tables verified after logical restore	Not production PITR/hardware failure

Area	Observed/reported result	Important boundary
		certification
Soak	405 batches in ~120.31 s, 3 writers, zero failures	Synthetic local workload
Promotion	Blocked; four unmet approval gates	Exact-target pilot and human approvals remain missing

A B C D E F G H

Capabilities and intentional non-capabilities

- Supports typed row validation, dataset constraints, rejection files, lineage and run summaries.
- Supports transactional MySQL target writes and a durable replay ledger where database prerequisites are satisfied.
- Provides CLI commands for validation, execution, profiling, metadata initialization, batch status and explicit SCD2 schema upgrade.
- Provides a browser-facing synthetic CSV demo that shares the Python validation core.
- Does not itself establish production scheduler authorization, real backup policy, PITR, replication safety, alert paging or human sign-off.
- Global dataset checks materialize records in memory; the framework is not an out-of-core distributed ETL engine.

Independent archive integrity inspection

The delivered packet contains a SHA256SUMS manifest covering 413 source, documentation and evidence files. During authoring, all 413 listed entries were independently hash-checked against the extracted packet and matched. This confirms consistency with the included manifest, not the provenance or trustworthiness of the original signer.

2. Architecture and component boundaries

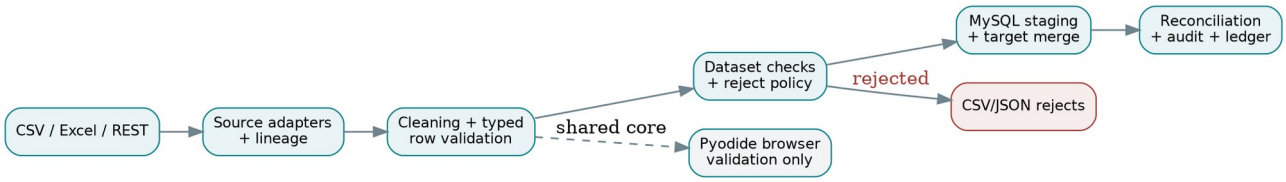


Figure 1. Logical ETL Guard processing architecture (derived from v0.1.3 source).

Sources produce SourceRecord objects with immutable per-record lineage. The shared ValidationEngine performs cleaning, Pydantic row checks and global dataset checks without importing the database loader. The orchestration Pipeline evaluates failure policies, resolves the configured Pydantic model, computes a request hash and, if not a dry run, asks MysqlLoader to stage and merge data. Local reject writers and summaries are independent artifacts; the durable success marker is the database ledger.

Component responsibilities

Layer	Primary module(s)	Responsibility
Configuration	config.py	Parse strict YAML models, safe identifiers and relative paths

Layer	Primary module(s)	Responsibility
Source connectors	sources/csv.py, excel.py, api.py	Read and enumerate source records with lineage
Transform	cleaning.py	Normalize fields, nulls, dates, numbers and cases
Validation	validation.py, checks.py, core.py	Pydantic, FK/uniqueness/null-count and policy-ready counts
Orchestration	pipeline.py	Run state machine, request hash, reports and audit coordination
Target & history	loaders/mysql.py, scd2.py	Stage, merge, lock, schema checks, and history transitions
Audit/ reconciliation	audit.py, reconcile.py	Ledger, audit, transactional count invariants
CLI	cli.py	Operator entrypoints and command exit behavior
Browser	browser_demo.py, demo/	Pyodide adaptation and browser-only validation

Deployment boundaries

- Python CLI may run locally, under CI or a supervised scheduler; it obtains a MySQL URL from an environment variable, not from the YAML literal.
- MySQL writes are absent from `etlguard validate` and from the client-side browser demo.
- The browser startup requires download of interpreter and library assets, but synthetic CSV content is processed locally in its client environment.
- Source-relative paths are resolved from the YAML directory. Target table identifiers are restricted; generated staging names begin with `stg_`.
- The explicit operator safety and release helpers reside in scripts/ and operations/; they are not implicitly executed by ordinary validation.

3. End-to-end execution sequence and state transitions



Figure 2. Simplified commit path. DDL/preflight precede atomic DML, audit and ledger writes.

1. Load and validate YAML as PipelineSettings, identifying the source, model, cleaning rules, global checks, optional target and output path.
2. Resolve the Pydantic model; instantiate source adapter with the canonical batch UUID and emit lineage-tagged SourceRecord objects.
3. Normalize each record, collect row validation errors and preserve a complete source digest. A partial source read yields a failure condition.
4. Perform foreign-key and duplicate/business-key checks, followed by dataset-level expected-count and null-percentage checks.
5. Derive RunResult metrics and a request hash covering normalized config, JSON schema and source digest.
6. Enforce global failure thresholds and replacement safeguards before any MySQL target DML.

7. For non-dry runs, initialize/check metadata and target/stage schemas; acquire target lock; check replay ledger; enter transactional staging and merge.
8. Verify reconciliation, cleanup staging rows, write the success audit and batch commit marker, commit once, and release the advisory lock.
9. Write reject CSV/JSON and summary JSON, treating post-commit report failures as warnings rather than a trigger for double load.
10. On a **failed** database action, record best-effort failure audit, expose **failed** status, and require batch-status investigation for uncertain commits.

State model

Run status	Meaning	Permitted action
dry_run	Validation completed without database target changes	Inspect reports; request approval for any later load
failed	Policy, source, config or write operation did not finish successfully	Read failures; determine durable ledger outcome before retry
success	Target and success marker completed, including replayed success	Check audit/reconciliation; avoid changing the UUID for identical retry
replayed=true	Same ID and hash found in existing committed ledger	No new rows inserted on the replay

4. Installation footprints and dependency boundaries

The **`pyproject.toml`** declares a setuptools build requiring Python 3.11+, direct dependencies on Pydantic, pandas, NumPy, openpyxl, httpx, SQLAlchemy, PyMySQL, Typer, PyYAML and Rich, and development extras for pytest, pytest-cov, Hypothesis, respx, Ruff, mypy, pandas-stubs, types-PyYAML and build. Review the exact installed versions in deployment lockfiles; the project specifies compatibility ranges, not a fully pinned universal production lock.

Interface	Needs	Does not need
Validate/profile CLI	Python environment, package and source files	MySQL server
MySQL run/init-db/status	Python dependencies plus MySQL 8.4/compatible MySQL 8, configured privileges	Browser/Pyodide
Browser demo	Modern browser, local HTTP server, startup CDN downloads	MySQL target
Staging gate	Disposable MySQL instance, dev dependencies, explicit schema consent	Production credentials
Operator drills	Dedicated disposable loopback instance with isolated identities	Live schema access

Canonical setup commands

```
python -m venv .venv
# macOS/Linux:
. .venv/bin/activate
# Windows PowerShell:
# .venv\Scripts\Activate.ps1
python -m pip install -e ".[dev]"
etlguard --help
etlguard validate src/etlguard/examples/pipelines/customers_clean.yaml
```

> Use the supplied wheel for runtime deployment if that is the approved artifact. Do not use local-only example credentials in shared or production installations.

PART B

CONFIGURATION, SOURCES AND QUALITY RULES



Technical reference and operational procedures. Use the heading navigation pane to locate topics.

5. Strict YAML configuration system

All declared pipeline configuration objects inherit from StrictSettings and prohibit unknown fields. Load errors, malformed YAML, invalid settings, unsafe names and missing file/URL selectors **fail** at configuration time. File paths are resolved relative to the YAML file. The model reference uses `module.path:ClassName` and must point to a Pydantic BaseModel subclass.

```
pipeline: customers_clean
source: {type: csv, path: ../data/customers_clean.csv}
model: etlguard.examples.models:Customer
cleaning:
  date_columns: [signup_date, end_date]
  number_columns: [credit_limit]
checks: {unique: [customer_id], max_null_pct: {email: 0.10}}
target: {table: customers_clean, mode: upsert, key: [customer_id]}
policy: {fail_if_reject_rate_above: 0.05}
output: {directory: ../../../../reports}
```

Configuration sections and contracts

Section	Type	Core purpose
pipeline	string	Filename-safe pipeline identifier, 1-128 characters
source	SourceSettings	csv/excel/api source and I/O controls
model	module:Class string	Pydantic row schema import path
cleaning	CleaningSettings	Canonicalization and duplicate suppression
checks	ChecksSettings	Foreign keys, uniqueness, cardinality and null percentages
target	TargetSettings optional	Table, mode, keys, chunk and lock controls
policy	PolicySettings	Reject gate and destructive replacement exceptions

Section	Type	Core purpose
output	OutputSettings	Report folder
database_url_env	string	Name of environment variable carrying MySQL URL

Configuration validation behavior

- Extra/unknown settings are forbidden by Pydantic ConfigDict(extra="forbid").
- CSV and Excel require `source.path`; API requires `source.url`.
- Foreign key references must use exactly one of `values` or `path`; CSV file references also require `column`.
- Target upsert/**SCD2** requires at least one key; keys cannot repeat or contain invalid SQL identifiers.
- Target table is a restricted SQL identifier <=48 characters; names beginning with stg_ and the audit table name are rejected.
- The pipeline name is filename-safe, preventing report path traversal through an unsafe value.
- Reject-rate and replace-ratio fractions use 0..1, not percentage values 0..100.
- Unknown/legacy configuration attributes **fail** fast rather than being silently ignored.

6. CSV and Excel ingestion

The CSV reader applies the configured encoding, delimiter and header handling while maintaining source-row lineage. The Excel connector handles sheet selection and header offsets. Both adapters provide SourceRecord values to the shared engine, not independent validation implementations. The source digest is accumulated from original input records for replay identity; unchanged-looking cleaned values from different raw inputs should not be assumed interchangeable.

CSV reader operating guidance

- Specify utf-8-sig for feeds that may include a UTF-8 BOM (the default).
- Match the delimiter to the actual CSV, and verify headers before a write-enabled run.
- Use `chunksize` to control pandas read chunking, but remember global checks still materialize full candidate sets.
- Capture an immutable snapshot or record source file **SHA-256** before assigning a durable batch UUID.
- Malformed input and source read exceptions must **fail** the run; do not treat a partially read feed as a valid complete dataset.
- Do not infer that parsing a 100,000-row synthetic CSV proves production export correctness.

Excel reader operating guidance

- Set `source.type: excel`, `path`, `sheet` and `header_row` to reflect actual workbook layout.
- Use the same Pydantic model and cleaning rules as the corresponding CSV feed.
- Normalize date and monetary columns explicitly because spreadsheet values may arrive as native dates/numbers or formatted strings.
- Be aware that worksheet formulas, cached calculations and data types depend on workbook provenance; inspect the workbook during acceptance.
- Large workbooks require representative memory testing; the framework is not an out-of-core workbook validator.

Lineage fields

Field	Meaning	Impact
batch_id	Canonical UUID of the execution/retry	Join rejected rows and audit to a stable batch
source_name	Source reader identifier	Provenance across files/API
source_row_number	Row/record ordinal assigned by adapter	Pinpoint problematic record
ingested_at	Capture timestamp	Operational chronology

7. REST API ingestion and retry/pagination controls

The REST connector uses httpx and supports authorization via an environment-variable name, a bounded timeout, retry/backoff behavior, and pagination. Pagination modes include none, page, offset and next-link extraction. `records_path` identifies an array nested inside response JSON; `next_path` points to the next-page cursor or link. A downstream ETL load must never proceed if the source terminates with an unreadable page after yielding earlier records.

Control	Declared default	Operational meaning
source.type	api	Select API adapter
url	None (required)	HTTP endpoint for source records
bearer_token_env	None	Optional token read from environment
timeout_seconds	15	HTTP timeout per request
max_retries	3	Maximum configured retry count, 0-10
backoff_seconds	0.5	Initial configured delay control
records_path	empty	Nested JSON record-array path
pagination.mode	none	none/page/offset/next
pagination.page_size	100	Records requested per page
pagination.max_pages	100	Limit on pagination iteration
pagination.next_path	links.next	Next cursor/link path

API source checklist

- Use server-supported pagination semantics, stable sort keys or snapshot tokens to avoid duplicates/missing rows during a changing feed.
- Confirm API authentication uses a dedicated, least-privilege secret; never print the bearer token or put it in YAML.

A B C D E F G H

- Plan retry budgets for 429, transient HTTP failures and timeouts; measure the source rate limit separately from MySQL throughput.
- Validate that `records_path` resolves to the intended record list, and that all pages are present before permitting a write.
- Persist the exact exported response/snapshot when deterministic replay is required.
- Inspect actual source adapter code when designing non-standard authentication or relative next links.

8. Deterministic cleaning and normalization

Cleaner iterates every field once: optionally normalizes column names, applies explicit rename_map, trims string values, converts configured null tokens, converts configured dates and numeric values, changes character case for designated fields, and rejects collisions in normalized output names. It does not silently drop bad dates or invalid monetary values; those reach Pydantic for explicit rejection.

Cleaning order

1. Resolve name as snake_case or original name
2. Apply mapped rename
3. Trim value if configured
4. Map configured null tokens case-insensitively to None
5. Parse configured date strings to ISO date
6. Parse configured amounts to Decimal (including negative parentheses/currency glyphs)
7. Lowercase/uppercase designated string columns
8. **Fail** on any two columns mapping to the same normalized name

Transformation	Example input	Result or behavior
snake_case_columns	Customer ID	customer_id
trim	C001	C001
null_tokens	N/A	None
date_columns	31/12/2026	2026-12-31 if format configured
number_columns	(1,234.50)	Decimal(-1234.50)
lowercase	USER@EXAMPLE.COM	user@example.com
uppercase	c001	C001
rename_map	Customer Number → customer_id	Target name under declared normalization policy

Important edge cases

- Boolean values are not treated as numeric values by decimal_number.
- Non-finite decimals and invalid numeric strings retain their original representation and should **fail** schema validation if incompatible.
- When snake_case_columns is false, rename_map keys must match raw incoming column names.

- Duplicate resulting column names are an error, not silently overwritten.
- The `drop_duplicates` rule applies only after a candidate passes row validation and reference/business-key checks.
- For auditability, clean values and raw values are stored separately in candidate objects.

9. Pydantic row validation and errors

RowValidator calls model.model_validate(cleaned). Accepted candidates retain Python-native date and Decimal values to support correct SQL parameter binding. Rejected rows preserve independent Pydantic error items rather than reducing them to one generic invalid-record flag; each error references the original raw record and its lineage.

Rejection semantics

Concept	Precise meaning
rows_read	Number of SourceRecord objects delivered by source adapter
rows_cleaned	Records transformed before candidate/global checks
rows_valid	Accepted candidates after checks and duplicate suppression
rows_rejected	Distinct source record identities with at least one reject
reject-file lines	One line per error; may exceed rows_rejected
duplicates_dropped	Exactly matching previously accepted records suppressed under policy
dataset_issues	Global conditions that fail the batch, even if individual rows are otherwise valid
source_error	Partial-feed or source read error; fails the run

Example Pydantic model fields

Customer

Field	Type	Constraint/default from source
customer_id	str	Field(min_length=4, max_length=8, pattern='^C\\d{3,7}$')
full_name	str	Field(min_length=2, max_length=80)
email	str | None	Field(default=None, max_length=150, pattern='^[^\\s@]+@[^\\s@]+\\.[^\\s@]+$')
signup_date	date	Required
end_date	date | None	None
status	Literal['active', 'inactive']	Required
credit_limit	Decimal	Field(ge=0, le=1000000, max_digits=14, decimal_places=2)

A B C D E F G H

Transaction

Field	Type	Constraint/default from source
transaction_id	str	Field(min_length=4, pattern='^T\\d{3,}$')
customer_id	str	Field(pattern='^C\\d{3,7}$')
transaction_date	date	Required
type	Literal['debit', 'credit']	Required
amount	Decimal	Field(ge=0, le=10000000, max_digits=16, decimal_places=2)
description	str | None	Field(default=None, max_length=200)

10. Dataset checks, rejection thresholds and counting invariants

DatasetChecker handles specified business-key uniqueness, allowed foreign-key references, accepted-row duplicate suppression and dataset-level row-count/null-rate checks. Foreign key references can be an inline set or an external CSV column loaded relative to the YAML directory. Dataset issues are distinct from row rejects and cause batch failure.

- Unique key: a later candidate with the same configured business-key tuple is rejected, not counted as a valid duplicate.
- Reference check: non-null FK values absent from the loaded reference set are rejected.
- Exact duplicate suppression is permitted only after earlier matching row was accepted.
- Expected row count bounds apply to the cleaned record set, not necessarily the number of accepted candidates.
- Max null percentage is the fraction of cleaned rows with a null field; values must fall between 0 and 1.
- Reject-rate gate compares distinct rejected source records with rows_read, and fails only when the rate exceeds the configured threshold.
- A row rejected for multiple fields still contributes one rejected source row and multiple error details.
- Failures are evaluated before enabling any write; partial source reads or dataset issues abort DB execution.

Illustrative acceptance calculations

```
reject_rate = rows_rejected / rows_read   # when rows_read > 0
# 100 repeated invalid records => 100 rejected, not 1 rejected + 99 duplicates
# Reconciliation: rows_valid == rows_inserted + rows_updated
# SCD2 unchanged matches count as updated/processed, separately disclosed unchanged.
```

> A batch can contain individually valid records but still fail global dataset checks. Inspect both rejected rows and dataset_issues before deciding whether to fix input or configuration.

11. Reject reports and run-summary schema

RejectWriter creates <pipeline>_<batch-uuid>_rejects.csv and _rejects.json. CSV rows are guarded against spreadsheet formula execution by prefixing suspicious cell values; JSON keeps exact values for machine processing. A run summary JSON includes the stable UUID, status, request hash, timing, counts, error aggregates, issues, failures and output locations.

Report	Primary audience	Security implication
_rejects.csv	Data correction/operator spreadsheet review	Formula escaping; can contain raw input data
_rejects.json	Programmatic review and quarantine tooling	Exact raw values may contain PII
_summary.json	Scheduler and monitoring service	Batch ID and failure details; avoid exposing secrets
etl_run_audit	Operations/DBA	Durable status and aggregate counts
etl_batch_commits	Commit-outcome and replay analysis	Request hash is an identity token, not a secret or signature

Reject CSV columns

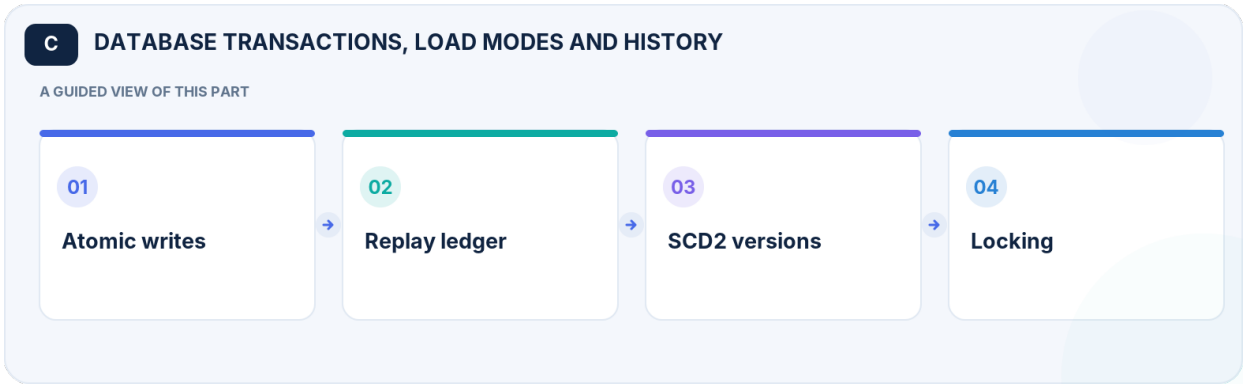
```
batch_id,source_row_number,field,error_type,error_message,raw_value,raw_row_json
```

Privacy and export controls

- Restrict report directory access; reject JSON may preserve all original source fields including personal data.
- Apply a retention/deletion policy reviewed by the data owner.
- Do not ship rejects or entire source snapshots to public CI artifacts for real customer datasets.
- Do not use spreadsheet exports as trusted executable content; CSV formula escaping is one control, not a substitute for safe handling.
- Treat generated summary paths as operational artifacts; an unavailable report after a committed DB transaction must not cause blind replay.

PART C

DATABASE TRANSACTIONS, LOAD MODES AND HISTORY



Technical reference and operational procedures. Use the heading navigation pane to locate topics.

12. MySQL 8 schema and table contracts

The loader builds SQLAlchemy tables from Pydantic model fields. Every target gets a UUID string __etl_id primary key; staging adds batch ID, source row number, and ingestion timestamp. MySQL tables must use InnoDB. The loader creates or checks schema outside the data transaction because MySQL DDL is not transactional in the same way as target DML.

Table / column	Function
etl_run_audit	One row per batch; status, timestamps and aggregate counts, compatible with older audit shape
etl_batch_commits	Unique durable batch ID, request hash, target identity, commit time and successful load counts
stg_<target>	Reusable per-target staging table with batch-scoped rows
<target>.__etl_id	UUID record identifier
<target>.business key	Upsert uniqueness or SCD2 matching key
SCD2 effective_from	UTC date/time inclusive validity start
SCD2 effective_to	Exclusive validity end; NULL for current
SCD2 is_current	Current-version status
SCD2 __etl_current_marker	Computed 1 for current/NULL for historical, used in unique guard

Storage and schema prerequisites

- Use **MySQL 8** and InnoDB for target, staging, audit and ledger.
- Grant the runtime only necessary DML on approved targets plus narrowly scoped table creation/maintenance if bootstrap is delegated.
- A pre-existing upsert target requires a unique index on exactly the configured business key and no competing unique index that could trigger an unrelated match.
- Business key columns must be NOT NULL; MySQL UNIQUE constraints permit multiple NULL entries otherwise.
- **SCD2** target requires the computed active-row marker and corresponding unique constraint; older tables need explicit maintenance-window upgrade.
- No silent schema alteration is applied to existing models; compare expected fields/types with target catalog before production.

13. Append, transactional replacement, upsert and SCD2

Mode	Database operation	Count and recovery notes
append	Insert all accepted candidates	Replay ledger prevents identical batch re-insertion
truncate_insert	DELETE target, insert accepted snapshot	Transactional DML, not TRUNCATE; default blocks empty/partial input
upsert	INSERT ON DUPLICATE KEY UPDATE	A matching business key counts as updated even if unchanged
scd2	Close current version and insert next if tracked fields change	Unchanged match counts as processed; historical versions retained

Replacement-specific operational requirements

Replacement is destructive from a business point of view even when database DELETE is rollback-capable. The default pipeline blocks empty sources and any rejected records, regardless of the general reject-rate policy, unless the operator explicitly opts in with allow_empty_replace or allow_partial_replace. The target may also require min_replace_ratio, comparing the accepted snapshot with the preexisting table size. Treat a subset sample as unsafe for production replacement.

- Pin the complete approved source snapshot and compare its row count with target baseline.
- Confirm whether foreign key references or downstream readers depend on uninterrupted complete contents.
- Set min_replace_ratio based on an approved expected lower bound; never substitute it for actual reconciliation.
- Keep legacy blank-source behavior disabled unless a complete business justification authorizes deletion.
- Test rollback of target DELETE + inserts using a disposable schema before approving replacement.

14. Atomic audit, commit ledger, idempotent replay

The durable batch ledger is keyed by canonical UUID. Its row stores a request hash derived from the source digest, validated configuration and Pydantic model schema. Successful target DML, stage cleanup, success audit and ledger reservation are committed in one database transaction. A repeated identical UUID/hash reads stored counts and does not perform new target DML. The same UUID with changed input, model, settings or target is refused.

Why both audit and ledger exist

- The audit row describes execution status and counts, including failures that may never have committed.
- The commit-ledger row asserts an actually committed successful load and is authoritative when an acknowledgement is lost.
- One successful audit without a commit ledger can indicate older release behavior or inconsistency; investigate rather than guessing.
- A failure audit is best-effort and cannot overwrite an existing committed success.
- If JSON/CSV report writing fails after the database commit, the run still records durable success and emits a warning rather than retryable failure.

Safe retry decision

```
# Stable UUID is persisted before first attempt.
etlguard batch-status 11111111-1111-4111-8111-111111111111
# Outcome A: committed + identical hash => no new load, return persisted counts.
# Outcome B: no ledger marker + verified rollback => retry same immutable batch.
# Outcome C: marker exists with different request hash => stop, investigate.
# Outcome D: database unavailable or ambiguous => stop; no fresh UUID.
```

> The UUID displayed above is a placeholder example. The operator must use the canonical UUID already assigned to the approved immutable batch.

15. MySQL isolation, target locks and v0.1.3 fix

The supplied v0.1.3 patch addresses a confirmed contention deadlock: at MySQL REPEATABLE READ, locking reads on missing audit/ledger UUID keys could create gap locks in shared metadata indexes even for separate target tables. Separate table-scoped advisory locks do not serialize those shared metadata accesses. Under coordinated concurrency, competing insert intentions caused error 1213 in the older code.

The v0.1.3 loader executes MySQL load connections under READ COMMITTED, retaining per-target GET_LOCK serialization, existing-row FOR UPDATE locking, unique batch reservations and the **SCD2** active uniqueness constraint. The package evidence includes deterministic regression before/after and an isolation reset probe; SQLAlchemy returns the pooled connection to the configured default afterward.

Locking topology

- Per-target advisory lock name is based on target table, acquired using GET_LOCK with bounded timeout.
- Missing-row metadata reads no longer acquire the same REPEATABLE READ gap-lock pattern under READ COMMITTED.
- Existing matching rows still use explicit row-level locks for **SCD2** update ordering.
- Unique constraints remain necessary for batch UUID reservations and active **SCD2** versions.
- Advisory locks are connection-scoped and must be released; on release error the connection is invalidated.
- Uncoordinated external writers may still cause deadlocks, uniqueness conflicts or history inconsistencies; production write ownership must be designed.

Change provenance

Item	Evidence
Bug	v0.1.2 cross-target shared metadata deadlock, MySQL 1213
Correction	READ COMMITTED on MySQL loader connection for transaction path

A B C D E F G H

Item	Evidence
Modified runtime files	etlguard/__init__.py; etlguard/loaders/mysql.py
Regression	Distinct-target deterministic test plus isolation reset probe
Evidence	changes/v0.1.2-to-v0.1.3.patch and evidence/qa/*regression*.log

16. SCD Type 2 time intervals and concurrency

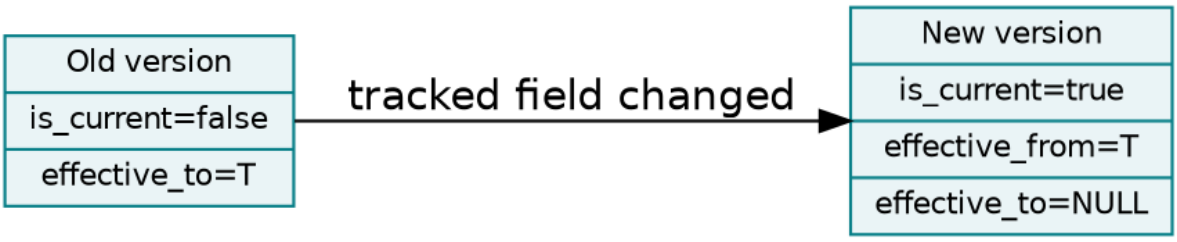


Figure 3. **SCD2** version closure and new active version, using UTC boundary T.

SCD2 keeps historical versions per business key. An unseen key inserts a current version. A key with changed tracked values closes the old version at T and inserts a new current version starting at exactly T; effective_to is exclusive. If no tracked field changed, the row is counted as processed but no physical version change occurs. UTC naive MySQL DATETIME(6) values are used consistently.

- The active marker is a persisted computed column: 1 for is_current=1, NULL otherwise.
- A UNIQUE constraint over business key plus active marker allows multiple closed history versions but blocks multiple current rows.
- The loader locks the active row before deciding whether to close it and serializes participating writers per table.
- The package reports an eight-writer/50-key exercise with 450 contiguous versions and exactly 50 active rows.
- Before production, review timezone interpretation, microsecond precision, historical backfills and interaction with external table writers.
- A schema upgrade is explicit (`etlguard upgrade-scd2`) and requires backup and change-window review.

SCD2 invariant checklist

- At most one is_current=1 row per non-null business key.
- Every closed version has non-NULL effective_to.
- The next version starts at the exact prior version end for a changed key.
- No overlapping active intervals or unexpected gaps due to concurrent updates.
- Unchanged input does not produce spurious versions.
- Effective date values are interpreted as UTC throughout readers and consumers.

17. Count reconciliation and transaction failure modes

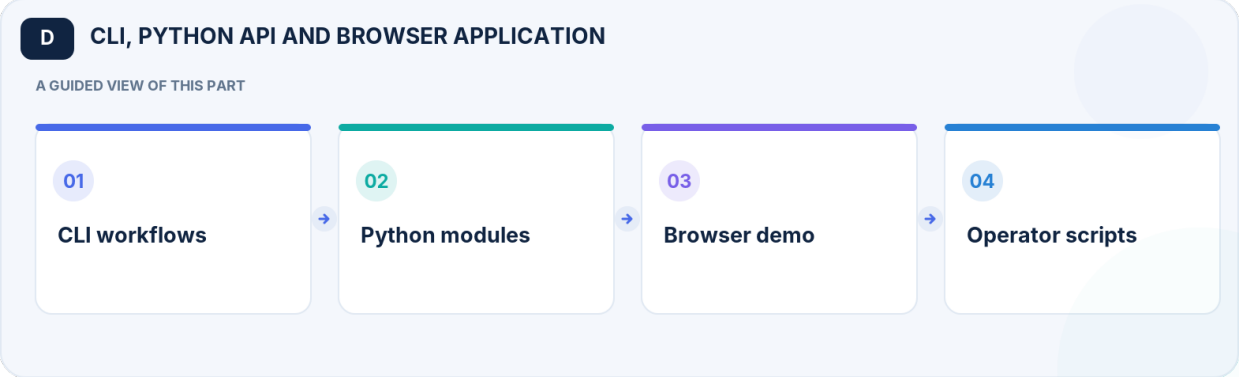
LoadCounts defines inserted, updated, unchanged, versions_created and replayed. Its loaded property equals inserted + updated, where `updated` includes processed existing keys for upsert and **SCD2**. The reconcile function enforces rows_valid == loaded and raises ReconciliationError on mismatch. A database transaction exception rolls back the stage and target DML along with success audit/ledger changes.

Failure matrix

Failure	Expected disposition	Operator response
Invalid YAML/model	Configuration failure before load	Correct config; rerun validation
Source stops mid-read	Failed; no target load	Restore/verify full source snapshot
Reject threshold exceeded	Failed; no merge	Correct data or explicitly review policy
Empty/partial replacement	Blocked by default	Approve snapshot and policy only after review
Schema/engine conflict	Fail closed	DBA fixes schema or privileges
Lock timeout	No blind retry	Inspect contention, hold batch UUID
Reconciliation mismatch	Abort transaction	Investigate rows/loader logic
Audit write exception before commit	Rollback target, staging and ledger	Resolve audit issue; verify ledger
COMMIT acknowledgement lost	Unknown to caller, durable outcome may exist	batch-status + reconcile before retry
Report file I/O after committed load	Warning but durable success remains	Repair report path; do not reload

PART D

CLI, PYTHON API AND BROWSER APPLICATION



Technical reference and operational procedures. Use the heading navigation pane to locate topics.

18. Command-line interface guide

The installed entrypoint is `etlguard` (Typer). The CLI exposes validate, run, profile, init-db, batch-status and upgrade-scd2. The main validate/run path uses exit 0 for successful/dry-run results, 1 for failed processing, and 2 for explicit configuration errors; individual subcommands document their own failure handling in cli.py.

Validate without DB

```
etlguard validate path/to/pipeline.yaml
```

Runs all source and quality checks, writes reports, never loads MySQL.

Transactional load

```
etlguard run path/to/pipeline.yaml --batch-id <canonical-uuid>
```

Execute approved immutable batch with stable retry ID.

Inspect CSV

```
etlguard profile path/to/feed.csv
```

Print inferred column types, null rate, distinct count and ranges.

Prepare audit metadata

```
etlguard init-db --database-url-env ETLGUARD_DATABASE_URL
```

Create audit and ledger tables; may perform DDL.

Find durable outcome

```
etlguard batch-status <canonical-uuid> --database-url-env ETLGUARD_DATABASE_URL
```

Query batch ledger and associated audit.

Upgrade SCD2 guard

```
etlguard upgrade-scd2 path/to/scd2.yaml --yes
```

Explicit DDL migration; maintenance approval required.

Exit, logs and automation handling

- Treat a nonzero status as a gate; never ignore it in a scheduler.
- Inspect summary JSON and durable batch-status rather than parsing only console colors.
- Persist the canonical UUID and immutable input hash before invoking run.
- After timeouts or lost acknowledgements, read the ledger before deciding to retry.
- The CLI does not implement a production scheduler; scheduling policy is outside the command.
- Keep database credentials out of shell history, source control and report attachments.

19. Public Python integration surface

The package exposes Pipeline and the shared ValidationEngine/typed configuration objects for application integration. The exact methods and signatures are catalogued in Part G and fully reproduced with line references in the source appendix. A Python integration should load config with `load_config`, obtain a pipeline through `Pipeline.from_yaml`, and treat RunResult as the status contract. Do not bypass write gates by calling internal MysqlLoader methods casually.

```
from etlguard.pipeline import Pipeline

outcome = Pipeline.from_yaml('path/to/pipeline.yaml').run(dry_run=True)
assert outcome.status in {'dry_run', 'failed'}
print(outcome.rows_read, outcome.rows_rejected, outcome.failures)
```

Core types

Type	Fields/behavior
PipelineSettings	Strict Pydantic nested configuration
SourceRecord / Lineage	Raw record plus source, ordinal, timestamp and batch ID
Candidate	Parsed data, cleaned/raw copies and lineage
CoreResult	Read/valid/rejected counts, issues and source digest
RunResult	Status, request hash, replay/commit, counts, failures and reports
LoadCounts	Reconciled inserted/updated/unchanged/version/replay counts
BatchConflictError	Same canonical UUID reused for a different committed request
ReconciliationError	Load counts fail the accepted-row invariant

20. Browser demo: shared Python core, local data handling

The included browser demo is designed for synthetic CSV validation. A local Python HTTP server serves the static UI; JavaScript boots Pyodide and loads the shared Python validation core. The browser accepts Customer or Transaction models, sample datasets or a UTF-8 CSV up to 5 MB, and allows reject CSV export. MySQL is not accessed by the browser demo.

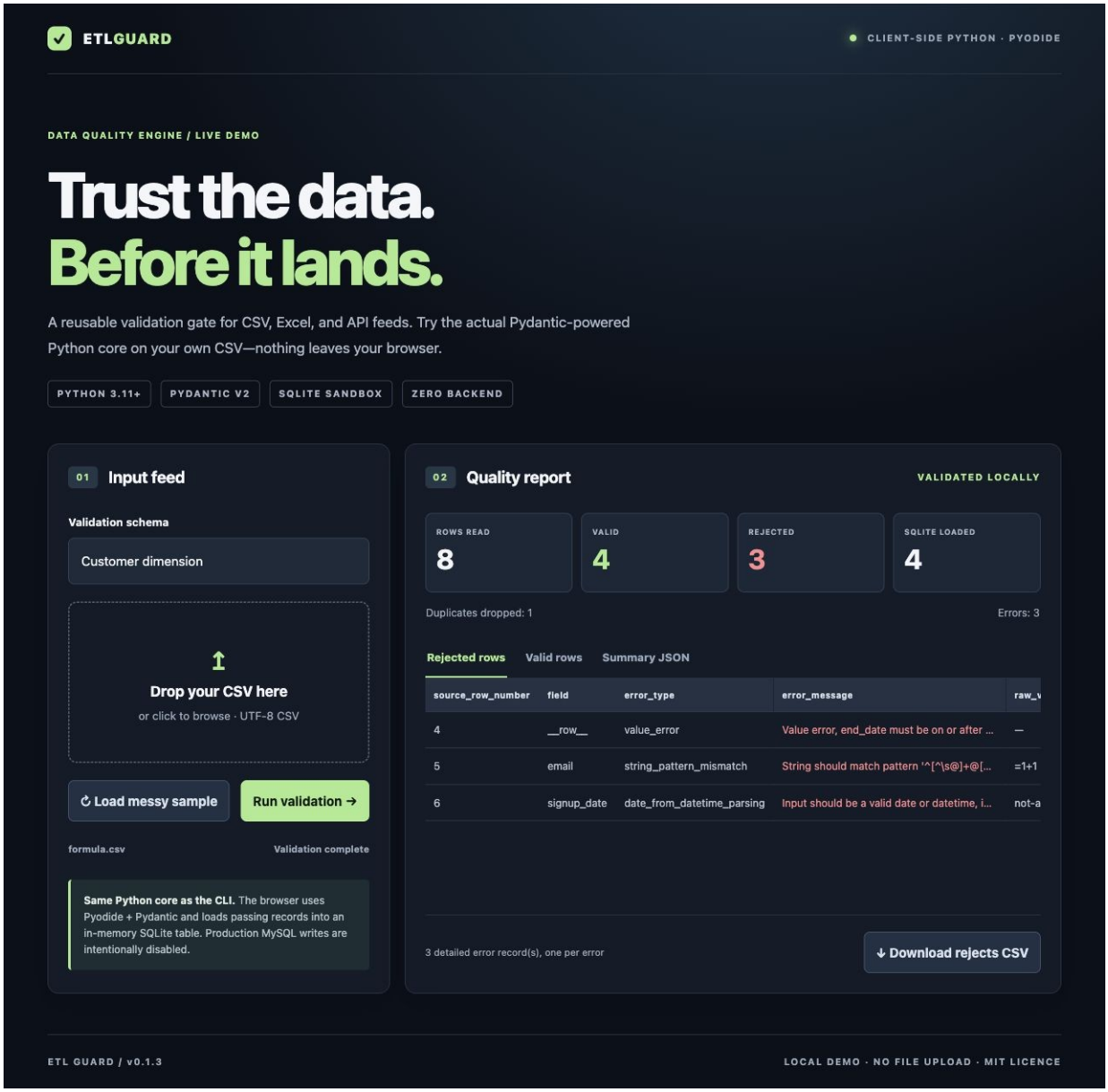


Figure 4. Screenshot included in the v0.1.3 browser verification evidence.

Recorded browser cases

Case	Reported observation
customer_sample	loaded=4, read=8, rejected=3, status=Validation complete, valid=4
transaction_sample	loaded=3, read=6, rejected=2, status=Validation complete, valid=3

A B C D E F G H

SECTION MAP

Case	Reported observation
empty.csv	downloadDisabled=True, read=—, status=Could not read CSV: The selected CSV is empty
too-large.csv	downloadDisabled=True, read=—, status=Browser demo accepts CSV files up to 5 MB
unsupported.txt	downloadDisabled=True, read=—, status=Select a .csv file
malformed.csv	downloadDisabled=True, read=—, status=Validation error: ValueError: source read failed: ValueError: A CSV row has more fields than its header
formula.csv	downloadDisabled=False, read=8, status=Validation complete

Security boundary

- Only synthetic/test data should be used unless a security/data owner approves the browser deployment and dependency supply chain.
- Interpreter/library downloads during startup create network trust and availability dependencies.
- Reject exports can contain raw field values; formula escaping mitigates spreadsheet formula injection.
- The 5 MB client-side file size cap is a browser demo limit, not an ETL loader ingestion cap.
- Do not characterize browser SQLite as the same persistence path as the MySQL CLI.

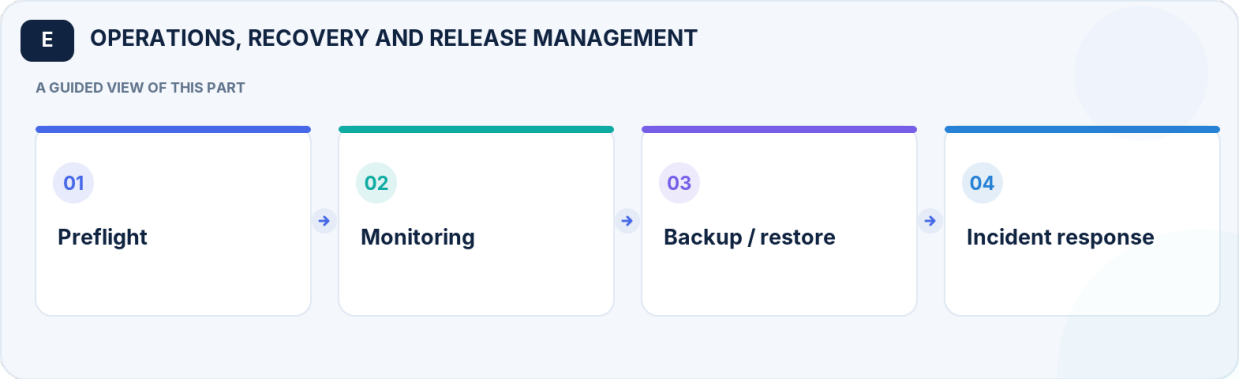
21. Developer extension points and versioning

New source adapters should preserve the SourceRecord/Lineage contract; new models should inherit Pydantic BaseModel; new dataset checks should distinguish row-level errors from dataset issues; and new database load modes must preserve the ledger/atomic audit/reconciliation invariants. Python semantic version, wheel contents and source manifests should change together after verified fixes.

- Implement a new source with `read(batch_id)` yielding lineage-tagged SourceRecord objects.
- Introduce new cleaning options through strict Pydantic settings and explicit tests.
- Protect report serialization of new Pydantic value types and raw error data.
- For new target modes, define counts/reconciliation semantics and idempotent replay behavior.
- Add tests for input corruption, unchanged UUID retry, changed hash conflict and an interrupted commit.
- Rebuild wheel/sdist and rerun full QA on the actual supported version matrix.

PART E

OPERATIONS, RECOVERY AND RELEASE MANAGEMENT



Technical reference and operational procedures. Use the heading navigation pane to locate topics.

22. Developer workstation and isolated MySQL staging

The delivered packet offers Docker **MySQL 8**.4 staging and standalone local MySQL provisioning helpers. The local run documented in FINAL_STATUS.md used Homebrew **MySQL 8**.4.13 in an isolated loopback-only instance with scoped schemas and credentials. The portable Docker Compose staging target binds localhost port 3308 with tmpfs database storage. Neither mode is a production database.

```
python -m pip install -e ".[dev]"
docker compose -f docker-compose.mysql8-staging.yml up -d
# Use a disposable schema and only local test credentials
export
  ETLGUARD_TEST_MYSQL_URL='mysql+pymysql://<testuser>:<password>@127.0.0.1:3308/etlguard_ci'
python scripts/verify_mysql8_staging.py \
  --confirm-disposable-schema etlguard_ci \
  --execute-tests --report staging-gate-report.json
```

Safety boundaries enforced by verification scripts

- Require explicit acknowledgement of disposable schema; never treat an arbitrary database URL as safe.
- Reject root/global-privilege or production-style targets under staging assumptions.
- Keep test schemas isolated and credentials ephemeral.
- Do not rerun a write with a newly generated UUID merely because acknowledgement was lost.
- Record JSON evidence and test log status, including any skipped tests.
- Preserve an immutable report directory for each serious acceptance run.

23. Controlled shadow pilot procedure

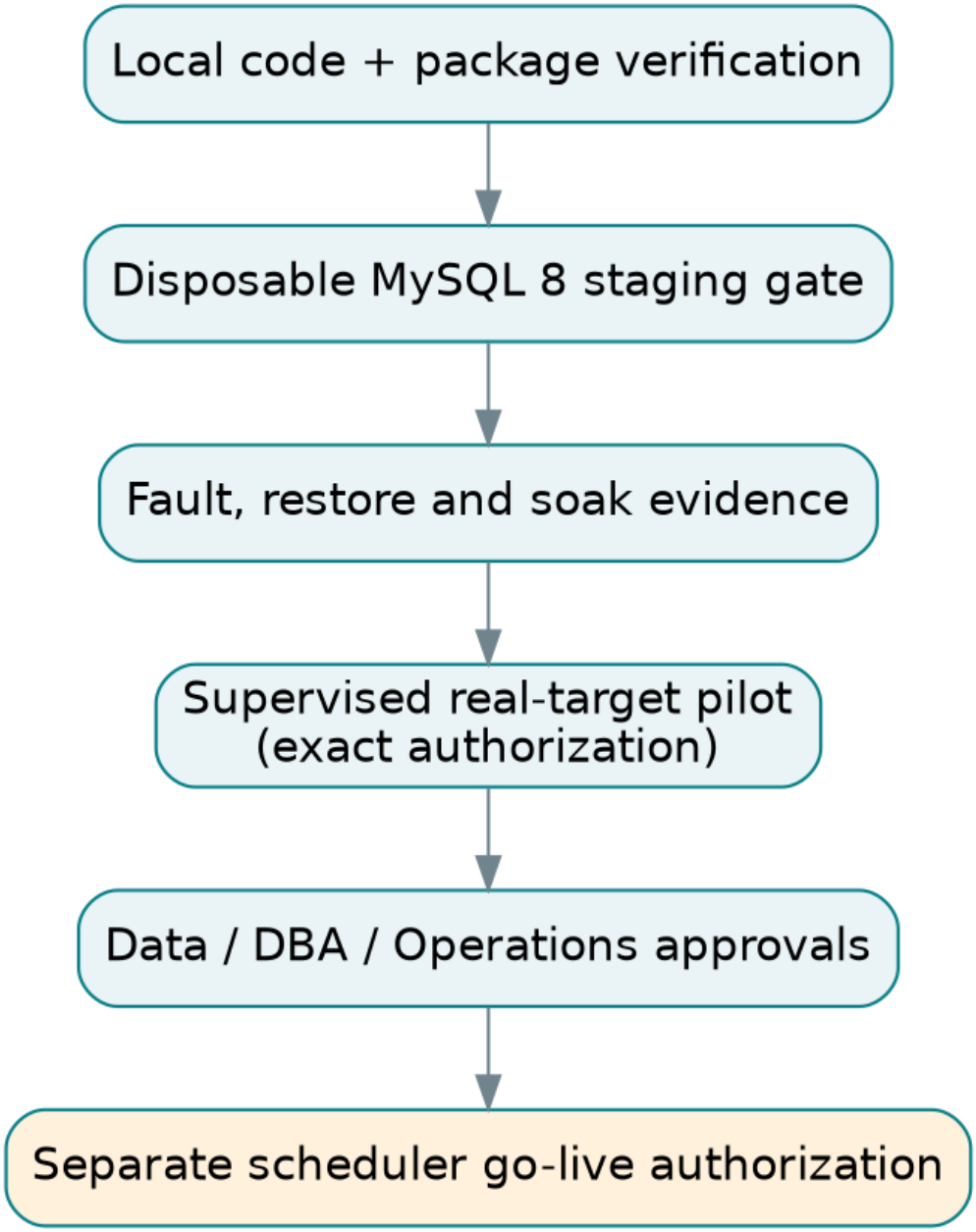


Figure 5. Technical verification gates followed by exact-target human approvals.

1. Obtain a representative, approved, immutable source snapshot and a reviewed YAML configuration.
2. Prepare the shadow manifest using prepare_shadow_pilot.py; pin source/config hashes and one canonical UUID.
3. Validate locally without a database and inspect rejects, dataset issues, counts and schema.
4. Verify that the destination is an explicitly disposable shadow schema with least-privilege grants.
5. Run the MySQL staging gate and require zero skipped integration tests before the shadow write.
6. Execute one load with the pinned manifest and UUID, record ledger, audit and target fingerprints.
7. Run independent shadow reconciliation against the accepted source and physical target.
8. Replay the identical UUID and input; confirm the target fingerprint remains identical.
9. Attempt a changed-input same-ID replay in staging; expect a conflict, not a second load.

10. Review monitoring, locks, batch status, backup restore readiness and ownership before requesting a real-target pilot.

> The included controlled-pilot/pilot-template.yaml is deliberately non-executing. An approved host, table, ticket, owner and rollback plan must be supplied separately.

24. Operational monitoring and incident triage

Monitoring correlates one stable UUID with the source/config hashes, start/end UTC, read/cleaned/valid/rejected/loaded counts, duration, rejection rate, per-target lock time, durable ledger status and independent reconciliation. Database measurements include CPU/RSS, disk free, running transactions, InnoDB lock/deadlock/log wait counters and indexed business-key query behavior.

Suggested signal families

Signal family	Trigger	Response
Correctness	Ledger/audit mismatch, target fingerprint mismatch, unexplained SCD2 history	Immediate stop, preserve artifacts, DBA/data-owner incident
Source integrity	Partial feed, changed immutable source, wrong schema	Stop; re-export/verify snapshot
Write safety	Replacement guard, engine/unique-key failure	Do not bypass without owner approval
Latency/locks	Proposed p95, lock timeout or batch latency breached	Pause affected batch and investigate bottleneck
Capacity	Memory/disk pressure, excessive transactions or wait counters	Rate-limit, resize after evidence, assess backpressure
Alerting	Production receiver unreachable or acknowledgement missing	Fail over via approved on-call process

Sample monitoring record fields

```
batch_id,started_at_utc,finished_at_utc,source_sha256,config_sha256,
rows_read,rows_valid,rows_rejected,rows_loaded,rows_inserted,rows_updated,
reject_rate,duration_seconds,durable_commit,replayed,target_fingerprint,
reconciliation_status,lock_wait_seconds,alert_delivery_status
```

> These are proposed monitoring fields for an external operational record, not a claim that every key exists as a native RunResult property.

25. Failure, uncertain commit, rollback and compensation

On a **failed** or interrupted run, first determine whether the database commit completed. Do not infer rollback from a crashed client or lost acknowledgement. Inspect the durable batch ledger and audit using the stable UUID after restoring DB connectivity. A committed transaction cannot be reversed through a later SQL ROLLBACK; compensating SQL or restore/PITR requires DBA-led approval and attention to legitimate subsequent writes.

1. Disable the specific scheduler job/producer; preserve source hash, config, batch ID, logs and error status.

2. Check DB connectivity and query etlguard batch-status for the original UUID.
3. If ledger exists, verify request hash and target identity and reconcile physical rows; do not reload.
4. If there is no ledger and verified rollback/no target change, remediate cause and retry the SAME immutable batch UUID.
5. If outcome remains uncertain or DB unavailable, hold writes and escalate; never guess by changing the UUID.
6. For committed incorrect data, open approved DBA change with snapshot, backup, rollback owner and downstream-impact plan.
7. Restore to a separate schema and independently verify before any exact-target compensation.
8. Document incident timeline and compare before/after counts, keys, audit and ledger.

Tested local fault families and limits

Fault exercise	Evidence filename	Scope limitation
Audit write injection	evidence/operations/audit_rollback.json	Synthetic disposable DB
Lost COMMIT acknowledgement	evidence/operations/lost_commit_ack.json	Injected post-commit client failure
Application kill	evidence/operations/process_kill.json	OS process termination scenario
MySQL server SIGKILL	evidence/operations/server-crash-recovery.json	Local server crash, not physical power loss
Lock timeout	evidence/operations/lock_timeout.json	Bounded test conditions
Changed immutable input	evidence/operations/changed_input.json	Hash/UUID conflict response
Logical backup/restore	evidence/operations/backup_restore.json	Snapshot restore, not PITR/replica failover

26. Backup, restore, retention and recovery objectives

The packet records a logical backup of 24 tables / 160,510 rows, restored into an empty separate schema and verified through row fingerprints. Reported backup time was 0.308 seconds, restore 1.787 seconds, independent verification 1.614 seconds, for 3.401 seconds restore plus verification. A deliberately post-backup transaction was absent in the restored database, establishing the difference between snapshot recovery and point-in-time recovery.

Recovery planning requirements

- Document backup scope, snapshot consistency method, encryption, keys, retention and off-host protection.
- Choose an owner-approved RPO and RTO against real data size, storage, replication topology and failure modes.
- Prove restore to a separate schema before applying recovery in place.
- Verify row counts plus content fingerprints and business key histories, not merely server start-up.
- Preserve the ledger/audit consistency point across target tables.
- Do not assume local logical recovery time predicts large production database recovery.
- Review retention of audit/ledger/reject artifacts so historical recovery and compliance expectations are aligned.

27. Performance envelopes and capacity limits

Synthetic rows	E2E seconds	Rows/sec	Combined RSS MiB	Retry seconds
10,000	1.327	7,534	249.546875	0.285
50,000	6.999	7,144	383.09375	1.507
100,000	15.731	6,357	563.703125	2.918

The release reports synthetic upsert tests at 10,000, 50,000 and 100,000 rows on an 8-core/8-GiB local host and 256-MiB MySQL buffer pool, with 1,000-row SQL chunks. The measured E2E duration includes ingest, transformation, validation, staging, merge, reconciliation and audit/ledger commit. The reported 120.31-second soak used three concurrent writers, each submitting 2,000-record batches, and completed 405 batches without failure.

Metric	Observation
Soak duration	120.31 seconds
Concurrent writers	3
Completed batches	405
Batch size	2000
Peak combined RSS	264.1 MiB
Probe p95	45.68 ms
Production soak?	False

Interpretation cautions

- The synthetic Customer schema, local disk and MySQL tuning are not necessarily representative of production.
- Global checks materialize rows in memory; model complexity and field widths can change memory/performance scaling.
- No production SLO or throughput target is approved solely from this benchmark.
- Monitor tail latency, lock wait, retry rates and transaction side effects, not throughput alone.
- Production capacity acceptance requires representative source distribution, indexes, downstream readers, concurrent external writers, network RTT and replication.

28. Release process and promotion decision

The current v0.1.3 package is complete for the tested local technical envelope. It has NOT been authorized to write to a real target or enable an unattended scheduler. The read-only promotion assessor returned **blocked** with four missing gates: supervised real-target pilot, genuine data-owner approval, genuine database-owner approval, and genuine operations-owner approval. The archived operational evidence does not replace those human decisions.

Gate	Current status	Required completion evidence
Source/wheel/sdist integrity	Passed in packet and independently manifest-checked	Preserve provenance/checksums
Local MySQL 8.4 staging	Passed in supplied evidence	Repeat for exact chosen target/version if required
Operational fault/soak/restore	Passed for synthetic local envelope	Evaluate on representative infrastructure
Approved real-target pilot	PENDING	Specific host/schema/table, snapshot, UUID, ticket, window and owner
Data-owner sign-off	PENDING	Validated business totals/quality expectations
DBA sign-off	PENDING	Schema, security, backup/PITR, isolation, RPO/RTO
Operations sign-off	PENDING	Alerts, monitoring, escalation, rollback plan
Unattended scheduler	NO-GO	Separate written go-live authorization

PART F

QUALITY EVIDENCE, TEST DESIGN AND AUDITABILITY



Technical reference and operational procedures. Use the heading navigation pane to locate topics.

29. Verification provenance and exact tested environment

The packet explicitly separates historical v0.1.2 GitHub CI jobs from fresh v0.1.3 tests. The latter were performed on 11 November 2026 using Python 3.13.16 and a private **MySQL 8**.4.13 instance, and are documented by JSON, XML and text logs under evidence/. The packet does not claim a fresh v0.1.3 **MySQL 8**.0/Python 3.11 matrix or independent production-target reproduction.

Evidence category	Declared result	Canonical artifact
pytest	137 passed / 0 skipped	evidence/qa/pytest.xml
Operator/safety unittest	39 passed	evidence/staging/operator_safety_tests.log
Coverage	91.19% branch coverage	evidence/qa/coverage.json
Style/type	Ruff and mypy passed	evidence/qa/ plus operators logs
MySQL staging	Passed, source hashes and replay	evidence/staging/staging-gate-report.json
Browser	7 cases; downloaded reject output escaped	evidence/browser/browser-report.json
Wheel CLI	9 checks	evidence/package/
Fault/recovery	Audit rollback, process/server crashes, lost ack	evidence/operations/*.json
Promotion	Blocked, exactly 4 missing gates	evidence/promotion-review.json

Independent verification performed for this manual

SHA256SUMS was checked against the extracted packet: 413 entries **passed** and none **failed**. This confirms internal file integrity of the provided archive manifest. No claims are made that the documentation generation process re-executed the local MySQL crash drills.

30. Testing strategy and regression taxonomy

The test suite covers row models, cleaning, API transport, CSV and Excel sources, CLI and pipeline lifecycle, SQLite-backed loader simulations, **MySQL 8** integration, **SCD2**/DDL, production safety, property-based cleaning and regression edges. Separate `scripts/test_*.py` exercises shadow-pilot and staging-assessor guardrails. The detailed test-by-test index in Part G links every defined test function to its source line.

Layer	Typical checks	Release requirement
Unit	Normalizers, model errors, config constraints	Fast, repeatable on every commit
Property-based	Cleaning idempotence / generated inputs	Hypothesis in clean Python environment
Integration	Real MySQL transaction/replay/SCD2	Zero skipped on chosen supported runtime
Transport	HTTP retry and malformed API output	Mocked boundary errors and pagination
Safety	Replacement, UUID conflict, source tampering	Fail closed under stress/failure
Browser	Input types, 5MB gate, exports, shared model	Real headless browser acceptance
Operations	Logical restore, process crash, database crash, soak	Authenticated evidence, synthetic limits clear
Release	Wheel install, hashes, promotion assessor	No unsupported production auto-approval

Acceptance rubric

- No skipped critical integration tests for the exact release candidate and intended runtime matrix.
- Every fault injection proves no duplicate committed target changes on an identical retry.
- Changed source/config/target with the same UUID is rejected.
- Audit, ledger and target reconcile independently after success and failure.
- Recoverable incidents produce enough durable information to make an unambiguous retry decision.
- Human approval is still required for a production target, even when all disposable tests **pass**.

31. Security, privacy and supply-chain controls

Security controls need to be applied to the context where ETL Guard is deployed. The package shows SQL identifier validation, environment variable credential references, browser CSV formula escaping, **fail**-closed staging test scripts and report content preservation. These controls do not substitute for network isolation, credentials rotation, secrets management, privacy policies, code signing or production IAM review.

- Secrets: store DB URLs and API bearer values in secret-managed environment variables, never in YAML, screenshots or CI artifact logs.

- Least privilege: give the runtime narrowly scoped target DML and read operations; reserve table DDL and **SCD2** upgrades for controlled maintenance identities.
- Integrity: use immutable source/config digests, stable UUIDs, build checksums and independent target reconciliation.
- Injection: restrict SQL table/column identifiers and escape CSV formula-leading characters in report exports.
- Personal data: raw rejects may reproduce sensitive source fields; apply retention, access control, encryption and regulated-data processing policies.
- Browser dependencies: pin/review client-side dependency supply chain, serve over trusted origin and scope data types supported.
- Operational evidence: prevent logs from leaking full passwords, raw tokens or user data; grant review access by role.
- Threat boundaries: **SHA-256** file matching is not a cryptographic signature or proof of trusted origin.

32. Troubleshooting playbooks

Config refused / exit 2

Inspect YAML location, strict fields, model import, target identifier, path resolution

> Operator action: Run etlguard validate and inspect ConfigError message

Unexpected rejected rows

Inspect per-field Pydantic errors, case/trim/date/number configuration

> Operator action: Compare raw_row_json and cleaned field values

Duplicate conflict

Check `checks.unique` and accepted-row duplicate semantics

> Operator action: Avoid weakening business-key gate to force load

API feed interrupted

Verify pagination, records_path, retries and HTTP credentials

> Operator action: Re-export immutable full source, not a partial feed

No database URL

Check configured database_url_env name and environment secret injection

> Operator action: Do not put credentials into source YAML

MySQL engine check failed

Confirm every participating table uses InnoDB

> Operator action: DBA migration/maintenance review

Upsert unique index conflict

Confirm exact unique business key and remove competing uniqueness where appropriate

> Operator action: Validate DDL under DBA supervision

SCD2 guard missing

Run dry-run and schema inspection; use upgrade-**scd2** in maintenance only

> Operator action: Backup and verify no multiple active rows

Deadlock 1213 / timeout

Inspect isolation, external writers, lock wait, release and deadlock trace

> Operator action: Never blind retry uncertain committed batches

Lost commit acknowledgement

Query ledger by original batch UUID and request hash

> Operator action: Decide based on durable row and physical reconciliation

Report creation failed

Inspect file permissions/disk, summary warning and durable success flag

> Operator action: Repair artifact path; do not duplicate load

Browser blank or blocked

Verify local server, internet CDN fetch, supported CSV type/size

> Operator action: Inspect browser console, do not presume MySQL problem

Performance regression

Compare row width, Pydantic model, indexes, chunk sizes, locks, memory

> Operator action: Reproduce representative workload and inspect MySQL status

Production promotion blocked

Read reasons in promotion-review.json and original owner evidence

> Operator action: Obtain genuine pilot sign-off and separate scheduler approval

33. Exact-target production change packet template

Before a controlled production pilot, the following information must be filled by authorized owners. No values should be inferred from the local disposable environment. Even after the technical gate passes, the release assessor is only a review aid.

Change ticket

Approved identifier, window/timezone and rollback owner

Target

Host/database/schema/table and approved MySQL minor version

Source snapshot

Approved dataset identity, **SHA-256** and expected counts

Pipeline

Reviewed YAML hash, source adapter, model and target load mode

Batch ID

Persisted canonical UUID assigned before first attempt

Security

Runtime grants, encryption, secret delivery, evidence retention

Backup/restore

Real target snapshot, verified independent restore and RPO/RTO

Quality

Business thresholds, acceptable rejection rate and key uniqueness

Performance

Target SLO, allowed memory/CPU/disk load and observation window

Monitoring

Actual receiver, alert owner, escalation contacts and success/failure signals

Rollback

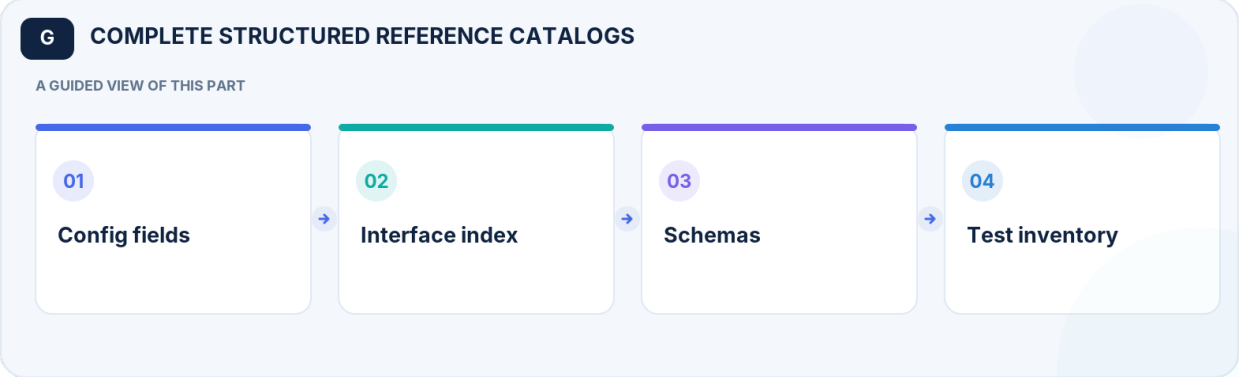
Stop conditions, compensating change or PITR procedure and reconciler

Approvals

Data owner, DBA and operations signatures, followed by separate scheduler authorization

PART G

COMPLETE STRUCTURED REFERENCE CATALOGS



Technical reference and operational procedures. Use the heading navigation pane to locate topics.

34. Field-by-field configuration reference

StrictSettings

Field	Declared type	Declared default

PaginationSettings

Field	Declared type	Declared default
mode	Literal['none', 'page', 'offset', 'next']	'none'
page_param	str	'page'
offset_param	str	'offset'
limit_param	str	'limit'
page_size	int	Field(default=100, ge=1)
start	int	Field(default=1, ge=0)
max_pages	int	Field(default=100, ge=1)
next_path	str	'links.next'

PaginationSettings.mode

Choose none, page, offset or next. Must match actual API pagination.

> Source: src/etlguard/config.py:L23. Declared type Literal['none', 'page', 'offset', 'next']; default/constraint 'none'.

PaginationSettings.page_param

Query parameter for page-based traversal.

> Source: src/etlguard/config.py:L24. Declared type str; default/constraint 'page'.

PaginationSettings.offset_param

Query parameter for offset traversal.

> Source: src/etlguard/config.py:L25. Declared type str; default/constraint 'offset'.

PaginationSettings.limit_param

Query parameter for limit/page size.

> Source: src/etlguard/config.py:L26. Declared type str; default/constraint 'limit'.

PaginationSettings.page_size

Requested page size; service may cap independently.

> Source: src/etlguard/config.py:L27. Declared type int; default/constraint Field(default=100, ge=1).

PaginationSettings.start

Initial page/offset value; >=0.

> Source: src/etlguard/config.py:L28. Declared type int; default/constraint Field(default=1, ge=0).

PaginationSettings.max_pages

Upper bound on API pagination cycles; failure of complete feed remains a stop.

> Source: src/etlguard/config.py:L29. Declared type int; default/constraint Field(default=100, ge=1).

PaginationSettings.next_path

Nested JSON path to next-link value.

> Source: src/etlguard/config.py:L30. Declared type str; default/constraint 'links.next'.

SourceSettings

Field	Declared type	Declared default
type	Literal['csv', 'excel', 'api']	(required)
name	str | None	None
path	str | None	None
url	str | None	None
delimiter	str	','
encoding	str	'utf-8-sig'
header_row	int	Field(default=1, ge=1)
chunksize	int	Field(default=10000, ge=1)

A B C D E F G H

Field	Declared type	Declared default
sheet	str | int	0
skip_rows	int	Field(default=0, ge=0)
records_path	str	''
bearer_token_env	str | None	None
timeout_seconds	float	Field(default=15, gt=0)
max_retries	int	Field(default=3, ge=0, le=10)
backoff_seconds	float	Field(default=0.5, ge=0)
pagination	PaginationSettings	Field(default_factory=PaginationSettings)

SourceSettings.type

One of csv, excel, api; chooses adapter.

> Source: src/etlguard/config.py:L34. Declared type Literal['csv', 'excel', 'api']; default/constraint required.

SourceSettings.name

Optional source label overriding inferred path/URL for reporting.

> Source: src/etlguard/config.py:L35. Declared type str | None; default/constraint None.

SourceSettings.path

Required for csv/excel; interpreted relative to pipeline YAML.

> Source: src/etlguard/config.py:L36. Declared type str | None; default/constraint None.

SourceSettings.url

Required for API; verify scheme, credentials and allowlist externally.

> Source: src/etlguard/config.py:L37. Declared type str | None; default/constraint None.

SourceSettings.delimiter

CSV field separator; must be nonempty.

> Source: src/etlguard/config.py:L38. Declared type str; default/constraint ','.

SourceSettings.encoding

CSV encoding, defaults to utf-8-sig for BOM tolerance.

> Source: src/etlguard/config.py:L39. Declared type str; default/constraint 'utf-8-sig'.

SourceSettings.header_row

One-based Excel/CSV header position, >=1.

> Source: src/etlguard/config.py:L40. Declared type int; default/constraint Field(default=1, ge=1).

SourceSettings.chunksize

Rows per read chunk; does not make global checks out-of-core.

> Source: src/etlguard/config.py:L41. Declared type int; default/constraint Field(default=10000, ge=1).

SourceSettings.sheet

Excel sheet name or index, 0 default.

> Source: src/etlguard/config.py:L42. Declared type str | int; default/constraint 0.

SourceSettings.skip_rows

Number of initial rows to skip, >=0.

> Source: src/etlguard/config.py:L43. Declared type int; default/constraint Field(default=0, ge=0).

SourceSettings.records_path

Nested path for records in API JSON.

> Source: src/etlguard/config.py:L44. Declared type str; default/constraint ''.

SourceSettings.bearer_token_env

Environment variable name for API bearer secret.

> Source: src/etlguard/config.py:L45. Declared type str | None; default/constraint None.

SourceSettings.timeout_seconds

HTTP request timeout, >0.

> Source: src/etlguard/config.py:L46. Declared type float; default/constraint Field(default=15, gt=0).

SourceSettings.max_retries

Bounded API retry count between 0 and 10.

> Source: src/etlguard/config.py:L47. Declared type int; default/constraint Field(default=3, ge=0, le=10).

SourceSettings.backoff_seconds

API retry backoff parameter, >=0.

> Source: src/etlguard/config.py:L48. Declared type float; default/constraint Field(default=0.5, ge=0).

SourceSettings.pagination

Nested PaginationSettings object.

> Source: src/etlguard/config.py:L49. Declared type PaginationSettings; default/constraint Field(default_factory=PaginationSettings).

CleaningSettings

Field	Declared type	Declared default
trim	bool	True
snake_case_columns	bool	True
null_tokens	list[str]	Field(default_factory=lambda: ['', 'NA', 'N/A',

A B C D E F G H

Field	Declared type	Declared default
		'null', '-'])
rename_map	dict[str, str]	Field(default_factory=dict)
date_columns	list[str]	Field(default_factory=list)
date_formats	list[str]	Field(default_factory=lambda: ['%Y-%m-%d', '%Y/%m/%d', '%d/%m/%Y', '%m/%d/%Y', '%d-%m-%Y', '%Y.
number_columns	list[str]	Field(default_factory=list)
lowercase	list[str]	Field(default_factory=list)
uppercase	list[str]	Field(default_factory=list)
drop_duplicates	bool	True

CleaningSettings.trim

Trim strings before type validation.

> Source: src/etlguard/config.py:L63. Declared type bool; default/constraint True.

CleaningSettings.snake_case_columns

Canonicalize incoming field names to snake_case.

> Source: src/etlguard/config.py:L64. Declared type bool; default/constraint True.

CleaningSettings.null_tokens

Case-insensitive strings mapped to None.

> Source: src/etlguard/config.py:L65. Declared type list[str]; default/constraint Field(default_factory=lambda: ['', 'NA', 'N/A', 'null', '-']).

CleaningSettings.rename_map

Source-to-model column name changes; normalization-sensitive.

> Source: src/etlguard/config.py:L66. Declared type dict[str, str]; default/constraint Field(default_factory=dict).

CleaningSettings.date_columns

Columns to parse via date_formats.

> Source: src/etlguard/config.py:L67. Declared type list[str]; default/constraint Field(default_factory=list).

CleaningSettings.date_formats

Ordered strptime-compatible date formats.

> Source: src/etlguard/config.py:L68. Declared type list[str]; default/constraint Field(default_factory=lambda: ['%Y-%m-%d', '%Y/%m/%d', '%d/%m/%Y', '%m/%d/%Y', '%d-%m-%Y', '%Y.%m.%d']).

CleaningSettings.number_columns

Columns converted to Decimal where possible.

> Source: src/etlguard/config.py:L71. Declared type list[str]; default/constraint Field(default_factory=list).

CleaningSettings.lowercase

Columns converted to lowercase when string.

> Source: src/etlguard/config.py:L72. Declared type list[str]; default/constraint Field(default_factory=list).

CleaningSettings.uppercase

Columns converted to uppercase when string.

> Source: src/etlguard/config.py:L73. Declared type list[str]; default/constraint Field(default_factory=list).

CleaningSettings.drop_duplicates

Suppress only exact repeats of previously accepted rows.

> Source: src/etlguard/config.py:L74. Declared type bool; default/constraint True.

ForeignKeySettings

Field	Declared type	Declared default
values	list[str | int] | None	None
path	str | None	None
column	str | None	None

ForeignKeySettings.values

Allowed reference set supplied inline; mutually exclusive with path.

> Source: src/etlguard/config.py:L78. Declared type list[str | int] | None; default/constraint None.

ForeignKeySettings.path

External CSV reference file; mutually exclusive with values.

> Source: src/etlguard/config.py:L79. Declared type str | None; default/constraint None.

ForeignKeySettings.column

Required reference CSV column when path is set.

> Source: src/etlguard/config.py:L80. Declared type str | None; default/constraint None.

RowCountSettings

Field	Declared type	Declared default
min	int | None	Field(default=None, ge=0)
max	int | None	Field(default=None, ge=0)

RowCountSettings.min

Minimum cleaned record count, nonnegative.

A B C D E F G H

Source: src/etlguard/config.py:L92. Declared type int | None; default/constraint Field(default=None, ge=0).

RowCountSettings.max

Maximum cleaned record count, nonnegative and not below min.

Source: src/etlguard/config.py:L93. Declared type int | None; default/constraint Field(default=None, ge=0).

ChecksSettings

Field	Declared type	Declared default
unique	list[str]	Field(default_factory=list)
foreign_keys	dict[str, ForeignKeySettings]	Field(default_factory=dict)
expected_row_count	RowCountSettings | None	None
max_null_pct	dict[str, float]	Field(default_factory=dict)

ChecksSettings.unique

Global per-batch business key uniqueness columns.

Source: src/etlguard/config.py:L103. Declared type list[str]; default/constraint Field(default_factory=list).

ChecksSettings.foreign_keys

Field-to-reference dictionary; row-level rejections.

Source: src/etlguard/config.py:L104. Declared type dict[str, ForeignKeySettings]; default/constraint Field(default_factory=dict).

ChecksSettings.expected_row_count

Nested global count bounds; dataset-level gate.

Source: src/etlguard/config.py:L105. Declared type RowCountSettings | None; default/constraint None.

ChecksSettings.max_null_pct

Field-to-fraction upper limits in 0..1.

Source: src/etlguard/config.py:L106. Declared type dict[str, float]; default/constraint Field(default_factory=dict).

TargetSettings

Field	Declared type	Declared default
table	str	(required)
mode	Literal['append', 'truncate_insert', 'upsert', 'scd2']	'append'
key	list[str]	Field(default_factory=list)
tracked_columns	list[str]	Field(default_factory=list)

A B C D E F G H

Field	Declared type	Declared default
batch_size	int	Field(default=500, ge=1)
lock_timeout_seconds	int	Field(default=30, ge=0, le=300)
min_replace_ratio	float | None	Field(default=None, ge=0, le=1)

TargetSettings.table

SQL table name <=48 chars; safe identifier; cannot use stg_ prefix.

> Source: src/etlguard/config.py:L117. Declared type str; default/constraint required.

TargetSettings.mode

append, truncate_insert, upsert or **scd2**.

> Source: src/etlguard/config.py:L118. Declared type Literal['append', 'truncate_insert', 'upsert', 'scd2']; default/constraint 'append'.

TargetSettings.key

Business key, required for upsert and **scd2**; nonnull in DB.

> Source: src/etlguard/config.py:L119. Declared type list[str]; default/constraint Field(default_factory=list).

TargetSettings.tracked_columns

SCD2 columns compared for change detection.

> Source: src/etlguard/config.py:L120. Declared type list[str]; default/constraint Field(default_factory=list).

TargetSettings.batch_size

Per-SQL-insert chunk size, >=1.

> Source: src/etlguard/config.py:L121. Declared type int; default/constraint Field(default=500, ge=1).

TargetSettings.lock_timeout_seconds

Bounded per-target advisory lock acquisition wait.

> Source: src/etlguard/config.py:L122. Declared type int; default/constraint Field(default=30, ge=0, le=300).

TargetSettings.min_replace_ratio

Optional lower bound for replacement snapshot/old-target size.

> Source: src/etlguard/config.py:L123. Declared type float | None; default/constraint Field(default=None, ge=0, le=1).

PolicySettings

Field	Declared type	Declared default
fail_if_reject_rate_above	float | None	Field(default=None, ge=0, le=1)
allow_partial_replace	bool	False

Field	Declared type	Declared default
allow_empty_replace	bool	False

PolicySettings.fail_if_reject_rate_above

Optional distinct rejected-row fraction threshold.

> Source: src/etlguard/config.py:L147. Declared type float | None; default/constraint Field(default=None, ge=0, le=1).

PolicySettings.allow_partial_replace

Explicitly permit replacement with rejected source rows.

> Source: src/etlguard/config.py:L149. Declared type bool; default/constraint False.

PolicySettings.allow_empty_replace

Explicitly permit empty replacement source.

> Source: src/etlguard/config.py:L150. Declared type bool; default/constraint False.

OutputSettings

Field	Declared type	Declared default
directory	str	'reports'

OutputSettings.directory

Where CSV, JSON and summary reports are written.

> Source: src/etlguard/config.py:L154. Declared type str; default/constraint 'reports'.

PipelineSettings

Field	Declared type	Declared default
pipeline	str	(required)
source	SourceSettings	(required)
model	str	(required)
cleaning	CleaningSettings	Field(default_factory=CleaningSettings)
checks	ChecksSettings	Field(default_factory=ChecksSettings)
target	TargetSettings | None	None
policy	PolicySettings	Field(default_factory=PolicySettings)
output	OutputSettings	Field(default_factory=OutputSettings)

A B C D E F G H

Field	Declared type	Declared default
database_url_env	str	'ETLGUARD_DATABASE_URL'

PipelineSettings.pipeline

Safe filename identifier, 1..128 chars.

> Source: src/etlguard/config.py:L158. Declared type str; default/constraint required.

PipelineSettings.source

Required source descriptor.

> Source: src/etlguard/config.py:L159. Declared type SourceSettings; default/constraint required.

PipelineSettings.model

Required module.path:PydanticClass reference.

> Source: src/etlguard/config.py:L160. Declared type str; default/constraint required.

PipelineSettings.cleaning

Optional transformation settings.

> Source: src/etlguard/config.py:L161. Declared type CleaningSettings; default/constraint Field(default_factory=CleaningSettings).

PipelineSettings.checks

Optional row-global data checks.

> Source: src/etlguard/config.py:L162. Declared type ChecksSettings; default/constraint Field(default_factory=ChecksSettings).

PipelineSettings.target

Optional for validation; required for write-enabled run.

> Source: src/etlguard/config.py:L163. Declared type TargetSettings | None; default/constraint None.

PipelineSettings.policy

Failure thresholds and replacement opt-ins.

> Source: src/etlguard/config.py:L164. Declared type PolicySettings; default/constraint Field(default_factory=PolicySettings).

PipelineSettings.output

Local report path configuration.

> Source: src/etlguard/config.py:L165. Declared type OutputSettings; default/constraint Field(default_factory=OutputSettings).

PipelineSettings.database_url_env

Environment variable name with MySQL connection URL.

> Source: src/etlguard/config.py:L166. Declared type str; default/constraint 'ETLGUARD_DATABASE_URL'.

35. Module and callable reference - runtime

src/etlguard/__init__.py

Package helper and internal declarations; inspect implementation appendix for exact behavior.

> Module statement: ETL Guard public API; lazy imports keep the browser core database-free.

__getattr__

```
__getattr__(name: str) -> Any
```

Implementation reference: src/etlguard/__init__.py, lines 9-14; 6 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

src/etlguard/audit.py

Creates and updates durable metadata. Success writes occur on the target transaction connection; best-effort failure writes never overwrite committed success.

> Module statement: Durable ETL audits and an atomic, idempotent batch-commit ledger.

Class BatchConflictError

A batch ID already belongs to a different source/configuration/target.

Declared at src/etlguard/audit.py:L62-L63; bases: ValueError.

Class AuditWriter

Declared at src/etlguard/audit.py:L66-L131; bases: object.

audit_table

```
audit_table() -> Table
```

Keep the v0.1.1 audit schema compatible with existing installations.

Implementation reference: src/etlguard/audit.py, lines 16-32; 17 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

commits_table

```
commits_table() -> Table
```

Rows become visible only if target data and the audit commit together.

Implementation reference: src/etlguard/audit.py, lines 35-51; 17 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

persisted_counts

```
persisted_counts(row: Any) -> LoadCounts
```

Implementation reference: src/etlguard/audit.py, lines 54-59; 6 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

AuditWriter.__init__

```
AuditWriter.__init__(self, engine: Engine)
```

Implementation reference: src/etlguard/audit.py, lines 67-70; 4 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

A B C D E F G H

AuditWriter.init_db

AuditWriter.init_db(self) -> None

Implementation reference: src/etlguard/audit.py, lines 72-75; 4 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

AuditWriter._values

AuditWriter._values(self, result: Any, source: str, status: str, counts: LoadCounts | None=None) -> dict[str, Any]

Implementation reference: src/etlguard/audit.py, lines 77-95; 19 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

AuditWriter.record_success

AuditWriter.record_success(self, conn: Connection, result: Any, source: str, counts: LoadCounts) -> None

Write success on the SAME transaction/connection as the target merge.

Implementation reference: src/etlguard/audit.py, lines 97-110; 14 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

AuditWriter.record_failure

AuditWriter.record_failure(self, result: Any, source: str) -> None

Best-effort failure audit, never overwriting a committed success.

Implementation reference: src/etlguard/audit.py, lines 112-125; 14 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

AuditWriter.write

AuditWriter.write(self, result: Any, source: str) -> None

Backward-compatible one-off audit API; not for transactional load success.

Implementation reference: src/etlguard/audit.py, lines 127-131; 5 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

src/etlguard/browser_demo.py

Python payload used in the client-side validation demo.

> Module statement: Pyodide demo adapter: calls production ValidationEngine; loads valid rows to SQLite.

validate_csv

validate_csv(csv_text: str, selected_model: str='Customer') -> str

Implementation reference: src/etlguard/browser_demo.py, lines 20-84; 65 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

src/etlguard/checks.py

Performs reference checks, business-key uniqueness, accepted-only duplicate suppression and dataset-level count/null percentage checks.

> Module statement: Dataset-level checks and reference-data checks independent of the database.

Class DatasetIssue

Declared at src/etlguard/checks.py:L16-L19; bases: object.

Class DatasetChecker

Declared at src/etlguard/checks.py:L35-L103; bases: object.

load_reference

load_reference(ref: ForeignKeySettings, base_dir: Path) -> set[str]

Implementation reference: src/etlguard/checks.py, lines 22-32; 11 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

DatasetChecker.__init__

DatasetChecker.__init__(self, settings: ChecksSettings, base_dir: Path, drop_duplicates: bool=False)

Implementation reference: src/etlguard/checks.py, lines 36-42; 7 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

DatasetChecker.filter_candidates

DatasetChecker.filter_candidates(self, items: list[Candidate]) -> tuple[list[Candidate], list[RejectedRow]]

Implementation reference: src/etlguard/checks.py, lines 44-83; 40 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

DatasetChecker.dataset_issues

DatasetChecker.dataset_issues(self, clean_rows: list[dict[str, Any]]) -> list[DatasetIssue]

Implementation reference: src/etlguard/checks.py, lines 85-103; 19 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

src/etlguard/cleaning.py

Pure Python record-level normalization with deterministic field conversion and collision checks.

> Module statement: Deterministic, idempotent record cleaning; no pandas or database dependency.

Class Cleaner

Declared at src/etlguard/cleaning.py:L50-L82; bases: object.

snake_case

snake_case(value: str) -> str

Implementation reference: src/etlguard/cleaning.py, lines 13-16; 4 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

date_iso

date_iso(value: Any, formats: list[str]) -> Any

Implementation reference: src/etlguard/cleaning.py, lines 19-31; 13 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

decimal_number

decimal_number(value: Any) -> Any

Implementation reference: src/etlguard/cleaning.py, lines 34-47; 14 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

A B C D E F G H

Cleaner.__init__

Cleaner.__init__(self, rules: CleaningSettings)

Implementation reference: src/etlguard/cleaning.py, lines 51-61; 11 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

Cleaner.clean

Cleaner.clean(self, row: Mapping[str, Any]) -> dict[str, Any]

Implementation reference: src/etlguard/cleaning.py, lines 63-82; 20 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

src/etlguard/cli.py

User-facing Typer commands, rich summaries and exit handling; database imports are command-specific.

> Module statement: Typer CLI: validate, run, profile, init-db; 0/1/2 exit contract.

show_summary

show_summary(result: RunResult) -> None

Implementation reference: src/etlguard/cli.py, lines 22-46; 25 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

_execute

_execute(path: Path, dry_run: bool, batch_id: str | None=None) -> None

Implementation reference: src/etlguard/cli.py, lines 49-57; 9 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

validate

validate(config: Annotated[Path, typer.Argument(help='YAML pipeline configuration')]) -> None

Dry run: write rejects + summary; never touch the database.

Implementation reference: src/etlguard/cli.py, lines 61-63; 3 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

run

run(config: Annotated[Path, typer.Argument(help='YAML pipeline configuration')], batch_id: str | None=typer.Option(None, '--batch-id', help='Stable UUID for safe retry after an unknown commit outcome')) -> None

Validate, stage, transactionally merge and audit a pipeline run.

Implementation reference: src/etlguard/cli.py, lines 67-74; 8 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

profile_command

profile_command(path: Annotated[Path, typer.Argument(help='CSV feed to inspect')]) -> None

Show types, null %, distinct count and min/max by column.

Implementation reference: src/etlguard/cli.py, lines 78-94; 17 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

init_db

init_db(database_url_env: str=typer.Option('ETLGUARD_DATABASE_URL', help='Environment variable for MySQL URL')) -> None

A B C D E F G H

Create the durable audit and atomic batch-commit tables, if absent.

Implementation reference: src/etlguard/cli.py, lines 98-116; 19 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

batch_status

batch_status(batch_id: str=typer.Argument(..., help='Canonical batch UUID'), database_url_env: str=typer.Option('ETLGUARD_DATABASE_URL')) -> None

Read the durable outcome before deciding whether to retry a run.

Implementation reference: src/etlguard/cli.py, lines 121-175; 55 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

upgrade_scd2

upgrade_scd2(config: Annotated[Path, typer.Argument(help='**SCD2** YAML pipeline configuration')], yes: bool=typer.Option(False, '--yes', help='Confirm maintenance-window schema migration')) → None

Explicitly add the **MySQL 8** active-row uniqueness guard to an existing **SCD2** table.

Implementation reference: src/etlguard/cli.py, lines 179-209; 31 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

main

main() -> None

Implementation reference: src/etlguard/cli.py, lines 211-212; 2 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

src/etlguard/config.py

Strict Pydantic settings types and YAML parsing with **fail**-closed identifier/path checks.

> Module statement: Strictly validated YAML config, with paths resolved relative to the YAML file.

Class ConfigError

Invalid user-supplied pipeline configuration (CLI exit status 2).

Declared at src/etlguard/config.py:L14-L15; bases: ValueError.

Class StrictSettings

Declared at src/etlguard/config.py:L18-L19; bases: BaseModel.

Class PaginationSettings

Declared at src/etlguard/config.py:L22-L30; bases: StrictSettings.

Class SourceSettings

Declared at src/etlguard/config.py:L33-L59; bases: StrictSettings.

Class CleaningSettings

Declared at src/etlguard/config.py:L62-L74; bases: StrictSettings.

Class ForeignKeySettings

Declared at src/etlguard/config.py:L77-L88; bases: StrictSettings.

A B C D E F G H

Class RowCountSettings

Declared at src/etlguard/config.py:L91-L99; bases: StrictSettings.

Class ChecksSettings

Declared at src/etlguard/config.py:L102-L113; bases: StrictSettings.

Class TargetSettings

Declared at src/etlguard/config.py:L116-L143; bases: StrictSettings.

Class PolicySettings

Declared at src/etlguard/config.py:L146-L150; bases: StrictSettings.

Class OutputSettings

Declared at src/etlguard/config.py:L153-L154; bases: StrictSettings.

Class PipelineSettings

Declared at src/etlguard/config.py:L157-L173; bases: StrictSettings.

SourceSettings.check_location

SourceSettings.check_location(self) -> SourceSettings

Implementation reference: src/etlguard/config.py, lines 52-59; 8 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

ForeignKeySettings.check_reference

ForeignKeySettings.check_reference(self) -> ForeignKeySettings

Implementation reference: src/etlguard/config.py, lines 83-88; 6 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

RowCountSettings.check_bounds

RowCountSettings.check_bounds(self) -> RowCountSettings

Implementation reference: src/etlguard/config.py, lines 96-99; 4 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

ChecksSettings.check_null_pct

ChecksSettings.check_null_pct(cls, value: dict[str, float]) -> dict[str, float]

Implementation reference: src/etlguard/config.py, lines 110-113; 4 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

TargetSettings.safe_identifier

TargetSettings.safe_identifier(cls, value: str) -> str

Implementation reference: src/etlguard/config.py, lines 127-132; 6 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

TargetSettings.check_mode

TargetSettings.check_mode(self) -> TargetSettings

Implementation reference: src/etlguard/config.py, lines 135-143; 9 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

PipelineSettings.safe_pipeline_name

PipelineSettings.safe_pipeline_name(cls, value: str) -> str

Implementation reference: src/etlguard/config.py, lines 170-173; 4 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

load_config

load_config(path: str | Path) -> tuple[PipelineSettings, Path]

Implementation reference: src/etlguard/config.py, lines 176-187; 12 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

src/etlguard/core.py

Validation orchestration independent of SQLAlchemy/MySQL; preserves source digest and lineage.

> Module statement: Shared CSV/Excel/API + Pyodide cleaning/validation orchestration, no SQL imports.

Class CoreResult

Declared at src/etlguard/core.py:L23-L52; bases: object.

Class ValidationEngine

Declared at src/etlguard/core.py:L55-L123; bases: object.

CoreResult.rows_valid

CoreResult.rows_valid(self) -> int

Implementation reference: src/etlguard/core.py, lines 35-36; 2 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

CoreResult.rows_rejected

CoreResult.rows_rejected(self) -> int

Implementation reference: src/etlguard/core.py, lines 39-40; 2 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

CoreResult.error_counts_by_type

CoreResult.error_counts_by_type(self) -> dict[str, int]

Implementation reference: src/etlguard/core.py, lines 43-46; 4 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

CoreResult.error_counts_by_field

CoreResult.error_counts_by_field(self) -> dict[str, int]

Implementation reference: src/etlguard/core.py, lines 49-52; 4 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

ValidationEngine.__init__

ValidationEngine.__init__(self, model: type[BaseModel], cleaning: CleaningSettings, checks: ChecksSettings, base_dir: Path=Path('.'))

Implementation reference: src/etlguard/core.py, lines 56-65; 10 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

ValidationEngine.run

ValidationEngine.run(self, source: Iterable[SourceRecord]) -> CoreResult

Implementation reference: src/etlguard/core.py, lines 67-123; 57 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

src/etlguard/examples/__init__.py

Package helper and internal declarations; inspect implementation appendix for exact behavior.

> Module statement: Public synthetic datasets and models; never use customer production records.

src/etlguard/examples/models.py

Package helper and internal declarations; inspect implementation appendix for exact behavior.

> Module statement: Demonstration models for customer dimensions and transaction facts.

Class Customer

Declared at src/etlguard/examples/models.py:L12-L27; bases: BaseModel.

Class Transaction

Declared at src/etlguard/examples/models.py:L30-L44; bases: BaseModel.

Customer.check_dates

Customer.check_dates(self) -> Customer

Implementation reference: src/etlguard/examples/models.py, lines 24-27; 4 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

Transaction.debit_positive

Transaction.debit_positive(self) -> Transaction

Implementation reference: src/etlguard/examples/models.py, lines 41-44; 4 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

src/etlguard/loaders/__init__.py

Package helper and internal declarations; inspect implementation appendix for exact behavior.

> Module statement: Database loading implementations are imported only when needed.

src/etlguard/loaders/base.py

Structural protocol and shared source/loader contracts.

> Module statement: Target loader contract.

Class Loader

Declared at src/etlguard/loaders/base.py:L14-L18; bases: Protocol.

Loader.load

Loader.load(self, candidates: list[Candidate], batch_id: str, model: type[BaseModel], target: TargetSettings) -> LoadCounts

Implementation reference: src/etlguard/loaders/base.py, lines 15-18; 4 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

src/etlguard/loaders/mysql.py

MySQL DDL/schema checks, advisory locks, transactional stage/merge, replay ledger and **SCD2** upgrade.

> Module statement: MySQL 8 loader: transactional batch staging, DML merge, type-aware DDL.

Class MysqlLoader

Declared at src/etlguard/loaders/mysql.py:L128-L468; bases: object.

sqlalchemy_type

```
sqlalchemy_type(annotation: Any) -> Any
```

Map concrete Pydantic field types to portable SQLAlchemy column types.

Implementation reference: src/etlguard/loaders/mysql.py, lines 33-59; 27 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

_model_columns

```
_model_columns(model: type[BaseModel]) -> list[Column[Any]]
```

Implementation reference: src/etlguard/loaders/mysql.py, lines 62-78; 17 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

build_tables

```
build_tables(model: type[BaseModel], target: TargetSettings) -> tuple[Table, Table]
```

Implementation reference: src/etlguard/loaders/mysql.py, lines 81-121; 41 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

batches

```
batches(items: list[Any], size: int) -> list[list[Any]]
```

Implementation reference: src/etlguard/loaders/mysql.py, lines 124-125; 2 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

MysqlLoader.__init__

```
MysqlLoader.__init__(self, database_url: str | None=None, engine: Engine | None=None)
```

Implementation reference: src/etlguard/loaders/mysql.py, lines 129-134; 6 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

MysqlLoader._locked_connection

```
MysqlLoader._locked_connection(self, target: TargetSettings)
```

Serialize every writer to a target, including FIRST **SCD2** inserts. MySQL GET_LOCK is connection-scoped and must be released explicitly. The lock spans all transactional work, not just the **SCD2** row SELECT.

Implementation reference: src/etlguard/loaders/mysql.py, lines 137-174; 38 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

MysqlLoader._digest

```
MysqlLoader._digest(candidates: list[Candidate], model: type[BaseModel], target: TargetSettings) -> str
```

Implementation reference: src/etlguard/loaders/mysql.py, lines 177-185; 9 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

A B C D E F G H

MysqlLoader._validate_id

MysqlLoader._validate_id(batch_id: str, request_hash: str) -> None

Implementation reference: src/etlguard/loaders/mysql.py, lines 188-195; 8 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

MysqlLoader._check_replay

MysqlLoader._check_replay(row: Any, batch_id: str, request_hash: str, target: TargetSettings, pipeline: str) -> LoadCounts

Implementation reference: src/etlguard/loaders/mysql.py, lines 198-206; 9 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

MysqlLoader.lookup_commit

MysqlLoader.lookup_commit(self, batch_id: str, request_hash: str, target: TargetSettings, pipeline: str) -> LoadCounts | None

Recover the durable outcome after a lost connection/response at COMMIT.

Implementation reference: src/etlguard/loaders/mysql.py, lines 208-218; 11 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

MysqlLoader._check_transactional_storage

MysqlLoader._check_transactional_storage(self, table_names: list[str]) -> None

MyISAM tables silently break rollback and atomic success guarantees.

Implementation reference: src/etlguard/loaders/mysql.py, lines 220-232; 13 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

MysqlLoader._check_schema

MysqlLoader._check_schema(self, model: type[BaseModel], target: TargetSettings, target_table: Table, stage_table: Table) -> None

Implementation reference: src/etlguard/loaders/mysql.py, lines 234-277; 44 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

MysqlLoader.load

MysqlLoader.load(self, candidates: list[Candidate], batch_id: str, model: type[BaseModel], target: TargetSettings, *, request_hash: str | None=None, audit_result: Any=None, allow_empty_replace: bool=False) -> LoadCounts

Commit data, unique batch marker and success audit in ONE DB transaction. An identical batch_id+hash replays recorded counts without touching data. A changed feed/configuration using that UUID is rejected, not reloaded.

Implementation reference: src/etlguard/loaders/mysql.py, lines 279-389; 111 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

MysqlLoader.upgrade_scd2

MysqlLoader.upgrade_scd2(self, model: type[BaseModel], target: TargetSettings) -> None

Explicit, operator-approved **MySQL 8** migration for pre-0.1.2 **SCD2** tables. Invoke in a maintenance window. It is NOT called by ordinary loads.

Implementation reference: src/etlguard/loaders/mysql.py, lines 391-432; 42 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

MysqlLoader._merge

MysqlLoader._merge(self, conn: Connection, table: Table, records: list[dict[str, Any]], target: TargetSettings) -> LoadCounts

A B C D E F G H

Implementation reference: src/etlguard/loaders/mysql.py, lines 434-468; 35 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

src/etlguard/pipeline.py

Main state machine, source adapter selection, request hashing, policy gating and report writing.

> Module statement: End-to-end pipeline orchestration, policies, summaries, audit and lazy DB loading.

Class RunResult

Declared at src/etlguard/pipeline.py:L23-L55; bases: BaseModel.

Class Pipeline

Declared at src/etlguard/pipeline.py:L72-L228; bases: object.

RunResult.ok

```
RunResult.ok(self) -> bool
```

Implementation reference: src/etlguard/pipeline.py, lines 54-55; 2 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

resolve_model

```
resolve_model(reference: str) -> type[BaseModel]
```

Implementation reference: src/etlguard/pipeline.py, lines 58-69; 12 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

Pipeline.__init__

```
Pipeline.__init__(self, settings: PipelineSettings, base_dir: Path)
```

Implementation reference: src/etlguard/pipeline.py, lines 73-75; 3 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

Pipeline.from_yaml

```
Pipeline.from_yaml(cls, path: str | Path) -> Pipeline
```

Implementation reference: src/etlguard/pipeline.py, lines 78-80; 3 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

Pipeline._source

```
Pipeline._source(self) -> CsvSource | ExcelSource | ApiSource
```

Implementation reference: src/etlguard/pipeline.py, lines 82-88; 7 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

Pipeline.run

```
Pipeline.run(self, dry_run: bool=False, batch_id: str | None=None) -> RunResult
```

Implementation reference: src/etlguard/pipeline.py, lines 90-228; 139 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

src/etlguard/profile.py

CSV profiling and column-level statistics for exploratory QA.

> Module statement: Quick per-column quality profile for ad-hoc feed triage.

A B C D E F G H

profile_csv

profile_csv(path: str | Path, encoding: str='utf-8-sig') -> list[dict[str, Any]]

Implementation reference: src/etlguard/profile.py, lines 11-30; 20 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

src/etlguard/reconcile.py

Load count contract and mismatch exception.

> Module statement: ETL count reconciliation; a failure aborts the database transaction.

Class ReconciliationError

Declared at src/etlguard/reconcile.py:L8-L9; bases: RuntimeError.

Class LoadCounts

Declared at src/etlguard/reconcile.py:L13-L24; bases: object.

LoadCounts.loaded

LoadCounts.loaded(self) -> int

Implementation reference: src/etlguard/reconcile.py, lines 21-24; 4 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

reconcile

reconcile(rows_valid: int, counts: LoadCounts) -> None

Implementation reference: src/etlguard/reconcile.py, lines 27-32; 6 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

src/etlguard/rejects.py

Structured error details and formula-safe CSV/precise JSON output.

> Module statement: One CSV/JSON record *per validation error*, preserving original source values.

Class RejectedRow

Declared at src/etlguard/rejects.py:L20-L27; bases: BaseModel.

Class RejectWriter

Declared at src/etlguard/rejects.py:L72-L87; bases: object.

raw_json

raw_json(row: dict[str, Any]) -> str

Implementation reference: src/etlguard/rejects.py, lines 30-31; 2 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

reject

reject(batch_id: str, source_row_number: int, field: str, error_type: str, error_message: str, raw_row: dict[str, Any], cleaned_row: dict[str, Any]) -> RejectedRow

Implementation reference: src/etlguard/rejects.py, lines 34-62; 29 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

A B C D E F G H

_safe_csv_cell

_safe_csv_cell(value: Any) -> Any

Prevent spreadsheet-formula execution without changing exact JSON rejects.

Implementation reference: src/etlguard/rejects.py, lines 65-69; 5 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

RejectWriter.write

RejectWriter.write(self, path_prefix: Path, errors: list[RejectedRow]) -> tuple[Path, Path]

Implementation reference: src/etlguard/rejects.py, lines 73-87; 15 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

src/etlguard/scd2.py

Per-key **SCD2** row closure/new version insertion using caller-provided SQL transaction.

> Module statement: SCD Type 2 history transition using a single supplied SQLAlchemy connection.

scd2_merge

scd2_merge(conn: Connection, table: Table, rows: list[dict[str, Any]], keys: list[str], tracked_columns: list[str]) -> LoadCounts

Implementation reference: src/etlguard/**scd2**.py, lines 15-53; 39 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

src/etlguard/sources/__init__.py

Package helper and internal declarations; inspect implementation appendix for exact behavior.

> Module statement: Source connectors.

src/etlguard/sources/api.py

httpx REST ingestion with retries, paging and result extraction.

> Module statement: GET JSON connector: retries, bearer auth, page/offset/next-link pagination.

Class SourceError

Declared at src/etlguard/sources/api.py:L17-L18; bases: RuntimeError.

Class ApiSource

Declared at src/etlguard/sources/api.py:L30-L126; bases: object.

dig

dig(data: Any, path: str) -> Any

Implementation reference: src/etlguard/sources/api.py, lines 21-27; 7 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

ApiSource.__init__

ApiSource.__init__(self, settings: SourceSettings, client: httpx.Client | None=None, sleeper: Callable[[float], None]=time.sleep)

Implementation reference: src/etlguard/sources/api.py, lines 31-43; 13 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

ApiSource._get

ApiSource._get(self, client: httpx.Client, url: str, params: dict[str, Any], headers: dict[str, str]) -> Any

Implementation reference: src/etlguard/sources/api.py, lines 45-62; 18 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

ApiSource.read

ApiSource.read(self, batch_id: str) -> Iterator[SourceRecord]

Implementation reference: src/etlguard/sources/api.py, lines 64-126; 63 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

src/etlguard/sources/base.py

Structural protocol and shared source/loader contracts.

> Module statement: Source protocol and per-record lineage shared across all connectors.

Class Lineage

Declared at src/etlguard/sources/base.py:L11-L15; bases: object.

Class SourceRecord

Declared at src/etlguard/sources/base.py:L19-L21; bases: object.

Class Source

Declared at src/etlguard/sources/base.py:L24-L25; bases: Protocol.

Source.read

Source.read(self, batch_id: str) -> Iterator[SourceRecord]

Implementation reference: src/etlguard/sources/base.py, lines 25-25; 1 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

lineage

lineage(source: str, row: int, batch_id: str) -> Lineage

Implementation reference: src/etlguard/sources/base.py, lines 28-29; 2 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

src/etlguard/sources/csv.py

CSV rows with configured parsing/lineage.

> Module statement: Chunked pandas CSV reader that preserves raw strings and blank rows.

Class CsvSource

Declared at src/etlguard/sources/csv.py:L12-L38; bases: object.

CsvSource.__init__

CsvSource.__init__(self, settings: SourceSettings, base_dir: Path)

Implementation reference: src/etlguard/sources/csv.py, lines 13-17; 5 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

CsvSource.read

CsvSource.read(self, batch_id: str) -> Iterator[SourceRecord]

Implementation reference: src/etlguard/sources/csv.py, lines 19-38; 20 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

src/etlguard/sources/excel.py

Excel worksheet rows with configured header/lineage.

> Module statement: Excel reader using pandas/openpyxl, with physical worksheet lineage.

Class ExcelSource

Declared at src/etlguard/sources/excel.py:L12-L40; bases: object.

ExcelSource.__init__

ExcelSource.__init__(self, settings: SourceSettings, base_dir: Path)

Implementation reference: src/etlguard/sources/excel.py, lines 13-17; 5 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

ExcelSource.read

ExcelSource.read(self, batch_id: str) -> Iterator[SourceRecord]

Implementation reference: src/etlguard/sources/excel.py, lines 19-40; 22 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

src/etlguard/validation.py

Pydantic typed row validation with independent field errors.

> Module statement: Pydantic v2 typed row validation, preserving all independent field errors.

Class Candidate

Declared at src/etlguard/validation.py:L15-L19; bases: object.

Class ValidationOutcome

Declared at src/etlguard/validation.py:L23-L25; bases: object.

Class RowValidator

Declared at src/etlguard/validation.py:L28-L47; bases: object.

RowValidator.__init__

RowValidator.__init__(self, model: type[BaseModel])

Implementation reference: src/etlguard/validation.py, lines 29-30; 2 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

RowValidator.validate

RowValidator.validate(self, cleaned: dict[str, Any], raw: dict[str, Any], line: Lineage) -> ValidationOutcome

Implementation reference: src/etlguard/validation.py, lines 32-47; 16 source lines. Verify all exceptions and side effects against this exact release before extending or overriding this callable.

36. Exact test-case register

Every test function defined in the archived tests/ directory is indexed here. This is a source register, not a claim that the documentation process reran each test. Fresh execution claims are taken from the packaged JUnit/coverage evidence.

tests/test_api.py - 8 definitions

All network calls are intercepted by respx; no live API dependencies.

Line	Test purpose (derived from declared function name)
L14	test nested json path
L21	test page pagination
L39	test offset pagination
L52	test next link and bearer env
L67	test 500 retry then success
L79	test retries exhausted and nonretryable 400
L92	test next link cross origin blocked
L101	test missing token

tests/test_api_transport.py - 8 definitions

Additional real httpx adapter tests that run without respx or a network.

Line	Test purpose (derived from declared function name)
L16	test httpx mock page pagination and lineage
L32	test offset pagination httpx mock
L46	test next link pagination and auth
L66	test httpx mock retry and backoff
L83	test api failure and wrong json shapes
L99	test api retries exhausted and network failure
L114	test bad next link loop wrong origin and max pages
L131	test missing token next link non string and api url validation

tests/test_badge.py - 1 definitions

CI badge generator does not depend on optional tooling or a live server.

Line	Test purpose (derived from declared function name)
L7	test badge thresholds

tests/test_browser.py - 4 definitions

Ensure the browser's SQLite demonstration uses the production validation core.

Line	Test purpose (derived from declared function name)
L10	test browser customer feed and sqlite
L19	test browser transaction debit rule
L27	test browser invalid csv headers
L38	test browser reject download escapes formulas but preserves json

tests/test_cleaning.py - 8 definitions

Line	Test purpose (derived from declared function name)
L12	test snake case camel spaces and hyphens
L18	test all cleaning steps
L34	test null tokens
L46	test decimal normalization
L50	test dates with errors and native types
L60	test rename collision becomes a row reject
L65	test trim disabled
L71	test case conversion and idempotence

tests/test_config.py - 6 definitions

Line	Test purpose (derived from declared function name)
L15	test unsafe table names rejected
L20	test upsert and scd2 require keys

Line	Test purpose (derived from declared function name)
L28	test invalid config ranges
L41	test build tables contain target and staging metadata
L50	test model target columns exist
L55	test sqlalchemy types unsupported

tests/test_coverage_edges.py - 6 definitions

Edge cases for API contract, model typing and dataset checks.

Line	Test purpose (derived from declared function name)
L16	test batching splits work
L21	test decimal types
L27	test dataset reference values are strings
L35	test core no duplicate drop can fail uniqueness
L46	test raw unsupported type preserved as error
L54	test cleaner custom null token

tests/test_mysql_ddl_compile.py - 3 definitions

Compile critical MySQL DDL offline; NOT a substitute for **MySQL 8** execution.

Line	Test purpose (derived from declared function name)
L11	test scd2 generated active marker compiles for mysql8
L25	test upsert requires mysql unique business key ddl
L34	test audit and commit ledger require innodb

tests/test_mysql_integration.py - 14 definitions

Real **MySQL 8** integration tests; CI provisions a service container.

Line	Test purpose (derived from declared function name)
L62	test upsert insert and update
L75	test scd2 tracks history and unchanged

A B C D E F G H

Line	Test purpose (derived from declared function name)
L94	test truncate insert is transactional delete
L103	test append inserts and reconciles
L111	test reconciliation mismatch rolls back stage and target
L127	test transaction rollback on merge failure
L143	test audit table rows persist
L161	test pipeline creates success audit
L189	test atomic batch commit and append retry on mysql8
L212	test mysql8 audit failure rolls back data and ledger
L237	test mysql8 concurrent scd2 updates have one current
L260	test mysql8 scd2 unique active index blocks external duplicate
L277	test mysql8 upgrade legacy scd2 requires explicit migration
L298	test distinct target writers do not deadlock shared metadata

tests/test_pipeline.py - 13 definitions

Line	Test purpose (derived from declared function name)
L46	test python api returns typed result and rejects
L63	test good dry run no database writes
L75	test dataset failure causes nonzero
L85	test cli validate exit codes and report
L94	test cli config error is 2
L102	test cli run missing environment is config 2
L109	test cli profile
L121	test resolve model invalid and config validation
L131	test empty config yaml rejected
L138	test reconciliation counts
L144	test reject writer

Line	Test purpose (derived from declared function name)
L155	test sample messy feed has expected counts
L164	test reject csv escapes spreadsheet formulas but json keeps raw

tests/test_production_safety.py - 18 definitions

Regression checks for atomic auditing, retry safety and **SCD2** uniqueness.

Line	Test purpose (derived from declared function name)
L47	test direct append is idempotent and audited
L59	test batch conflict rejects changed input
L68	test audit exception rolls back target and ledger
L87	test lost commit response is recovered from durable record
L104	test failure audit then success same id
L119	test safe replace minimum size enforced
L131	test generated scd2 active unique constraint
L148	test legacy scd2 without db guard fails closed
L161	test invalid batch id does not write
L166	test partial source failure preserves counts without loading
L182	test full pipeline replays identical batch and preserves success audit
L227	test pipeline run tracks partial source and refuses to load
L249	test cli exposes stable batch retry and status commands
L262	test cli db status and upgrade guard with sqlite
L299	test post commit summary io problem is warning not failed load
L338	test mysql key with nullable column is rejected even with unique index
L345	test target rejects repeated business key fields
L351	test direct loader rejects duplicate business keys before writing

tests/test_property_cleaning.py - 1 definitions

Property-based cleaning invariant: f(f(x)) == f(x) across varied strings.

A B C D E F G H

Line	Test purpose (derived from declared function name)
L15	test cleaning is idempotent

tests/test_safety_regressions.py - 10 definitions

Regression tests for source-row accuracy and destructive load safety.

Line	Test purpose (derived from declared function name)
L41	test identical invalid records are all rejected
L53	test only identical valid rows are dropped
L61	test empty target replacement is blocked by default
L68	test empty target replacement requires explicit opt in
L73	test partial target replacement blocked unless explicitly allowed
L84	test rejects path traversal in pipeline identifier
L90	test upsert rejects competing unique constraint before load
L112	test identical invalid foreign keys are not deduplicated
L124	test repeated business key violations are all rejected
L136	test rename map without column normalization

tests/test_sources.py - 6 definitions

Line	Test purpose (derived from declared function name)
L12	test csv bom chunking and physical lineage
L25	test csv non utf8 encoding and delimiter
L33	test csv blank record is counted
L41	test csv header offset
L50	test excel sheet name skip rows and header
L65	test excel sheet index

tests/test_sqlite_loader.py - 7 definitions

Fast SQLAlchemy transactional smoke tests (CI also runs against real **MySQL 8**).

A
B
C
D
E
F
G
H

Line	Test purpose (derived from declared function name)
L48	test mysql loader append stage and replace smoke
L60	test scd2 versions and unchanged smoke
L75	test reconciliation rollback smoke
L90	test missing business unique constraint fails without writing
L100	test model type mapping and safe table validation
L127	test audit writer smoke
L137	test empty truncate insert clears target

tests/test_validation.py - 10 definitions

Line	Test purpose (derived from declared function name)
L11	test valid customer model and python types
L19	test multiple independent field errors are collected
L28	test customer row level end date rule
L37	test debit row level positive rule
L46	test dataset uniqueness and foreign keys
L63	test dataset null pct and row count threshold
L76	test empty feed and upper count limit
L90	test duplicate rows drop after cleaning
L102	test column collision does not stop feed
L111	test reference csv support

37. Operator script, staging helper and CLI safety reference

operations/assemble_delivery.py

Build a secret-free, checksummed review packet from authentic local results.

Top-level entrypoint	Line	Size
write	L18	4 LOC

Top-level entrypoint	Line	Size
read	L24	2 LOC

Complete implementation is included in the source archive. Treat this helper as a bounded operator tool, not as authorization for a real production target.

operations/bootstrap_mysql.py

Provision a new loopback-only disposable instance, without Homebrew services.

Top-level entrypoint	Line	Size
start	L52	13 LOC

Complete implementation is included in the source archive. Treat this helper as a bounded operator tool, not as authorization for a real production target.

operations/package_cli_acceptance.py

Exercise the installed wheel from outside the source checkout.

Top-level entrypoint	Line	Size
run	L36	17 LOC

Complete implementation is included in the source archive. Treat this helper as a bounded operator tool, not as authorization for a real production target.

operations/prepare_approval_packet.py

Prepare a portable, non-executing pilot template and an honest evidence manifest.

Complete implementation is included in the source archive. Treat this helper as a bounded operator tool, not as authorization for a real production target.

operations/run_operational_acceptance.py

Actual recovery, contention, capacity and local monitoring drills on private MySQL. Requires bootstrap_mysql.py's NEW disposable instance. No production URL input.

Top-level entrypoint	Line	Size
now	L45	2 LOC
save	L49	2 LOC
candidate	L53	17 LOC
table_name	L72	2 LOC
rows	L76	4 LOC

A B C D E F G H

Top-level entrypoint	Line	Size
fingerprint	L82	3 LOC
committed	L87	4 LOC
check	L93	25 LOC
preflight	L120	15 LOC
audit_rollback	L137	33 LOC
lost_commit_ack	L172	31 LOC
crash_worker	L205	11 LOC
process_kill	L218	32 LOC
changed_input	L252	14 LOC
lock_timeout	L268	27 LOC
scd_concurrency	L297	52 LOC
replacement	L351	43 LOC
directory_bytes	L447	2 LOC
performance	L451	116 LOC
monitoring	L569	71 LOC
snapshot	L642	8 LOC
backup_restore	L652	81 LOC
main	L735	29 LOC

Complete implementation is included in the source archive. Treat this helper as a bounded operator tool, not as authorization for a real production target.

operations/run_soak.py

Two-minute synthetic local contention and monitoring observation.

Top-level entrypoint	Line	Size
worker	L41	14 LOC

Complete implementation is included in the source archive. Treat this helper as a bounded operator tool, not as authorization for a real production target.

operations/run_staging.py

Run the supplied one-shot using only this run's generated disposable identities.

Complete implementation is included in the source archive. Treat this helper as a bounded operator tool, not as authorization for a real production target.

operations/server_crash_drill.py

Stage or verify SIGKILL recovery of this run's private MySQL instance.

Complete implementation is included in the source archive. Treat this helper as a bounded operator tool, not as authorization for a real production target.

operations/start_existing_mysql.py

Restart only the private disposable instance provisioned by this handoff.

Complete implementation is included in the source archive. Treat this helper as a bounded operator tool, not as authorization for a real production target.

local_staging_one_shot.py

One-shot ISOLATED staging gate and same-batch shadow reconciliation only. No production mode exists. Never removes containers, existing tables or volumes. Run only after installing the source's [dev] dependencies and reviewing the local disposable MySQL Docker Compose configuration.

Top-level entrypoint	Line	Size
run_step	L20	8 LOC
main	L30	92 LOC

Complete implementation is included in the source archive. Treat this helper as a bounded operator tool, not as authorization for a real production target.

source/etl-validation-toolkit/scripts/archive_ci_evidence.py

Retain synthetic staging evidence in Actions logs when artifact storage is full.

Top-level entrypoint	Line	Size
main	L9	10 LOC

Complete implementation is included in the source archive. Treat this helper as a bounded operator tool, not as authorization for a real production target.

source/etl-validation-toolkit/scripts/assess_promotion.py

Read-only evidence assessor: never promotes or modifies a database. Passing this assessor means *ready for human release review*, NOT approved to run production. All operational evidence must be independently reviewed.

Top-level entrypoint	Line	Size
digest_file	L30	6 LOC
check_staging	L38	42 LOC
check_operational	L82	28 LOC
assess	L112	20 LOC
main	L134	16 LOC

Complete implementation is included in the source archive. Treat this helper as a bounded operator tool, not as authorization for a real production target.

source/etl-validation-toolkit/scripts/build_coverage_badge.py

Render Shields endpoint JSON from the branch-aware coverage.py summary. Usage: python scripts/build_coverage_badge.py coverage-summary.json demo/coverage.json

Top-level entrypoint	Line	Size
badge_payload	L13	11 LOC
main	L26	8 LOC

Complete implementation is included in the source archive. Treat this helper as a bounded operator tool, not as authorization for a real production target.

source/etl-validation-toolkit/scripts/build_demo_bundle.py

Bundle only MySQL-free, pandas-free production Python modules for GitHub Pages.

Complete implementation is included in the source archive. Treat this helper as a bounded operator tool, not as authorization for a real production target.

source/etl-validation-toolkit/scripts/ci_shadow_smoke.py

CI-only **MySQL 8** shadow smoke: separate disposable database and stable retry. Requires an explicitly acknowledged LOCAL disposable MySQL container with the root password used only for initial test-schema creation. NOT a production tool.

Top-level entrypoint	Line	Size
require_disposable_ci	L27	12 LOC
bootstrap	L41	32 LOC
smoke	L75	48 LOC
main	L125	23 LOC

Complete implementation is included in the source archive. Treat this helper as a bounded operator tool, not as authorization for a real production target.

source/etl-validation-toolkit/scripts/prepare_shadow_pilot.py

Prepare and verify a shadow-only pilot manifest. Never connects to a database. Intentionally does not execute ``etlguard run``. Operators must verify the isolated database and table before allowing any write, using least privilege.

Top-level entrypoint	Line	Size
sha256_file	L28	6 LOC
shadow_table_name	L36	9 LOC
plan	L47	74 LOC
verify	L123	29 LOC
main	L154	19 LOC

Complete implementation is included in the source archive. Treat this helper as a bounded operator tool, not as authorization for a real production target.

source/etl-validation-toolkit/scripts/run_shadow_pilot.py

Fail-closed shadow pilot launcher using a pinned manifest and disposable schema. ``validate`` is database-free. ``execute`` is only permitted for a locally reachable, isolated MySQL 8 shadow schema with matching least-privilege grants. Never use real customer/application/production schemas or credentials.

Top-level entrypoint	Line	Size
check_shadow_destination	L24	17 LOC
preflight_shadow	L43	60 LOC
run_pilot	L105	37 LOC
main	L144	29 LOC

Complete implementation is included in the source archive. Treat this helper as a bounded operator tool, not as authorization for a real production target.

source/etl-validation-toolkit/scripts/serve_demo.py

Launch the browser application with Python's standard library.

Top-level entrypoint	Line	Size
main	L10	16 LOC

A
B
C
D
E
F
G
H

Complete implementation is included in the source archive. Treat this helper as a bounded operator tool, not as authorization for a real production target.

source/etl-validation-toolkit/scripts/test_pilot_release_gate.py

Offline regression tests for shadow snapshot and human promotion review gates.

Complete implementation is included in the source archive. Treat this helper as a bounded operator tool, not as authorization for a real production target.

source/etl-validation-toolkit/scripts/test_shadow_reconciliation.py

Offline tests for independent pilot reconciliation; SQLite is NOT MySQL certification.

Complete implementation is included in the source archive. Treat this helper as a bounded operator tool, not as authorization for a real production target.

source/etl-validation-toolkit/scripts/test_verify_mysql8_staging.py

Offline safety regression tests for the independent staging gate.

Complete implementation is included in the source archive. Treat this helper as a bounded operator tool, not as authorization for a real production target.

source/etl-validation-toolkit/scripts/verify_mysql8_staging.py

Fail-closed **MySQL 8** staging gate for ETL Guard v0.1.2. This runner creates/drops test tables indirectly through pytest integration tests. It is intentionally restricted to disposable schemas that are explicitly confirmed. It never connects to the production target database.

Top-level entrypoint	Line	Size
validate_destination	L27	20 LOC
is_mysql8	L49	2 LOC
disallowed_tables	L53	4 LOC
grants_are_disposable_schema_only	L59	13 LOC
strict_sql_mode_is_enabled	L74	4 LOC
redact_secrets	L80	14 LOC
preflight	L96	50 LOC
execute_gate	L148	31 LOC
main	L181	65 LOC

Complete implementation is included in the source archive. Treat this helper as a bounded operator tool, not as authorization for a real production target.

source/etl-validation-toolkit/scripts/verify_shadow_reconciliation.py

Independent read-only, row-level verification of an isolated **MySQL 8** shadow pilot. The CLI refuses non-local, non-disposable MySQL URLs. A separate passing **MySQL 8** staging report is mandatory. No source records, secrets, or database rows are included in the JSON evidence; only counts, fingerprints and mismatch types. Offline unit tests exercise compare_shadow() against SQLite. This does NOT substitute for executing the CLI against genuine disposable **MySQL 8** staging.

Top-level entrypoint	Line	Size
canonical	L39	21 LOC
serialized	L62	2 LOC
row_fingerprint	L66	9 LOC
compare_shadow	L77	113 LOC
reconcile_shadow	L192	55 LOC
main	L249	22 LOC

Complete implementation is included in the source archive. Treat this helper as a bounded operator tool, not as authorization for a real production target.

38. Evidence inventory and interpretation guide

All evidence paths below are relative to the v0.1.3 release packet. Binary fixtures and private keys/password directories are not reproduced in the manual. Large performance JSON files contain many samples and should be opened in structured analysis tools for raw time-series investigation.

Quality and regression evidence

Artifact path (some long paths abbreviated)	Bytes	Type
evidence/qa/build.log	20,948	LOG
evidence/qa/coverage.json	135,292	JSON
evidence/qa/coverage.xml	62,098	XML
evidence/qa/distinct-target-regression-after.log	98	LOG
evidence/qa/distinct-target-regression-before-deterministic.log	14,971	LOG
evidence/qa/distinct-target-regression-before.log	98	LOG
evidence/qa/environment.txt	1,014	TXT
evidence/qa/operations-format.log	23	LOG
evidence/qa/operations-ruff.log	19	LOG

A B C D E F G H

Artifact path (some long paths abbreviated)	Bytes	Type
evidence/qa/pilot-template-validation.log	1,699	LOG
evidence/qa/pip-check.log	30	LOG
evidence/qa/pytest.xml	15,673	XML

Staging and shadow evidence

Artifact path (some long paths abbreviated)	Bytes	Type
evidence/staging/immutable_pilot/reports/customers_clean_shadow_462cc3db-b19b-4fb4-8cdb-e9c5fd328873_r...	82	CSV
evidence/staging/immutable_pilot/reports/customers_clean_shadow_462cc3db-b19b-4fb4-8cdb-e9c5fd328873_r...	2	JSON
evidence/staging/immutable_pilot/reports/customers_clean_shadow_462cc3db-b19b-4fb4-8cdb-e9c5fd328873_s...	1,461	JSON
evidence/staging/immutable_pilot/reports/customers_clean_shadow_a45b9d90-a378-40f2-96b4-aede243f5867_r...	82	CSV
evidence/staging/immutable_pilot/reports/customers_clean_shadow_a45b9d90-a378-40f2-96b4-aede243f5867_r...	2	JSON
evidence/staging/immutable_pilot/reports/customers_clean_shadow_a45b9d90-a378-40f2-96b4-aede243f5867_s...	1,461	JSON
evidence/staging/immutable_pilot/reports/customers_clean_shadow_eb70ab16-deac-4482-af62-ad4a29e0e017_r...	82	CSV
evidence/staging/immutable_pilot/reports/customers_clean_shadow_eb70ab16-deac-4482-af62-ad4a29e0e017_r...	2	JSON
evidence/staging/immutable_pilot/reports/customers_clean_shadow_eb70ab16-deac-4482-af62-ad4a29e0e017_s...	1,461	JSON
evidence/staging/immutable_pilot/reports/customers_clean_shadow_fdd2e6ed-57d6-4a1e-a04e-a6107d701b3b_r...	82	CSV
evidence/staging/immutable_pilot/reports/customers_clean_shadow_fdd2e6ed-57d6-4a1e-a04e-a6107d701b3b_r...	2	JSON
evidence/staging/immutable_pilot/reports/customers_clean_shadow_fdd2e6ed-57d6-4a1e-a04e-a6107d701b3b_s...	1,461	JSON
evidence/staging/immutable_pilot/shadow-manifest.json	2,017	JSON
evidence/staging/mysql8-gate-logs/complete_suite.log	2,557	LOG
evidence/staging/mysql8-gate-logs/mypy.log	44	LOG

A
B
C
D
E
F
G
H

SECTION MAP

Artifact path (some long paths abbreviated)	Bytes	Type
evidence/staging/mysql8-gate-logs/mysql8_integration.log	99	LOG
evidence/staging/mysql8-gate-logs/ruff.log	19	LOG
evidence/staging/mysql8_staging_gate.log	979	LOG
evidence/staging/one-shot-staging-summary.json	549	JSON
evidence/staging/operator_safety_tests.log	5,468	LOG
evidence/staging/shadow-load-first.json	685	JSON
evidence/staging/shadow-load-identical_retry.json	682	JSON
evidence/staging/shadow-reconciliation-first.json	1,012	JSON
evidence/staging/shadow-reconciliation-identical_retry.json	1,012	JSON
evidence/staging/shadow-validation.json	187	JSON
evidence/staging/shadow_load_first.log	111	LOG
evidence/staging/shadow_load_identical_retry.log	121	LOG
evidence/staging/shadow_manifest_verify.log	219	LOG
evidence/staging/shadow_plan.log	2,017	LOG
evidence/staging/shadow_reconciliation_first.log	152	LOG
evidence/staging/shadow_reconciliation_identical_retry.log	162	LOG
evidence/staging/shadow_validate_no_writes.log	111	LOG
evidence/staging/staging-gate-report.json	1,631	JSON

Recovery and performance evidence

Artifact path (some long paths abbreviated)	Bytes	Type
evidence/operations/audit_rollback.json	308	JSON
evidence/operations/backup-restore/synthetic-ops.sql	21,336,534	SQL
evidence/operations/backup_manifest.json	7,418	JSON
evidence/operations/backup_restore.json	8,008	JSON
evidence/operations/changed_input.json	299	JSON

Artifact path (some long paths abbreviated)	Bytes	Type
evidence/operations/connection-isolation.json	113	JSON
evidence/operations/final-instance-health.json	446	JSON
evidence/operations/lock_timeout.json	315	JSON
evidence/operations/lost_commit_ack.json	512	JSON
evidence/operations/metrics_10000.json	1,101	JSON
evidence/operations/metrics_100000.json	14,046	JSON
evidence/operations/metrics_50000.json	6,819	JSON
evidence/operations/metrics_soak120.json	118,557	JSON
evidence/operations/monitoring.json	712	JSON
evidence/operations/op_dbcrash_f128797f.signal	35	SIGNAL
evidence/operations/op_kill_9bea02c0.log	0	LOG
evidence/operations/op_kill_9bea02c0.signal	35	SIGNAL
evidence/operations/performance/input-10000.csv	747,840	CSV
evidence/operations/performance/input-100000.csv	7,677,840	CSV
evidence/operations/performance/input-50000.csv	3,827,840	CSV
evidence/operations/performance/reports/performance_100000_61b8e2ee-5793-404a-873c-3bbdd73e9720_reject...	82	CSV
evidence/operations/performance/reports/performance_100000_61b8e2ee-5793-404a-873c-3bbdd73e9720_reject...	2	JSON
evidence/operations/performance/reports/performance_100000_61b8e2ee-5793-404a-873c-3bbdd73e9720_summar...	1,443	JSON
evidence/operations/performance/reports/performance_10000_d6419230-7eb1-4dfa-8e35-3d12cd9d7ed9_rejects...	82	CSV
evidence/operations/performance/reports/performance_10000_d6419230-7eb1-4dfa-8e35-3d12cd9d7ed9_rejects...	2	JSON
evidence/operations/performance/reports/performance_10000_d6419230-7eb1-4dfa-8e35-3d12cd9d7ed9_summary...	1,433	JSON
evidence/operations/performance/reports/performance_50000_3564db2e-0527-4bb8-b1c0-87ea58c81ef6_rejects...	82	CSV
evidence/operations/performance/reports/performance_50000_3564db2e-0527-4bb8-b1c0-87ea58c81ef6_rejects...	2	JSON

Artifact path (some long paths abbreviated)	Bytes	Type
evidence/operations/performance/reports/performance_50000_3564db2e-0527-4bb8-b1c0-87ea58c81ef6_summary...	1,433	JSON
evidence/operations/performance.json	5,293	JSON
evidence/operations/performance_progress.json	4,637	JSON
evidence/operations/preflight.json	564	JSON
evidence/operations/process_kill.json	376	JSON
evidence/operations/replacement/reports/replacement_guard_a205e9e8-8dfe-4f47-8080-b2f04db7fc19_rejects...	383	CSV
evidence/operations/replacement/reports/replacement_guard_a205e9e8-8dfe-4f47-8080-b2f04db7fc19_rejects...	461	JSON
evidence/operations/replacement/reports/replacement_guard_a205e9e8-8dfe-4f47-8080-b2f04db7fc19_summary...	1,585	JSON
evidence/operations/replacement/reports/replacement_guard_def44a70-8d87-424e-ac4a-198642ce61be_rejects...	82	CSV
evidence/operations/replacement/reports/replacement_guard_def44a70-8d87-424e-ac4a-198642ce61be_rejects...	2	JSON
evidence/operations/replacement/reports/replacement_guard_def44a70-8d87-424e-ac4a-198642ce61be_summary...	1,521	JSON
evidence/operations/replacement/reports/replacement_guard_e9f12f6f-5ee2-4d08-b365-5bc0a37dffdb_rejects...	82	CSV
evidence/operations/replacement/reports/replacement_guard_e9f12f6f-5ee2-4d08-b365-5bc0a37dffdb_rejects...	2	JSON
evidence/operations/replacement/reports/replacement_guard_e9f12f6f-5ee2-4d08-b365-5bc0a37dffdb_summary...	1,517	JSON
evidence/operations/replacement/source.csv	118	CSV
evidence/operations/replacement_guards.json	1,025	JSON
evidence/operations/scd2_concurrency.json	1,576	JSON
evidence/operations/server-crash-pending.json	461	JSON
evidence/operations/server-crash-recovery.json	775	JSON
evidence/operations/server-crash-recovery.log	1,373	LOG
evidence/operations/soak.json	60,577	JSON
evidence/operations/summary.json	21,353	JSON

SECTION MAP

Browser acceptance evidence

Artifact path (some long paths abbreviated)	Bytes	Type
evidence/browser/actual-download.csv	1,015	CSV
evidence/browser/browser-report.json	1,614	JSON
evidence/browser/browser-validated.png	119,063	PNG
evidence/browser/empty.csv	0	CSV
evidence/browser/formula.csv	516	CSV
evidence/browser/malformed.csv	44	CSV
evidence/browser/too-large.csv	5,000,001	CSV
evidence/browser/unsupported.txt	17	TXT

Installed-wheel CLI evidence

Artifact path (some long paths abbreviated)	Bytes	Type
evidence/package/batch_status.log	1,763	LOG
evidence/package/clean.log	1,717	LOG
evidence/package/dirty.log	1,934	LOG
evidence/package/final-wheel-install.log	356	LOG
evidence/package/final-wheel-validation.log	1,705	LOG
evidence/package/first_load.log	1,691	LOG
evidence/package/help.log	2,692	LOG
evidence/package/identical_retry.log	1,744	LOG
evidence/package/init_db.log	47	LOG
evidence/package/invalid_config.log	266	LOG
evidence/package/package-installation.json	1,085	JSON
evidence/package/profile.log	1,586	LOG
evidence/package/reports/customers_clean_cbb77dce-a720-4e29-948a-3bc4278f29b4_rejects.csv	82	CSV
evidence/package/reports/customers_clean_cbb77dce-a720-4e29-	2	JSON

A
B
C
D
E
F
G
H

Artifact path (some long paths abbreviated)	Bytes	Type
948a-3bc4278f29b4_rejects.json		
evidence/package/reports/customers_clean_cbb77dce-a720-4e29-948a-3bc4278f29b4_summary.json	1,398	JSON

Build/runtime identity evidence

Artifact path (some long paths abbreviated)	Bytes	Type
evidence/integrity/authenticated-ci-jobs.json	10,031	JSON
evidence/integrity/handoff-verification.json	9,206	JSON
evidence/integrity/v013-runtime-match.json	5,466	JSON

39. Full release file register

This manifest is a navigation and review aid. The canonical full-length integrity digests remain in SHA256SUMS in the supplied packet. All 413 listed entries **passed SHA-256** matching during documentation preparation. The following groups include every listed path and byte count, grouped to remain legible in print.

DEPLOYMENT_CHECKLIST.md/ - 1 entries

Release-relative file path	Bytes
DEPLOYMENT_CHECKLIST.md	1,859

FINAL_STATUS.md/ - 1 entries

Release-relative file path	Bytes
FINAL_STATUS.md	5,130

GO_NO_GO_DECISION.md/ - 1 entries

Release-relative file path	Bytes
GO_NO_GO_DECISION.md	1,855

MONITORING_PROCEDURE.md/ - 1 entries

Release-relative file path	Bytes
MONITORING_PROCEDURE.md	1,995

README_START_HERE.md/ - 1 entries

Release-relative file path	Bytes
README_START_HERE.md	3,488

ROLLBACK_PLAN.md/ - 1 entries

Release-relative file path	Bytes
ROLLBACK_PLAN.md	2,236

TECHNICAL_REPORT.md/ - 1 entries

Release-relative file path	Bytes
TECHNICAL_REPORT.md	4,301

artifacts/ - 3 entries

Release-relative file path	Bytes
artifacts/etl-validation-toolkit-v0.1.3-source.zip	175,211
artifacts/etl_validation_toolkit-0.1.3-py3-none-any.whl	53,195
artifacts/etl_validation_toolkit-0.1.3.tar.gz	147,351

changes/ - 1 entries

Release-relative file path	Bytes
changes/v0.1.2-to-v0.1.3.patch	13,154

controlled-pilot/ - 9 entries

Release-relative file path	Bytes
controlled-pilot/data/approved-snapshot-example.csv	251
controlled-pilot/pilot-template.yaml	622
controlled-pilot/prepared-manifest.json	664
controlled-pilot/reports/controlled_customer_pilot_379d1f85-cd99-4346-bc66-8d2a81d44924_rejects.csv	82
controlled-pilot/reports/controlled_customer_pilot_379d1f85-cd99-4346-bc66-8d2a81d44924_rejects.json	2
controlled-pilot/reports/controlled_customer_pilot_379d1f85-cd99-4346-bc66-8d2a81d44924_summary.json	1,308
controlled-pilot/reports/controlled_customer_pilot_7f50e4a7-a072-4944-b96c-5779b1f4871c_rejects.csv	82
controlled-pilot/reports/controlled_customer_pilot_7f50e4a7-a072-4944-b96c-5779b1f4871c_rejects.json	2
controlled-pilot/reports/controlled_customer_pilot_7f50e4a7-a072-4944-b96c-5779b1f4871c_summary.json	1,308

evidence/ - 132 entries

Release-relative file path	Bytes
evidence/INDEX.md	1,333
evidence/browser/actual-download.csv	1,015
evidence/browser/browser-report.json	1,614
evidence/browser/browser-validated.png	119,063
evidence/browser/empty.csv	0
evidence/browser/formula.csv	516
evidence/browser/malformed.csv	44
evidence/browser/too-large.csv	5,000,001
evidence/browser/unsupported.txt	17
evidence/integrity/authenticated-ci-jobs.json	10,031
evidence/integrity/handoff-verification.json	9,206
evidence/integrity/v013-runtime-match.json	5,466

A
B
C
D
E
F
G
H

Release-relative file path	Bytes
evidence/mysql/instance.json	307
evidence/operations/audit_rollback.json	308
evidence/operations/backup-restore/synthetic-ops.sql	21,336,534
evidence/operations/backup_manifest.json	7,418
evidence/operations/backup_restore.json	8,008
evidence/operations/changed_input.json	299
evidence/operations/connection-isolation.json	113
evidence/operations/final-instance-health.json	446
evidence/operations/lock_timeout.json	315
evidence/operations/lost_commit_ack.json	512
evidence/operations/metrics_10000.json	1,101
evidence/operations/metrics_100000.json	14,046
evidence/operations/metrics_50000.json	6,819
evidence/operations/metrics_soak120.json	118,557
evidence/operations/monitoring.json	712
evidence/operations/op_dbcrash_f128797f.signal	35
evidence/operations/op_kill_9bea02c0.log	0
evidence/operations/op_kill_9bea02c0.signal	35
evidence/operations/performance/input-10000.csv	747,840
evidence/operations/performance/input-100000.csv	7,677,840
evidence/operations/performance/input-50000.csv	3,827,840
evidence/operations/performance/pipeline-10000.yaml	580
evidence/operations/performance/pipeline-100000.yaml	583
evidence/operations/performance/pipeline-50000.yaml	580
evidence/operations/performance/reports/performance_100000_61b8e2ee-5793-404a-873c-3bbdd73e9720_rejects.csv	82

Release-relative file path	Bytes
evidence/operations/performance/reports/performance_100000_61b8e2ee-5793-404a-873c-3bbdd73e9720_rejects.json	2
evidence/operations/performance/reports/performance_100000_61b8e2ee-5793-404a-873c-3bbdd73e9720_summary.json	1,443
evidence/operations/performance/reports/performance_10000_d6419230-7eb1-4dfa-8e35-3d12cd9d7ed9_rejects.csv	82
evidence/operations/performance/reports/performance_10000_d6419230-7eb1-4dfa-8e35-3d12cd9d7ed9_rejects.json	2
evidence/operations/performance/reports/performance_10000_d6419230-7eb1-4dfa-8e35-3d12cd9d7ed9_summary.json	1,433
evidence/operations/performance/reports/performance_50000_3564db2e-0527-4bb8-b1c0-87ea58c81ef6_rejects.csv	82
evidence/operations/performance/reports/performance_50000_3564db2e-0527-4bb8-b1c0-87ea58c81ef6_rejects.json	2
evidence/operations/performance/reports/performance_50000_3564db2e-0527-4bb8-b1c0-87ea58c81ef6_summary.json	1,433
evidence/operations/performance.json	5,293
evidence/operations/performance_progress.json	4,637
evidence/operations/preflight.json	564
evidence/operations/process_kill.json	376
evidence/operations/replacement/pipeline.yaml	649
evidence/operations/replacement/reports/replacement_guard_a205e9e8-8dfe-4f47-8080-b2f04db7fc19_rejects.csv	383
evidence/operations/replacement/reports/replacement_guard_a205e9e8-8dfe-4f47-8080-b2f04db7fc19_rejects.json	461
evidence/operations/replacement/reports/replacement_guard_a205e9e8-8dfe-4f47-8080-b2f04db7fc19_summary.json	1,585
evidence/operations/replacement/reports/replacement_guard_def44a70-8d87-424e-ac4a-198642ce61be_rejects.csv	82
evidence/operations/replacement/reports/replacement_guard_def44a70-8d87-424e-ac4a-198642ce61be_rejects.json	2
evidence/operations/replacement/reports/replacement_guard_def44a70-8d87-424e-ac4a-198642ce61be_summary.json	1,521
evidence/operations/replacement/reports/replacement_guard_e9f12f6f-5ee2-4d08-b365-5bc0a37dffdb_rejects.csv	82

Release-relative file path	Bytes
evidence/operations/replacement/reports/replacement_guard_e9f12f6f-5ee2-4d08-b365-5bc0a37dffdb_rejects.json	2
evidence/operations/replacement/reports/replacement_guard_e9f12f6f-5ee2-4d08-b365-5bc0a37dffdb_summary.json	1,517
evidence/operations/replacement/source.csv	118
evidence/operations/replacement_guards.json	1,025
evidence/operations/scd2_concurrency.json	1,576
evidence/operations/server-crash-pending.json	461
evidence/operations/server-crash-recovery.json	775
evidence/operations/server-crash-recovery.log	1,373
evidence/operations/soak.json	60,577
evidence/operations/summary.json	21,353
evidence/package/batch_status.log	1,763
evidence/package/clean.log	1,717
evidence/package/dirty.log	1,934
evidence/package/final-wheel-install.log	356
evidence/package/final-wheel-validation.log	1,705
evidence/package/first_load.log	1,691
evidence/package/help.log	2,692
evidence/package/identical_retry.log	1,744
evidence/package/init_db.log	47
evidence/package/invalid_config.log	266
evidence/package/package-installation.json	1,085
evidence/package/profile.log	1,586
evidence/package/reports/customers_clean_cbb77dce-a720-4e29-948a-3bc4278f29b4_rejects.csv	82
evidence/package/reports/customers_clean_cbb77dce-a720-4e29-948a-3bc4278f29b4_rejects.json	2
evidence/package/reports/customers_clean_cbb77dce-a720-4e29-948a-3bc4278f29b4_summary.json	1,398

Release-relative file path	Bytes
evidence/package/wheel-pipeline.yaml	664
evidence/pilot-evidence.json	4,000
evidence/promotion-review.json	1,742
evidence/qa/.coverage	122,880
evidence/qa/build.log	20,948
evidence/qa/coverage.json	135,292
evidence/qa/coverage.xml	62,098
evidence/qa/distinct-target-regression-after.log	98
evidence/qa/distinct-target-regression-before-deterministic.log	14,971
evidence/qa/distinct-target-regression-before.log	98
evidence/qa/environment.txt	1,014
evidence/qa/operations-format.log	23
evidence/qa/operations-ruff.log	19
evidence/qa/pilot-template-validation.log	1,699
evidence/qa/pip-check.log	30
evidence/qa/pytest.xml	15,673
evidence/staging/immutable_pilot/reports/customers_clean_shadow_462cc3db-b19b-4fb4-8cdb-e9c5fd328873_rejects.csv	82
evidence/staging/immutable_pilot/reports/customers_clean_shadow_462cc3db-b19b-4fb4-8cdb-e9c5fd328873_rejects.json	2
evidence/staging/immutable_pilot/reports/customers_clean_shadow_462cc3db-b19b-4fb4-8cdb-e9c5fd328873_summary.json	1,461
evidence/staging/immutable_pilot/reports/customers_clean_shadow_a45b9d90-a378-40f2-96b4-aede243f5867_rejects.csv	82
evidence/staging/immutable_pilot/reports/customers_clean_shadow_a45b9d90-a378-40f2-96b4-aede243f5867_rejects.json	2
evidence/staging/immutable_pilot/reports/customers_clean_shadow_a45b9d90-a378-40f2-96b4-aede243f5867_summary.json	1,461
evidence/staging/immutable_pilot/reports/customers_clean_shadow_eb70ab16-deac-4482-af62-ad4a29e0e017_rejects.csv	82

Release-relative file path	Bytes
evidence/staging/immutable_pilot/reports/customers_clean_shadow_eb70ab16-deac-4482-af62-ad4a29e0e017_rejects.json	2
evidence/staging/immutable_pilot/reports/customers_clean_shadow_eb70ab16-deac-4482-af62-ad4a29e0e017_summary.json	1,461
evidence/staging/immutable_pilot/reports/customers_clean_shadow_fdd2e6ed-57d6-4a1e-a04e-a6107d701b3b_rejects.csv	82
evidence/staging/immutable_pilot/reports/customers_clean_shadow_fdd2e6ed-57d6-4a1e-a04e-a6107d701b3b_rejects.json	2
evidence/staging/immutable_pilot/reports/customers_clean_shadow_fdd2e6ed-57d6-4a1e-a04e-a6107d701b3b_summary.json	1,461
evidence/staging/immutable_pilot/shadow-manifest.json	2,017
evidence/staging/immutable_pilot/shadow-pipeline.yaml	1,561
evidence/staging/mysql8-gate-logs/complete_suite.log	2,557
evidence/staging/mysql8-gate-logs/mypy.log	44
evidence/staging/mysql8-gate-logs/mysql8_integration.log	99
evidence/staging/mysql8-gate-logs/ruff.log	19
evidence/staging/mysql8_staging_gate.log	979
evidence/staging/one-shot-staging-summary.json	549
evidence/staging/operator_safety_tests.log	5,468
evidence/staging/shadow-load-first.json	685
evidence/staging/shadow-load-identical_retry.json	682
evidence/staging/shadow-reconciliation-first.json	1,012
evidence/staging/shadow-reconciliation-identical_retry.json	1,012
evidence/staging/shadow-validation.json	187
evidence/staging/shadow_load_first.log	111
evidence/staging/shadow_load_identical_retry.log	121
evidence/staging/shadow_manifest_verify.log	219
evidence/staging/shadow_plan.log	2,017
evidence/staging/shadow_reconciliation_first.log	152

Release-relative file path	Bytes
evidence/staging/shadow_reconciliation_identical_retry.log	162
evidence/staging/shadow_validate_no_writes.log	111
evidence/staging/staging-gate-report.json	1,631

historical/ - 152 entries

Release-relative file path	Bytes
historical/ETLGuard_v0.1.2_Final_Local_Handoff.zip	413,320
historical/offline_acceptance_v012.py	13,164
historical/v0.1.2-evidence/browser/actual-download.csv	1,015
historical/v0.1.2-evidence/browser/browser-report.json	1,634
historical/v0.1.2-evidence/browser/browser-validated.png	119,066
historical/v0.1.2-evidence/browser/empty.csv	0
historical/v0.1.2-evidence/browser/formula.csv	516
historical/v0.1.2-evidence/browser/malformed.csv	44
historical/v0.1.2-evidence/browser/too-large.csv	5,000,001
historical/v0.1.2-evidence/browser/unsupported.txt	17
historical/v0.1.2-evidence/integrity/authenticated-ci-jobs.json	10,031
historical/v0.1.2-evidence/integrity/handoff-verification.json	9,206
historical/v0.1.2-evidence/mysql/instance.json	307
historical/v0.1.2-evidence/offline/immutable_input/config.yaml	595
historical/v0.1.2-evidence/offline/immutable_input/source.csv	252
historical/v0.1.2-evidence/offline/mysql_gate_no_database.json	414
historical/v0.1.2-evidence/offline/not_authorized_pilot_evidence.json	2,521
historical/v0.1.2-evidence/offline/offline-acceptance.json	3,471
historical/v0.1.2-evidence/offline/pilot_offline/reports/customers_clean_shadow_d0810858-e5df-4aec-8709-d5c407c9d703_rejects.csv	82

Release-relative file path	Bytes
historical/v0.1.2-evidence/offline/pilot_offline/reports/customers_clean_shadow_d0810858-e5df-4aec-8709-d5c407c9d703_rejects.json	2
historical/v0.1.2-evidence/offline/pilot_offline/reports/customers_clean_shadow_d0810858-e5df-4aec-8709-d5c407c9d703_summary.json	1,428
historical/v0.1.2-evidence/offline/pilot_offline/shadow-manifest.json	1,950
historical/v0.1.2-evidence/offline/pilot_offline/shadow-pipeline.yaml	1,532
historical/v0.1.2-evidence/offline/promotion_no_go.json	1,915
historical/v0.1.2-evidence/offline/sample_configs/customers.yaml	858
historical/v0.1.2-evidence/offline/sample_configs/customers_clean.yaml	629
historical/v0.1.2-evidence/offline/sample_configs/customers_excel.yaml	696
historical/v0.1.2-evidence/offline/sample_configs/customers_scd2.yaml	751
historical/v0.1.2-evidence/offline/sample_configs/transactions.yaml	749
historical/v0.1.2-evidence/offline/sample_reports/customers/customers_daily_4088a3d3-a3e7-49cf-a41c-4125bf34ac0c_rejects.csv	1,099
historical/v0.1.2-evidence/offline/sample_reports/customers/customers_daily_4088a3d3-a3e7-49cf-a41c-4125bf34ac0c_rejects.json	1,483
historical/v0.1.2-evidence/offline/sample_reports/customers/customers_daily_4088a3d3-a3e7-49cf-a41c-4125bf34ac0c_summary.json	1,630
historical/v0.1.2-evidence/offline/sample_reports/customers_clean/customers_clean_2751b6d1-3938-4264-ae17-f6ba2fdc716d_rejects.csv	82
historical/v0.1.2-evidence/offline/sample_reports/customers_clean/customers_clean_2751b6d1-3938-4264-ae17-f6ba2fdc716d_rejects.json	2
historical/v0.1.2-evidence/offline/sample_reports/customers_clean/customers_clean_2751b6d1-3938-4264-ae17-f6ba2fdc716d_summary.json	1,454
historical/v0.1.2-evidence/offline/sample_reports/customers_excel/customers_excel_36b7f84c-a75c-4354-a61c-a244f27a183c_rejects.csv	428
historical/v0.1.2-evidence/offline/sample_reports/customers_excel/customers_excel_36b7f84c-a75c-4354-a61c-a244f27a183c_rejects.json	500
historical/v0.1.2-evidence/offline/sample_reports/customers_excel/customers_excel_36b7f84c-a75c-4354-a61c-a244f27a183c_summary.json	1,559
historical/v0.1.2-evidence/offline/sample_reports/customers_scd2/customers_history_922d2c62-3346-477c-8fc4-d36944b6e750_rejects.csv	82
historical/v0.1.2-evidence/offline/sample_reports/customers_scd2/customers_history_922d2c62-3346-477c-8fc4-d36944b6e750_rejects.json	2

A
B
C
D
E
F
G
H

Release-relative file path	Bytes
historical/v0.1.2-evidence/offline/sample_reports/customers_scd2/customers_history_922d2c62-3346-477c-8fc4-d36944b6e750_summary.json	1,459
historical/v0.1.2-evidence/offline/sample_reports/transactions/transactions_daily_0f157897-bde4-46d4-a854-7f57ef6144dd_rejects.csv	1,007
historical/v0.1.2-evidence/offline/sample_reports/transactions/transactions_daily_0f157897-bde4-46d4-a854-7f57ef6144dd_rejects.json	1,387
historical/v0.1.2-evidence/offline/sample_reports/transactions/transactions_daily_0f157897-bde4-46d4-a854-7f57ef6144dd_summary.json	1,653
historical/v0.1.2-evidence/operations/audit_rollback.json	308
historical/v0.1.2-evidence/operations/backup-restore/synthetic-ops.sql	21,333,484
historical/v0.1.2-evidence/operations/backup_manifest.json	6,873
historical/v0.1.2-evidence/operations/backup_restore.json	7,430
historical/v0.1.2-evidence/operations/changed_input.json	300
historical/v0.1.2-evidence/operations/lock_timeout.json	313
historical/v0.1.2-evidence/operations/lost_commit_ack.json	512
historical/v0.1.2-evidence/operations/metrics_10000.json	1,319
historical/v0.1.2-evidence/operations/metrics_100000.json	13,631
historical/v0.1.2-evidence/operations/metrics_50000.json	6,813
historical/v0.1.2-evidence/operations/metrics_soak120.json	121,439
historical/v0.1.2-evidence/operations/monitoring.json	713
historical/v0.1.2-evidence/operations/op_kill_18b9cc96.log	0
historical/v0.1.2-evidence/operations/op_kill_18b9cc96.signal	35
historical/v0.1.2-evidence/operations/performance/input-10000.csv	747,840
historical/v0.1.2-evidence/operations/performance/input-100000.csv	7,677,840
historical/v0.1.2-evidence/operations/performance/input-50000.csv	3,827,840
historical/v0.1.2-evidence/operations/performance/pipeline-10000.yaml	580
historical/v0.1.2-evidence/operations/performance/pipeline-100000.yaml	583
historical/v0.1.2-evidence/operations/performance/pipeline-50000.yaml	580
historical/v0.1.2-evidence/operations/performance/reports/performance_100000_7e85481b-f695-4361-	82

Release-relative file path	Bytes
a07b-f7586387b193_rejects.csv	
historical/v0.1.2-evidence/operations/performance/reports/performance_100000_7e85481b-f695-4361-a07b-f7586387b193_rejects.json	2
historical/v0.1.2-evidence/operations/performance/reports/performance_100000_7e85481b-f695-4361-a07b-f7586387b193_summary.json	1,443
historical/v0.1.2-evidence/operations/performance/reports/performance_10000_c9a12101-fcf3-4e32-93db-757f0eec81ff_rejects.csv	82
historical/v0.1.2-evidence/operations/performance/reports/performance_10000_c9a12101-fcf3-4e32-93db-757f0eec81ff_rejects.json	2
historical/v0.1.2-evidence/operations/performance/reports/performance_10000_c9a12101-fcf3-4e32-93db-757f0eec81ff_summary.json	1,433
historical/v0.1.2-evidence/operations/performance/reports/performance_50000_7cdf4a1e-21ba-44dd-a927-17ff14e007e6_rejects.csv	82
historical/v0.1.2-evidence/operations/performance/reports/performance_50000_7cdf4a1e-21ba-44dd-a927-17ff14e007e6_rejects.json	2
historical/v0.1.2-evidence/operations/performance/reports/performance_50000_7cdf4a1e-21ba-44dd-a927-17ff14e007e6_summary.json	1,433
historical/v0.1.2-evidence/operations/performance.json	5,293
historical/v0.1.2-evidence/operations/performance_progress.json	4,638
historical/v0.1.2-evidence/operations/preflight.json	564
historical/v0.1.2-evidence/operations/process_kill.json	377
historical/v0.1.2-evidence/operations/replacement/pipeline.yaml	649
historical/v0.1.2-evidence/operations/replacement/reports/replacement_guard_15396f47-eab5-47d4-8f3b-c4f23db576ab_rejects.csv	383
historical/v0.1.2-evidence/operations/replacement/reports/replacement_guard_15396f47-eab5-47d4-8f3b-c4f23db576ab_rejects.json	461
historical/v0.1.2-evidence/operations/replacement/reports/replacement_guard_15396f47-eab5-47d4-8f3b-c4f23db576ab_summary.json	1,585
historical/v0.1.2-evidence/operations/replacement/reports/replacement_guard_921cbe6b-7a47-453c-86db-fcbfcbfb0340_rejects.csv	82
historical/v0.1.2-evidence/operations/replacement/reports/replacement_guard_921cbe6b-7a47-453c-86db-fcbfcbfb0340_rejects.json	2
historical/v0.1.2-evidence/operations/replacement/reports/replacement_guard_921cbe6b-7a47-453c-86db-fcbfcbfb0340_summary.json	1,515

Release-relative file path	Bytes
historical/v0.1.2-evidence/operations/replacement/reports/replacement_guard_c6781e74-9c5e-4281-8e6f-1bf17718fcd2_rejects.csv	82
historical/v0.1.2-evidence/operations/replacement/reports/replacement_guard_c6781e74-9c5e-4281-8e6f-1bf17718fcd2_rejects.json	2
historical/v0.1.2-evidence/operations/replacement/reports/replacement_guard_c6781e74-9c5e-4281-8e6f-1bf17718fcd2_summary.json	1,521
historical/v0.1.2-evidence/operations/replacement/source.csv	118
historical/v0.1.2-evidence/operations/replacement_guards.json	1,025
historical/v0.1.2-evidence/operations/scd2_concurrency.json	1,576
historical/v0.1.2-evidence/operations/soak-incomplete.json	284
historical/v0.1.2-evidence/operations/summary.json	20,712
historical/v0.1.2-evidence/package/batch_status.log	1,763
historical/v0.1.2-evidence/package/clean.log	1,717
historical/v0.1.2-evidence/package/dirty.log	1,934
historical/v0.1.2-evidence/package/first_load.log	1,691
historical/v0.1.2-evidence/package/help.log	2,692
historical/v0.1.2-evidence/package/identical_retry.log	1,744
historical/v0.1.2-evidence/package/init_db.log	47
historical/v0.1.2-evidence/package/invalid_config.log	266
historical/v0.1.2-evidence/package/package-installation.json	1,085
historical/v0.1.2-evidence/package/profile.log	1,586
historical/v0.1.2-evidence/package/reports/customers_clean_7680675c-0707-4529-9ba2-1678a23749a0_rejects.csv	82
historical/v0.1.2-evidence/package/reports/customers_clean_7680675c-0707-4529-9ba2-1678a23749a0_rejects.json	2
historical/v0.1.2-evidence/package/reports/customers_clean_7680675c-0707-4529-9ba2-1678a23749a0_summary.json	1,398
historical/v0.1.2-evidence/package/wheel-pipeline.yaml	664
historical/v0.1.2-evidence/python-environment.txt	1,014
historical/v0.1.2-evidence/qa/.coverage	122,880

A
B
C
D
E
F
G
H

Release-relative file path	Bytes
historical/v0.1.2-evidence/qa/coverage.json	135,267
historical/v0.1.2-evidence/qa/coverage.xml	62,062
historical/v0.1.2-evidence/qa/distinct-target-regression-after.log	98
historical/v0.1.2-evidence/qa/distinct-target-regression-before-deterministic.log	14,971
historical/v0.1.2-evidence/qa/distinct-target-regression-before.log	98
historical/v0.1.2-evidence/qa/pytest.xml	15,539
historical/v0.1.2-evidence/staging/mysql8_staging_gate.log	390
historical/v0.1.2-evidence/staging/one-shot-staging-summary.json	367
historical/v0.1.2-evidence/staging/operator_safety_tests.log	5,468
historical/v0.1.2-evidence/staging/staging-gate-report.json	561
historical/v0.1.2-evidence/staging-02/immutable_pilot/reports/customers_clean_shadow_116d55d4-bed0-46f9-becc-dbbf0b0b80c5_rejects.csv	82
historical/v0.1.2-evidence/staging-02/immutable_pilot/reports/customers_clean_shadow_116d55d4-bed0-46f9-becc-dbbf0b0b80c5_rejects.json	2
historical/v0.1.2-evidence/staging-02/immutable_pilot/reports/customers_clean_shadow_116d55d4-bed0-46f9-becc-dbbf0b0b80c5_summary.json	1,470
historical/v0.1.2-evidence/staging-02/immutable_pilot/reports/customers_clean_shadow_75a02ff8-bb1a-480a-9df7-80522797bd27_rejects.csv	82
historical/v0.1.2-evidence/staging-02/immutable_pilot/reports/customers_clean_shadow_75a02ff8-bb1a-480a-9df7-80522797bd27_rejects.json	2
historical/v0.1.2-evidence/staging-02/immutable_pilot/reports/customers_clean_shadow_75a02ff8-bb1a-480a-9df7-80522797bd27_summary.json	1,470
historical/v0.1.2-evidence/staging-02/immutable_pilot/reports/customers_clean_shadow_a2c6e43f-211d-456f-9756-d8fc38408815_rejects.csv	82
historical/v0.1.2-evidence/staging-02/immutable_pilot/reports/customers_clean_shadow_a2c6e43f-211d-456f-9756-d8fc38408815_rejects.json	2
historical/v0.1.2-evidence/staging-02/immutable_pilot/reports/customers_clean_shadow_a2c6e43f-211d-456f-9756-d8fc38408815_summary.json	1,470
historical/v0.1.2-evidence/staging-02/immutable_pilot/reports/customers_clean_shadow_a3c4f0ec-deed-437b-a89d-97e703bedf6b_rejects.csv	82
historical/v0.1.2-evidence/staging-02/immutable_pilot/reports/customers_clean_shadow_a3c4f0ec-deed-437b-a89d-97e703bedf6b_rejects.json	2
historical/v0.1.2-evidence/staging-02/immutable_pilot/reports/customers_clean_shadow_a3c4f0ec-	1,470

Release-relative file path	Bytes
deed-437b-a89d-97e703bedf6b_summary.json	
historical/v0.1.2-evidence/staging-02/immutable_pilot/shadow-manifest.json	2,029
historical/v0.1.2-evidence/staging-02/immutable_pilot/shadow-pipeline.yaml	1,564
historical/v0.1.2-evidence/staging-02/mysql8-gate-logs/complete_suite.log	2,557
historical/v0.1.2-evidence/staging-02/mysql8-gate-logs/mypy.log	44
historical/v0.1.2-evidence/staging-02/mysql8-gate-logs/mysql8_integration.log	99
historical/v0.1.2-evidence/staging-02/mysql8-gate-logs/ruff.log	19
historical/v0.1.2-evidence/staging-02/mysql8_staging_gate.log	982
historical/v0.1.2-evidence/staging-02/one-shot-staging-summary.json	549
historical/v0.1.2-evidence/staging-02/operator_safety_tests.log	5,468
historical/v0.1.2-evidence/staging-02/shadow-load-first.json	688
historical/v0.1.2-evidence/staging-02/shadow-load-identical_retry.json	685
historical/v0.1.2-evidence/staging-02/shadow-reconciliation-first.json	1,012
historical/v0.1.2-evidence/staging-02/shadow-reconciliation-identical_retry.json	1,012
historical/v0.1.2-evidence/staging-02/shadow-validation.json	187
historical/v0.1.2-evidence/staging-02/shadow_load_first.log	114
historical/v0.1.2-evidence/staging-02/shadow_load_identical_retry.log	124
historical/v0.1.2-evidence/staging-02/shadow_manifest_verify.log	219
historical/v0.1.2-evidence/staging-02/shadow_plan.log	2,029
historical/v0.1.2-evidence/staging-02/shadow_reconciliation_first.log	155
historical/v0.1.2-evidence/staging-02/shadow_reconciliation_identical_retry.log	165
historical/v0.1.2-evidence/staging-02/shadow_validate_no_writes.log	114
historical/v0.1.2-evidence/staging-02/staging-gate-report.json	1,643

local_staging_one_shot.py/ - 1 entries

Release-relative file path	Bytes
local_staging_one_shot.py	7,952

operations/ - 10 entries

Release-relative file path	Bytes
operations/assemble_delivery.py	29,179
operations/bootstrap_mysql.py	4,152
operations/package_cli_acceptance.py	3,084
operations/prepare_approval_packet.py	4,959
operations/ruff.toml	88
operations/run_operational_acceptance.py	26,887
operations/run_soak.py	2,414
operations/run_staging.py	1,433
operations/server_crash_drill.py	3,873
operations/start_existing_mysql.py	1,085

requirements-operations.txt/ - 1 entries

Release-relative file path	Bytes
requirements-operations.txt	14

requirements-qa-lock.txt/ - 1 entries

Release-relative file path	Bytes
requirements-qa-lock.txt	849

source/ - 96 entries

Release-relative file path	Bytes
source/etl-validation-toolkit/.env.example	289
source/etl-validation-toolkit/.github/workflows/ci.yml	2,926
source/etl-validation-toolkit/.github/workflows/mysql8-staging-gate.yml	2,484
source/etl-validation-toolkit/.github/workflows/release.yml	577
source/etl-validation-toolkit/.gitignore	322
source/etl-validation-toolkit/CHANGELOG.md	4,783
source/etl-validation-toolkit/LICENSE	1,079
source/etl-validation-toolkit/Launch ETL Guard.command	73
source/etl-validation-toolkit/MANIFEST.in	267
source/etl-validation-toolkit/README.md	19,488
source/etl-validation-toolkit/demo/app.js	8,123
source/etl-validation-toolkit/demo/etlguard_core.zip	15,614
source/etl-validation-toolkit/demo/index.html	4,225
source/etl-validation-toolkit/demo/samples/customers_messy.csv	524
source/etl-validation-toolkit/demo/samples/transactions_messy.csv	367
source/etl-validation-toolkit/demo/styles.css	5,687
source/etl-validation-toolkit/docker-compose.mysql8-staging.yml	700
source/etl-validation-toolkit/docker-compose.yml	545
source/etl-validation-toolkit/docs/completion-report.md	3,248
source/etl-validation-toolkit/docs/configuration.md	8,129
source/etl-validation-toolkit/docs/deployment.md	3,560
source/etl-validation-toolkit/docs/design-decisions.md	4,567
source/etl-validation-toolkit/docs/mysql8-staging-and-pilot-gate.md	8,278
source/etl-validation-toolkit/docs/phase-2-verification-results.md	6,316
source/etl-validation-toolkit/docs/phase-3-shadow-reconciliation.md	5,438

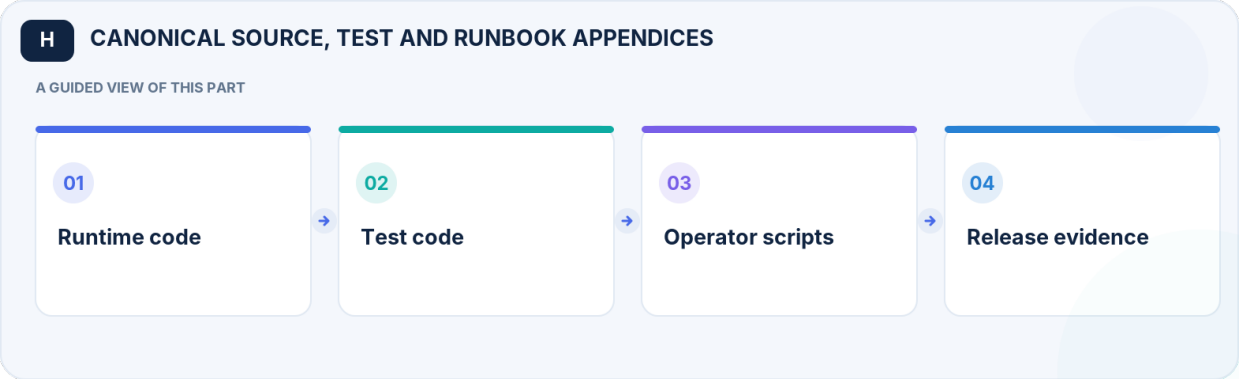
Release-relative file path	Bytes
source/etl-validation-toolkit/docs/pilot-evidence-template.json	2,521
source/etl-validation-toolkit/docs/production-v0.1.2.md	9,701
source/etl-validation-toolkit/docs/review-v0.1.1.md	7,360
source/etl-validation-toolkit/docs/sample_rejects.csv	1,099
source/etl-validation-toolkit/docs/sample_rejects.json	1,483
source/etl-validation-toolkit/docs/shadow-pilot-and-promotion.md	9,824
source/etl-validation-toolkit/pyproject.toml	2,022
source/etl-validation-toolkit/scripts/archive_ci_evidence.py	747
source/etl-validation-toolkit/scripts/assess_promotion.py	7,423
source/etl-validation-toolkit/scripts/build_coverage_badge.py	1,165
source/etl-validation-toolkit/scripts/build_demo_bundle.py	1,098
source/etl-validation-toolkit/scripts/ci_shadow_smoke.py	7,031
source/etl-validation-toolkit/scripts/mysql8_init_shadow.sql	348
source/etl-validation-toolkit/scripts/prepare_shadow_pilot.py	8,448
source/etl-validation-toolkit/scripts/run_shadow_pilot.py	9,645
source/etl-validation-toolkit/scripts/serve_demo.py	1,073
source/etl-validation-toolkit/scripts/test_pilot_release_gate.py	12,103
source/etl-validation-toolkit/scripts/test_shadow_reconciliation.py	10,294
source/etl-validation-toolkit/scripts/test_verify_mysql8_staging.py	2,073
source/etl-validation-toolkit/scripts/verify_mysql8_staging.py	12,016
source/etl-validation-toolkit/scripts/verify_shadow_reconciliation.py	12,601
source/etl-validation-toolkit/src/etlguard/__init__.py	413
source/etl-validation-toolkit/src/etlguard/audit.py	5,749
source/etl-validation-toolkit/src/etlguard/browser_demo.py	3,691
source/etl-validation-toolkit/src/etlguard/checks.py	4,529
source/etl-validation-toolkit/src/etlguard/cleaning.py	3,294
source/etl-validation-toolkit/src/etlguard/cli.py	8,829

Release-relative file path	Bytes
source/etl-validation-toolkit/src/etlguard/config.py	7,174
source/etl-validation-toolkit/src/etlguard/core.py	5,238
source/etl-validation-toolkit/src/etlguard/examples/__init__.py	83
source/etl-validation-toolkit/src/etlguard/examples/data/customers_clean.csv	251
source/etl-validation-toolkit/src/etlguard/examples/data/customers_messy.csv	524
source/etl-validation-toolkit/src/etlguard/examples/data/customers_messy.xlsx	5,058
source/etl-validation-toolkit/src/etlguard/examples/data/transactions_messy.csv	367
source/etl-validation-toolkit/src/etlguard/examples/models.py	1,631
source/etl-validation-toolkit/src/etlguard/examples/pipelines/customers.yaml	628
source/etl-validation-toolkit/src/etlguard/examples/pipelines/customers_clean.yaml	416
source/etl-validation-toolkit/src/etlguard/examples/pipelines/customers_excel.yaml	479
source/etl-validation-toolkit/src/etlguard/examples/pipelines/customers_scd2.yaml	525
source/etl-validation-toolkit/src/etlguard/examples/pipelines/transactions.yaml	513
source/etl-validation-toolkit/src/etlguard/loaders/__init__.py	70
source/etl-validation-toolkit/src/etlguard/loaders/base.py	434
source/etl-validation-toolkit/src/etlguard/loaders/mysql.py	24,481
source/etl-validation-toolkit/src/etlguard/pipeline.py	10,411
source/etl-validation-toolkit/src/etlguard/profile.py	1,300
source/etl-validation-toolkit/src/etlguard/reconcile.py	907
source/etl-validation-toolkit/src/etlguard/rejects.py	2,813
source/etl-validation-toolkit/src/etlguard/scd2.py	2,045
source/etl-validation-toolkit/src/etlguard/sources/__init__.py	212
source/etl-validation-toolkit/src/etlguard/sources/api.py	5,406
source/etl-validation-toolkit/src/etlguard/sources/base.py	679
source/etl-validation-toolkit/src/etlguard/sources/csv.py	1,229
source/etl-validation-toolkit/src/etlguard/sources/excel.py	1,382
source/etl-validation-toolkit/src/etlguard/validation.py	1,527

Release-relative file path	Bytes
source/etl-validation-toolkit/tests/conftest.py	914
source/etl-validation-toolkit/tests/test_api.py	4,599
source/etl-validation-toolkit/tests/test_api_transport.py	6,625
source/etl-validation-toolkit/tests/test_badge.py	746
source/etl-validation-toolkit/tests/test_browser.py	1,867
source/etl-validation-toolkit/tests/test_cleaning.py	2,658
source/etl-validation-toolkit/tests/test_config.py	2,026
source/etl-validation-toolkit/tests/test_coverage_edges.py	2,159
source/etl-validation-toolkit/tests/test_mysql_ddl_compile.py	1,526
source/etl-validation-toolkit/tests/test_mysql_integration.py	15,254
source/etl-validation-toolkit/tests/test_pipeline.py	6,770
source/etl-validation-toolkit/tests/test_production_safety.py	15,546
source/etl-validation-toolkit/tests/test_property_cleaning.py	772
source/etl-validation-toolkit/tests/test_safety_regressions.py	6,642
source/etl-validation-toolkit/tests/test_sources.py	2,781
source/etl-validation-toolkit/tests/test_sqlite_loader.py	6,808
source/etl-validation-toolkit/tests/test_validation.py	5,250

PART H

CANONICAL SOURCE, TEST AND RUNBOOK APPENDICES



Technical reference and operational procedures. Use the heading navigation pane to locate topics.

40. Appendix A - Complete v0.1.3 runtime Python source

The following listings reproduce all Python runtime modules in the supplied `source/etl-validation-toolkit/src/etlguard` tree. Numbered lines are navigation aids. The source ZIP remains the executable authority; these printed listings are for review and maintainability, not direct copy/paste when manual line wrapping occurs.

src/etlguard/__init__.py

14 source lines. Path: src/etlguard/__init__.py

```
0001  """ETL Guard public API; lazy imports keep the browser core database-free."""
0002
0003  from typing import Any
0004
0005  __version__ = "0.1.3"
0006  __all__ = ["Pipeline", "RunResult", "__version__"]
0007
0008
0009  def __getattr__(name: str) -> Any:
0010      if name in {"Pipeline", "RunResult"}:
0011          from etlguard.pipeline import Pipeline, RunResult
0012
0013          return {"Pipeline": Pipeline, "RunResult": RunResult}[name]
0014      raise AttributeError(name)
```

src/etlguard/audit.py

131 source lines. Path: src/etlguard/audit.py

```
0001  """Durable ETL audits and an atomic, idempotent batch-commit ledger."""
0002
0003  from __future__ import annotations
0004
0005  import json
0006  from datetime import datetime, timezone
0007  from typing import Any
0008
0009  from sqlalchemy import BigInteger, Column, MetaData, String, Table, Text, select
0010  from sqlalchemy.dialects.mysql import DATETIME
```

```
from sqlalchemy.engine import Connection, Engine

from etlguard.reconcile import LoadCounts


def audit_table() -> Table:
    """Keep the v0.1.1 audit schema compatible with existing installations."""
    return Table(
        "etl_run_audit", MetaData(),
        Column("batch_id", String(36), primary_key=True),
        Column("pipeline", String(128), nullable=False),
        Column("source", String(512), nullable=False),
        Column("started_at", DATETIME(fsp=6), nullable=False),
        Column("finished_at", DATETIME(fsp=6), nullable=False),
        Column("status", String(32), nullable=False),
        Column("rows_read", BigInteger(), nullable=False),
        Column("rows_valid", BigInteger(), nullable=False),
        Column("rows_rejected", BigInteger(), nullable=False),
        Column("rows_loaded", BigInteger(), nullable=False),
        Column("error_summary", Text(), nullable=False),
        mysql_engine="InnoDB",
    )


def commits_table() -> Table:
    """Rows become visible only if target data and the audit commit together."""
    return Table(
        "etl_batch_commits", MetaData(),
        Column("batch_id", String(36), primary_key=True),
        Column("request_hash", String(64), nullable=False),
        Column("pipeline", String(128), nullable=False),
        Column("target_table", String(64), nullable=False),
        Column("target_mode", String(32), nullable=False),
        Column("committed_at", DATETIME(fsp=6), nullable=False),
        Column("rows_inserted", BigInteger(), nullable=False),
        Column("rows_updated", BigInteger(), nullable=False),
        Column("rows_unchanged", BigInteger(), nullable=False),
        Column("scd2_versions_created", BigInteger(), nullable=False),
        Column("rows_valid", BigInteger(), nullable=False),
        mysql_engine="InnoDB",
    )


def persisted_counts(row: Any) -> LoadCounts:
    return LoadCounts(
        inserted=row["rows_inserted"], updated=row["rows_updated"],
        unchanged=row["rows_unchanged"], versions_created=row["scd2_versions_created"],
        replayed=True,
    )


class BatchConflictError(ValueError):
    """A batch ID already belongs to a different source/configuration/target."""


class AuditWriter:
    def __init__(self, engine: Engine):
        self.engine = engine
        self.table = audit_table()
        self.commits = commits_table()

    def init_db(self) -> None:
        # Both DDL calls must happen *outside* the data transaction (MySQL DDL commits).
        self.table.metadata.create_all(self.engine, checkfirst=True)
        self.commits.metadata.create_all(self.engine, checkfirst=True)

    def _values(self, result: Any, source: str, status: str, counts: LoadCounts | None =
None) -> dict[str, Any]:
        return {
            "batch_id": result.batch_id,
            "pipeline": result.pipeline,
            "source": source[:512],
            "started_at": result.started_at.replace(tzinfo=None),
            "finished_at": datetime.now(timezone.utc).replace(tzinfo=None),
            "status": status,
            "rows_read": result.rows_read,
            "rows_valid": result.rows_valid,
            "rows_rejected": result.rows_rejected,
            "rows_loaded": counts.loaded if counts else result.rows_loaded,
            "error_summary": json.dumps({
                "error_counts_by_type": result.error_counts_by_type,
                "error_counts_by_field": result.error_counts_by_field,
                "issues": result.dataset_issues,
                "failures": result.failures,
```

```
            }),
        }

    def record_success(self, conn: Connection, result: Any, source: str, counts: LoadCounts) -> None:
        """Write success on the SAME transaction/connection as the target merge."""
        values = self._values(result, source, "success", counts)
        existing = conn.execute(
            select(self.table.c.status).where(self.table.c.batch_id == result.batch_id)
            .with_for_update()
        ).first()
        if existing is None:
            conn.execute(self.table.insert().values(**values))
        elif existing.status != "success":
            conn.execute(self.table.update().where(self.table.c.batch_id == result.batch_id)
                         .values(**values))
        else:
            raise BatchConflictError("Audit already claims a successful batch without a new commit")

    def record_failure(self, result: Any, source: str) -> None:
        """Best-effort failure audit, never overwriting a committed success."""
        self.init_db()
        values = self._values(result, source, "failed")
        with self.engine.begin() as conn:
            existing = conn.execute(
                select(self.table.c.status).where(self.table.c.batch_id == result.batch_id)
                .with_for_update()
            ).first()
            if existing is None:
                conn.execute(self.table.insert().values(**values))
            elif existing.status != "success":
                conn.execute(self.table.update().where(self.table.c.batch_id == result.batch_id)
                             .values(**values))

    def write(self, result: Any, source: str) -> None:
        """Backward-compatible one-off audit API; not for transactional load success."""
        self.init_db()
        with self.engine.begin() as conn:
            conn.execute(self.table.insert().values(**self._values(result, source, result.status)))
```

src/etlguard/browser_demo.py

84 source lines. Path: src/etlguard/browser_demo.py

```
"""Pyodide demo adapter: calls production ValidationEngine; loads valid rows to SQLite."""

from __future__ import annotations

import csv
import io
import json
import sqlite3
from datetime import datetime, timezone
from pathlib import Path
from typing import Any

from etlguard.config import ChecksSettings, CleaningSettings
from etlguard.core import ValidationEngine
from etlguard.examples.models import Customer, Transaction
from etlguard.rejects import REJECT_COLUMNS, _safe_csv_cell
from etlguard.sources.base import Lineage, SourceRecord


def validate_csv(csv_text: str, selected_model: str = "Customer") -> str:
    if selected_model not in {"Customer", "Transaction"}:
        raise ValueError("Unknown demonstration model")
    model = Customer if selected_model == "Customer" else Transaction
    is_customer = selected_model == "Customer"
    settings = CleaningSettings(
        date_columns=["signup_date", "end_date"] if is_customer else ["transaction_date"],
        number_columns=["credit_limit"] if is_customer else ["amount"],
        lowercase=["email", "status"] if is_customer else ["type"],
        uppercase=["customer_id"] if is_customer else ["customer_id", "transaction_id"],
    )
```

```
    checks = ChecksSettings(unique=["customer_id"] if is_customer else ["transaction_id"])
    reader = csv.DictReader(io.StringIO(csv_text.lstrip("\ufeff")))
    if not reader.fieldnames:
        raise ValueError("CSV is missing a header row")
    batch_id = "browser-demo"

    def records() -> Any:
        for row in reader:
            if None in row:
                raise ValueError("A CSV row has more fields than its header")
            yield SourceRecord(
                row, Lineage("browser-upload", reader.line_num, datetime.now(timezone.utc), batch_id)
            )

    core = ValidationEngine(model, settings, checks, Path(".")).run(records())
    if core.source_error:
        raise ValueError(core.source_error)  # Never show a partial CSV as a valid feed.
    valid_rows = []
    for candidate in core.valid:
        valid_rows.append(model.model_validate(candidate.data).model_dump(mode="json"))

    # Browser-only destination: SQLite in Pyodide virtual memory, never MySQL.
    conn = sqlite3.connect(":memory:")
    columns = list(model.model_fields)
    ddl = ", ".join('"' + column + '" TEXT' for column in columns)
    conn.execute(f"CREATE TABLE validated_feed ({ddl})")
    quoted = ", ".join('"' + column + '"' for column in columns)
    slots = ", ".join("?" for _ in columns)
    conn.executemany(
        f"INSERT INTO validated_feed ({quoted}) VALUES ({slots})",
        [[None if row[col] is None else str(row[col]) for col in columns] for row in valid_rows],
    )
    loaded = conn.execute("SELECT COUNT(*) FROM validated_feed").fetchone()[0]
    conn.close()
    reject_rows = [row.model_dump() for row in core.rejects]
    rejects_buffer = io.StringIO()
    writer = csv.DictWriter(rejects_buffer, fieldnames=REJECT_COLUMNS)
    writer.writeheader()
    writer.writerows({key: _safe_csv_cell(value) for key, value in row.items()}
                     for row in reject_rows)
    return json.dumps({
        "model": selected_model,
        "batch_id": batch_id,
        "rows_read": core.rows_read,
        "rows_cleaned": core.rows_cleaned,
        "rows_valid": core.rows_valid,
        "rows_rejected": core.rows_rejected,
        "duplicates_dropped": core.duplicates_dropped,
        "rows_loaded_sqlite": loaded,
        "error_counts_by_type": core.error_counts_by_type,
        "valid_rows": valid_rows,
        "rejected_rows": reject_rows,
        "rejects_csv": rejects_buffer.getvalue(),
    }, ensure_ascii=False, default=str)
```

src/etlguard/checks.py

103 source lines. Path: src/etlguard/checks.py

```
"""Dataset-level checks and reference-data checks independent of the database."""

from __future__ import annotations

import csv
from dataclasses import dataclass
from pathlib import Path
from typing import Any

from etlguard.config import ChecksSettings, ForeignKeySettings
from etlguard.rejects import RejectedRow, raw_json, reject
from etlguard.validation import Candidate


@dataclass(frozen=True)
class DatasetIssue:
    field: str
    error_type: str
    message: str
```

```
0020
0021
0022  def load_reference(ref: ForeignKeySettings, base_dir: Path) -> set[str]:
0023      if ref.values is not None:
0024          return {str(value) for value in ref.values}
0025      if ref.path is None or ref.column is None:
0026          raise ValueError("missing foreign key reference settings")
0027      path = (base_dir / ref.path).resolve()
0028      with path.open(encoding="utf-8-sig", newline="") as handle:
0029          reader = csv.DictReader(handle)
0030          if not reader.fieldnames or ref.column not in reader.fieldnames:
0031              raise ValueError(f"Reference file {path} missing column {ref.column}")
0032          return {str(row[ref.column]) for row in reader if row[ref.column] is not None}
0033
0034
0035  class DatasetChecker:
0036      def __init__(self, settings: ChecksSettings, base_dir: Path, drop_duplicates: bool =
      False):
0037          self.settings = settings
0038          self.drop_duplicates = drop_duplicates
0039          self.duplicates_dropped = 0
0040          self.references = {
0041              field: load_reference(ref, base_dir) for field, ref in
      settings.foreign_keys.items()
0042          }
0043
0044      def filter_candidates(self, items: list[Candidate]) -> tuple[list[Candidate],
      list[RejectedRow]]:
0045          valid: list[Candidate] = []
0046          rejected: list[RejectedRow] = []
0047          seen: set[tuple[Any, ...]] = set()
0048          seen_valid_rows: set[str] = set()
0049          self.duplicates_dropped = 0
0050          for item in items:
0051              problems: list[RejectedRow] = []
0052              for field, reference in self.references.items():
0053                  value = item.data.get(field)
0054                  if value is not None and str(value) not in reference:
0055                      problems.append(reject(
0056                          item.lineage.batch_id, item.lineage.source_row_number,
0057                          field, "foreign_key", "value not present in reference set",
0058                          item.raw, item.cleaned,
0059                      ))
0060              # Only previously ACCEPTED records can justify de-duplicating a row.
0061              # A row with invalid foreign keys or a duplicate business key must
0062              # remain an independently counted reject, even when identical to
0063              # another rejected row.
0064              fingerprint = raw_json(item.cleaned)
0065              if not problems and self.drop_duplicates and fingerprint in seen_valid_rows:
0066                  self.duplicates_dropped += 1
0067                  continue
0068              if self.settings.unique:
0069                  key = tuple(item.data.get(col) for col in self.settings.unique)
0070                  if key in seen:
0071                      problems.append(reject(
0072                          item.lineage.batch_id, item.lineage.source_row_number,
0073                          ",".join(self.settings.unique), "unique", "duplicate dataset
      business key",
0074                          item.raw, item.cleaned,
0075                      ))
0076                  elif not problems:
0077                      seen.add(key)
0078              if problems:
0079                  rejected.extend(problems)
0080              else:
0081                  valid.append(item)
0082                  seen_valid_rows.add(fingerprint)
0083          return valid, rejected
0084
0085      def dataset_issues(self, clean_rows: list[dict[str, Any]]) -> list[DatasetIssue]:
0086          issues: list[DatasetIssue] = []
0087          count = len(clean_rows)
0088          bounds = self.settings.expected_row_count
0089          if bounds:
0090              if bounds.min is not None and count < bounds.min:
0091                  issues.append(DatasetIssue("__dataset__", "row_count", f"{count} below
      minimum {bounds.min}"))
0092              if bounds.max is not None and count > bounds.max:
0093                  issues.append(DatasetIssue("__dataset__", "row_count", f"{count} above
      maximum {bounds.max}"))
0094          for field, threshold in self.settings.max_null_pct.items():
0095              if not count:
0096                  continue
0097              nulls = sum(row.get(field) is None for row in clean_rows)
0098              pct = nulls / count
```

A
B
C
D
E
F
G
H

```
            if pct > threshold:
                issues.append(DatasetIssue(
                    field, "null_pct", f"{pct:.2%} null exceeds limit {threshold:.2%}",
                ))
        return issues
```

src/etlguard/cleaning.py

82 source lines. Path: src/etlguard/cleaning.py

```
"""Deterministic, idempotent record cleaning; no pandas or database dependency."""

from __future__ import annotations

import re
from datetime import date, datetime
from decimal import Decimal, InvalidOperation
from typing import Any, Mapping

from etlguard.config import CleaningSettings


def snake_case(value: str) -> str:
    value = re.sub(r"([a-z0-9])([A-Z])", r"\1_\2", value.strip())
    value = re.sub(r"[^a-zA-Z0-9]+", "_", value)
    return value.strip("_").lower()


def date_iso(value: Any, formats: list[str]) -> Any:
    if isinstance(value, datetime):
        return value.date().isoformat()
    if isinstance(value, date):
        return value.isoformat()
    if not isinstance(value, str):
        return value
    for fmt in formats:
        try:
            return datetime.strptime(value, fmt).date().isoformat()
        except ValueError:
            continue
    return value  # Pydantic reports a bad date, rather than silently dropping it.


def decimal_number(value: Any) -> Any:
    if isinstance(value, Decimal):
        return value
    if isinstance(value, bool) or not isinstance(value, (int, float, str)):
        return value
    number = str(value).strip()
    if number.startswith("(") and number.endswith(")"):
        number = "-" + number[1:-1]
    number = re.sub(r"[,$€£₹\s]", "", number)
    try:
        parsed = Decimal(number)
        return parsed if parsed.is_finite() else value
    except InvalidOperation:
        return value


class Cleaner:
    def __init__(self, rules: CleaningSettings):
        self.rules = rules
        self.null_tokens = {token.strip().casefold() for token in rules.null_tokens}
        # Rename rules use normalized names only when normalization is enabled.
        # Without snake_case_columns, mapping keys must match raw column names.
        name_format = snake_case if rules.snake_case_columns else str
        self.rename = {name_format(key): name_format(val) for key, val in
rules.rename_map.items()}
        self.dates = set(rules.date_columns)
        self.numbers = set(rules.number_columns)
        self.lowercase = set(rules.lowercase)
        self.uppercase = set(rules.uppercase)

    def clean(self, row: Mapping[str, Any]) -> dict[str, Any]:
        result: dict[str, Any] = {}
        for raw_name, raw_value in row.items():
            base = snake_case(str(raw_name)) if self.rules.snake_case_columns else
str(raw_name)
            name = self.rename.get(base, base)
            value = raw_value.strip() if self.rules.trim and isinstance(raw_value, str)
```

```
    else raw_value
0069             if isinstance(value, str) and value.casefold() in self.null_tokens:
0070                 value = None
0071             if value is not None and name in self.dates:
0072                 value = date_iso(value, self.rules.date_formats)
0073             if value is not None and name in self.numbers:
0074                 value = decimal_number(value)
0075             if isinstance(value, str) and name in self.lowercase:
0076                 value = value.lower()
0077             if isinstance(value, str) and name in self.uppercase:
0078                 value = value.upper()
0079             if name in result:
0080                 raise ValueError(f"Two incoming columns normalise to the same name:
    {name}")
0081             result[name] = value
0082         return result
```

src/etlguard/cli.py

216 source lines. Path: src/etlguard/cli.py

```
0001 """Typer CLI: validate, run, profile, init-db; 0/1/2 exit contract."""
0002
0003 from __future__ import annotations
0004
0005 import os
0006 from pathlib import Path
0007 from typing import Annotated
0008 from uuid import UUID
0009
0010 import typer
0011 from rich.console import Console
0012 from rich.table import Table
0013
0014 from etlguard.config import ConfigError, load_config
0015 from etlguard.pipeline import Pipeline, RunResult
0016 from etlguard.profile import profile_csv
0017
0018 app = typer.Typer(help="ETL Guard: fail bad data before it reaches MySQL.",
    no_args_is_help=True)
0019 console = Console()
0020
0021
0022 def show_summary(result: RunResult) -> None:
0023     table = Table(title=f"ETL Guard: {result.pipeline} [{result.status}]")
0024     table.add_column("Metric")
0025     table.add_column("Value", justify="right")
0026     for label, value in [
0027         ("Batch ID", result.batch_id), ("Rows read", result.rows_read),
0028         ("Rows cleaned", result.rows_cleaned), ("Valid", result.rows_valid),
0029         ("Rejected", result.rows_rejected), ("Duplicates dropped",
    result.duplicates_dropped),
0030         ("Loaded", result.rows_loaded), ("Inserted", result.rows_inserted),
0031         ("Updated/matched", result.rows_updated), ("Duration (s)",
    result.duration_seconds),
0032     ]:
0033         table.add_row(str(label), str(value))
0034     console.print(table)
0035     if result.error_counts_by_type:
0036         console.print("Errors by type:", result.error_counts_by_type)
0037     if result.error_counts_by_field:
0038         console.print("Errors by field:", result.error_counts_by_field)
0039     if result.replayed:
0040         console.print("[yellow]Already committed: source data was NOT loaded
    again.[/yellow]")
0041     if result.warnings:
0042         console.print("[yellow]Warnings:[/yellow]", "; ".join(result.warnings))
0043     if result.failures:
0044         console.print("[red]Failures:[/red]", "; ".join(result.failures))
0045     console.print(f"Summary: {result.summary_json}")
0046     console.print(f"Rejects: {result.reject_csv}")
0047
0048
0049 def _execute(path: Path, dry_run: bool, batch_id: str | None = None) -> None:
0050     try:
0051         result = Pipeline.from_yaml(path).run(dry_run=dry_run, batch_id=batch_id)
0052     except (ConfigError, ValueError) as exc:
0053         console.print(f"[red]Configuration error:[/red] {exc}")
0054         raise typer.Exit(code=2) from exc
0055     show_summary(result)
0056     if not result.ok:
```

A B C D E F G H

```
0057          raise typer.Exit(code=1)
0058
0059
0060  @app.command()
0061  def validate(config: Annotated[Path, typer.Argument(help="YAML pipeline configuration")])
       -> None:
0062      """Dry run: write rejects + summary; never touch the database."""
0063      _execute(config, dry_run=True)
0064
0065
0066  @app.command()
0067  def run(
0068      config: Annotated[Path, typer.Argument(help="YAML pipeline configuration")],
0069      batch_id: str | None = typer.Option(
0070          None, "--batch-id", help="Stable UUID for safe retry after an unknown commit
      outcome"
0071      ),
0072  ) -> None:
0073      """Validate, stage, transactionally merge and audit a pipeline run."""
0074      _execute(config, dry_run=False, batch_id=batch_id)
0075
0076
0077  @app.command("profile")
0078  def profile_command(path: Annotated[Path, typer.Argument(help="CSV feed to inspect")]) ->
       None:
0079      """Show types, null %, distinct count and min/max by column."""
0080      try:
0081          profiles = profile_csv(path)
0082      except (OSError, ValueError) as exc:
0083          console.print(f"[red]Profile error:[/red] {exc}")
0084          raise typer.Exit(code=1) from exc
0085      table = Table(title=f"Column profile: {path.name}")
0086      for heading in ["Column", "Type", "Rows", "Null %", "Distinct", "Min", "Max"]:
0087          table.add_column(heading)
0088      for entry in profiles:
0089          table.add_row(
0090              entry["column"], entry["inferred_type"], str(entry["rows"]),
0091              f"{entry['null_pct']:.2%}", str(entry["distinct"]),
0092              str(entry["min"]), str(entry["max"]),
0093          )
0094      console.print(table)
0095
0096
0097  @app.command("init-db")
0098  def init_db(
0099      database_url_env: str = typer.Option("ETLGUARD_DATABASE_URL", help="Environment
      variable for MySQL URL"),
0100  ) -> None:
0101      """Create the durable audit and atomic batch-commit tables, if absent."""
0102      from etlguard.audit import AuditWriter
0103      from etlguard.loaders.mysql import MysqlLoader
0104
0105      url = os.getenv(database_url_env)
0106      if not url:
0107          console.print(f"[red]Missing database URL env:[/red] {database_url_env}")
0108          raise typer.Exit(code=2)
0109      try:
0110          loader = MysqlLoader(url)
0111          AuditWriter(loader.engine).init_db()
0112          loader.engine.dispose()
0113      except Exception as exc:
0114          console.print(f"[red]Database initialization failed:[/red] {type(exc).__name__}")
0115          raise typer.Exit(code=1) from exc
0116      console.print("[green]etl_run_audit and etl_batch_commits are ready.[/green]")
0117
0118
0119
0120  @app.command("batch-status")
0121  def batch_status(
0122      batch_id: str = typer.Argument(..., help="Canonical batch UUID"),
0123      database_url_env: str = typer.Option("ETLGUARD_DATABASE_URL"),
0124  ) -> None:
0125      """Read the durable outcome before deciding whether to retry a run."""
0126      from sqlalchemy import inspect, select
0127      from etlguard.audit import AuditWriter
0128      from etlguard.loaders.mysql import MysqlLoader
0129
0130      try:
0131          if str(UUID(batch_id)) != batch_id:
0132              raise ValueError("non-canonical UUID")
0133      except ValueError as exc:
0134          console.print(f"[red]Invalid batch UUID:[/red] {exc}")
0135          raise typer.Exit(code=2) from exc
0136      url = os.getenv(database_url_env)
```

```
0137      if not url:
0138          console.print(f"[red]Missing database URL env:[/red] {database_url_env}")
0139          raise typer.Exit(code=2)
0140      loader = MysqlLoader(url)
0141      try:
0142          writer = AuditWriter(loader.engine)
0143          if not inspect(loader.engine).has_table(writer.commits.name):
0144              console.print("[yellow]Batch ledger has not been initialized.[/yellow]")
0145              raise typer.Exit(code=1)
0146          has_audit = inspect(loader.engine).has_table(writer.table.name)
0147          with loader.engine.connect() as conn:
0148              row = conn.execute(select(writer.commits).where(
0149                  writer.commits.c.batch_id == batch_id
0150              )).mappings().first()
0151              audit_row = (conn.execute(select(writer.table).where(
0152                  writer.table.c.batch_id == batch_id
0153              )).mappings().first() if has_audit else None)
0154          if row is None:
0155              console.print("[yellow]No v0.1.2 committed load found for this batch
      ID.[/yellow]")
0156              if audit_row is not None:
0157                  console.print(f"Audit status: {audit_row['status']}")
0158                  console.print(f"Audit details: {audit_row['error_summary']}")
0159              console.print("Legacy v0.1.1 loads may have committed without a ledger
      entry.")
0160              raise typer.Exit(code=1)
0161          table = Table(title=f"Committed batch: {batch_id}")
0162          table.add_column("Field")
0163          table.add_column("Value")
0164          for key in ("pipeline", "target_table", "target_mode", "committed_at",
0165                      "rows_valid", "rows_inserted", "rows_updated", "rows_unchanged",
0166                      "scd2_versions_created", "request_hash"):
0167              table.add_row(key, str(row[key]))
0168          console.print(table)
0169      except typer.Exit:
0170          raise
0171      except Exception as exc:
0172          console.print(f"[red]Status lookup failed:[/red] {type(exc).__name__}: {exc}")
0173          raise typer.Exit(code=1) from exc
0174      finally:
0175          loader.engine.dispose()
0176
0177
0178  @app.command("upgrade-scd2")
0179  def upgrade_scd2(
0180      config: Annotated[Path, typer.Argument(help="SCD2 YAML pipeline configuration")],
0181      yes: bool = typer.Option(False, "--yes", help="Confirm maintenance-window schema
      migration"),
0182  ) -> None:
0183      """Explicitly add the MySQL 8 active-row uniqueness guard to an existing SCD2
      table."""
0184      from etlguard.loaders.mysql import MysqlLoader
0185      from etlguard.pipeline import resolve_model
0186
0187      if not yes:
0188          console.print("[red]Schema migration requires --yes and a database maintenance
      window.[/red]")
0189          raise typer.Exit(code=2)
0190      try:
0191          settings, _ = load_config(config)
0192          if settings.target is None or settings.target.mode != "scd2":
0193              raise ConfigError("The YAML must specify target.mode: scd2")
0194          url = os.getenv(settings.database_url_env)
0195          if not url:
0196              raise ConfigError(f"Set environment variable {settings.database_url_env}")
0197          model = resolve_model(settings.model)
0198          loader = MysqlLoader(url)
0199          try:
0200              loader.upgrade_scd2(model, settings.target)
0201          finally:
0202              loader.engine.dispose()
0203      except (ConfigError, ValueError) as exc:
0204          console.print(f"[red]Migration rejected:[/red] {exc}")
0205          raise typer.Exit(code=2) from exc
0206      except Exception as exc:
0207          console.print(f"[red]Migration failed:[/red] {type(exc).__name__}: {exc}")
0208          raise typer.Exit(code=1) from exc
0209      console.print("[green]SCD2 active-row uniqueness guard verified.[/green]")
0210
0211  def main() -> None:
0212      app()
0213
0214
0215  if __name__ == "__main__":
0216      main()
```

src/etlguard/config.py

187 source lines. Path: src/etlguard/config.py

```
0001 """Strictly validated YAML config, with paths resolved relative to the YAML file."""
0002
0003 from __future__ import annotations
0004
0005 import re
0006 from pathlib import Path
0007 from typing import Literal
0008
0009 from pydantic import BaseModel, ConfigDict, Field, field_validator, model_validator
0010
0011 IDENTIFIER = re.compile(r"^[A-Za-z_][A-Za-z0-9_]*$")
0012
0013
0014 class ConfigError(ValueError):
0015     """Invalid user-supplied pipeline configuration (CLI exit status 2)."""
0016
0017
0018 class StrictSettings(BaseModel):
0019     model_config = ConfigDict(extra="forbid")
0020
0021
0022 class PaginationSettings(StrictSettings):
0023     mode: Literal["none", "page", "offset", "next"] = "none"
0024     page_param: str = "page"
0025     offset_param: str = "offset"
0026     limit_param: str = "limit"
0027     page_size: int = Field(default=100, ge=1)
0028     start: int = Field(default=1, ge=0)
0029     max_pages: int = Field(default=100, ge=1)
0030     next_path: str = "links.next"
0031
0032
0033 class SourceSettings(StrictSettings):
0034     type: Literal["csv", "excel", "api"]
0035     name: str | None = None
0036     path: str | None = None
0037     url: str | None = None
0038     delimiter: str = ","
0039     encoding: str = "utf-8-sig"
0040     header_row: int = Field(default=1, ge=1)
0041     chunksize: int = Field(default=10000, ge=1)
0042     sheet: str | int = 0
0043     skip_rows: int = Field(default=0, ge=0)
0044     records_path: str = ""
0045     bearer_token_env: str | None = None
0046     timeout_seconds: float = Field(default=15, gt=0)
0047     max_retries: int = Field(default=3, ge=0, le=10)
0048     backoff_seconds: float = Field(default=0.5, ge=0)
0049     pagination: PaginationSettings = Field(default_factory=PaginationSettings)
0050
0051     @model_validator(mode="after")
0052     def check_location(self) -> SourceSettings:
0053         if self.type in {"csv", "excel"} and not self.path:
0054             raise ValueError("CSV/Excel sources require path")
0055         if self.type == "api" and not self.url:
0056             raise ValueError("API sources require url")
0057         if not self.delimiter:
0058             raise ValueError("delimiter must not be empty")
0059         return self
0060
0061
0062 class CleaningSettings(StrictSettings):
0063     trim: bool = True
0064     snake_case_columns: bool = True
0065     null_tokens: list[str] = Field(default_factory=lambda: ["", "NA", "N/A", "null",
"-"])
0066     rename_map: dict[str, str] = Field(default_factory=dict)
0067     date_columns: list[str] = Field(default_factory=list)
0068     date_formats: list[str] = Field(default_factory=lambda: [
0069         "%Y-%m-%d", "%Y/%m/%d", "%d/%m/%Y", "%m/%d/%Y", "%d-%m-%Y", "%Y.%m.%d"
0070     ])
0071     number_columns: list[str] = Field(default_factory=list)
0072     lowercase: list[str] = Field(default_factory=list)
0073     uppercase: list[str] = Field(default_factory=list)
0074     drop_duplicates: bool = True
0075
```

A
B
C
D
E
F
G
H

```python

class ForeignKeySettings(StrictSettings):
    values: list[str | int] | None = None
    path: str | None = None
    column: str | None = None

    @model_validator(mode="after")
    def check_reference(self) -> ForeignKeySettings:
        if (self.values is None) == (self.path is None):
            raise ValueError("foreign key reference requires exactly one of values or path")
        if self.path is not None and not self.column:
            raise ValueError("file-based foreign key reference requires column")
        return self


class RowCountSettings(StrictSettings):
    min: int | None = Field(default=None, ge=0)
    max: int | None = Field(default=None, ge=0)

    @model_validator(mode="after")
    def check_bounds(self) -> RowCountSettings:
        if self.min is not None and self.max is not None and self.min > self.max:
            raise ValueError("expected_row_count min cannot exceed max")
        return self


class ChecksSettings(StrictSettings):
    unique: list[str] = Field(default_factory=list)
    foreign_keys: dict[str, ForeignKeySettings] = Field(default_factory=dict)
    expected_row_count: RowCountSettings | None = None
    max_null_pct: dict[str, float] = Field(default_factory=dict)

    @field_validator("max_null_pct")
    @classmethod
    def check_null_pct(cls, value: dict[str, float]) -> dict[str, float]:
        if any(not 0 <= pct <= 1 for pct in value.values()):
            raise ValueError("max_null_pct must be between 0 and 1")
        return value


class TargetSettings(StrictSettings):
    table: str
    mode: Literal["append", "truncate_insert", "upsert", "scd2"] = "append"
    key: list[str] = Field(default_factory=list)
    tracked_columns: list[str] = Field(default_factory=list)
    batch_size: int = Field(default=500, ge=1)
    lock_timeout_seconds: int = Field(default=30, ge=0, le=300)
    min_replace_ratio: float | None = Field(default=None, ge=0, le=1)

    @field_validator("table")
    @classmethod
    def safe_identifier(cls, value: str) -> str:
        if not IDENTIFIER.fullmatch(value) or len(value) > 48:
            raise ValueError("table must be a SQL identifier of at most 48 characters")
        if value.startswith("stg_") or value == "etl_run_audit":
            raise ValueError("table is a reserved ETL Guard name")
        return value

    @model_validator(mode="after")
    def check_mode(self) -> TargetSettings:
        if self.mode in {"upsert", "scd2"} and not self.key:
            raise ValueError("upsert/scd2 needs at least one business key")
        if len(set(self.key)) != len(self.key):
            raise ValueError("target.key must not contain duplicate column names")
        for name in [*self.key, *self.tracked_columns]:
            if not IDENTIFIER.fullmatch(name):
                raise ValueError(f"invalid target column identifier: {name}")
        return self


class PolicySettings(StrictSettings):
    fail_if_reject_rate_above: float | None = Field(default=None, ge=0, le=1)
    # Replacement mode is destructive: both overrides require explicit opt-in.
    allow_partial_replace: bool = False
    allow_empty_replace: bool = False


class OutputSettings(StrictSettings):
    directory: str = "reports"


class PipelineSettings(StrictSettings):
    pipeline: str
    source: SourceSettings
```

```
    model: str
    cleaning: CleaningSettings = Field(default_factory=CleaningSettings)
    checks: ChecksSettings = Field(default_factory=ChecksSettings)
    target: TargetSettings | None = None
    policy: PolicySettings = Field(default_factory=PolicySettings)
    output: OutputSettings = Field(default_factory=OutputSettings)
    database_url_env: str = "ETLGUARD_DATABASE_URL"

    @field_validator("pipeline")
    @classmethod
    def safe_pipeline_name(cls, value: str) -> str:
        if not re.fullmatch(r"[A-Za-z0-9][A-Za-z0-9_.-]{0,127}", value):
            raise ValueError("pipeline must be a 1-128 character filename-safe
identifier")
        return value


def load_config(path: str | Path) -> tuple[PipelineSettings, Path]:
    file_path = Path(path).expanduser().resolve()
    import yaml

    try:
        data = yaml.safe_load(file_path.read_text(encoding="utf-8"))
        if not isinstance(data, dict):
            raise ConfigError("YAML must contain a mapping")
        settings = PipelineSettings.model_validate(data)
        return settings, file_path.parent
    except (OSError, yaml.YAMLError, ValueError) as exc:
        raise ConfigError(f"Invalid config {file_path}: {exc}") from exc
```

src/etlguard/core.py

123 source lines. Path: src/etlguard/core.py

```
"""Shared CSV/Excel/API + Pyodide cleaning/validation orchestration, no SQL imports."""

from __future__ import annotations

from collections import Counter
from hashlib import sha256
import json
from dataclasses import dataclass, field
from pathlib import Path
from typing import Any, Iterable

from pydantic import BaseModel

from etlguard.checks import DatasetChecker, DatasetIssue
from etlguard.cleaning import Cleaner
from etlguard.config import ChecksSettings, CleaningSettings
from etlguard.rejects import RejectedRow, reject
from etlguard.sources.base import SourceRecord
from etlguard.validation import Candidate, RowValidator


@dataclass
class CoreResult:
    rows_read: int = 0
    source_digest: str = ""
    source_error: str | None = None
    rows_cleaned: int = 0
    duplicates_dropped: int = 0
    valid: list[Candidate] = field(default_factory=list)
    rejects: list[RejectedRow] = field(default_factory=list)
    dataset_issues: list[DatasetIssue] = field(default_factory=list)
    rejected_source_rows: set[tuple[str, int]] = field(default_factory=set)

    @property
    def rows_valid(self) -> int:
        return len(self.valid)

    @property
    def rows_rejected(self) -> int:
        return len(self.rejected_source_rows)

    @property
    def error_counts_by_type(self) -> dict[str, int]:
        counts = Counter(row.error_type for row in self.rejects)
        counts.update(issue.error_type for issue in self.dataset_issues)
        return dict(sorted(counts.items()))
```

```
0047
0048      @property
0049      def error_counts_by_field(self) -> dict[str, int]:
0050          counts = Counter(row.field for row in self.rejects)
0051          counts.update(issue.field for issue in self.dataset_issues)
0052          return dict(sorted(counts.items()))
0053
0054
0055  class ValidationEngine:
0056      def __init__(
0057          self,
0058          model: type[BaseModel],
0059          cleaning: CleaningSettings,
0060          checks: ChecksSettings,
0061          base_dir: Path = Path("."),
0062      ):
0063          self.cleaner = Cleaner(cleaning)
0064          self.validator = RowValidator(model)
0065          self.checker = DatasetChecker(checks, base_dir,
      drop_duplicates=cleaning.drop_duplicates)
0066
0067      def run(self, source: Iterable[SourceRecord]) -> CoreResult:
0068          result = CoreResult()
0069          candidates: list[Candidate] = []
0070          clean_rows: list[dict[str, Any]] = []
0071          fingerprint = sha256()
0072          def guarded_source() -> Iterable[SourceRecord]:
0073              try:
0074                  yield from source
0075              except Exception as exc:
0076                  # Preserve all already-read counts/rejects for a failed partial feed.
0077                  result.source_error = f"source read failed: {type(exc).__name__}: {exc}"
0078
0079          for record in guarded_source():
0080              result.rows_read += 1
0081              # Include ALL incoming records, including invalid ones, in the retry
      identity.
0082              # A length prefix avoids ambiguous concatenations of adjacent JSON records.
0083              encoded = json.dumps({
0084                  "row_number": record.lineage.source_row_number,
0085                  "data": record.data,
0086              }, sort_keys=True, separators=(",", ":"), default=str).encode("utf-8")
0087              fingerprint.update(len(encoded).to_bytes(8, "big"))
0088              fingerprint.update(encoded)
0089              try:
0090                  cleaned = self.cleaner.clean(record.data)
0091              except ValueError as exc:
0092                  result.rejects.append(reject(
0093                      record.lineage.batch_id, record.lineage.source_row_number,
0094                      "__row__", "cleaning", str(exc), record.data, record.data,
0095                  ))
0096                  result.rejected_source_rows.add(
0097                      (record.lineage.source_name, record.lineage.source_row_number)
0098                  )
0099                  continue
0100              result.rows_cleaned += 1
0101              clean_rows.append(cleaned)
0102              outcome = self.validator.validate(cleaned, record.data, record.lineage)
0103              if outcome.rejects:
0104                  # Do not deduplicate invalid rows: every physical bad source
0105                  # record must count toward rejection thresholds and audit totals.
0106                  result.rejects.extend(outcome.rejects)
0107                  result.rejected_source_rows.add(
0108                      (record.lineage.source_name, record.lineage.source_row_number)
0109                  )
0110              elif outcome.candidate:
0111                  candidates.append(outcome.candidate)
0112          result.valid, dataset_rejects = self.checker.filter_candidates(candidates)
0113          result.duplicates_dropped = self.checker.duplicates_dropped
0114          result.rejects.extend(dataset_rejects)
0115          # The same physical source row can produce multiple errors, but is one reject.
0116          rejected_numbers = {row.source_row_number for row in dataset_rejects}
0117          for candidate in candidates:
0118              if candidate.lineage.source_row_number in rejected_numbers:
0119                  result.rejected_source_rows.add((candidate.lineage.source_name,
      candidate.lineage.source_row_number))
0120          result.dataset_issues = self.checker.dataset_issues(clean_rows)
0121          result.source_digest = fingerprint.hexdigest()
0122          assert result.rows_read == result.rows_valid + result.rows_rejected +
      result.duplicates_dropped
0123          return result
```

src/etlguard/examples/__init__.py

1 source lines. Path: src/etlguard/examples/__init__.py

```
"""Public synthetic datasets and models; never use customer production records."""
```

src/etlguard/examples/models.py

44 source lines. Path: src/etlguard/examples/models.py

```
"""Demonstration models for customer dimensions and transaction facts."""

from __future__ import annotations

from datetime import date
from decimal import Decimal
from typing import Literal

from pydantic import BaseModel, ConfigDict, Field, model_validator


class Customer(BaseModel):
    model_config = ConfigDict(extra="forbid")

    customer_id: str = Field(min_length=4, max_length=8, pattern=r"^C\d{3,7}$")
    full_name: str = Field(min_length=2, max_length=80)
    email: str | None = Field(default=None, max_length=150,
pattern=r"^[^\s@]+@[^\s@]+\.[^\s@]+$")
    signup_date: date
    end_date: date | None = None
    status: Literal["active", "inactive"]
    credit_limit: Decimal = Field(ge=0, le=1000000, max_digits=14, decimal_places=2)

    @model_validator(mode="after")
    def check_dates(self) -> Customer:
        if self.end_date is not None and self.end_date < self.signup_date:
            raise ValueError("end_date must be on or after signup_date")
        return self


class Transaction(BaseModel):
    model_config = ConfigDict(extra="forbid")

    transaction_id: str = Field(min_length=4, pattern=r"^T\d{3,}$")
    customer_id: str = Field(pattern=r"^C\d{3,7}$")
    transaction_date: date
    type: Literal["debit", "credit"]
    amount: Decimal = Field(ge=0, le=10000000, max_digits=16, decimal_places=2)
    description: str | None = Field(default=None, max_length=200)

    @model_validator(mode="after")
    def debit_positive(self) -> Transaction:
        if self.type == "debit" and self.amount <= 0:
            raise ValueError("debit amount must be greater than zero")
        return self
```

src/etlguard/loaders/__init__.py

1 source lines. Path: src/etlguard/loaders/__init__.py

```
"""Database loading implementations are imported only when needed."""
```

src/etlguard/loaders/base.py

18 source lines. Path: src/etlguard/loaders/base.py

```
"""Target loader contract."""

from __future__ import annotations

from typing import Protocol

```

A
B
C
D
E
F
G
H

```python
from pydantic import BaseModel

from etlguard.config import TargetSettings
from etlguard.reconcile import LoadCounts
from etlguard.validation import Candidate


class Loader(Protocol):
    def load(
        self, candidates: list[Candidate], batch_id: str,
        model: type[BaseModel], target: TargetSettings,
    ) -> LoadCounts: ...
```

src/etlguard/loaders/mysql.py

468 source lines. Path: src/etlguard/loaders/mysql.py

```python
"""MySQL 8 loader: transactional batch staging, DML merge, type-aware DDL."""

from __future__ import annotations

from contextlib import contextmanager
from datetime import date, datetime, timezone
from hashlib import sha256
import json
import re
from types import SimpleNamespace
from uuid import UUID
from decimal import Decimal
from typing import Any, Sequence, get_args, get_origin
from uuid import uuid4

from pydantic import BaseModel
from sqlalchemy import (
    BigInteger, Boolean, Column, Computed, Date, Float, Index, MetaData,
    Numeric, SmallInteger, String, Table, UniqueConstraint, create_engine, func, inspect,
    select, text, tuple_,
)
from sqlalchemy.dialects.mysql import DATETIME, insert as mysql_insert
from sqlalchemy.engine import Connection, Engine
from sqlalchemy.schema import SchemaItem

from etlguard.audit import AuditWriter, BatchConflictError, persisted_counts
from etlguard.config import IDENTIFIER, TargetSettings
from etlguard.reconcile import LoadCounts, reconcile
from etlguard.scd2 import scd2_merge
from etlguard.validation import Candidate


def sqlalchemy_type(annotation: Any) -> Any:
    """Map concrete Pydantic field types to portable SQLAlchemy column types."""
    import types
    import typing

    origin = get_origin(annotation)
    if origin in (typing.Union, types.UnionType):
        non_none = [arg for arg in get_args(annotation) if arg is not type(None)]
        if len(non_none) == 1:
            return sqlalchemy_type(non_none[0])
    if origin is typing.Literal:
        return sqlalchemy_type(type(get_args(annotation)[0]))
    if annotation is str:
        return String(255)
    if annotation is int:
        return BigInteger()
    if annotation is float:
        return Float()
    if annotation is bool:
        return Boolean()
    if annotation is Decimal:
        return Numeric(18, 2)
    if annotation is date:
        return Date()
    if annotation is datetime:
        return DATETIME(fsp=6)
    raise ValueError(f"Unsupported model field type for MySQL: {annotation}")


def _model_columns(model: type[BaseModel]) -> list[Column[Any]]:
    cols = []
    for name, info in model.model_fields.items():
```

```
        if (not IDENTIFIER.fullmatch(name) or name.startswith("__etl_") or
                name in {"effective_from", "effective_to", "is_current"}):
            raise ValueError(f"Unsafe or reserved model column name: {name}")
        if info.annotation is None:
            raise ValueError(f"Field {name} has no type")
        nullable = not info.is_required() or type(None) in get_args(info.annotation)
        column_type = sqlalchemy_type(info.annotation)
        if isinstance(column_type, String) and info.metadata:
            for constraint in info.metadata:
                max_length = getattr(constraint, "max_length", None)
                if isinstance(max_length, int) and max_length > 0:
                    column_type = String(max_length)
        cols.append(Column(name, column_type, nullable=nullable))
    return cols


def build_tables(model: type[BaseModel], target: TargetSettings) -> tuple[Table, Table]:
    fields = set(model.model_fields)
    if any(key not in fields for key in [*target.key, *target.tracked_columns]):
        raise ValueError("target.key and target.tracked_columns must exist on the
Pydantic model")
    meta = MetaData()
    unique = [UniqueConstraint(*target.key, name=f"uq_{target.table}_business")] if (
        target.key and target.mode != "scd2"
    ) else []
    scd_items: list[SchemaItem] = [
            Column("effective_from", DATETIME(fsp=6), nullable=False),
            Column("effective_to", DATETIME(fsp=6)),
            Column("is_current", Boolean(), nullable=False),
            # UNIQUE(key, marker) allows many NULL markers for historical rows,
            # but exactly one marker=1 for an active version (MySQL 8 + SQLite).
            Column("__etl_current_marker", SmallInteger(), Computed(
                "CASE WHEN is_current = 1 THEN 1 ELSE NULL END", persisted=True,
            )),
            UniqueConstraint(*target.key, "__etl_current_marker",
                             name=f"uq_{target.table}_active"),
            Index(f"ix_{target.table}_current", *target.key, "is_current"),
    ] if target.mode == "scd2" else []
    target_table = Table(
        target.table, meta,
        Column("__etl_id", String(36), primary_key=True),
        *_model_columns(model),
        *scd_items,
        *unique,
        mysql_engine="InnoDB",
    )
    stage_name = f"stg_{target.table}"
    stage_table = Table(
        stage_name, meta,
        Column("__etl_id", String(36), primary_key=True),
        Column("__etl_batch_id", String(36), nullable=False),
        Column("__etl_source_row_number", BigInteger(), nullable=False),
        Column("__etl_ingested_at", DATETIME(fsp=6), nullable=False),
        *_model_columns(model),
        Index(f"ix_{stage_name}_batch", "__etl_batch_id"),
        mysql_engine="InnoDB",
    )
    return target_table, stage_table


def batches(items: list[Any], size: int) -> list[list[Any]]:
    return [items[index:index + size] for index in range(0, len(items), size)]


class MysqlLoader:
    def __init__(self, database_url: str | None = None, engine: Engine | None = None):
        if engine is None:
            if not database_url or not database_url.startswith("mysql+pymysql://"):
                raise ValueError("Supply a mysql+pymysql:// database URL")
            engine = create_engine(database_url, pool_pre_ping=True)
        self.engine = engine

    @contextmanager
    def _locked_connection(self, target: TargetSettings):
        """Serialize every writer to a target, including FIRST SCD2 inserts.

        MySQL GET_LOCK is connection-scoped and must be released explicitly.
        The lock spans all transactional work, not just the SCD2 row SELECT.
        """
        with self.engine.connect() as conn:
            lock = f"etlguard:table:{target.table}"
            acquired = False
            if conn.dialect.name == "mysql":
                # Missing ledger/audit rows must not take REPEATABLE READ gap
                # locks: independent target writers would deadlock when they
```

```
                # reserve different UUIDs in the same primary-index gap.
                # Existing-row locks, unique batch reservations and the target
                # advisory lock still protect retries and SCD2 ordering.
                conn.execution_options(isolation_level="READ COMMITTED")
                value = conn.execute(text("SELECT GET_LOCK(:name, :timeout)"), {
                    "name": lock, "timeout": target.lock_timeout_seconds,
                }).scalar()
                conn.commit()
                if value != 1:
                    raise TimeoutError(f"Could not acquire MySQL ETL lock {lock!r}")
                acquired = True
            try:
                yield conn
            finally:
                if acquired:
                    try:
                        if conn.in_transaction():
                            conn.rollback()
                        released = conn.execute(text("SELECT RELEASE_LOCK(:name)"),
                                                {"name": lock}).scalar()
                        conn.commit()
                        if released != 1:
                            raise RuntimeError(f"MySQL ETL lock {lock!r} was not
released")
                    except Exception:
                        conn.invalidate()  # Never return a lock-holding session to the
pool.
                        raise

    @staticmethod
    def _digest(candidates: list[Candidate], model: type[BaseModel],
                target: TargetSettings) -> str:
        payload = {
            "model": f"{model.__module__}:{model.__qualname__}",
            "target": target.model_dump(mode="json"),
            "rows": [item.data for item in candidates],
        }
        return sha256(json.dumps(payload, sort_keys=True, default=str,
                                 separators=(",", ":")).encode("utf-8")).hexdigest()

    @staticmethod
    def _validate_id(batch_id: str, request_hash: str) -> None:
        try:
            if str(UUID(batch_id)) != batch_id:
                raise ValueError("non-canonical UUID")
        except ValueError as exc:
            raise ValueError("batch_id must be a canonical UUID") from exc
        if not re.fullmatch(r"[0-9a-f]{64}", request_hash):
            raise ValueError("request_hash must be a lowercase SHA-256 hex digest")

    @staticmethod
    def _check_replay(row: Any, batch_id: str, request_hash: str,
                      target: TargetSettings, pipeline: str) -> LoadCounts:
        if (row["request_hash"] != request_hash or row["target_table"] != target.table
                or row["target_mode"] != target.mode or row["pipeline"] != pipeline):
            raise BatchConflictError(
                f"Batch {batch_id} was committed for different
input/configuration/target; "
                "use a new UUID for a different load, or restore the original input to
retry"
            )
        return persisted_counts(row)

    def lookup_commit(self, batch_id: str, request_hash: str, target: TargetSettings,
                      pipeline: str) -> LoadCounts | None:
        """Recover the durable outcome after a lost connection/response at COMMIT."""
        audit = AuditWriter(self.engine)
        if not inspect(self.engine).has_table(audit.commits.name):
            return None
        with self.engine.connect() as conn:
            row = conn.execute(select(audit.commits).where(
                audit.commits.c.batch_id == batch_id
            )).mappings().first()
        return self._check_replay(row, batch_id, request_hash, target, pipeline) if row
else None

    def _check_transactional_storage(self, table_names: list[str]) -> None:
        """MyISAM tables silently break rollback and atomic success guarantees."""
        if self.engine.dialect.name != "mysql":
            return
        inspector = inspect(self.engine)
        for name in table_names:
            options = inspector.get_table_options(name)
            engine_name = options.get("mysql_engine", "").lower()
            if engine_name != "innodb":
```

```
0229                 raise ValueError(
0230                     f"Table {name} uses storage engine {engine_name or 'unknown'}; "
0231                     "atomic ETL loads require InnoDB for ALL target/stage/audit/ledger
      tables"
0232                 )
0233
0234     def _check_schema(self, model: type[BaseModel], target: TargetSettings,
0235                       target_table: Table, stage_table: Table) -> None:
0236         fields = list(model.model_fields)
0237         for table in (target_table, stage_table):
0238             missing = [name for name in fields if name not in table.c]
0239             if missing:
0240                 raise ValueError(f"Existing table {table.name} missing model columns:
      {missing}")
0241         if target.mode not in {"upsert", "scd2"}:
0242             return
0243         inspector = inspect(self.engine)
0244         uniques: list[Sequence[str | None]] = [constraint.get("column_names") or []
0245                    for constraint in inspector.get_unique_constraints(target.table)]
0246         uniques += [index.get("column_names") or [] for index in
      inspector.get_indexes(target.table)
0247                     if index.get("unique")]
0248         uniques += [inspector.get_pk_constraint(target.table).get("constrained_columns")
      or []]
0249         if any(target_table.c[key].nullable for key in target.key):
0250             raise ValueError(
0251                 f"{target.mode} business keys must be NOT NULL; MySQL UNIQUE keys "
0252                 "permit duplicate NULL values"
0253             )
0254         if target.mode == "scd2":
0255             required = set([*target.key, "__etl_current_marker"])
0256             if ("__etl_current_marker" not in target_table.c or
0257                     target_table.c["__etl_current_marker"].computed is None or
0258                     not any(set(keys) == required and len(keys) == len(required)
0259                             for keys in uniques)):
0260                 raise ValueError(
0261                     f"Existing SCD2 table {target.table} lacks its UNIQUE active-row
      guard; "
0262                     "run 'etlguard upgrade-scd2 path/to/pipeline.yaml' during
      maintenance"
0263                 )
0264             return
0265         business_key = set(target.key)
0266         if not any(set(keys) == business_key and len(keys) == len(target.key) for keys in
       uniques):
0267             raise ValueError(
0268                 f"Upsert target {target.table} has no UNIQUE key on {target.key}; "
0269                 "add a unique index or use a fresh target table"
0270             )
0271         competing = [keys for keys in uniques
0272                      if keys and set(keys) not in (business_key, {"__etl_id"})]
0273         if competing:
0274             raise ValueError(
0275                 f"Upsert target {target.table} has competing UNIQUE keys {competing}; "
0276                 "ON DUPLICATE KEY UPDATE could change a different business record"
0277             )
0278
0279     def load(
0280         self, candidates: list[Candidate], batch_id: str,
0281         model: type[BaseModel], target: TargetSettings, *,
0282         request_hash: str | None = None, audit_result: Any = None,
0283         allow_empty_replace: bool = False,
0284     ) -> LoadCounts:
0285         """Commit data, unique batch marker and success audit in ONE DB transaction.
0286
0287         An identical batch_id+hash replays recorded counts without touching data.
0288         A changed feed/configuration using that UUID is rejected, not reloaded.
0289         """
0290         request_hash = request_hash or self._digest(candidates, model, target)
0291         self._validate_id(batch_id, request_hash)
0292         if audit_result is None:
0293             # Low-level callers get the SAME atomic audit guarantee as Pipeline.run.
0294             current = datetime.now(timezone.utc)
0295             audit_result = SimpleNamespace(
0296                 batch_id=batch_id, pipeline="direct_api", source="direct_api",
0297                 started_at=current, rows_read=len(candidates),
      rows_valid=len(candidates),
0298                 rows_rejected=0, rows_loaded=0, error_counts_by_type={},
0299                 error_counts_by_field={}, dataset_issues=[], failures=[],
0300             )
0301         pipeline = audit_result.pipeline
0302         if target.mode == "truncate_insert" and not candidates and not
      allow_empty_replace:
0303             raise ValueError("truncate_insert blocked: no valid records; explicit
      approval is required")
```

```
0304          if target.mode in {"upsert", "scd2"}:
0305              keys = [tuple(item.data[key] for key in target.key) for item in candidates]
0306              if len(set(keys)) != len(keys):
0307                  raise ValueError("Duplicate business keys within one load are not
      allowed")
0308          # DDL must occur BEFORE the data transaction; audit schema stays
      v0.1.1-compatible.
0309          auditor = AuditWriter(self.engine)
0310          auditor.init_db()
0311          target_def, _stage_def = build_tables(model, target)
0312          try:
0313              with self._locked_connection(target) as conn:
0314                  target_def.metadata.create_all(conn, checkfirst=True)
0315                  conn.commit()  # MySQL CREATE TABLE can implicitly commit.
0316                  target_table = Table(target.table, MetaData(), autoload_with=conn)
0317                  stage_table = Table(f"stg_{target.table}", MetaData(),
      autoload_with=conn)
0318                  self._check_schema(model, target, target_table, stage_table)
0319                  self._check_transactional_storage([
0320                      target.table, stage_table.name, auditor.table.name,
      auditor.commits.name,
0321                  ])
0322                  # Reflections/inspections may implicitly start a read transaction.
0323                  conn.commit()
0324                  with conn.begin():
0325                      previous = conn.execute(select(auditor.commits).where(
0326                          auditor.commits.c.batch_id == batch_id
0327                      ).with_for_update()).mappings().first()
0328                      if previous:
0329                          return self._check_replay(previous, batch_id, request_hash,
0330                                                    target, pipeline)
0331                      # Unique reservation is not visible until data + audit commit;
0332                      # concurrent reuse of this UUID blocks or fails without double
      loading.
0333                      conn.execute(auditor.commits.insert().values(
0334                          batch_id=batch_id, request_hash=request_hash, pipeline=pipeline,
0335                          target_table=target.table, target_mode=target.mode,
0336                          committed_at=datetime.now(timezone.utc).replace(tzinfo=None),
0337                          rows_inserted=0, rows_updated=0, rows_unchanged=0,
0338                          scd2_versions_created=0, rows_valid=len(candidates),
0339                      ))
0340                      stage_rows = [{
0341                          "__etl_id": str(uuid4()),
0342                          "__etl_batch_id": batch_id,
0343                          "__etl_source_row_number": item.lineage.source_row_number,
0344                          "__etl_ingested_at":
      item.lineage.ingested_at.replace(tzinfo=None),
0345                          **item.data,
0346                      } for item in candidates]
0347                      conn.execute(stage_table.delete().where(
0348                          stage_table.c["__etl_batch_id"] == batch_id
0349                      ))
0350                      for chunk in batches(stage_rows, target.batch_size):
0351                          conn.execute(stage_table.insert(), chunk)
0352                      staged = conn.execute(select(stage_table).where(
0353                          stage_table.c["__etl_batch_id"] == batch_id
0354                      )).mappings().all()
0355                      fields = list(model.model_fields)
0356                      records = [{name: row[name] for name in fields} for row in staged]
0357                      if target.mode == "truncate_insert" and target.min_replace_ratio is
      not None:
0358                          previous_rows =
      conn.execute(select(func.count()).select_from(target_table)).scalar_one()
0359                          if previous_rows and len(records) / previous_rows <
      target.min_replace_ratio:
0360                              raise ValueError(
0361                                  f"truncate_insert blocked: {len(records)} valid rows vs "
0362                                  f"{previous_rows} existing rows below min_replace_ratio="
0363                                  f"{target.min_replace_ratio:.2f}"
0364                              )
0365                      counts = self._merge(conn, target_table, records, target)
0366                      reconcile(len(candidates), counts)
0367                      conn.execute(stage_table.delete().where(
0368                          stage_table.c["__etl_batch_id"] == batch_id
0369                      ))
0370                      conn.execute(auditor.commits.update().where(
0371                          auditor.commits.c.batch_id == batch_id
0372
      ).values(committed_at=datetime.now(timezone.utc).replace(tzinfo=None),
0373                               rows_inserted=counts.inserted, rows_updated=counts.updated,
0374                               rows_unchanged=counts.unchanged,
0375                               scd2_versions_created=counts.versions_created))
0376                      auditor.record_success(conn, audit_result, audit_result.source,
      counts)
0377                      return counts
```

```
0378          except Exception:
0379              # The connection can be lost while a COMMIT reply is in flight.
0380              # Only a durable matching commit proves success; otherwise fail closed.
0381              try:
0382                  recovered = self.lookup_commit(batch_id, request_hash, target, pipeline)
0383              except BatchConflictError:
0384                  raise  # Distinguish a changed batch from a transient database error.
0385              except Exception:
0386                  recovered = None  # Preserve the original failure (commit state unknown).
0387              if recovered is not None:
0388                  return recovered
0389              raise
0390
0391      def upgrade_scd2(self, model: type[BaseModel], target: TargetSettings) -> None:
0392          """Explicit, operator-approved MySQL 8 migration for pre-0.1.2 SCD2 tables.
0393
0394          Invoke in a maintenance window. It is NOT called by ordinary loads.
0395          """
0396          if self.engine.dialect.name != "mysql" or target.mode != "scd2":
0397              raise ValueError("upgrade-scd2 requires MySQL 8 and an SCD2 target")
0398          build_tables(model, target)  # Validate model and identifiers.
0399          inspector = inspect(self.engine)
0400          if not inspector.has_table(target.table):
0401              raise ValueError(f"SCD2 table {target.table} does not exist")
0402          with self._locked_connection(target) as conn:
0403              table = Table(target.table, MetaData(), autoload_with=conn)
0404              if any(key not in table.c or table.c[key].nullable for key in target.key):
0405                  raise ValueError("SCD2 key columns must exist and be NOT NULL before
      migration")
0406              if "is_current" not in table.c:
0407                  raise ValueError("SCD2 table is missing is_current")
0408              active = select(*[table.c[key] for key in target.key]).where(
0409                  table.c.is_current.is_(True)
0410              ).group_by(*(table.c[key] for key in target.key)).having(func.count() >
      1).limit(1)
0411              if conn.execute(active).first() is not None:
0412                  raise ValueError("Cannot add active-row uniqueness: duplicate active SCD2
       keys exist")
0413              conn.commit()
0414              if "__etl_current_marker" not in table.c:
0415                  conn.execute(text(
0416                      f"ALTER TABLE `{target.table}` ADD COLUMN `__etl_current_marker` "
0417                      "TINYINT GENERATED ALWAYS AS (CASE WHEN is_current = 1 THEN 1 ELSE
      NULL END) STORED"
0418                  ))
0419                  conn.commit()  # Explicit DDL migration; MySQL auto-commits anyway.
0420              inspector = inspect(conn)
0421              unique_sets = [set(c.get("column_names") or [])
0422                             for c in inspector.get_unique_constraints(target.table)]
0423              unique_sets.extend(set(i.get("column_names") or [])
0424                                 for i in inspector.get_indexes(target.table) if
      i.get("unique"))
0425              wanted = set([*target.key, "__etl_current_marker"])
0426              if wanted not in unique_sets:
0427                  fields = ", ".join(f"`{key}`" for key in [*target.key,
      "__etl_current_marker"])
0428                  conn.execute(text(
0429                      f"ALTER TABLE `{target.table}` ADD UNIQUE INDEX "
0430                      f"`uq_{target.table}_active` ({fields})"
0431                  ))
0432                  conn.commit()
0433
0434      def _merge(
0435          self, conn: Connection, table: Table,
0436          records: list[dict[str, Any]], target: TargetSettings,
0437      ) -> LoadCounts:
0438          if target.mode == "truncate_insert":
0439              # Transactional DELETE, not MySQL TRUNCATE (TRUNCATE implicitly commits).
0440              conn.execute(table.delete())
0441          if not records:
0442              return LoadCounts(0, 0)
0443          if target.mode == "scd2":
0444              return scd2_merge(conn, table, records, target.key, target.tracked_columns)
0445          if target.mode in {"append", "truncate_insert"}:
0446              for chunk in batches(records, target.batch_size):
0447                  conn.execute(table.insert(), [{"__etl_id": str(uuid4()), **row} for row
      in chunk])
0448              return LoadCounts(len(records), 0)
0449          existing: set[tuple[Any, ...]] = set()
0450          for chunk in batches(records, target.batch_size):
0451              keys = [tuple(row[key] for key in target.key) for row in chunk]
0452              if len(target.key) == 1:
0453                  condition = table.c[target.key[0]].in_([value[0] for value in keys])
0454              else:
0455                  condition = tuple_(*(table.c[key] for key in target.key)).in_(keys)
```

A
B
C
D
E
F
G
H

```
            selected = conn.execute(select(*(table.c[key] for key in target.key)).where(condition))
            existing.update(tuple(row) for row in selected)
        for chunk in batches(records, target.batch_size):
            values = [{"__etl_id": str(uuid4()), **row} for row in chunk]
            statement = mysql_insert(table).values(values)
            update_fields = [name for name in chunk[0] if name not in target.key]
            if not update_fields:
                update_fields = target.key[:1]
            conn.execute(statement.on_duplicate_key_update(**{
                name: statement.inserted[name] for name in update_fields
            }))
        updated = sum(tuple(row[key] for key in target.key) in existing for row in records)
        return LoadCounts(len(records) - updated, updated)
```

src/etlguard/pipeline.py

228 source lines. Path: src/etlguard/pipeline.py

```
"""End-to-end pipeline orchestration, policies, summaries, audit and lazy DB loading."""

from __future__ import annotations

import importlib
import json
from hashlib import sha256
import os
from datetime import datetime, timezone
from pathlib import Path
from time import perf_counter
from typing import Any, Literal
from uuid import UUID, uuid4

from pydantic import BaseModel, Field

from etlguard.config import ConfigError, PipelineSettings, load_config
from etlguard.core import CoreResult, ValidationEngine
from etlguard.rejects import RejectWriter
from etlguard.sources import ApiSource, CsvSource, ExcelSource


class RunResult(BaseModel):
    batch_id: str
    pipeline: str
    source: str
    status: Literal["success", "dry_run", "failed"] = "failed"
    request_hash: str | None = None
    replayed: bool = False
    durable_commit: bool = False
    warnings: list[str] = Field(default_factory=list)
    started_at: datetime
    finished_at: datetime
    duration_seconds: float = 0
    rows_read: int = 0
    rows_cleaned: int = 0
    rows_valid: int = 0
    rows_rejected: int = 0
    rows_loaded: int = 0
    rows_inserted: int = 0
    rows_updated: int = 0
    rows_unchanged: int = 0
    scd2_versions_created: int = 0
    duplicates_dropped: int = 0
    error_counts_by_type: dict[str, int] = Field(default_factory=dict)
    error_counts_by_field: dict[str, int] = Field(default_factory=dict)
    dataset_issues: list[str] = Field(default_factory=list)
    failures: list[str] = Field(default_factory=list)
    reject_csv: str | None = None
    reject_json: str | None = None
    summary_json: str | None = None

    @property
    def ok(self) -> bool:
        return self.status in {"success", "dry_run"}


def resolve_model(reference: str) -> type[BaseModel]:
    if ":" not in reference:
        raise ConfigError("model must be a module.path:ClassName reference")
    module_name, class_name = reference.split(":", maxsplit=1)
```

```
    try:
        imported = importlib.import_module(module_name)
        model = getattr(imported, class_name)
    except (ImportError, AttributeError) as exc:
        raise ConfigError(f"Cannot import Pydantic model: {reference}: {exc}") from exc
    if not isinstance(model, type) or not issubclass(model, BaseModel):
        raise ConfigError(f"{reference} must be a Pydantic BaseModel subclass")
    return model


class Pipeline:
    def __init__(self, settings: PipelineSettings, base_dir: Path):
        self.settings = settings
        self.base_dir = base_dir

    @classmethod
    def from_yaml(cls, path: str | Path) -> Pipeline:
        settings, base_dir = load_config(path)
        return cls(settings, base_dir)

    def _source(self) -> CsvSource | ExcelSource | ApiSource:
        cfg = self.settings.source
        if cfg.type == "csv":
            return CsvSource(cfg, self.base_dir)
        if cfg.type == "excel":
            return ExcelSource(cfg, self.base_dir)
        return ApiSource(cfg)

    def run(self, dry_run: bool = False, batch_id: str | None = None) -> RunResult:
        if batch_id is not None:
            try:
                if str(UUID(batch_id)) != batch_id:
                    raise ValueError("non-canonical UUID")
            except ValueError as exc:
                raise ConfigError("--batch-id must be a canonical UUID") from exc
        start = perf_counter()
        now = datetime.now(timezone.utc)
        cfg = self.settings
        source = cfg.source.name or cfg.source.path or cfg.source.url or "unknown"
        result = RunResult(
            batch_id=batch_id or str(uuid4()), pipeline=cfg.pipeline, source=source,
            started_at=now, finished_at=now,
        )
        prefix = (self.base_dir / cfg.output.directory).resolve() /
f"{cfg.pipeline}_{result.batch_id}"
        core = CoreResult()
        engine: Any = None
        try:
            model = resolve_model(cfg.model)
            checks = cfg.checks.model_copy(deep=True)
            if cfg.target is not None and cfg.target.mode in {"upsert", "scd2"} and not
checks.unique:
                checks.unique = cfg.target.key[:]
            core = ValidationEngine(model, cfg.cleaning, checks, self.base_dir).run(
                self._source().read(result.batch_id)
            )
            result.rows_read = core.rows_read
            # Repeatable hash of the complete incoming feed + validated settings.
            # This prevents a changed input/configuration silently reusing a batch UUID.
            digest_input = {
                "config": cfg.model_dump(mode="json"),
                "model_schema": model.model_json_schema(),
                "source_digest": core.source_digest,
            }
            result.request_hash = sha256(json.dumps(
                digest_input, sort_keys=True, default=str, separators=(",", ":")
            ).encode("utf-8")).hexdigest()
            result.rows_cleaned = core.rows_cleaned
            result.rows_valid = core.rows_valid
            result.rows_rejected = core.rows_rejected
            result.duplicates_dropped = core.duplicates_dropped
            result.error_counts_by_type = core.error_counts_by_type
            result.error_counts_by_field = core.error_counts_by_field
            result.dataset_issues = [f"{issue.field}: {issue.message}" for issue in
core.dataset_issues]
            if core.dataset_issues:
                result.failures.append("dataset-level validation failed")
            if core.source_error is not None:
                result.failures.append(core.source_error)
            max_rate = cfg.policy.fail_if_reject_rate_above
            if max_rate is not None and result.rows_read and result.rows_rejected /
result.rows_read > max_rate:
                result.failures.append(
                    f"reject rate {result.rows_rejected / result.rows_read:.2%} exceeds
{max_rate:.2%}"
```

A
B
C
D
E
F
G
H

SECTION MAP

```
                )
            if cfg.target is not None and cfg.target.mode == "truncate_insert":
                if result.rows_rejected and not cfg.policy.allow_partial_replace:
                    result.failures.append(
                        "truncate_insert blocked: rejected source records; "
                        "set policy.allow_partial_replace=true to explicitly allow this"
                    )
                if result.rows_read == 0 and not cfg.policy.allow_empty_replace:
                    result.failures.append(
                        "truncate_insert blocked: source is empty; "
                        "set policy.allow_empty_replace=true to explicitly allow this"
                    )
            if result.failures:
                result.status = "failed"
            elif dry_run:
                result.status = "dry_run"
            else:
                if cfg.target is None:
                    raise ConfigError("etlguard run requires target settings")
                db_url = os.getenv(cfg.database_url_env)
                if not db_url:
                    raise ConfigError(f"Set database URL environment variable: {cfg.database_url_env}")
                from etlguard.loaders.mysql import MysqlLoader

                loader = MysqlLoader(db_url)
                engine = loader.engine
                counts = loader.load(
                    core.valid, result.batch_id, model, cfg.target,
                    request_hash=result.request_hash, audit_result=result,
                    allow_empty_replace=cfg.policy.allow_empty_replace,
                )
                result.replayed = counts.replayed
                result.durable_commit = True
                result.rows_inserted = counts.inserted
                result.rows_updated = counts.updated
                result.rows_unchanged = counts.unchanged
                result.scd2_versions_created = counts.versions_created
                result.rows_loaded = counts.loaded
                result.status = "success"
        except ConfigError:
            raise
        except Exception as exc:
            # DB errors must yield a failed job, not partial uncommitted data.
            result.status = "failed"
            result.failures.append(f"{type(exc).__name__}: {exc}")
        finally:
            result.finished_at = datetime.now(timezone.utc)
            result.duration_seconds = round(perf_counter() - start, 4)
            if not dry_run and result.status != "success":
                try:
                    if engine is None and os.getenv(cfg.database_url_env):
                        from etlguard.loaders.mysql import MysqlLoader
                        engine = MysqlLoader(os.environ[cfg.database_url_env]).engine
                    if engine is not None:
                        from etlguard.audit import AuditWriter
                        AuditWriter(engine).record_failure(result, source)
                except Exception as exc:
                    result.failures.append(f"failure audit unavailable: {type(exc).__name__}: {exc}")
            try:
                csv_path, json_path = RejectWriter().write(prefix, core.rejects)
                result.reject_csv = str(csv_path)
                result.reject_json = str(json_path)
            except Exception as exc:
                # The DB success marker is authoritative. A local file I/O error
                # must NOT transform an already-committed batch into a retryable load.
                message = f"reject report unavailable: {type(exc).__name__}: {exc}"
                if result.durable_commit:
                    result.warnings.append(message)
                else:
                    result.status = "failed"
                    result.failures.append(message)
            summary_path = prefix.with_name(prefix.name + "_summary.json")
            try:
                result.summary_json = str(summary_path)
                summary_path.parent.mkdir(parents=True, exist_ok=True)
                summary_path.write_text(result.model_dump_json(indent=2), encoding="utf-8")
            except Exception as exc:
                message = f"summary report unavailable: {type(exc).__name__}: {exc}"
                if result.durable_commit:
                    result.warnings.append(message)
                else:
                    result.status = "failed"
```

```
                    result.failures.append(message)
                result.summary_json = None
            if engine is not None:
                engine.dispose()
        return result
```

src/etlguard/profile.py

30 source lines. Path: src/etlguard/profile.py

```python
"""Quick per-column quality profile for ad-hoc feed triage."""

from __future__ import annotations

from pathlib import Path
from typing import Any

import pandas as pd


def profile_csv(path: str | Path, encoding: str = "utf-8-sig") -> list[dict[str, Any]]:
    frame = pd.read_csv(path, dtype=str, keep_default_na=False, encoding=encoding)
    blank = {"", "na", "n/a", "null", "-"}
    profiles = []
    for column in frame.columns:
        series = frame[column].astype("string").str.strip()
        series = series.mask(series.str.casefold().isin(blank))
        nonnull = series.dropna()
        numbers = pd.to_numeric(nonnull.str.replace(",", "", regex=False),
errors="coerce")
        all_numeric = len(nonnull) > 0 and numbers.notna().all()
        profiles.append({
            "column": str(column),
            "inferred_type": "number" if all_numeric else "string",
            "rows": int(len(series)),
            "null_pct": round(float(series.isna().mean()), 4) if len(series) else 0.0,
            "distinct": int(nonnull.nunique()),
            "min": (float(numbers.min()) if all_numeric else str(nonnull.min())) if
len(nonnull) else None,
            "max": (float(numbers.max()) if all_numeric else str(nonnull.max())) if
len(nonnull) else None,
        })
    return profiles
```

src/etlguard/reconcile.py

32 source lines. Path: src/etlguard/reconcile.py

```python
"""ETL count reconciliation; a failure aborts the database transaction."""

from __future__ import annotations

from dataclasses import dataclass, field


class ReconciliationError(RuntimeError):
    pass


@dataclass(frozen=True)
class LoadCounts:
    inserted: int
    updated: int
    unchanged: int = 0
    versions_created: int = 0
    replayed: bool = field(default=False, compare=False)

    @property
    def loaded(self) -> int:
        # "updated" means an existing business key was processed; for SCD2 it
        # includes unchanged matches, with unchanged disclosed separately.
        return self.inserted + self.updated


def reconcile(rows_valid: int, counts: LoadCounts) -> None:
    if rows_valid != counts.loaded:
        raise ReconciliationError(
            f"reconciliation mismatch: valid={rows_valid}, "
```

A
B
C
D
E
F
G
H

```
            f"inserted={counts.inserted}, updated={counts.updated}"
        )
```

src/etlguard/rejects.py

87 source lines. Path: src/etlguard/rejects.py

```python
"""One CSV/JSON record *per validation error*, preserving original source values."""

from __future__ import annotations

import csv
import json
from pathlib import Path
from typing import Any

from pydantic import BaseModel

from etlguard.cleaning import snake_case

REJECT_COLUMNS = [
    "batch_id", "source_row_number", "field", "error_type",
    "error_message", "raw_value", "raw_row_json",
]


class RejectedRow(BaseModel):
    batch_id: str
    source_row_number: int
    field: str
    error_type: str
    error_message: str
    raw_value: str | None
    raw_row_json: str


def raw_json(row: dict[str, Any]) -> str:
    return json.dumps(row, ensure_ascii=False, sort_keys=True, default=str)


def reject(
    batch_id: str,
    source_row_number: int,
    field: str,
    error_type: str,
    error_message: str,
    raw_row: dict[str, Any],
    cleaned_row: dict[str, Any],
) -> RejectedRow:
    # Preserve the original incoming cell whenever possible. Model field names
    # may have been normalized/renamed, so fall back to the cleaned value.
    raw_value = raw_row.get(field)
    if field not in raw_row:
        for incoming_name, original in raw_row.items():
            normalized = snake_case(str(incoming_name))
            if normalized == field:
                raw_value = original
                break
        else:
            raw_value = cleaned_row.get(field)
    return RejectedRow(
        batch_id=batch_id,
        source_row_number=source_row_number,
        field=field,
        error_type=error_type,
        error_message=error_message,
        raw_value=None if raw_value is None else str(raw_value),
        raw_row_json=raw_json(raw_row),
    )


def _safe_csv_cell(value: Any) -> Any:
    """Prevent spreadsheet-formula execution without changing exact JSON rejects."""
    if isinstance(value, str) and value.startswith(("=", "+", "-", "@", "\t", "\r")):
        return "'" + value
    return value


class RejectWriter:
    def write(self, path_prefix: Path, errors: list[RejectedRow]) -> tuple[Path, Path]:
        path_prefix.parent.mkdir(parents=True, exist_ok=True)
```

A
B
C
D
E
F
G
H

```
0075          csv_path = path_prefix.with_name(path_prefix.name + "_rejects.csv")
0076          json_path = path_prefix.with_name(path_prefix.name + "_rejects.json")
0077          with csv_path.open("w", newline="", encoding="utf-8") as handle:
0078              writer = csv.DictWriter(handle, fieldnames=REJECT_COLUMNS)
0079              writer.writeheader()
0080              for row in errors:
0081                  writer.writerow({key: _safe_csv_cell(value)
0082                                   for key, value in row.model_dump().items()})
0083          json_path.write_text(
0084              json.dumps([item.model_dump() for item in errors], ensure_ascii=False,
      indent=2),
0085              encoding="utf-8",
0086          )
0087          return csv_path, json_path
```

src/etlguard/scd2.py

53 source lines. Path: src/etlguard/**scd2**.py

```
0001  """SCD Type 2 history transition using a single supplied SQLAlchemy connection."""
0002
0003  from __future__ import annotations
0004
0005  from datetime import datetime, timezone
0006  from typing import Any
0007  from uuid import uuid4
0008
0009  from sqlalchemy import Table, and_, select
0010  from sqlalchemy.engine import Connection
0011
0012  from etlguard.reconcile import LoadCounts
0013
0014
0015  def scd2_merge(
0016      conn: Connection,
0017      table: Table,
0018      rows: list[dict[str, Any]],
0019      keys: list[str],
0020      tracked_columns: list[str],
0021  ) -> LoadCounts:
0022      inserted = 0
0023      updated = 0
0024      unchanged = 0
0025      versions = 0
0026      track = (tracked_columns or [name for name in rows[0] if name not in keys]) if rows
      else []
0027      # UTC naive DATETIME(6) is stored consistently without local timezone conversion.
0028      for row in rows:
0029          predicate = and_(*(table.c[key] == row[key] for key in keys),
      table.c.is_current.is_(True))
0030          current =
      conn.execute(select(table).where(predicate).with_for_update()).mappings().first()
0031          if current is None:
0032              conn.execute(table.insert().values(
0033                  **row, __etl_id=str(uuid4()),
      effective_from=datetime.now(timezone.utc).replace(tzinfo=None),
0034                  effective_to=None, is_current=True,
0035              ))
0036              inserted += 1
0037          elif any(current[col] != row[col] for col in track):
0038              now = datetime.now(timezone.utc).replace(tzinfo=None)
0039              conn.execute(table.update().where(table.c.__etl_id ==
      current["__etl_id"]).values(
0040                  effective_to=now, is_current=False,
0041              ))
0042              conn.execute(table.insert().values(
0043                  **row, __etl_id=str(uuid4()), effective_from=now,
0044                  effective_to=None, is_current=True,
0045              ))
0046              updated += 1
0047              versions += 1
0048          else:
0049              # A matched, unchanged business key counts as processed for count
0050              # reconciliation. No physical write occurs for this SCD2 row.
0051              updated += 1
0052              unchanged += 1
0053      return LoadCounts(inserted, updated, unchanged, versions)
```

A B C D E F G H

src/etlguard/sources/__init__.py

7 source lines. Path: src/etlguard/sources/__init__.py

```
0001  """Source connectors."""
0002
0003  from etlguard.sources.api import ApiSource
0004  from etlguard.sources.csv import CsvSource
0005  from etlguard.sources.excel import ExcelSource
0006
0007  __all__ = ["ApiSource", "CsvSource", "ExcelSource"]
```

src/etlguard/sources/api.py

126 source lines. Path: src/etlguard/sources/api.py

```
0001  """GET JSON connector: retries, bearer auth, page/offset/next-link pagination."""
0002
0003  from __future__ import annotations
0004
0005  import os
0006  import time
0007  from contextlib import nullcontext
0008  from typing import Any, Callable, Iterator
0009  from urllib.parse import urljoin, urlparse
0010
0011  import httpx
0012
0013  from etlguard.config import SourceSettings
0014  from etlguard.sources.base import SourceRecord, lineage
0015
0016
0017  class SourceError(RuntimeError):
0018      pass
0019
0020
0021  def dig(data: Any, path: str) -> Any:
0022      current = data
0023      for part in filter(None, path.split(".")):
0024          if not isinstance(current, dict) or part not in current:
0025              raise SourceError(f"Missing JSON path: {path}")
0026          current = current[part]
0027      return current
0028
0029
0030  class ApiSource:
0031      def __init__(
0032          self,
0033          settings: SourceSettings,
0034          client: httpx.Client | None = None,
0035          sleeper: Callable[[float], None] = time.sleep,
0036      ):
0037          if settings.url is None:
0038              raise ValueError("API source missing URL")
0039          if urlparse(settings.url).scheme not in {"http", "https"}:
0040              raise ValueError("API URL must use http or https")
0041          self.settings = settings
0042          self.client = client
0043          self.sleeper = sleeper
0044
0045      def _get(self, client: httpx.Client, url: str, params: dict[str, Any], headers:
      dict[str, str]) -> Any:
0046          cfg = self.settings
0047          for attempt in range(cfg.max_retries + 1):
0048              try:
0049                  response = client.get(url, params=params, headers=headers,
      timeout=cfg.timeout_seconds)
0050                  response.raise_for_status()
0051                  return response.json()
0052              except (httpx.RequestError, httpx.HTTPStatusError) as exc:
0053                  retryable = isinstance(exc, httpx.RequestError) or (
0054                      isinstance(exc, httpx.HTTPStatusError)
0055                      and exc.response.status_code in {429, 500, 502, 503, 504}
0056                  )
0057                  if not retryable or attempt >= cfg.max_retries:
0058                      raise SourceError(f"API request failed ({type(exc).__name__})") from
      exc
0059                  self.sleeper(cfg.backoff_seconds * (2 ** attempt))
0060              except ValueError as exc:
0061                  raise SourceError("API returned invalid JSON") from exc
```

A
B
C
D
E
F
G
H

```
0062          raise SourceError("unreachable API retry state")  # pragma: no cover
0063
0064      def read(self, batch_id: str) -> Iterator[SourceRecord]:
0065          cfg = self.settings
0066          assert cfg.url is not None
0067          headers: dict[str, str] = {}
0068          if cfg.bearer_token_env:
0069              token = os.getenv(cfg.bearer_token_env)
0070              if not token:
0071                  raise SourceError(f"Missing API bearer token env:
      {cfg.bearer_token_env}")
0072              headers["Authorization"] = f"Bearer {token}"
0073          page_cfg = cfg.pagination
0074          url = cfg.url
0075          params: dict[str, Any] = {}
0076          row_number = 0
0077          seen_urls: set[str] = set()
0078          ctx = nullcontext(self.client) if self.client is not None else httpx.Client()
0079          with ctx as client:
0080              assert client is not None
0081              for page_index in range(page_cfg.max_pages):
0082                  if page_cfg.mode == "page":
0083                      params = {
0084                          page_cfg.page_param: page_cfg.start + page_index,
0085                          page_cfg.limit_param: page_cfg.page_size,
0086                      }
0087                  elif page_cfg.mode == "offset":
0088                      params = {
0089                          page_cfg.offset_param: page_cfg.start + page_index *
      page_cfg.page_size,
0090                          page_cfg.limit_param: page_cfg.page_size,
0091                      }
0092                  doc = self._get(client, url, params, headers)
0093                  items = dig(doc, cfg.records_path)
0094                  if not isinstance(items, list):
0095                      raise SourceError("records_path must point to a JSON array")
0096                  for item in items:
0097                      if not isinstance(item, dict):
0098                          raise SourceError("API array entries must be JSON objects")
0099                      row_number += 1
0100                      yield SourceRecord(item, lineage(cfg.name or cfg.url, row_number,
      batch_id))
0101                  if page_cfg.mode == "none":
0102                      return
0103                  if page_cfg.mode in {"page", "offset"}:
0104                      if len(items) < page_cfg.page_size:
0105                          return
0106                      continue
0107                  # Next-link style; missing/null/empty next is the natural end.
0108                  try:
0109                      next_url = dig(doc, page_cfg.next_path)
0110                  except SourceError:
0111                      return
0112                  if not next_url:
0113                      return
0114                  if not isinstance(next_url, str):
0115                      raise SourceError("next-link must be a string URL")
0116                  target = urljoin(url, next_url)
0117                  origin = urlparse(cfg.url)
0118                  destination = urlparse(target)
0119                  if (origin.scheme, origin.netloc) != (destination.scheme,
      destination.netloc):
0120                      raise SourceError("next-link changed origin; refusing to forward
      credentials")
0121                  if target in seen_urls or target == url:
0122                      raise SourceError("API pagination loop detected")
0123                  seen_urls.add(url)
0124                  url = target
0125                  params = {}
0126              raise SourceError(f"API exceeded max_pages={page_cfg.max_pages}")
```

src/etlguard/sources/base.py

29 source lines. Path: src/etlguard/sources/base.py

```
0001  """Source protocol and per-record lineage shared across all connectors."""
0002
0003  from __future__ import annotations
0004
0005  from dataclasses import dataclass
0006  from datetime import datetime, timezone
```

A B C D E F G H

SECTION MAP

```
from typing import Any, Iterator, Protocol


@dataclass(frozen=True)
class Lineage:
    source_name: str
    source_row_number: int
    ingested_at: datetime
    batch_id: str


@dataclass(frozen=True)
class SourceRecord:
    data: dict[str, Any]
    lineage: Lineage


class Source(Protocol):
    def read(self, batch_id: str) -> Iterator[SourceRecord]: ...


def lineage(source: str, row: int, batch_id: str) -> Lineage:
    return Lineage(source, row, datetime.now(timezone.utc), batch_id)
```

src/etlguard/sources/csv.py

38 source lines. Path: src/etlguard/sources/csv.py

```
"""Chunked pandas CSV reader that preserves raw strings and blank rows."""

from __future__ import annotations

from pathlib import Path
from typing import Iterator

from etlguard.config import SourceSettings
from etlguard.sources.base import SourceRecord, lineage


class CsvSource:
    def __init__(self, settings: SourceSettings, base_dir: Path):
        if settings.path is None:
            raise ValueError("CSV source missing path")
        self.settings = settings
        self.path = (base_dir / settings.path).resolve()

    def read(self, batch_id: str) -> Iterator[SourceRecord]:
        import pandas as pd

        cfg = self.settings
        reader = pd.read_csv(
            self.path,
            sep=cfg.delimiter,
            encoding=cfg.encoding,
            header=cfg.header_row - 1,
            chunksize=cfg.chunksize,
            dtype=str,
            keep_default_na=False,
            na_filter=False,
            skip_blank_lines=False,
        )
        row_number = cfg.header_row
        for chunk in reader:
            for row in chunk.to_dict(orient="records"):
                row_number += 1
                yield SourceRecord(row, lineage(cfg.name or self.path.name, row_number,
batch_id))
```

src/etlguard/sources/excel.py

40 source lines. Path: src/etlguard/sources/excel.py

```
"""Excel reader using pandas/openpyxl, with physical worksheet lineage."""

from __future__ import annotations

from pathlib import Path
from typing import Iterator
```

A
B
C
D
E
F
G
H

```
0007
0008  from etlguard.config import SourceSettings
0009  from etlguard.sources.base import SourceRecord, lineage
0010
0011
0012  class ExcelSource:
0013      def __init__(self, settings: SourceSettings, base_dir: Path):
0014          if settings.path is None:
0015              raise ValueError("Excel source missing path")
0016          self.settings = settings
0017          self.path = (base_dir / settings.path).resolve()
0018
0019      def read(self, batch_id: str) -> Iterator[SourceRecord]:
0020          import pandas as pd
0021
0022          cfg = self.settings
0023          frame = pd.read_excel(
0024              self.path,
0025              sheet_name=cfg.sheet,
0026              header=cfg.header_row - 1,
0027              skiprows=cfg.skip_rows,
0028              dtype=str,
0029              keep_default_na=False,
0030              na_filter=False,
0031              engine="openpyxl",
0032          )
0033          # With numeric skiprows, pandas' header is relative to the remaining sheet.
0034          first_data_row = cfg.skip_rows + cfg.header_row + 1
0035          for offset, (_, series) in enumerate(frame.iterrows()):
0036              record = {str(key): value for key, value in series.items()}
0037              yield SourceRecord(
0038                  record,
0039                  lineage(cfg.name or self.path.name, first_data_row + offset, batch_id),
0040              )
```

src/etlguard/validation.py

47 source lines. Path: src/etlguard/validation.py

```
0001  """Pydantic v2 typed row validation, preserving all independent field errors."""
0002
0003  from __future__ import annotations
0004
0005  from dataclasses import dataclass
0006  from typing import Any
0007
0008  from pydantic import BaseModel, ValidationError
0009
0010  from etlguard.rejects import RejectedRow, reject
0011  from etlguard.sources.base import Lineage
0012
0013
0014  @dataclass
0015  class Candidate:
0016      data: dict[str, Any]
0017      cleaned: dict[str, Any]
0018      raw: dict[str, Any]
0019      lineage: Lineage
0020
0021
0022  @dataclass
0023  class ValidationOutcome:
0024      candidate: Candidate | None
0025      rejects: list[RejectedRow]
0026
0027
0028  class RowValidator:
0029      def __init__(self, model: type[BaseModel]):
0030          self.model = model
0031
0032      def validate(self, cleaned: dict[str, Any], raw: dict[str, Any], line: Lineage) ->
      ValidationOutcome:
0033          try:
0034              # Retain Python date/Decimal objects for correct SQLAlchemy type binding.
0035              instance = self.model.model_validate(cleaned)
0036              return ValidationOutcome(
0037                  Candidate(instance.model_dump(mode="python"), cleaned, raw, line), [],
0038              )
0039          except ValidationError as exc:
0040              errors = []
0041              for detail in exc.errors(include_url=False, include_context=False,
```

A
B
C
D
E
F
G
H

```
    include_input=False):
                loc = ".".join(str(part) for part in detail["loc"]) or "__row__"
                errors.append(reject(
                    line.batch_id, line.source_row_number, loc, detail["type"],
                    detail["msg"], raw, cleaned,
                ))
            return ValidationOutcome(None, errors)
```

41. Appendix B - Complete developer and staging scripts

These listings document release preparation, operator-safety tests, independent reconciliation, shadow pilots, MySQL staging verification and the local web demo. They are intended to be read alongside the release approval rules and must not be launched against an unapproved database.

scripts/archive_ci_evidence.py

22 source lines.

```
"""Retain synthetic staging evidence in Actions logs when artifact storage is full."""

from base64 import b64encode
from io import BytesIO
from pathlib import Path
from zipfile import ZIP_DEFLATED, ZipFile


def main() -> None:
    root = Path.cwd()
    paths = [root / "staging-gate-report.json", root / "shadow-smoke-report.json"]
    paths.extend(sorted((root / "mysql8-gate-logs").glob("*")))
    buffer = BytesIO()
    with ZipFile(buffer, "w", ZIP_DEFLATED) as archive:
        for path in paths:
            if path.is_file() and not path.is_symlink():
                archive.write(path, path.relative_to(root))
    print("ETLGUARD_EVIDENCE_BASE64=" + b64encode(buffer.getvalue()).decode("ascii"))


if __name__ == "__main__":
    main()
```

scripts/assess_promotion.py

153 source lines.

```
#!/usr/bin/env python3
"""Read-only evidence assessor: never promotes or modifies a database.

Passing this assessor means *ready for human release review*, NOT approved to
run production. All operational evidence must be independently reviewed.
"""
from __future__ import annotations

import argparse
from etlguard import __version__

from datetime import datetime, timedelta, timezone
from hashlib import sha256
import json
from pathlib import Path
import re
import sys
from typing import Any

REQUIRED_AUTOMATED = ("mysql8_integration", "complete_suite", "ruff", "mypy")
REQUIRED_OPERATIONAL = (
    "immutable_source", "backup_restore_rehearsal", "shadow_row_reconciliation",
    "same_batch_retry", "changed_input_rejected", "commit_outcome_recovery",
    "scd2_concurrency", "rollback_rehearsal", "monitoring_observation",
    "supervised_real_target_pilot",
)
REQUIRED_APPROVERS = ("data_owner", "database_owner", "operations_owner")
```

A B C D E F G H

```
0028
0029
0030  def digest_file(path: Path) -> str:
0031      digest = sha256()
0032      with path.open("rb") as handle:
0033          for chunk in iter(lambda: handle.read(1024 * 1024), b""):
0034              digest.update(chunk)
0035      return digest.hexdigest()
0036
0037
0038  def check_staging(report: dict[str, Any], *, now: datetime | None = None) -> list[str]:
0039      reasons: list[str] = []
0040      if report.get("toolkit_version") != __version__:
0041          reasons.append(f"Staging report version is not v{__version__}")
0042      if (report.get("status") != "passed" or not report.get("mysql8_server_verified")
0043              or not report.get("mysql8_integration_executed")):
0044          reasons.append("A completed, passing real MySQL 8 staging gate is required")
0045      preflight = report.get("preflight") or {}
0046      version = str(preflight.get("server_version", ""))
0047      if not re.match(r"^8\.\d+\.", version) or "mariadb" in version.lower():
0048          reasons.append("Staging server was not verified as Oracle MySQL 8")
0049      if str(preflight.get("default_storage_engine", "")).lower() != "innodb":
0050          reasons.append("Staging server's default engine was not InnoDB")
0051      if preflight.get("unexpected_tables"):
0052          reasons.append("Staging schema contained unexpected tables")
0053      if preflight.get("read_only") is not False:
0054          reasons.append("Staging preflight did not verify read/write test schema")
0055      if preflight.get("strict_sql_mode") is not True:
0056          reasons.append("Staging preflight did not verify STRICT SQL mode")
0057      if preflight.get("schema_only_privileges") is not True:
0058          reasons.append("Staging preflight did not verify least-privilege test
      credentials")
0059      if preflight.get("non_innodb_tables"):
0060          reasons.append("Staging preflight found non-transactional tables")
0061      timestamp = report.get("timestamp_utc")
0062      try:
0063          generated = datetime.fromisoformat(str(timestamp).replace("Z", "+00:00"))
0064          if generated.tzinfo is None:
0065              raise ValueError("Missing timezone")
0066          current = now or datetime.now(timezone.utc)
0067          if generated > current + timedelta(minutes=5) or current - generated >
      timedelta(days=30):
0068              reasons.append("Staging report is outside the 30-day validity window")
0069      except ValueError:
0070          reasons.append("Staging report does not contain a valid timezone-aware
      timestamp")
0071      checks = report.get("checks") or []
0072      if not isinstance(checks, list):
0073          checks = []
0074      by_name = {item.get("name"): item for item in checks if isinstance(item, dict)}
0075      for key in REQUIRED_AUTOMATED:
0076          entry = by_name.get(key)
0077          if not entry or entry.get("status") != "passed" or entry.get("exit_code") != 0 or
       entry.get("skipped") != 0:
0078              reasons.append(f"Automated gate '{key}' did not pass with zero skips")
0079      return reasons
0080
0081
0082  def check_operational(evidence: dict[str, Any], evidence_file: Path) -> tuple[list[str],
      dict[str, str]]:
0083      reasons: list[str] = []
0084      hashes: dict[str, str] = {}
0085      if evidence.get("toolkit_version") != __version__:
0086          reasons.append(f"Pilot evidence version is not v{__version__}")
0087      for section, keys in [("operational", REQUIRED_OPERATIONAL), ("approvals",
      REQUIRED_APPROVERS)]:
0088          entries = evidence.get(section) or {}
0089          if not isinstance(entries, dict):
0090              entries = {}
0091          for key in keys:
0092              record = entries.get(key) or {}
0093              if not isinstance(record, dict) or record.get("verified") is not True:
0094                  reasons.append(f"Missing verified {section}: {key}")
0095                  continue
0096              path_text = record.get("evidence_file")
0097              if not isinstance(path_text, str) or not path_text.strip():
0098                  reasons.append(f"Missing artifact for {section}.{key}")
0099                  continue
0100              file_path = (evidence_file.parent / path_text).resolve()
0101              # Do not allow manifest relative traversal to unintentionally read private
      files.
0102              if not file_path.is_relative_to(evidence_file.parent.resolve()):
0103                  reasons.append(f"Evidence path outside directory for {section}.{key}")
0104                  continue
0105              if not file_path.is_file() or file_path.stat().st_size == 0:
```

```
                reasons.append(f"Missing/empty evidence file for {section}.{key}")
                continue
            hashes[f"{section}.{key}"] = digest_file(file_path)
    return reasons, hashes


def assess(staging: Path, evidence: Path) -> dict[str, Any]:
    staging_data = json.loads(staging.read_text(encoding="utf-8"))
    evidence_data = json.loads(evidence.read_text(encoding="utf-8"))
    if not isinstance(staging_data, dict) or not isinstance(evidence_data, dict):
        raise ValueError("Both JSON files must contain objects")
    staging_reasons = check_staging(staging_data)
    operational_reasons, files = check_operational(evidence_data, evidence)
    reasons = staging_reasons + operational_reasons
    return {
        "assessed_utc": datetime.now(timezone.utc).isoformat(),
        "toolkit_version": __version__,
        "decision": "blocked" if reasons else "review_ready_not_authorized",
        "production_approved": False,
        "database_writes_executed": False,
        "reasons": reasons,
        "staging_report_sha256": digest_file(staging),
        "evidence_manifest_sha256": digest_file(evidence),
        "evidence_artifact_sha256": files,
        "disclaimer": "Artifact presence and user assertions are not cryptographic
signatures. Human approval is still required.",
    }


def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--staging", type=Path, required=True)
    parser.add_argument("--evidence", type=Path, required=True)
    parser.add_argument("--report", type=Path, default=Path("promotion-review.json"))
    args = parser.parse_args()
    try:
        result = assess(args.staging.resolve(), args.evidence.resolve())
        out = args.report.resolve()
        out.parent.mkdir(parents=True, exist_ok=True)
        out.write_text(json.dumps(result, indent=2, sort_keys=True) + "\n",
encoding="utf-8")
        print(f"Decision: {result['decision']}; unmet gates: {len(result['reasons'])};
report: {out}")
        return 0 if result["decision"] == "review_ready_not_authorized" else 2
    except (OSError, ValueError, TypeError, KeyError) as exc:
        print(f"BLOCKED: {type(exc).__name__}: {exc}", file=sys.stderr)
        return 2


if __name__ == "__main__":
    raise SystemExit(main())
```

scripts/build_coverage_badge.py

37 source lines.

```
"""Render Shields endpoint JSON from the branch-aware coverage.py summary.

Usage: python scripts/build_coverage_badge.py coverage-summary.json demo/coverage.json
"""

from __future__ import annotations

import json
import sys
from pathlib import Path


def badge_payload(total_percent: float) -> dict[str, str | int]:
    value = max(0, min(100, round(total_percent)))
    if value >= 90:
        color = "brightgreen"
    elif value >= 85:
        color = "green"
    elif value >= 75:
        color = "yellow"
    else:
        color = "red"
    return {"schemaVersion": 1, "label": "coverage", "message": f"{value}%", "color":
color}
```

```
0024
0025
0026  def main() -> None:
0027      if len(sys.argv) != 3:
0028          raise SystemExit("Usage: build_coverage_badge.py coverage-summary.json
      demo/coverage.json")
0029      source, destination = Path(sys.argv[1]), Path(sys.argv[2])
0030      summary = json.loads(source.read_text(encoding="utf-8"))
0031      percent = float(summary["totals"]["percent_covered"])
0032      destination.parent.mkdir(parents=True, exist_ok=True)
0033      destination.write_text(json.dumps(badge_payload(percent), indent=2) + "\n",
      encoding="utf-8")
0034
0035
0036  if __name__ == "__main__":
0037      main()
```

scripts/build_demo_bundle.py

23 source lines.

```
0001  """Bundle only MySQL-free, pandas-free production Python modules for GitHub Pages."""
0002
0003  from pathlib import Path
0004  from zipfile import ZIP_DEFLATED, ZipFile
0005
0006  ROOT = Path(__file__).resolve().parents[1]
0007  SRC = ROOT / "src" / "etlguard"
0008  DEST = ROOT / "demo" / "etlguard_core.zip"
0009  FILES = [
0010      "__init__.py", "config.py", "cleaning.py", "validation.py", "checks.py",
0011      "rejects.py", "core.py", "browser_demo.py",
0012      "sources/__init__.py", "sources/base.py", "sources/api.py", "sources/csv.py",
0013      "sources/excel.py", "examples/__init__.py", "examples/models.py",
0014  ]
0015  # sources/__init__.py is not import-safe in the browser because it imports httpx/pandas;
0016  # explicitly provide the lightweight package init instead of its desktop variant.
0017  with ZipFile(DEST, "w", compression=ZIP_DEFLATED) as archive:
0018      for name in FILES:
0019          if name == "sources/__init__.py":
0020              archive.writestr("etlguard/sources/__init__.py", '"""Shared browser-safe
      lineage package."""\n')
0021          else:
0022              archive.write(SRC / name, f"etlguard/{name}")
0023  print(f"Created {DEST} ({DEST.stat().st_size:,} bytes)")
```

scripts/ci_shadow_smoke.py

151 source lines.

```
0001  #!/usr/bin/env python3
0002  """CI-only MySQL 8 shadow smoke: separate disposable database and stable retry.
0003
0004  Requires an explicitly acknowledged LOCAL disposable MySQL container with the
0005  root password used only for initial test-schema creation. NOT a production tool.
0006  """
0007  from __future__ import annotations
0008
0009  import argparse
0010  from datetime import datetime, timezone
0011  import json
0012  import os
0013  from pathlib import Path
0014  import sys
0015  import traceback
0016  from tempfile import TemporaryDirectory
0017
0018  from prepare_shadow_pilot import plan
0019  from run_shadow_pilot import run_pilot
0020  from verify_shadow_reconciliation import reconcile_shadow
0021  from verify_mysql8_staging import is_mysql8
0022
0023  SHADOW_DB = "etlguard_shadow_ci"
0024  SHADOW_PASSWORD = "etlguard_shadow_local_only"
0025
0026
0027  def require_disposable_ci(root_url: str, consent: str):
0028      from sqlalchemy.engine import make_url
```

A B C D E F G H

```
0029
0030      if consent != "DISPOSABLE_MYSQL_ONLY":
0031          raise ValueError("Explicit disposable CI acknowledgment required")
0032      if not root_url:
0033          raise ValueError("ETLGUARD_CI_MYSQL_ROOT_URL is required")
0034      url = make_url(root_url)
0035      if (url.drivername != "mysql+pymysql" or url.username != "root" or
0036              url.host not in {"127.0.0.1", "localhost"} or url.database != "etlguard_ci"):
0037          raise ValueError("CI bootstrap restricted to local root@etlguard_ci disposable
      database")
0038      return url
0039
0040
0041  def bootstrap(url):
0042      from sqlalchemy import create_engine, text
0043
0044      engine = create_engine(url, pool_pre_ping=True, connect_args={"connect_timeout": 10})
0045      try:
0046          with engine.connect() as conn:
0047              version = conn.execute(text("SELECT VERSION()")).scalar_one()
0048              existing = conn.execute(text(
0049                  "SELECT SCHEMA_NAME FROM information_schema.SCHEMATA "
0050                  "WHERE SCHEMA_NAME = :db"
0051              ), {"db": SHADOW_DB}).first()
0052              if not is_mysql8(version) or existing:
0053                  raise ValueError("CI bootstrap requires MySQL 8 and an UNUSED shadow
      database")
0054              conn.commit()
0055              conn.execute(text(
0056                  "CREATE DATABASE `etlguard_shadow_ci` "
0057                  "CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci"
0058              ))
0059              conn.commit()
0060              # Password is a fixed *public* disposable-CI-only fixture, not external
      input.
0061              conn.execute(text(
0062                  "CREATE USER 'etlguard_shadow'@'%' "
0063                  "IDENTIFIED BY 'etlguard_shadow_local_only'"
0064              ))
0065              conn.commit()
0066              conn.execute(text(
0067                  "GRANT ALL PRIVILEGES ON `etlguard_shadow_ci`.* TO 'etlguard_shadow'@'%'"
0068              ))
0069              conn.commit()
0070          return str(version)
0071      finally:
0072          engine.dispose()
0073
0074
0075  def smoke(project: Path, root_url: str, consent: str, staging_report: Path) -> dict:
0076      from assess_promotion import check_staging
0077
0078      report = json.loads(staging_report.read_text(encoding="utf-8"))
0079      if not isinstance(report, dict) or check_staging(report):
0080          raise ValueError("CI shadow smoke requires a passing MySQL 8 staging report")
0081      url = require_disposable_ci(root_url, consent)
0082      version = bootstrap(url)
0083      shadow_url = url.set(username="etlguard_shadow", password=SHADOW_PASSWORD,
0084                           database=SHADOW_DB)
0085      os.environ["ETLGUARD_SHADOW_MYSQL_URL"] =
      shadow_url.render_as_string(hide_password=False)
0086      with TemporaryDirectory() as temporary:
0087          work = Path(temporary)
0088          plan(project / "src/etlguard/examples/pipelines/customers_clean.yaml", work /
      "shadow", None, False)
0089          manifest = work / "shadow/shadow-manifest.json"
0090          first = run_pilot(manifest, execute=True, confirmed_schema=SHADOW_DB,
0091                            staging_report=staging_report)
0092          first_reconciliation = reconcile_shadow(
0093              manifest, staging_report, SHADOW_DB,
0094              shadow_url.render_as_string(hide_password=False),
0095          )
0096          retry = run_pilot(manifest, execute=True, confirmed_schema=SHADOW_DB,
0097                            staging_report=staging_report)
0098          retry_reconciliation = reconcile_shadow(
0099              manifest, staging_report, SHADOW_DB,
0100              shadow_url.render_as_string(hide_password=False),
0101          )
0102          if (first_reconciliation["status"] != "passed" or
0103                  retry_reconciliation["status"] != "passed" or
0104                  first_reconciliation["target_physical_sha256"] !=
0105                  retry_reconciliation["target_physical_sha256"]):
0106              raise AssertionError("Independent row-level shadow reconciliation or retry
      fingerprint failed")
0107          if (first["status"] != "shadow_committed_not_production" or
```

```
                first["replayed"] or retry["replayed"] is not True or
                first["rows_loaded"] < 1 or first["rows_loaded"] != retry["rows_loaded"]
or
                first["request_hash"] != retry["request_hash"]):
            raise AssertionError("Shadow retry did not prove stable idempotence")
        return {"status": "passed_disposable_shadow_only", "timestamp_utc":
datetime.now(timezone.utc).isoformat(),
                "mysql_version": version, "shadow_schema": SHADOW_DB,
                "batch_id": first["batch_id"], "rows_loaded": first["rows_loaded"],
                "identical_retry_replayed": retry["replayed"],
                "source_and_target_rows_equal": True,
                "audit_and_ledger_reconciled": True,
                "target_unchanged_after_retry": True,
                "target_physical_sha256": retry_reconciliation["target_physical_sha256"],
                "reconciliation_before_retry": first_reconciliation,
                "reconciliation_after_retry": retry_reconciliation,
                "production_approved": False}


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--project", type=Path,
default=Path(__file__).resolve().parent.parent)
    parser.add_argument("--report", type=Path, default=Path("shadow-smoke-report.json"))
    parser.add_argument("--staging-report", type=Path, required=True)
    args = parser.parse_args()
    try:
        result = smoke(args.project.resolve(),
os.environ.get("ETLGUARD_CI_MYSQL_ROOT_URL", ""),
                       os.environ.get("ETLGUARD_CI_ALLOW_BOOTSTRAP", ""),
                       args.staging_report.resolve())
        code = 0
    except Exception as exc:
        result = {"status": "blocked", "error_type": type(exc).__name__,
                  "production_approved": False}
        print(f"CI shadow smoke blocked: {type(exc).__name__}", file=sys.stderr)
        for frame in traceback.extract_tb(exc.__traceback__):
            print(f"  {Path(frame.filename).name}:{frame.lineno} in {frame.name}",
file=sys.stderr)
        code = 2
    out = args.report.resolve()
    out.parent.mkdir(parents=True, exist_ok=True)
    out.write_text(json.dumps(result, indent=2, sort_keys=True) + "\n", encoding="utf-8")
    print(f"CI smoke status: {result['status']}; report: {out}")
    return code


if __name__ == "__main__":
    raise SystemExit(main())
```

scripts/prepare_shadow_pilot.py

176 source lines.

```
#!/usr/bin/env python3
"""Prepare and verify a shadow-only pilot manifest. Never connects to a database.

Intentionally does not execute ``etlguard run``. Operators must verify the
isolated database and table before allowing any write, using least privilege.
"""
from __future__ import annotations

import argparse
from etlguard import __version__

from datetime import datetime, timezone
from hashlib import sha256
import json
from pathlib import Path
import shlex
import sys
from uuid import UUID, uuid4
from typing import Any

import yaml

from etlguard.config import IDENTIFIER, load_config

SHADOW_ENV = "ETLGUARD_SHADOW_MYSQL_URL"


```

```
0028  def sha256_file(path: Path) -> str:
0029      digest = sha256()
0030      with path.open("rb") as fh:
0031          for block in iter(lambda: fh.read(1024 * 1024), b""):
0032              digest.update(block)
0033      return digest.hexdigest()
0034
0035
0036  def shadow_table_name(original: str, specified: str | None = None) -> str:
0037      name = specified or ("pilot_" + original[:42])
0038      if not IDENTIFIER.fullmatch(name) or len(name) > 48 or not name.startswith("pilot_"):
0039          raise ValueError("Shadow target must be a SQL identifier <=48 characters
      beginning pilot_")
0040      if name == original:
0041          raise ValueError("Shadow target must differ from the source configuration
      target")
0042      if name.startswith("pilot_stg_"):
0043          raise ValueError("Do not use staging-table-like names for shadow target")
0044      return name
0045
0046
0047  def plan(config: Path, outdir: Path, requested_table: str | None, allow_shadow_replace:
      bool) -> dict[str, Any]:
0048      settings, base_dir = load_config(config)
0049      if settings.target is None:
0050          raise ValueError("A target is required to plan a database shadow pilot")
0051      if settings.source.type not in {"csv", "excel"}:
0052          raise ValueError("Shadow pilot requires an immutable local CSV/Excel snapshot,
      not an API feed")
0053      source_path = (base_dir / (settings.source.path or "")).resolve()
0054      if not source_path.is_file():
0055          raise ValueError(f"Source snapshot does not exist: {source_path}")
0056      if source_path.stat().st_size == 0:
0057          raise ValueError("Source snapshot is empty; review it instead of planning a
      database load")
0058      if settings.target.mode == "truncate_insert" and not allow_shadow_replace:
0059          raise ValueError("Snapshot replacement requires explicit --allow-shadow-replace,
      on an ISOLATED database")
0060      table_name = shadow_table_name(settings.target.table, requested_table)
0061      outdir = outdir.resolve()
0062      pilot_yaml = outdir / "shadow-pipeline.yaml"
0063      manifest_path = outdir / "shadow-manifest.json"
0064      if pilot_yaml.exists() or manifest_path.exists():
0065          raise FileExistsError("Pilot already prepared: refusing to overwrite its stable
      batch ID or snapshot metadata")
0066      snapshot_hash = sha256_file(source_path)
0067      config_hash = sha256_file(config.resolve())
0068      pilot = settings.model_dump(mode="json")
0069      pilot["pipeline"] = (settings.pipeline[:115] + "_shadow")
0070      pilot["source"]["path"] = str(source_path)
0071      pilot["target"]["table"] = table_name
0072      pilot["database_url_env"] = SHADOW_ENV
0073      pilot["output"]["directory"] = str(outdir / "reports")
0074      # Never carry dangerous live replacement overrides into a new pilot.
0075      pilot["policy"]["allow_partial_replace"] = False
0076      pilot["policy"]["allow_empty_replace"] = False
0077      pilot["target"]["min_replace_ratio"] = pilot["target"].get("min_replace_ratio") or (
0078          0.9 if pilot["target"]["mode"] == "truncate_insert" else None
0079      )
0080      outdir.mkdir(parents=True, exist_ok=True)
0081      pilot_yaml.write_text(yaml.safe_dump(pilot, sort_keys=False), encoding="utf-8")
0082      batch_id = str(uuid4())
0083      manifest: dict[str, Any] = {
0084          "format_version": 1,
0085          "toolkit_version": __version__,
0086          "created_at_utc": datetime.now(timezone.utc).isoformat(),
0087          "status": "prepared_not_executed",
0088          "batch_id": batch_id,
0089          "source_path": str(source_path),
0090          "source_bytes": source_path.stat().st_size,
0091          "source_sha256": snapshot_hash,
0092          "original_yaml": str(config.resolve()),
0093          "original_yaml_sha256": config_hash,
0094          "shadow_yaml": str(pilot_yaml),
0095          "shadow_yaml_sha256": sha256_file(pilot_yaml),
0096          "original_target": settings.target.table,
0097          "shadow_target": table_name,
0098          "target_mode": settings.target.mode,
0099          "database_url_env": SHADOW_ENV,
0100          "warning": "No DB connection was made. Set a dedicated SHADOW MySQL URL; never a
      live-target URL.",
0101          "commands": {
0102              "verify_snapshot": f"python scripts/prepare_shadow_pilot.py verify --manifest
       {shlex.quote(str(manifest_path))}",
0103              "validate_only": f"etlguard validate {shlex.quote(str(pilot_yaml))}",
```

```
            "shadow_load_after_approval": (
                f"python scripts/run_shadow_pilot.py --manifest {shlex.quote(str(manifest_path))} "
                f"--execute --confirm-shadow-schema etlguard_shadow_ci "
                f"--staging-report staging-gate-report.json"
            ),
            "check_commit": f"etlguard batch-status {batch_id} --database-url-env {SHADOW_ENV}",
        },
    }
    # Restrict reading of pipeline locations and operational metadata where possible.
    fd = manifest_path.open("x", encoding="utf-8")
    try:
        json.dump(manifest, fd, indent=2, sort_keys=True)
        fd.write("\n")
    finally:
        fd.close()
    manifest_path.chmod(0o600)
    return manifest


def verify(manifest_file: Path) -> dict[str, Any]:
    manifest = json.loads(manifest_file.read_text(encoding="utf-8"))
    required = {"source_path", "source_sha256", "source_bytes", "original_yaml", "original_yaml_sha256",
                "shadow_yaml", "shadow_yaml_sha256", "batch_id", "shadow_target", "database_url_env"}
    if not required.issubset(manifest) or manifest.get("format_version") != 1:
        raise ValueError("Invalid/incomplete shadow pilot manifest")
    if str(UUID(manifest["batch_id"])) != manifest["batch_id"]:
        raise ValueError("Batch ID is not a canonical UUID")
    if manifest["database_url_env"] != SHADOW_ENV:
        raise ValueError("Shadow database environment variable was altered")
    shadow_table_name(manifest["original_target"], manifest["shadow_target"])
    checked = []
    for label, pathname, recorded in [
        ("source", "source_path", "source_sha256"),
        ("original_yaml", "original_yaml", "original_yaml_sha256"),
        ("shadow_yaml", "shadow_yaml", "shadow_yaml_sha256"),
    ]:
        path = Path(manifest[pathname])
        if not path.is_file() or sha256_file(path) != manifest[recorded]:
            raise ValueError(f"{label} changed or disappeared; do NOT reuse this batch ID")
        checked.append(label)
    if Path(manifest["source_path"]).stat().st_size != manifest["source_bytes"]:
        raise ValueError("Source length changed; do NOT reuse this batch ID")
    settings, _ = load_config(manifest["shadow_yaml"])
    if (settings.target is None or settings.target.table != manifest["shadow_target"] or
            settings.database_url_env != SHADOW_ENV):
        raise ValueError("Shadow configuration does not match the manifest")
    return {"status": "snapshot_verified_not_executed", "checks": checked,
            "batch_id": manifest["batch_id"], "shadow_target": manifest["shadow_target"]}


def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    commands = parser.add_subparsers(dest="command", required=True)
    p = commands.add_parser("plan", help="Create a non-overwriting shadow configuration and stable manifest")
    p.add_argument("--config", required=True, type=Path)
    p.add_argument("--output", required=True, type=Path)
    p.add_argument("--shadow-table")
    p.add_argument("--allow-shadow-replace", action="store_true")
    v = commands.add_parser("verify", help="Confirm source/config checksums before any manual execution")
    v.add_argument("--manifest", required=True, type=Path)
    args = parser.parse_args()
    try:
        result = (plan(args.config, args.output, args.shadow_table, args.allow_shadow_replace)
                  if args.command == "plan" else verify(args.manifest))
        print(json.dumps(result, indent=2))
        return 0
    except (OSError, ValueError, TypeError, KeyError, yaml.YAMLError) as exc:
        print(f"BLOCKED: {type(exc).__name__}: {exc}", file=sys.stderr)
        return 2


if __name__ == "__main__":
    raise SystemExit(main())
```

scripts/run_shadow_pilot.py

176 source lines.

```
#!/usr/bin/env python3
"""Fail-closed shadow pilot launcher using a pinned manifest and disposable schema.

``validate`` is database-free. ``execute`` is only permitted for a locally
reachable, isolated MySQL 8 shadow schema with matching least-privilege grants.
Never use real customer/application/production schemas or credentials.
"""
from __future__ import annotations

import argparse
import json
import os
from pathlib import Path
import re
import sys
from typing import Any

from prepare_shadow_pilot import SHADOW_ENV, verify
from verify_mysql8_staging import grants_are_disposable_schema_only, is_mysql8, strict_sql_mode_is_enabled

SHADOW_SCHEMA = re.compile(r"^etlguard_shadow_(?:ci|test|pilot)(?:_[a-z0-9]+)*$")


def check_shadow_destination(text: str, confirmed: str) -> Any:
    from sqlalchemy.engine import make_url

    if not text:
        raise ValueError(f"Set {SHADOW_ENV}; it must point to a DISPOSABLE shadow database")
    url = make_url(text)
    if url.drivername != "mysql+pymysql":
        raise ValueError("Shadow database URL must use mysql+pymysql")
    if not url.host or url.host not in {"127.0.0.1", "localhost", "::1"}:
        raise ValueError("Only localhost/SSH-tunneled shadow MySQL is permitted")
    if not url.database or not SHADOW_SCHEMA.fullmatch(url.database):
        raise ValueError("Shadow schema must be etlguard_shadow_ci/test/pilot (optional suffix)")
    if confirmed != url.database:
        raise ValueError("--confirm-shadow-schema must exactly match the URL schema")
    if not url.username or url.username.lower() in {"root", "admin", "administrator"}:
        raise ValueError("Use a dedicated non-root MySQL shadow user")
    return url


def preflight_shadow(url: Any, batch_id: str, shadow_table: str) -> dict[str, Any]:
    from sqlalchemy import create_engine, text

    engine = create_engine(url, pool_pre_ping=True, connect_args={"connect_timeout": 10})
    try:
        with engine.connect() as conn:
            row = conn.execute(text(
                "SELECT VERSION() AS v, DATABASE() AS db, @@default_storage_engine AS storage, "
                "@@read_only AS read_only, @@SESSION.sql_mode AS sql_mode"
            )).mappings().one()
            tables = list(conn.execute(text(
                "SELECT table_name AS table_name, engine AS engine FROM information_schema.tables "
                "WHERE table_schema = DATABASE()"
            )).mappings())
            grants = list(conn.execute(text("SHOW GRANTS")).scalars())
            if not is_mysql8(str(row["v"])):
                raise ValueError("A genuine MySQL 8 shadow server is required")
            if row["db"] != url.database or str(row["storage"]).lower() != "innodb" or row["read_only"]:
                raise ValueError("Shadow database/engine/read-write preflight did not pass")
            if not strict_sql_mode_is_enabled(str(row["sql_mode"])):
                raise ValueError("MySQL STRICT SQL mode required for shadow pilot")
            if not grants_are_disposable_schema_only(grants, url.database):
                raise ValueError("Shadow account has broad or inherited grants outside its disposable schema")
            allowed = {"etl_run_audit", "etl_batch_commits", shadow_table, f"stg_{shadow_table}"}
            if any(t["table_name"] not in allowed for t in tables):
                raise ValueError("Shadow schema has unrelated tables; use a fresh isolated schema")
            if any(str(t["engine"] or "").lower() != "innodb" for t in tables):
                raise ValueError("All existing shadow tables must use InnoDB")
```

A
B
C
D
E
F
G
H

```
            names = {str(t["table_name"]) for t in tables}
            committed = None
            if "etl_batch_commits" in names:
                committed = conn.execute(text(
                    "SELECT batch_id, target_table FROM etl_batch_commits"
                )).mappings().all()
                if any(item["batch_id"] != batch_id or item["target_table"] !=
shadow_table
                       for item in committed):
                    raise ValueError("Shadow ledger contains an unrelated batch; use a
fresh isolated schema")
            if shadow_table in names and not committed:
                raise ValueError("Shadow target pre-exists without a matching committed
batch; refuse write")
            if committed and shadow_table not in names:
                raise ValueError("Ledger indicates a committed batch but target table is
absent")
            if committed:
                if "etl_run_audit" not in names:
                    raise ValueError("Committed shadow batch is missing its success audit
 table")
                audit = conn.execute(text(
                    "SELECT status FROM etl_run_audit WHERE batch_id = :batch"
                ), {"batch": batch_id}).scalar_one_or_none()
                if audit != "success":
                    raise ValueError("Committed shadow batch is missing its success
audit")
            if f"stg_{shadow_table}" in names:
                staged = conn.execute(text(
                    f"SELECT COUNT(*) FROM `stg_{shadow_table}`"
                )).scalar_one()
                if staged:
                    raise ValueError("Shadow staging table contains leftover rows;
investigate before retrying")
            return {"server_version": str(row["v"]), "database": str(row["db"]),
                    "target": shadow_table, "replay": bool(committed),
                    "audit_or_ledger_tables": bool({"etl_run_audit", "etl_batch_commits"}
 & names)}
    finally:
        engine.dispose()


def run_pilot(manifest: Path, execute: bool, confirmed_schema: str | None = None,
              staging_report: Path | None = None) -> dict[str, Any]:
    from etlguard.pipeline import Pipeline

    verify(manifest)
    record = json.loads(manifest.read_text(encoding="utf-8"))
    yaml_path = Path(record["shadow_yaml"])
    # Validate before connecting to the DB. Failures mean no target writes.
    dry = Pipeline.from_yaml(yaml_path).run(dry_run=True)
    if not dry.ok:
        raise ValueError(f"Shadow dry validation failed: {dry.failures}")
    if not execute:
        return {"status": "validated_only_no_database_writes", "batch_id":
record["batch_id"],
                "rows_read": dry.rows_read, "rows_valid": dry.rows_valid,
                "shadow_target": record["shadow_target"]}
    if not confirmed_schema:
        raise ValueError("--confirm-shadow-schema required before any write")
    if staging_report is None:
        raise ValueError("A real passing --staging-report is mandatory before a shadow
write")
    from assess_promotion import check_staging
    stage_data = json.loads(staging_report.read_text(encoding="utf-8"))
    if not isinstance(stage_data, dict) or check_staging(stage_data):
        raise ValueError("Staging gate is not fully passed; no shadow database write
permitted")
    url = check_shadow_destination(os.environ.get(SHADOW_ENV, ""), confirmed_schema)
    preflight = preflight_shadow(url, record["batch_id"], record["shadow_target"])
    # Re-check immutable inputs immediately before the one authorized shadow write.
    verify(manifest)
    result = Pipeline.from_yaml(yaml_path).run(batch_id=record["batch_id"])
    if not result.ok or not result.durable_commit:
        raise ValueError(f"Shadow run failed (no success asserted): {result.failures}")
    # Verify canonical source snapshot once more after reading it.
    verify(manifest)
    return {"status": "shadow_committed_not_production", "batch_id": record["batch_id"],
            "replayed": result.replayed, "rows_read": result.rows_read,
            "rows_loaded": result.rows_loaded, "rows_inserted": result.rows_inserted,
            "rows_updated": result.rows_updated, "request_hash": result.request_hash,
            "shadow_preflight": preflight, "summary_json": result.summary_json}


def main() -> int:
```

```
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--manifest", type=Path, required=True)
    choice = parser.add_mutually_exclusive_group(required=True)
    choice.add_argument("--validate", action="store_true", help="Safe local data validation only")
    choice.add_argument("--execute", action="store_true", help="Perform one explicitly authorized SHADOW-only load")
    parser.add_argument("--confirm-shadow-schema", help="Exact disposable schema, required for --execute")
    parser.add_argument("--staging-report", type=Path, help="Passing real MySQL 8 JSON gate; required for --execute")
    parser.add_argument("--report", type=Path)
    args = parser.parse_args()
    report: dict[str, Any]
    try:
        report = run_pilot(args.manifest.resolve(), args.execute, args.confirm_shadow_schema,
                           args.staging_report.resolve() if args.staging_report else None)
        code = 0
    except Exception as exc:
        # Avoid accidentally printing DB exception strings/URLs which can contain secrets.
        report = {"status": "blocked", "error_type": type(exc).__name__,
                  "message": "Shadow pilot blocked; see operator logs and verify the isolated database."}
        print(f"BLOCKED: {report['error_type']}: {report['message']}", file=sys.stderr)
        code = 2
    if args.report:
        destination = args.report.resolve()
        destination.parent.mkdir(parents=True, exist_ok=True)
        destination.write_text(json.dumps(report, indent=2, sort_keys=True) + "\n", encoding="utf-8")
        print(f"Report: {destination}")
    else:
        print(json.dumps(report, indent=2, sort_keys=True))
    return code


if __name__ == "__main__":
    raise SystemExit(main())
```

scripts/serve_demo.py

29 source lines.

```
"""Launch the browser application with Python's standard library."""

import argparse
from functools import partial
from http.server import SimpleHTTPRequestHandler, ThreadingHTTPServer
from pathlib import Path
import webbrowser


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--port", type=int, default=0, help="Port (default: choose an available port)")
    parser.add_argument("--no-browser", action="store_true")
    args = parser.parse_args()
    demo = Path(__file__).resolve().parents[1] / "demo"
    handler = partial(SimpleHTTPRequestHandler, directory=str(demo))
    with ThreadingHTTPServer(("127.0.0.1", args.port), handler) as server:
        url = f"http://127.0.0.1:{server.server_port}/"
        print(f"ETL Guard is running at {url}\nPress Ctrl+C to stop.", flush=True)
        if not args.no_browser:
            webbrowser.open(url)
        try:
            server.serve_forever()
        except KeyboardInterrupt:
            print("\nETL Guard stopped.")


if __name__ == "__main__":
    main()
```

A
B
C
D
E
F
G
H

scripts/test_pilot_release_gate.py

235 source lines.

```
"""Offline regression tests for shadow snapshot and human promotion review gates."""
from __future__ import annotations

from datetime import datetime, timezone
import json
import os
from unittest.mock import patch
from pathlib import Path
from tempfile import TemporaryDirectory
import unittest

import yaml

from assess_promotion import (
    REQUIRED_APPROVERS, REQUIRED_AUTOMATED, REQUIRED_OPERATIONAL,
    assess, check_staging, check_operational,
)
from prepare_shadow_pilot import plan, verify, shadow_table_name
from run_shadow_pilot import check_shadow_destination, run_pilot
from ci_shadow_smoke import require_disposable_ci
from verify_mysql8_staging import grants_are_disposable_schema_only,
strict_sql_mode_is_enabled


class PilotGateTests(unittest.TestCase):
    def setUp(self):
        self.tmp = TemporaryDirectory()
        self.addCleanup(self.tmp.cleanup)
        self.root = Path(self.tmp.name)
        self.csv = self.root / "feed.csv"

    self.csv.write_text("customer_id,full_name,email,signup_date,status,credit_limit\n"
                        "C001,Alice
    Person,alice@example.com,2026-01-01,active,100\n",
                        encoding="utf-8")
        self.yaml = self.root / "pipeline.yaml"
        self.original = {
            "pipeline": "customers_daily", "source": {"type": "csv", "path": "feed.csv"},
            "model": "etlguard.examples.models:Customer",
            "target": {"table": "customers", "mode": "upsert", "key": ["customer_id"]},
            "output": {"directory": "reports"},
        }
        self.write_config()
        self.out = self.root / "pilot"

    def write_config(self):
        self.yaml.write_text(yaml.safe_dump(self.original), encoding="utf-8")

    def staging_ok(self):
        return {
            "toolkit_version": "0.1.3", "status": "passed", "mysql8_server_verified":
    True,
            "mysql8_integration_executed": True,
            "timestamp_utc": datetime.now(timezone.utc).isoformat(),
            "preflight": {"server_version": "8.4.7", "default_storage_engine": "InnoDB",
                          "unexpected_tables": [], "read_only": False,
                          "strict_sql_mode": True, "schema_only_privileges": True,
                          "non_innodb_tables": []},
            "checks": [{"name": key, "status": "passed", "skipped": 0, "exit_code": 0}
                       for key in REQUIRED_AUTOMATED],
        }

    def test_shadow_plan_never_executes_and_verifies_hashes(self):
        manifest = plan(self.yaml, self.out, None, False)
        self.assertEqual(manifest["status"], "prepared_not_executed")
        self.assertEqual(manifest["database_url_env"], "ETLGUARD_SHADOW_MYSQL_URL")
        self.assertEqual(manifest["shadow_target"], "pilot_customers")
        self.assertEqual(verify(self.out / "shadow-manifest.json")["status"],
                         "snapshot_verified_not_executed")
        settings = yaml.safe_load((self.out / "shadow-pipeline.yaml").read_text())
        self.assertEqual(settings["target"]["table"], "pilot_customers")
        self.assertNotEqual(settings["source"]["path"], "feed.csv")

    def test_does_not_overwrite_persisted_batch_id(self):
        plan(self.yaml, self.out, None, False)
        with self.assertRaises(FileExistsError):
            plan(self.yaml, self.out, None, False)

    def test_changed_source_blocks_same_batch_id(self):
        plan(self.yaml, self.out, None, False)
```

```
0077          self.csv.write_text(self.csv.read_text() + "C002,Bob\n")
0078          with self.assertRaisesRegex(ValueError, "changed or disappeared"):
0079              verify(self.out / "shadow-manifest.json")
0080
0081      def test_changed_yaml_blocks_same_batch_id(self):
0082          plan(self.yaml, self.out, None, False)
0083          self.original["pipeline"] = "changed"
0084          self.write_config()
0085          with self.assertRaisesRegex(ValueError, "changed or disappeared"):
0086              verify(self.out / "shadow-manifest.json")
0087
0088      def test_changed_shadow_yaml_blocks_same_batch_id(self):
0089          plan(self.yaml, self.out, None, False)
0090          with (self.out / "shadow-pipeline.yaml").open("a") as f:
0091              f.write("\n# changed\n")
0092          with self.assertRaisesRegex(ValueError, "changed or disappeared"):
0093              verify(self.out / "shadow-manifest.json")
0094
0095      def test_blocks_api_without_immutable_snapshot(self):
0096          self.original["source"] = {"type": "api", "url": "https://example.com/api"}
0097          self.write_config()
0098          with self.assertRaisesRegex(ValueError, "immutable local"):
0099              plan(self.yaml, self.out, None, False)
0100
0101      def test_replace_requires_explicit_shadow_opt_in(self):
0102          self.original["target"] = {"table": "customers", "mode": "truncate_insert"}
0103          self.original["policy"] = {"allow_empty_replace": True, "allow_partial_replace":
    True}
0104          self.write_config()
0105          with self.assertRaisesRegex(ValueError, "requires explicit"):
0106              plan(self.yaml, self.out, None, False)
0107          plan(self.yaml, self.out, None, True)
0108          configured = yaml.safe_load((self.out / "shadow-pipeline.yaml").read_text())
0109          self.assertFalse(configured["policy"]["allow_empty_replace"])
0110          self.assertFalse(configured["policy"]["allow_partial_replace"])
0111          self.assertEqual(configured["target"]["min_replace_ratio"], 0.9)
0112
0113      def test_shadow_table_identifier_guard(self):
0114          for name in ["customers", "orders", "prod_payments", "pilot_a;DROP TABLE",
    "pilot_stg_rows"]:
0115              with self.assertRaises(ValueError, msg=name):
0116                  shadow_table_name("customers", name)
0117          self.assertEqual(shadow_table_name("customers"), "pilot_customers")
0118
0119      def test_failed_or_skipped_stage_stays_blocked(self):
0120          good = self.staging_ok()
0121          self.assertEqual(check_staging(good), [])
0122          for key in ["mysql8_server_verified", "mysql8_integration_executed"]:
0123              report = self.staging_ok()
0124              report[key] = False
0125              self.assertTrue(check_staging(report))
0126          report = self.staging_ok()
0127          report["checks"][0]["skipped"] = 1
0128          self.assertTrue(check_staging(report))
0129          report = self.staging_ok()
0130          report["preflight"]["server_version"] = "10.11.4-MariaDB"
0131          self.assertTrue(check_staging(report))
0132
0133      def test_missing_operational_evidence_is_blocked(self):
0134          template = {"toolkit_version": "0.1.3", "operational": {}, "approvals": {}}
0135          reasons, hashes = check_operational(template, self.root / "evidence.json")
0136          self.assertEqual(len(reasons), len(REQUIRED_OPERATIONAL) +
    len(REQUIRED_APPROVERS))
0137          self.assertEqual(hashes, {})
0138
0139      def test_forged_evidence_file_paths_outside_folder_are_refused(self):
0140          outside = self.root.parent / "outside.log"
0141          outside.write_text("fake", encoding="utf-8")
0142          self.addCleanup(outside.unlink)
0143          evidence = {"toolkit_version": "0.1.3", "operational": {"immutable_source": {
0144              "verified": True, "evidence_file": "../outside.log"}}, "approvals": {}}
0145          reasons, _ = check_operational(evidence, self.root / "evidence.json")
0146          self.assertTrue(any("outside directory" in reason for reason in reasons))
0147
0148      def test_complete_papers_only_allow_review_not_production(self):
0149          staging = self.root / "stage.json"
0150          staging.write_text(json.dumps(self.staging_ok()))
0151          artifact = self.root / "proof.log"
0152          artifact.write_text("dummy test fixture; independent verification still
    required")
0153          data = {
0154              "toolkit_version": "0.1.3",
0155              "operational": {name: {"verified": True, "evidence_file": "proof.log"}
0156                              for name in REQUIRED_OPERATIONAL},
0157              "approvals": {name: {"verified": True, "evidence_file": "proof.log"}
```

A
B
C
D
E
F
G
H

```
                           for name in REQUIRED_APPROVERS},
        }
        evidence = self.root / "evidence.json"
        evidence.write_text(json.dumps(data))
        result = assess(staging, evidence)
        self.assertEqual(result["decision"], "review_ready_not_authorized")
        self.assertFalse(result["production_approved"])
        self.assertFalse(result["database_writes_executed"])

    def test_real_local_gate_blocked(self):
        self.assertTrue(check_staging({"status": "blocked", "preflight": None}))

    def test_grants_refuse_global_or_role_privileges(self):
        local = "GRANT ALL PRIVILEGES ON `etlguard_ci`.* TO `etlguard_ci`@`%`"
        basic = "GRANT USAGE ON *.* TO `etlguard_ci`@`%`"
        broad = "GRANT SELECT ON *.* TO `etlguard_ci`@`%`"
        role = "GRANT `reporting_role`@`%` TO `etlguard_ci`@`%`"
        self.assertTrue(grants_are_disposable_schema_only([basic, local], "etlguard_ci"))
        escaped = local.replace("etlguard_ci", r"etlguard\_ci")
        self.assertTrue(grants_are_disposable_schema_only([basic, escaped],
"etlguard_ci"))
        self.assertFalse(grants_are_disposable_schema_only([basic, escaped],
"etlguard_other"))
        self.assertFalse(grants_are_disposable_schema_only([basic, broad, local],
"etlguard_ci"))
        self.assertFalse(grants_are_disposable_schema_only([local, role], "etlguard_ci"))

    def test_shadow_run_validate_never_contacts_database(self):
        plan(self.yaml, self.out, None, False)
        record = run_pilot(self.out / "shadow-manifest.json", False)
        self.assertEqual(record["status"], "validated_only_no_database_writes")
        self.assertEqual(record["rows_valid"], 1)

    def test_shadow_execute_refuses_missing_database_url(self):
        plan(self.yaml, self.out, None, False)
        staging = self.root / "staging.json"
        staging.write_text(json.dumps(self.staging_ok()), encoding="utf-8")
        with patch.dict(os.environ, {"ETLGUARD_SHADOW_MYSQL_URL": ""}):
            with self.assertRaisesRegex(ValueError, "DISPOSABLE shadow"):
                run_pilot(self.out / "shadow-manifest.json", True, "etlguard_shadow_ci",
                          staging)

    def test_shadow_execute_refuses_failed_staging_gate(self):
        plan(self.yaml, self.out, None, False)
        staging = self.root / "staging.json"
        staging.write_text(json.dumps({"status": "blocked", "toolkit_version": "0.1.3"}),
                           encoding="utf-8")
        with self.assertRaisesRegex(ValueError, "Staging gate is not fully passed"):
            run_pilot(self.out / "shadow-manifest.json", True, "etlguard_shadow_ci",
                      staging)

    def test_shadow_destination_refuses_live_or_root_url(self):
        okay = "mysql+pymysql://shadow:shadow_pw@127.0.0.1:3308/etlguard_shadow_ci"
        self.assertEqual(check_shadow_destination(okay, "etlguard_shadow_ci").database,
                         "etlguard_shadow_ci")
        for url in ["mysql+pymysql://shadow:pw@localhost/prod",
                    "mysql+pymysql://root:pw@localhost/etlguard_shadow_ci",
                    "mysql+pymysql://shadow:pw@prod-db/etlguard_shadow_ci"]:
            with self.assertRaises(ValueError, msg=url):
                check_shadow_destination(url, "etlguard_shadow_ci")

    def test_ci_bootstrap_requires_consent_and_local_root(self):
        okay = "mysql+pymysql://root:ci_only@127.0.0.1:3306/etlguard_ci"
        self.assertEqual(require_disposable_ci(okay, "DISPOSABLE_MYSQL_ONLY").database,
                         "etlguard_ci")
        for url, consent in [(okay, ""),
                             ("mysql+pymysql://root:pw@production.example/etlguard_ci",
                              "DISPOSABLE_MYSQL_ONLY"),
                             ("mysql+pymysql://root:pw@localhost/production",
                              "DISPOSABLE_MYSQL_ONLY")]:
            with self.assertRaises(ValueError):
                require_disposable_ci(url, consent)

    def test_sql_mode_requires_strict(self):

self.assertTrue(strict_sql_mode_is_enabled("ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES"))
        self.assertTrue(strict_sql_mode_is_enabled("STRICT_ALL_TABLES"))
        self.assertFalse(strict_sql_mode_is_enabled("NO_ENGINE_SUBSTITUTION"))


if __name__ == "__main__":
    unittest.main()
```

A
B
C
D
E
F
G
H

scripts/test_shadow_reconciliation.py

211 source lines.

```
"""Offline tests for independent pilot reconciliation; SQLite is NOT MySQL certification."""
from __future__ import annotations

from datetime import date, datetime, timezone
from decimal import Decimal
import json
from pathlib import Path
import sys
from tempfile import TemporaryDirectory
import unittest
from uuid import uuid4
from unittest.mock import patch

import yaml
from sqlalchemy import MetaData, Table, create_engine, select

sys.path.insert(0, str(Path(__file__).resolve().parent))
from prepare_shadow_pilot import plan, verify
from verify_shadow_reconciliation import (
    canonical, compare_shadow, reconcile_shadow, row_fingerprint,
)
from etlguard.config import load_config
from etlguard.core import ValidationEngine
from etlguard.loaders.mysql import MysqlLoader
from etlguard.pipeline import Pipeline, resolve_model


class ReconciliationTests(unittest.TestCase):
    def setUp(self):
        self.temp = TemporaryDirectory()
        self.addCleanup(self.temp.cleanup)
        self.root = Path(self.temp.name)

    def setup_loaded(self, mode="append"):
        source = self.root / "source.csv"
        source.write_text(
            "customer_id,full_name,email,signup_date,status,credit_limit\n"
            "C001,Alice Doe,alice@example.com,2026-01-01,active,100.00\n"
            "C002,Bob Doe,bob@example.com,2026-01-02,active,200.00\n",
            encoding="utf-8",
        )
        config = self.root / "config.yaml"
        config.write_text(yaml.safe_dump({
            "pipeline": "integration_qa",
            "source": {"type": "csv", "path": "source.csv"},
            "model": "etlguard.examples.models:Customer",
            "target": {"table": "customers", "mode": mode, "key": ["customer_id"] if mode
 == "scd2" else []},
        }), encoding="utf-8")
        out = self.root / "pilot"
        manifest = plan(config, out, None, False)
        verify(out / "shadow-manifest.json")
        cfg, base = load_config(out / "shadow-pipeline.yaml")
        model = resolve_model(cfg.model)
        pipe = Pipeline(cfg, base)
        dry = pipe.run(dry_run=True, batch_id=manifest["batch_id"])
        self.assertTrue(dry.ok)
        checks = cfg.checks.model_copy(deep=True)
        if cfg.target.mode in {"scd2", "upsert"} and not checks.unique:
            checks.unique = cfg.target.key[:]
        core = ValidationEngine(model, cfg.cleaning, checks, base).run(
            pipe._source().read(manifest["batch_id"])
        )
        engine = create_engine(f"sqlite:///{self.root / 'database.sqlite'}")
        self.addCleanup(engine.dispose)
        loader = MysqlLoader(engine=engine)
        loader.load(core.valid, manifest["batch_id"], model, cfg.target,
                    request_hash=dry.request_hash, audit_result=dry)
        return engine, cfg, model, manifest, core, dry, loader

    def compare(self, state):
        engine, cfg, model, manifest, core, dry, _ = state
        return compare_shadow(engine, cfg, model, manifest["batch_id"], core.valid, dry)

    def test_clean_first_load_matches_full_rows_and_audits(self):
        result = self.compare(self.setup_loaded())
        self.assertEqual(result["status"], "passed", result)
        self.assertEqual(result["source_rows_valid"], 2)
        self.assertEqual(result["target_rows_observed"], 2)
```

A B C D E F G H

SECTION MAP

```
0079          self.assertEqual(result["missing_source_rows"], 0)
0080          self.assertFalse(result["database_writes_executed_by_verifier"])
0081          self.assertNotIn("Alice Doe", json.dumps(result))
0082
0083      def test_stable_retry_preserves_physical_row_fingerprint(self):
0084          state = self.setup_loaded()
0085          before = self.compare(state)
0086          engine, cfg, model, manifest, core, dry, loader = state
0087          retry = loader.load(core.valid, manifest["batch_id"], model, cfg.target,
0088                              request_hash=dry.request_hash, audit_result=dry)
0089          self.assertTrue(retry.replayed)
0090          after = self.compare(state)
0091          self.assertEqual(before["target_physical_sha256"],
      after["target_physical_sha256"])
0092          self.assertEqual(after["status"], "passed")
0093
0094      def test_same_count_but_corrupted_customer_is_detected(self):
0095          state = self.setup_loaded()
0096          engine, cfg, *_ = state
0097          target = Table(cfg.target.table, MetaData(), autoload_with=engine)
0098          with engine.begin() as conn:
0099              conn.execute(target.update().where(target.c.customer_id == "C001").values(
0100                  full_name="Altered Customer"
0101              ))
0102          result = self.compare(state)
0103          self.assertEqual(result["status"], "failed")
0104          self.assertIn("source_vs_persisted_row_values_mismatch",
      result["mismatch_types"])
0105          self.assertEqual(result["target_rows_observed"], 2)
0106          self.assertNotIn("Altered Customer", json.dumps(result))
0107
0108      def test_commit_hash_change_is_detected(self):
0109          state = self.setup_loaded()
0110          engine = state[0]
0111          commits = Table("etl_batch_commits", MetaData(), autoload_with=engine)
0112          with engine.begin() as conn:
0113              conn.execute(commits.update().values(request_hash="0" * 64))
0114          result = self.compare(state)
0115          self.assertIn("batch_commit_ledger_mismatch", result["mismatch_types"])
0116
0117      def test_audit_count_change_is_detected(self):
0118          state = self.setup_loaded()
0119          engine = state[0]
0120          audit = Table("etl_run_audit", MetaData(), autoload_with=engine)
0121          with engine.begin() as conn:
0122              conn.execute(audit.update().values(rows_valid=400))
0123          result = self.compare(state)
0124          self.assertIn("audit_counts_or_status_mismatch", result["mismatch_types"])
0125
0126      def test_staging_leftover_is_detected(self):
0127          state = self.setup_loaded()
0128          engine, cfg, *_ = state
0129          stage = Table(f"stg_{cfg.target.table}", MetaData(), autoload_with=engine)
0130          with engine.begin() as conn:
0131              conn.execute(stage.insert().values(
0132                  __etl_id=str(uuid4()), __etl_batch_id=str(uuid4()),
0133                  __etl_source_row_number=42, __etl_ingested_at=datetime.now(timezone.utc),
0134                  customer_id="C003", full_name="Injected Stage", email="a@example.com",
0135                  signup_date=date(2026, 1, 3), status="active", credit_limit=100,
0136              ))
0137          result = self.compare(state)
0138          self.assertIn("staging_rows_remain", result["mismatch_types"])
0139
0140      def test_scd2_first_load_has_one_current_per_key(self):
0141          state = self.setup_loaded("scd2")
0142          self.assertEqual(self.compare(state)["status"], "passed")
0143          engine, cfg, *_ = state
0144          target = Table(cfg.target.table, MetaData(), autoload_with=engine)
0145          with engine.begin() as conn:
0146              conn.execute(target.insert().values(
0147                  __etl_id=str(uuid4()), customer_id="C001", full_name="Earlier",
0148                  email="alice@example.com", signup_date=date(2026, 1, 1),
0149                  status="active", credit_limit=Decimal("100.00"),
0150                  effective_from=datetime(2020, 1, 1),
0151                  effective_to=datetime(2021, 1, 1), is_current=False,
0152              ))
0153          result = self.compare(state)
0154          self.assertIn("unexpected_scd2_history_during_first_batch",
      result["mismatch_types"])
0155
0156      def test_ledger_anomaly_and_extra_rows_detected(self):
0157          state = self.setup_loaded()
0158          engine, cfg, *_ = state
0159          ledger = Table("etl_batch_commits", MetaData(), autoload_with=engine)
0160          with engine.begin() as conn:
```

```
            existing = conn.execute(select(ledger)).mappings().first()
            conn.execute(ledger.insert().values({
                **dict(existing), "batch_id": str(uuid4()),
            }))
        result = self.compare(state)
        self.assertIn("ledger_batch_count_or_identity_mismatch",
result["mismatch_types"])

    def test_fingerprint_independent_of_observation_order(self):
        rows = [{"n": Decimal("100.00"), "id": "1"},
                {"n": Decimal("200.0"), "id": "2"}]
        self.assertEqual(row_fingerprint(rows), row_fingerprint(list(reversed(rows))))
        self.assertEqual(canonical(Decimal("100.00")), canonical(Decimal("100")))

    def test_wrapper_reconciles_using_offline_mocked_mysql_preflight(self):
        state = self.setup_loaded()
        engine = state[0]
        manifest_path = self.root / "pilot/shadow-manifest.json"
        staging = self.root / "stage.json"
        staging.write_text(json.dumps({"status": "passed"}), encoding="utf-8")
        with (patch("verify_shadow_reconciliation.check_staging", return_value=[]),
              patch("verify_shadow_reconciliation.check_shadow_destination",
return_value=object()),
              patch("verify_shadow_reconciliation.preflight_shadow",
return_value={"replay": True}),
              patch("verify_shadow_reconciliation.create_engine", return_value=engine)):
            result = reconcile_shadow(manifest_path, staging, "etlguard_shadow_ci",
"fake")
        self.assertEqual(result["status"], "passed", result)
        self.assertTrue(result["target_physical_sha256"])
        self.assertFalse(result["production_approved"])

    def test_changed_source_blocks_wrapper_before_database_access(self):
        self.setup_loaded()
        source = self.root / "source.csv"
        source.write_text(source.read_text(encoding="utf-8") +
                          "C003,Eve,eve@example.com,2026-01-03,active,300.00\n",
encoding="utf-8")
        with patch("verify_shadow_reconciliation.create_engine") as database:
            with self.assertRaisesRegex(ValueError, "changed or disappeared"):
                reconcile_shadow(self.root / "pilot/shadow-manifest.json",
                                 self.root / "missing-staging.json",
"etlguard_shadow_ci", "")
            database.assert_not_called()

    def test_cli_wrapper_rejects_missing_staging_evidence_before_db_connection(self):
        state = self.setup_loaded()
        *_, manifest, _, _, _ = state
        manifest_path = self.root / "pilot/shadow-manifest.json"
        stage = self.root / "staging.json"
        stage.write_text(json.dumps({"status": "blocked"}), encoding="utf-8")
        with self.assertRaisesRegex(ValueError, "passing MySQL 8 staging report"):
            reconcile_shadow(manifest_path, stage, "etlguard_shadow_ci", "")


if __name__ == "__main__":
    unittest.main()
```

scripts/test_verify_mysql8_staging.py

51 source lines.

```
"""Offline safety regression tests for the independent staging gate."""
import unittest
from pathlib import Path
import importlib.util

SCRIPT = Path(__file__).with_name("verify_mysql8_staging.py")
spec = importlib.util.spec_from_file_location("etlguard_staging_gate", SCRIPT)
module = importlib.util.module_from_spec(spec)
spec.loader.exec_module(module)


class TestSafetyGate(unittest.TestCase):
    good = "mysql+pymysql://etlguard_ci:local_only@127.0.0.1:3307/etlguard_ci"

    def test_safe_disposable_database(self):
        url = module.validate_destination(self.good, "etlguard_ci")
        self.assertEqual(url.database, "etlguard_ci")

    def test_production_style_name_blocked(self):
```

```
0020          with self.assertRaises(ValueError):
0021              module.validate_destination(self.good.replace("/etlguard_ci", "/customers"),
      "customers")
0022
0023      def test_mismatch_blocked(self):
0024          with self.assertRaises(ValueError):
0025              module.validate_destination(self.good, "etlguard_test")
0026
0027      def test_root_blocked(self):
0028          with self.assertRaises(ValueError):
0029              module.validate_destination(self.good.replace("etlguard_ci:local",
      "root:local"), "etlguard_ci")
0030
0031      def test_remote_blocked_without_override(self):
0032          remote = self.good.replace("127.0.0.1", "db.example.invalid")
0033          with self.assertRaises(ValueError):
0034              module.validate_destination(remote, "etlguard_ci")
0035          self.assertEqual(module.validate_destination(remote, "etlguard_ci", True).host,
0036                           "db.example.invalid")
0037
0038      def test_mysql8_only(self):
0039          self.assertTrue(module.is_mysql8("8.0.41"))
0040          self.assertTrue(module.is_mysql8("8.4.4"))
0041          self.assertFalse(module.is_mysql8("5.7.42"))
0042          self.assertFalse(module.is_mysql8("10.11.2-MariaDB"))
0043
0044      def test_only_temporary_integration_tables_allowed(self):
0045          self.assertEqual(module.disallowed_tables([
0046              "etl_run_audit", "etl_batch_commits", "it_abc", "stg_it_abc", "customers"
0047          ]), ["customers"])
0048
0049
0050  if __name__ == "__main__":
0051      unittest.main()
```

scripts/verify_mysql8_staging.py

249 source lines.

```
0001  #!/usr/bin/env python3
0002  """Fail-closed MySQL 8 staging gate for ETL Guard v0.1.2.
0003
0004  This runner creates/drops test tables indirectly through pytest integration tests.
0005  It is intentionally restricted to disposable schemas that are explicitly confirmed.
0006  It never connects to the production target database.
0007  """
0008  from __future__ import annotations
0009
0010  import argparse
0011  from etlguard import __version__
0012
0013  from datetime import datetime, timezone
0014  import json
0015  import os
0016  from pathlib import Path
0017  import re
0018  import subprocess
0019  import sys
0020  from urllib.parse import quote, unquote
0021  from typing import Any
0022
0023  SCHEMA_PATTERN = re.compile(r"^etlguard_(?:ci|test|staging_ci)(?:_[a-z0-9]+)*$")
0024  ALLOWED_TABLES = {"etl_run_audit", "etl_batch_commits"}
0025
0026
0027  def validate_destination(url_text: str, confirmed_schema: str, allow_remote: bool =
      False) -> Any:
0028      """Reject production-looking destinations before opening a connection."""
0029      from sqlalchemy.engine import make_url
0030
0031      if not url_text:
0032          raise ValueError("ETLGUARD_TEST_MYSQL_URL is not set")
0033      url = make_url(url_text)
0034      if url.drivername != "mysql+pymysql":
0035          raise ValueError("Test URL must use mysql+pymysql:// (not SQLite or MariaDB)")
0036      if not url.database or not SCHEMA_PATTERN.fullmatch(url.database):
0037          raise ValueError("Test database must be named etlguard_ci, etlguard_test, or
      etlguard_staging_ci (optional suffix)")
0038      if confirmed_schema != url.database:
0039          raise ValueError("--confirm-disposable-schema must EXACTLY match the URL
      database")
```

A B C D E F G H

```
    if (url.username or "").lower() in {"root", "admin", "administrator", ""}:
        raise ValueError("A non-root, dedicated disposable-schema MySQL user is required")
    if url.host not in {"127.0.0.1", "localhost", "::1"} and not allow_remote:
        raise ValueError("Remote MySQL hosts require explicit --allow-remote")
    if not url.host:
        raise ValueError("URL must specify the MySQL hostname")
    return url


def is_mysql8(version: str) -> bool:
    return bool(re.match(r"^8\.\d+\.", version)) and "mariadb" not in version.lower()


def disallowed_tables(names: list[str]) -> list[str]:
    return sorted(name for name in names if not (
        name in ALLOWED_TABLES or name.startswith("it_") or name.startswith("stg_it_")
    ))


def grants_are_disposable_schema_only(grants: list[str], schema: str) -> bool:
    """Allow only USAGE globally and direct grants on the disposable schema.

    Disallow role grants rather than trying to resolve inherited global privileges.
    """
    schema_sql = re.escape(schema)
    limited = re.compile(r"\bON\s+`?" + schema_sql + r"`?\.\*\s+TO\b", re.I)
    usage = re.compile(r"^GRANT\s+USAGE\s+ON\s+\*\.\*\s+TO\b", re.I)
    # SHOW GRANTS escapes literal underscores in database identifiers.
    return bool(grants) and all(
        usage.search(grant) or limited.search(grant.replace("\\_", "_"))
        for grant in grants
    )


def strict_sql_mode_is_enabled(sql_mode: str) -> bool:
    """Silent value coercion and truncation are unsafe for financial/data loads."""
    modes = {item.strip().upper() for item in sql_mode.split(",")}
    return bool({"STRICT_TRANS_TABLES", "STRICT_ALL_TABLES"} & modes)


def redact_secrets(message: str, url_text: str) -> str:
    """Do not leak database passwords into the gate status or persisted logs."""
    from sqlalchemy.engine import make_url

    secrets = [url_text]
    try:
        password = make_url(url_text).password or ""
        secrets.extend([password, quote(password, safe=""), unquote(password)])
    except Exception:
        pass
    for secret in sorted(set(secrets), key=len, reverse=True):
        if len(secret) >= 4:
            message = message.replace(secret, "<redacted>")
    return message


def preflight(url: Any) -> dict[str, Any]:
    from sqlalchemy import create_engine, text

    engine = create_engine(url, pool_pre_ping=True, connect_args={"connect_timeout": 10})
    try:
        with engine.connect() as conn:
            row = conn.execute(text(
                "SELECT VERSION() AS version, DATABASE() AS db_name, "
                "@@default_storage_engine AS storage_engine, @@read_only AS read_only, "
                "@@SESSION.sql_mode AS sql_mode"
            )).mappings().one()
            tables = list(conn.execute(text(
                "SELECT table_name AS table_name, engine AS engine, table_type AS table_type FROM information_schema.tables "
                "WHERE table_schema = DATABASE()"
            )).mappings())
            grants = list(conn.execute(text("SHOW GRANTS")).scalars())
        names = [str(row["table_name"]) for row in tables]
        bad_engines = [str(t["table_name"]) for t in tables
                       if str(t["engine"] or "").lower() != "innodb" or
                       str(t["table_type"]).upper() != "BASE TABLE"]
        info = {
            "server_version": str(row["version"]),
            "database": str(row["db_name"]),
            "default_storage_engine": str(row["storage_engine"]),
            "read_only": bool(row["read_only"]),
            "strict_sql_mode": strict_sql_mode_is_enabled(str(row["sql_mode"])),
            "schema_only_privileges": grants_are_disposable_schema_only(grants,
```

A
B
C
D
E
F
G
H

SECTION MAP

```
    url.database),
0123            "existing_table_count": len(names),
0124            "unexpected_tables": disallowed_tables(names),
0125            "non_innodb_tables": bad_engines,
0126        }
0127        if not is_mysql8(info["server_version"]):
0128            raise ValueError(f"Expected MySQL 8.x, found {info['server_version']}")
0129        if info["database"] != url.database:
0130            raise ValueError("Connected schema does not match the confirmed disposable
    schema")
0131        if info["default_storage_engine"].lower() != "innodb":
0132            raise ValueError("MySQL default storage engine must be InnoDB")
0133        if info["read_only"]:
0134            raise ValueError("Staging server is read-only; tests need disposable-schema
    DDL")
0135        if not info["strict_sql_mode"]:
0136            raise ValueError("MySQL strict SQL mode is required to avoid silent
    coercion/truncation")
0137        if not info["schema_only_privileges"]:
0138            raise ValueError("Test identity has non-disposable-schema or inherited global
     privileges")
0139        if info["unexpected_tables"]:
0140            raise ValueError(f"Unexpected tables in test schema:
    {info['unexpected_tables']}")
0141        if info["non_innodb_tables"]:
0142            raise ValueError(f"Existing tables are not InnoDB:
    {info['non_innodb_tables']}")
0143        return info
0144    finally:
0145        engine.dispose()
0146
0147
0148 def execute_gate(project: Path, environment: dict[str, str], logs: Path) ->
    list[dict[str, Any]]:
0149    tests = [
0150        ("mysql8_integration", [sys.executable, "-m", "pytest", "-q", "-ra", "-m",
    "integration", "tests/test_mysql_integration.py"]),
0151        ("complete_suite", [sys.executable, "-m", "pytest", "-q", "-ra",
    "--cov=etlguard", "--cov-branch", "--cov-fail-under=85"]),
0152        ("ruff", [sys.executable, "-m", "ruff", "check", "."]),
0153        ("mypy", [sys.executable, "-m", "mypy", "src/etlguard"]),
0154    ]
0155    results: list[dict[str, Any]] = []
0156    for name, cmd in tests:
0157        print(f"Running {name}: {' '.join(cmd)}", flush=True)
0158        try:
0159            completed = subprocess.run(cmd, cwd=project, env=environment,
0160                                       stdout=subprocess.PIPE, stderr=subprocess.STDOUT,
0161                                       text=True, errors="replace", timeout=900,
    check=False)
0162            output = completed.stdout
0163            code = completed.returncode
0164        except subprocess.TimeoutExpired as exc:
0165            output = str(exc)
0166            code = 124
0167        # No database URL or credentials are printed in normal gate logs.
0168        output = redact_secrets(output, environment.get("ETLGUARD_TEST_MYSQL_URL", ""))
0169        (logs / f"{name}.log").write_text(output, encoding="utf-8")
0170        skipped = sum(int(num) for num in re.findall(r"(?:^|\s)(\d+) skipped\b", output))
0171        result = {
0172            "name": name, "exit_code": code, "skipped": skipped,
0173            "status": "passed" if code == 0 and skipped == 0 else "failed",
0174            "log": str(logs / f"{name}.log"),
0175        }
0176        results.append(result)
0177        print(f"  {result['status']} (exit={code}, skipped={skipped})", flush=True)
0178    return results
0179
0180
0181 def main() -> int:
0182    parser = argparse.ArgumentParser(description=__doc__)
0183    parser.add_argument("--project", type=Path,
    default=Path(__file__).resolve().parent.parent,
0184                        help="Path to extracted v0.1.2 source project")
0185    parser.add_argument("--confirm-disposable-schema", required=True,
0186                        help="Exact disposable database name (for example etlguard_ci)")
0187    parser.add_argument("--allow-remote", action="store_true",
0188                        help="Permit a named remote test server (never a production DB)")
0189    parser.add_argument("--execute-tests", action="store_true",
0190                        help="Run destructive integration tests after the safe
    preflight")
0191    parser.add_argument("--report", type=Path, default=Path("staging-gate-report.json"))
0192    args = parser.parse_args()
0193
0194    report = {
```

```
0195            "timestamp_utc": datetime.now(timezone.utc).isoformat(),
0196            "toolkit_version": __version__,
0197            "gate": "mysql8_staging",
0198            "status": "blocked",
0199            "mysql8_server_verified": False,
0200            "mysql8_integration_executed": False,
0201            "production_pilot_executed": False,
0202            "preflight": None,
0203            "checks": [],
0204            "note": "A staging pass is NOT authorization for production deployment.",
0205        }
0206        exit_code = 2
0207        url_text = os.environ.get("ETLGUARD_TEST_MYSQL_URL", "")
0208        try:
0209            project = args.project.expanduser().resolve()
0210            if not (project / "tests/test_mysql_integration.py").is_file():
0211                raise ValueError("--project must point to extracted ETL Guard source with
    tests/")
0212            url = validate_destination(url_text, args.confirm_disposable_schema,
    args.allow_remote)
0213            print(f"Checking disposable MySQL destination: {url.host}:{url.port or
    3306}/{url.database}")
0214            report["preflight"] = preflight(url)
0215            report["mysql8_server_verified"] = True
0216            if not args.execute_tests:
0217                report["status"] = "preflight_passed_only"
0218                report["note"] = "Re-run with --execute-tests for the MySQL, quality, lint,
    and type gates."
0219                exit_code = 0
0220            else:
0221                env = os.environ.copy()
0222                env["ETLGUARD_DATABASE_URL"] = url_text
0223                # An editable package installation is preferred; PYTHONPATH also permits
    source tests.
0224                env["PYTHONPATH"] = str(project / "src") + os.pathsep + env.get("PYTHONPATH",
     "")
0225                logs = args.report.expanduser().resolve().parent / "mysql8-gate-logs"
0226                logs.mkdir(parents=True, exist_ok=True)
0227                checks = execute_gate(project, env, logs)
0228                report["mysql8_integration_executed"] = True
0229                report["checks"] = checks
0230                report["status"] = "passed" if all(c["status"] == "passed" for c in checks)
    else "failed"
0231                report["note"] = ("MySQL8 + static/functional staging gates passed; a
    separate "
0232                                  "approved controlled pilot is still required."
0233                                  if report["status"] == "passed" else
0234                                  "One or more gates failed or were skipped. Do not deploy.")
0235                exit_code = 0 if report["status"] == "passed" else 1
0236        except Exception as exc:
0237            reason = redact_secrets(str(exc), url_text)
0238            print(f"BLOCKED: {reason}", file=sys.stderr)
0239            report["error"] = reason
0240        finally:
0241            out = args.report.expanduser().resolve()
0242            out.parent.mkdir(parents=True, exist_ok=True)
0243            out.write_text(json.dumps(report, indent=2, sort_keys=True) + "\n",
    encoding="utf-8")
0244            print(f"Gate status: {report['status']}; report: {out}")
0245        return exit_code
0246
0247
0248    if __name__ == "__main__":
0249        raise SystemExit(main())
```

scripts/verify_shadow_reconciliation.py

274 source lines.

```
0001    #!/usr/bin/env python3
0002    """Independent read-only, row-level verification of an isolated MySQL 8 shadow pilot.
0003
0004    The CLI refuses non-local, non-disposable MySQL URLs. A separate passing MySQL 8
0005    staging report is mandatory. No source records, secrets, or database rows are
0006    included in the JSON evidence; only counts, fingerprints and mismatch types.
0007
0008    Offline unit tests exercise compare_shadow() against SQLite. This does NOT
0009    substitute for executing the CLI against genuine disposable MySQL 8 staging.
0010    """
0011    from __future__ import annotations
0012
```

```
0013  import argparse
0014  from etlguard import __version__
0015
0016  from collections import Counter
0017  from datetime import date, datetime, timezone
0018  from decimal import Decimal
0019  from math import isfinite
0020  from hashlib import sha256
0021  import json
0022  import os
0023  from pathlib import Path
0024  import sys
0025  from typing import Any
0026
0027  from sqlalchemy import MetaData, Table, create_engine, inspect, select
0028  from sqlalchemy.engine import Engine
0029
0030  from etlguard.config import load_config
0031  from etlguard.core import ValidationEngine
0032  from etlguard.pipeline import Pipeline, RunResult, resolve_model
0033  from etlguard.validation import Candidate
0034  from assess_promotion import check_staging
0035  from prepare_shadow_pilot import SHADOW_ENV, sha256_file, verify
0036  from run_shadow_pilot import check_shadow_destination, preflight_shadow
0037
0038
0039  def canonical(value: Any) -> Any:
0040      """Stable JSON values for typed MySQL / SQLite comparison; no raw output."""
0041      if isinstance(value, Decimal):
0042          if not value.is_finite():
0043              raise ValueError("Non-finite numeric values cannot be reconciled")
0044          return str(value.normalize())
0045      if isinstance(value, datetime):
0046          return value.isoformat(timespec="microseconds")
0047      if isinstance(value, date):
0048          return value.isoformat()
0049      if isinstance(value, bytes):
0050          return value.hex()
0051      if isinstance(value, dict):
0052          return {str(k): canonical(v) for k, v in sorted(value.items())}
0053      if isinstance(value, (list, tuple)):
0054          return [canonical(v) for v in value]
0055      if isinstance(value, float) and not isfinite(value):
0056          raise ValueError("Non-finite floating-point values cannot be reconciled")
0057      if value is None or isinstance(value, (str, int, bool, float)):
0058          return value
0059      raise ValueError(f"Unsupported database value type for reconciliation:
      {type(value).__name__}")
0060
0061
0062  def serialized(data: dict[str, Any]) -> str:
0063      return json.dumps(canonical(data), separators=(",", ":"), sort_keys=True,
      ensure_ascii=True)
0064
0065
0066  def row_fingerprint(rows: list[dict[str, Any]]) -> str:
0067      """Physical fingerprint includes ETL IDs and SCD2 history, not just counts."""
0068      ordered = sorted(serialized(row) for row in rows)
0069      digest = sha256()
0070      for value in ordered:
0071          encoded = value.encode("utf-8")
0072          digest.update(len(encoded).to_bytes(8, "big"))
0073          digest.update(encoded)
0074      return digest.hexdigest()
0075
0076
0077  def compare_shadow(engine: Engine, settings: Any, model: Any, batch_id: str,
0078                     expected: list[Candidate], dry: RunResult) -> dict[str, Any]:
0079      """Read-only evidence for one first-load + any identical retries.
0080
0081      All SQL statements are SELECT. This function can be tested with SQLite,
0082      while the CLI restricts actual operator execution to an isolated MySQL 8.
0083      """
0084      target = settings.target
0085      if target is None:
0086          raise ValueError("Shadow target must be configured")
0087      names = [target.table, f"stg_{target.table}", "etl_batch_commits", "etl_run_audit"]
0088      problems: list[str] = []
0089      with engine.connect() as conn:
0090          inspector = inspect(conn)
0091          absent = [name for name in names if not inspector.has_table(name)]
0092          if absent:
0093              raise ValueError("Shadow tables missing: " + ", ".join(absent))
0094          target_t, stage_t, ledger_t, audit_t = (
```

A
B
C
D
E
F
G
H

```
0095             Table(name, MetaData(), autoload_with=conn) for name in names
0096         )
0097         target_rows = [dict(item) for item in conn.execute(select(target_t)).mappings()]
0098         stage_rows = conn.execute(select(stage_t)).all()
0099         ledger_rows = conn.execute(select(ledger_t)).mappings().all()
0100         audit_rows = conn.execute(select(audit_t)).mappings().all()
0101
0102     # Never put raw source or persisted data into the returned evidence.
0103     if stage_rows:
0104         problems.append("staging_rows_remain")
0105     if len(ledger_rows) != 1 or ledger_rows[0]["batch_id"] != batch_id:
0106         problems.append("ledger_batch_count_or_identity_mismatch")
0107     if len(audit_rows) != 1 or audit_rows[0]["batch_id"] != batch_id:
0108         problems.append("audit_batch_count_or_identity_mismatch")
0109     if len(target_rows) != len(expected):
0110         problems.append("target_row_count_mismatch")
0111     if len(expected) == 0:
0112         problems.append("no_valid_source_rows_for_shadow_pilot")
0113
0114     active_rows = target_rows
0115     if target.mode == "scd2":
0116         active_rows = [row for row in target_rows if bool(row.get("is_current"))]
0117         if len(active_rows) != len(target_rows):
0118             problems.append("unexpected_scd2_history_during_first_batch")
0119         if any(row.get("effective_to") is not None for row in active_rows):
0120             problems.append("active_scd2_row_has_end_timestamp")
0121         keys = [tuple(row[key] for key in target.key) for row in active_rows]
0122         if len(keys) != len(set(keys)):
0123             problems.append("multiple_active_scd2_versions")
0124
0125     # Round-trip through the declared model to normalize SQL date/decimal columns.
0126     fields = list(model.model_fields)
0127     try:
0128         observed_data = [
0129             model.model_validate({name: row[name] for name in
    fields}).model_dump(mode="python")
0130             for row in active_rows
0131         ]
0132         expected_data = [model.model_validate(item.data).model_dump(mode="python")
0133                          for item in expected]
0134         actual_counter = Counter(serialized(item) for item in observed_data)
0135         expected_counter = Counter(serialized(item) for item in expected_data)
0136         missing_count = sum((expected_counter - actual_counter).values())
0137         unexpected_count = sum((actual_counter - expected_counter).values())
0138         if missing_count or unexpected_count:
0139             problems.append("source_vs_persisted_row_values_mismatch")
0140     except (ValueError, KeyError, TypeError):
0141         # Avoid exposing data through Pydantic's value-bearing validation errors.
0142         problems.append("persisted_row_schema_or_type_mismatch")
0143         missing_count = len(expected)
0144         unexpected_count = len(active_rows)
0145
0146     if len(ledger_rows) == 1:
0147         commit = ledger_rows[0]
0148         ledger_rules = {
0149             "request_hash": dry.request_hash,
0150             "pipeline": settings.pipeline,
0151             "target_table": target.table,
0152             "target_mode": target.mode,
0153             "rows_valid": dry.rows_valid,
0154             "rows_inserted": dry.rows_valid,
0155             "rows_updated": 0,
0156             "rows_unchanged": 0,
0157             "scd2_versions_created": 0,
0158         }
0159         if any(commit.get(key) != value for key, value in ledger_rules.items()):
0160             problems.append("batch_commit_ledger_mismatch")
0161     if len(audit_rows) == 1:
0162         audit = audit_rows[0]
0163         audit_rules = {
0164             "pipeline": settings.pipeline,
0165             "status": "success",
0166             "rows_read": dry.rows_read,
0167             "rows_valid": dry.rows_valid,
0168             "rows_rejected": dry.rows_rejected,
0169             "rows_loaded": dry.rows_valid,
0170         }
0171         if any(audit.get(key) != value for key, value in audit_rules.items()):
0172             problems.append("audit_counts_or_status_mismatch")
0173
0174     return {
0175         "status": "passed" if not problems else "failed",
0176         "source_rows_read": dry.rows_read,
0177         "source_rows_valid": dry.rows_valid,
0178         "source_rows_rejected": dry.rows_rejected,
```

A
B
C
D
E
F
G
H

```
        "target_rows_observed": len(target_rows),
        "staging_rows_observed": len(stage_rows),
        "audit_rows_observed": len(audit_rows),
        "commit_rows_observed": len(ledger_rows),
        "missing_source_rows": missing_count,
        "unexpected_target_rows": unexpected_count,
        "target_physical_sha256": row_fingerprint(target_rows),
        "mismatch_types": problems,
        "database_writes_executed_by_verifier": False,
        "production_approved": False,
    }


def reconcile_shadow(manifest_path: Path, staging_report: Path,
                     confirmed_schema: str, url_text: str) -> dict[str, Any]:
    """Fail closed and verify a committed disposable MySQL shadow batch."""
    verify(manifest_path)
    manifest = json.loads(manifest_path.read_text(encoding="utf-8"))
    staging = json.loads(staging_report.read_text(encoding="utf-8"))
    if not isinstance(staging, dict) or check_staging(staging):
        raise ValueError("A passing MySQL 8 staging report is required")
    url = check_shadow_destination(url_text, confirmed_schema)
    preflight = preflight_shadow(url, manifest["batch_id"], manifest["shadow_target"])
    if not preflight["replay"]:
        raise ValueError("No durable shadow commit exists; execute the approved pilot
first")
    cfg, base_dir = load_config(manifest["shadow_yaml"])
    model = resolve_model(cfg.model)
    pipe = Pipeline(cfg, base_dir)
    dry = pipe.run(dry_run=True, batch_id=manifest["batch_id"])
    if not dry.ok or dry.rows_valid <= 0:
        raise ValueError("Source validation failed or no valid records were available")
    checks = cfg.checks.model_copy(deep=True)
    if cfg.target is not None and cfg.target.mode in {"upsert", "scd2"} and not
checks.unique:
        checks.unique = cfg.target.key[:]
    core = ValidationEngine(model, cfg.cleaning, checks, base_dir).run(
        pipe._source().read(manifest["batch_id"])
    )
    digest_input = {
        "config": cfg.model_dump(mode="json"),
        "model_schema": model.model_json_schema(),
        "source_digest": core.source_digest,
    }
    second_hash = sha256(json.dumps(
        digest_input, sort_keys=True, default=str, separators=(",", ":")
    ).encode("utf-8")).hexdigest()
    if (core.source_error or core.dataset_issues or core.rows_valid != dry.rows_valid or
            core.rows_read != dry.rows_read or core.rows_rejected != dry.rows_rejected or
            second_hash != dry.request_hash):
        raise ValueError("Source changed or validation did not reproduce the dry run")
    engine = create_engine(url, pool_pre_ping=True, connect_args={"connect_timeout": 10})
    try:
        results = compare_shadow(engine, cfg, model, manifest["batch_id"], core.valid,
dry)
    finally:
        engine.dispose()
    # Verify source and both configs were unchanged during the read.
    verify(manifest_path)
    return {
        **results,
        "timestamp_utc": datetime.now(timezone.utc).isoformat(),
        "toolkit_version": __version__,
        "phase": "shadow_row_reconciliation",
        "schema": confirmed_schema,
        "shadow_target": manifest["shadow_target"],
        "batch_id": manifest["batch_id"],
        "manifest_sha256": sha256_file(manifest_path),
        "staging_report_sha256": sha256_file(staging_report),
        "note": "Read-only reconciliation on disposable MySQL 8; not production
authorization",
    }


def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--manifest", type=Path, required=True)
    parser.add_argument("--staging-report", type=Path, required=True)
    parser.add_argument("--confirm-shadow-schema", required=True)
    parser.add_argument("--report", type=Path,
default=Path("shadow-reconciliation.json"))
    args = parser.parse_args()
    try:
        result = reconcile_shadow(args.manifest.resolve(), args.staging_report.resolve(),
                                  args.confirm_shadow_schema, os.environ.get(SHADOW_ENV,
```

```
    ""))
0259          exit_code = 0 if result["status"] == "passed" else 1
0260      except Exception as exc:
0261          # No password, URL or raw source values should be returned in evidence.
0262          result = {"status": "blocked", "error_type": type(exc).__name__,
0263                    "database_writes_executed_by_verifier": False, "production_approved":
    False}
0264          exit_code = 2
0265          print(f"BLOCKED: {result['error_type']}; see isolated staging prerequisites",
    file=sys.stderr)
0266      report_path = args.report.resolve()
0267      report_path.parent.mkdir(parents=True, exist_ok=True)
0268      report_path.write_text(json.dumps(result, indent=2, sort_keys=True) + "\n",
    encoding="utf-8")
0269      print(f"Reconciliation status: {result['status']}; report: {report_path}")
0270      return exit_code
0271
0272
0273  if __name__ == "__main__":
0274      raise SystemExit(main())
```

42. Appendix C - Complete verification test source

This section reproduces the archived unit, transport, property-based, safety and MySQL integration test programs. A test appearing in source is not itself proof of successful execution; see the current JUnit and staging reports for that evidence.

tests/conftest.py

33 source lines.

```
0001  """Shared synthetic data and strongly typed source records."""
0002
0003  from __future__ import annotations
0004
0005  from datetime import datetime, timezone
0006  from pathlib import Path
0007  from typing import Any
0008
0009  import pytest
0010
0011  from etlguard.sources.base import Lineage, SourceRecord
0012
0013
0014  @pytest.fixture
0015  def customer_raw() -> dict[str, Any]:
0016      return {
0017          "customer_id": "C001", "full_name": "Alice Doe", "email": "alice@example.com",
0018          "signup_date": "2026-01-01", "end_date": None,
0019          "status": "active", "credit_limit": "1200.00",
0020      }
0021
0022
0023  @pytest.fixture
0024  def record_factory():
0025      def create(data: dict[str, Any], number: int = 2) -> SourceRecord:
0026          line = Lineage("fixture.csv", number, datetime.now(timezone.utc), "batch-123")
0027          return SourceRecord(data, line)
0028      return create
0029
0030
0031  @pytest.fixture
0032  def example_dir() -> Path:
0033      return Path(__file__).resolve().parents[1] / "src" / "etlguard" / "examples"
```

tests/test_api.py

106 source lines.

```
0001  """All network calls are intercepted by respx; no live API dependencies."""
0002
0003  from __future__ import annotations
```

```python

import httpx
import pytest

from etlguard.config import PaginationSettings, SourceSettings
from etlguard.sources.api import ApiSource, SourceError, dig

respx = pytest.importorskip("respx")


def test_nested_json_path():
    assert dig({"data": {"items": [1, 2]}}, "data.items") == [1, 2]
    with pytest.raises(SourceError, match="Missing"):
        dig({}, "data.items")


@respx.mock
def test_page_pagination():
    route = respx.get("https://api.example.test/items")
    route.side_effect = [
        httpx.Response(200, json={"data": {"items": [{"id": 1}, {"id": 2}]}}),
        httpx.Response(200, json={"data": {"items": [{"id": 3}]}}),
    ]
    settings = SourceSettings(
        type="api", url="https://api.example.test/items", records_path="data.items",
        pagination=PaginationSettings(mode="page", page_size=2),
    )
    records = list(ApiSource(settings, sleeper=lambda _: None).read("batch"))
    assert [record.data["id"] for record in records] == [1, 2, 3]
    assert [record.lineage.source_row_number for record in records] == [1, 2, 3]
    assert route.call_count == 2
    assert "page=1" in str(route.calls[0].request.url)


@respx.mock
def test_offset_pagination():
    route = respx.get("https://api.example.test/items")
    route.side_effect = [
        httpx.Response(200, json=[{"id": 1}]),
        httpx.Response(200, json=[]),
    ]
    cfg = SourceSettings(type="api", url="https://api.example.test/items",
                         pagination=PaginationSettings(mode="offset", page_size=1,
start=0))
    assert len(list(ApiSource(cfg, sleeper=lambda _: None).read("x"))) == 1
    assert "offset=1" in str(route.calls[1].request.url)


@respx.mock
def test_next_link_and_bearer_env(monkeypatch):
    monkeypatch.setenv("ETLGUARD_SAMPLE_TOKEN", "test-only")
    first = respx.get("https://api.example.test/items").mock(return_value=httpx.Response(
        200, json={"data": {"items": [{"id": 1}]}, "links": {"next": "/other"}}))
    respx.get("https://api.example.test/other").mock(return_value=httpx.Response(
        200, json={"data": {"items": [{"id": 2}]}, "links": {"next": None}}))
    cfg = SourceSettings(type="api", url="https://api.example.test/items",
                         records_path="data.items",
bearer_token_env="ETLGUARD_SAMPLE_TOKEN",
                         pagination=PaginationSettings(mode="next"))
    rows = list(ApiSource(cfg, sleeper=lambda _: None).read("b"))
    assert len(rows) == 2
    assert first.calls[0].request.headers["Authorization"] == "Bearer test-only"


@respx.mock
def test_500_retry_then_success():
    route = respx.get("https://api.example.test/items")
    route.side_effect = [httpx.Response(500), httpx.Response(502), httpx.Response(200,
json=[])]
    sleep_calls = []
    cfg = SourceSettings(type="api", url="https://api.example.test/items",
                         max_retries=2, backoff_seconds=0.1)
    assert list(ApiSource(cfg, sleeper=sleep_calls.append).read("b")) == []
    assert sleep_calls == [0.1, 0.2]
    assert route.call_count == 3


@respx.mock
def test_retries_exhausted_and_nonretryable_400():
    route =
respx.get("https://api.example.test/items").mock(return_value=httpx.Response(500))
    cfg = SourceSettings(type="api", url="https://api.example.test/items", max_retries=1)
    with pytest.raises(SourceError, match="request failed"):
        list(ApiSource(cfg, sleeper=lambda _: None).read("b"))
    assert route.call_count == 2
```

```
    route.mock(return_value=httpx.Response(400))
    with pytest.raises(SourceError):
        list(ApiSource(cfg, sleeper=lambda _: None).read("b"))
    assert route.call_count == 3


@respx.mock
def test_next_link_cross_origin_blocked():
    respx.get("https://api.example.test/items").mock(return_value=httpx.Response(
        200, json={"items": [], "links": {"next": "https://evil.example/steal"}}))
    cfg = SourceSettings(type="api", url="https://api.example.test/items",
                         records_path="items",
pagination=PaginationSettings(mode="next"))
    with pytest.raises(SourceError, match="changed origin"):
        list(ApiSource(cfg, sleeper=lambda _: None).read("b"))


def test_missing_token(monkeypatch):
    monkeypatch.delenv("DOES_NOT_EXIST_TOKEN", raising=False)
    cfg = SourceSettings(type="api", url="https://api.example.test/items",
                         bearer_token_env="DOES_NOT_EXIST_TOKEN")
    with pytest.raises(SourceError, match="Missing API bearer"):
        list(ApiSource(cfg).read("b"))
```

tests/test_api_transport.py

143 source lines.

```
"""Additional real httpx adapter tests that run without respx or a network."""

from __future__ import annotations

import httpx
import pytest

from etlguard.config import PaginationSettings, SourceSettings
from etlguard.sources.api import ApiSource, SourceError


def client_for(handler):
    return httpx.Client(transport=httpx.MockTransport(handler))


def test_httpx_mock_page_pagination_and_lineage():
    def handler(request):
        assert request.method == "GET"
        page = request.url.params["page"]
        if page == "1":
            return httpx.Response(200, json={"data": {"items": [{"id": 1}, {"id": 2}]}})
        return httpx.Response(200, json={"data": {"items": [{"id": 3}]}})

    settings = SourceSettings(type="api", url="https://api.test/feed",
records_path="data.items",
                              pagination=PaginationSettings(mode="page", page_size=2))
    records = list(ApiSource(settings, client=client_for(handler)).read("batch-abc"))
    assert [record.data["id"] for record in records] == [1, 2, 3]
    assert [record.lineage.source_row_number for record in records] == [1, 2, 3]
    assert all(record.lineage.batch_id == "batch-abc" for record in records)


def test_offset_pagination_httpx_mock():
    offsets = []

    def handler(request):
        offset = int(request.url.params["offset"])
        offsets.append(offset)
        return httpx.Response(200, json=[{"id": offset}] if offset == 0 else [])

    cfg = SourceSettings(type="api", url="https://api.test/feed",
                         pagination=PaginationSettings(mode="offset", page_size=1,
start=0))
    assert len(list(ApiSource(cfg, client=client_for(handler)).read("id"))) == 1
    assert offsets == [0, 1]


def test_next_link_pagination_and_auth(monkeypatch):
    monkeypatch.setenv("SYNTHETIC_FEED_TOKEN", "placeholder")
    urls = []

    def handler(request):
        assert request.headers["authorization"] == "Bearer placeholder"
```

```
        urls.append(str(request.url))
        if request.url.path == "/start":
            return httpx.Response(200, json={"data": {"items": [{"id": 1}]},
                                             "links": {"next": "/next"}})
        return httpx.Response(200, json={"data": {"items": [{"id": 2}]},
                                         "links": {"next": None}})

    cfg = SourceSettings(type="api", url="https://api.test/start",
records_path="data.items",
                         bearer_token_env="SYNTHETIC_FEED_TOKEN",
                         pagination=PaginationSettings(mode="next"))
    assert len(list(ApiSource(cfg, client=client_for(handler)).read("batch"))) == 2
    assert urls == ["https://api.test/start", "https://api.test/next"]


def test_httpx_mock_retry_and_backoff():
    attempts = []
    sleeps = []

    def handler(_request):
        attempts.append(1)
        if len(attempts) < 3:
            return httpx.Response(500)
        return httpx.Response(200, json=[])

    cfg = SourceSettings(type="api", url="https://api.test/feed", max_retries=2,
                         backoff_seconds=0.05)
    assert list(ApiSource(cfg, client=client_for(handler),
sleeper=sleeps.append).read("id")) == []
    assert len(attempts) == 3
    assert sleeps == [0.05, 0.1]


def test_api_failure_and_wrong_json_shapes():
    cfg = SourceSettings(type="api", url="https://api.test/feed", max_retries=0)
    bad_status = client_for(lambda request: httpx.Response(400))
    with pytest.raises(SourceError, match="request failed"):
        list(ApiSource(cfg, client=bad_status).read("id"))
    wrong_list = client_for(lambda request: httpx.Response(200, json={"not": "a list"}))
    with pytest.raises(SourceError, match="JSON array"):
        list(ApiSource(cfg, client=wrong_list).read("id"))
    wrong_row = client_for(lambda request: httpx.Response(200, json=[4]))
    with pytest.raises(SourceError, match="JSON objects"):
        list(ApiSource(cfg, client=wrong_row).read("id"))
    invalid_json = client_for(lambda request: httpx.Response(200, text="not-json"))
    with pytest.raises(SourceError, match="invalid JSON"):
        list(ApiSource(cfg, client=invalid_json).read("id"))


def test_api_retries_exhausted_and_network_failure():
    cfg = SourceSettings(type="api", url="https://api.test/feed", max_retries=1,
                         backoff_seconds=0)
    bad = client_for(lambda request: httpx.Response(503))
    with pytest.raises(SourceError, match="request failed"):
        list(ApiSource(cfg, client=bad, sleeper=lambda _: None).read("id"))

    def disconnected(request):
        raise httpx.ConnectError("offline", request=request)

    offline = client_for(disconnected)
    with pytest.raises(SourceError, match="request failed"):
        list(ApiSource(cfg, client=offline, sleeper=lambda _: None).read("id"))


def test_bad_next_link_loop_wrong_origin_and_max_pages():
    base = dict(type="api", url="https://api.test/feed", records_path="items")
    same = client_for(lambda request: httpx.Response(200, json={
        "items": [], "links": {"next": "/feed"}}))
    cfg = SourceSettings(**base, pagination=PaginationSettings(mode="next"))
    with pytest.raises(SourceError, match="loop"):
        list(ApiSource(cfg, client=same).read("id"))
    other = client_for(lambda request: httpx.Response(200, json={
        "items": [], "links": {"next": "https://other.test/private"}}))
    with pytest.raises(SourceError, match="origin"):
        list(ApiSource(cfg, client=other).read("id"))
    cfg = SourceSettings(**base, pagination=PaginationSettings(mode="page", max_pages=2,
page_size=1))
    endless = client_for(lambda request: httpx.Response(200, json={"items": [{"id":
1}]}))
    with pytest.raises(SourceError, match="max_pages"):
        list(ApiSource(cfg, client=endless).read("id"))


def test_missing_token_next_link_non_string_and_api_url_validation(monkeypatch):
    monkeypatch.delenv("NO_TOKEN_IN_TEST", raising=False)
```

A
B
C
D
E
F
G
H

```
    cfg = SourceSettings(type="api", url="https://api.test/feed",
bearer_token_env="NO_TOKEN_IN_TEST")
    with pytest.raises(SourceError, match="bearer"):
        list(ApiSource(cfg).read("id"))
    cfg = SourceSettings(type="api", url="https://api.test/feed", records_path="items",
                         pagination=PaginationSettings(mode="next"))
    bad = client_for(lambda request: httpx.Response(200, json={
        "links": {"next": 123}, "items": []}))
    with pytest.raises(SourceError, match="next-link"):
        list(ApiSource(cfg, client=bad).read("id"))
    with pytest.raises(ValueError, match="http"):
        ApiSource(SourceSettings(type="api", url="file:///etc/passwd"))
```

tests/test_badge.py

17 source lines.

```
"""CI badge generator does not depend on optional tooling or a live server."""

import importlib.util
from pathlib import Path


def test_badge_thresholds():
    script = Path(__file__).resolve().parents[1] / "scripts" / "build_coverage_badge.py"
    spec = importlib.util.spec_from_file_location("badge", script)
    assert spec is not None and spec.loader is not None
    module = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(module)
    assert module.badge_payload(92.4)["message"] == "92%"
    assert module.badge_payload(92.4)["color"] == "brightgreen"
    assert module.badge_payload(86)["color"] == "green"
    assert module.badge_payload(80)["color"] == "yellow"
    assert module.badge_payload(60)["color"] == "red"
```

tests/test_browser.py

46 source lines.

```
"""Ensure the browser's SQLite demonstration uses the production validation core."""

import json
import csv
import io

from etlguard.browser_demo import validate_csv


def test_browser_customer_feed_and_sqlite(example_dir):
    text = (example_dir / "data" / "customers_messy.csv").read_text()
    result = json.loads(validate_csv(text, "Customer"))
    assert (result["rows_read"], result["rows_valid"], result["rows_rejected"]) == (8, 4,
 3)
    assert result["rows_loaded_sqlite"] == result["rows_valid"]
    assert len(result["rejected_rows"]) == 3
    assert "raw_row_json" in result["rejects_csv"]


def test_browser_transaction_debit_rule(example_dir):
    text = (example_dir / "data" / "transactions_messy.csv").read_text()
    result = json.loads(validate_csv(text, "Transaction"))
    assert result["rows_valid"] == 3
    assert result["rows_rejected"] == 2
    assert result["duplicates_dropped"] == 1


def test_browser_invalid_csv_headers():
    import pytest

    with pytest.raises(ValueError, match="header"):
        validate_csv("", "Customer")
    with pytest.raises(ValueError, match="Unknown"):
        validate_csv("data\n", "Unknown")
    with pytest.raises(ValueError, match="more fields than its header"):
        validate_csv("customer_id,full_name\nC001,Alice,extra\n", "Customer")


def test_browser_reject_download_escapes_formulas_but_preserves_json(example_dir):
```

```
    text = (example_dir / "data" / "customers_messy.csv").read_text()
    text = text.replace("bad-at-email", "=1+1")
    result = json.loads(validate_csv(text, "Customer"))
    raw = next(row for row in result["rejected_rows"] if row["field"] == "email")
    exported = next(row for row in csv.DictReader(io.StringIO(result["rejects_csv"]))
                    if row["field"] == "email")
    assert raw["raw_value"] == "=1+1"
    assert exported["raw_value"] == "'=1+1"
```

tests/test_cleaning.py

76 source lines.

```
from __future__ import annotations

from datetime import date, datetime
from decimal import Decimal

import pytest

from etlguard.cleaning import Cleaner, date_iso, decimal_number, snake_case
from etlguard.config import CleaningSettings


def test_snake_case_camel_spaces_and_hyphens():
    assert snake_case("Customer ID") == "customer_id"
    assert snake_case("SignupDate") == "signup_date"
    assert snake_case(" Credit-Limit (€) ") == "credit_limit"


def test_all_cleaning_steps():
    cleaner = Cleaner(CleaningSettings(
        rename_map={"Client No": "Customer ID"},
        date_columns=["signup_date"], number_columns=["credit_limit"],
        lowercase=["email"], uppercase=["customer_id"],
    ))
    assert cleaner.clean({
        "Client No": "  c001 ", "E-mail": " NA ",
        "Signup Date": "13/02/2026", "Credit Limit": " ₹1,245.00 ",
    }) == {
        "customer_id": "C001", "e_mail": None,
        "signup_date": "2026-02-13", "credit_limit": Decimal("1245.00"),
    }


@pytest.mark.parametrize("token", ["", "   ", "NA", "N/A", "null", "NuLl", "-"])
def test_null_tokens(token):
    assert Cleaner(CleaningSettings()).clean({"Field": token}) == {"field": None}


@pytest.mark.parametrize("value,expected", [
    ("$1,200.90", Decimal("1200.90")),
    ("(1,200.90)", Decimal("-1200.90")),
    ("€2,000", Decimal("2000")),
    ("123", Decimal("123")),
    ("nope", "nope"),
    ("NaN", "NaN"),
])
def test_decimal_normalization(value, expected):
    assert decimal_number(value) == expected


def test_dates_with_errors_and_native_types():
    formats = CleaningSettings().date_formats
    assert date_iso("2026/01/05", formats) == "2026-01-05"
    assert date_iso("05/01/2026", formats) == "2026-01-05"
    assert date_iso("2026.01.05", formats) == "2026-01-05"
    assert date_iso("bad date", formats) == "bad date"
    assert date_iso(date(2026, 1, 5), formats) == "2026-01-05"
    assert date_iso(datetime(2026, 1, 5, 12), formats) == "2026-01-05"


def test_rename_collision_becomes_a_row_reject():
    with pytest.raises(ValueError, match="normalise"):
        Cleaner(CleaningSettings()).clean({"Customer ID": "C001", "customer_id": "C002"})


def test_trim_disabled():
    assert Cleaner(CleaningSettings(trim=False)).clean({"title": " hello "}) == {
        "title": " hello "
    }
```

A B C D E F G H

```
0069
0070
0071  def test_case_conversion_and_idempotence():
0072      cleaner = Cleaner(CleaningSettings(lowercase=["email"], uppercase=["status"]))
0073      raw = {" Email ": " Alice@EXAMPLE.COM ", "Status": " active "}
0074      once = cleaner.clean(raw)
0075      assert once == {"email": "alice@example.com", "status": "ACTIVE"}
0076      assert cleaner.clean(once) == once
```

tests/test_config.py

57 source lines.

```
0001  from __future__ import annotations
0002
0003  import pytest
0004  from pydantic import ValidationError
0005
0006  from etlguard.config import (
0007      ChecksSettings, ForeignKeySettings, RowCountSettings,
0008      SourceSettings, TargetSettings,
0009  )
0010  from etlguard.loaders.mysql import build_tables, sqlalchemy_type
0011  from etlguard.examples.models import Customer
0012
0013
0014  @pytest.mark.parametrize("bad", ["users;DROP TABLE x", "stg_test", "etl_run_audit",
      "bad-name"])
0015  def test_unsafe_table_names_rejected(bad):
0016      with pytest.raises(ValidationError):
0017          TargetSettings(table=bad)
0018
0019
0020  def test_upsert_and_scd2_require_keys():
0021      with pytest.raises(ValidationError):
0022          TargetSettings(table="users", mode="scd2")
0023      with pytest.raises(ValidationError):
0024          TargetSettings(table="users", mode="upsert")
0025      assert TargetSettings(table="users", mode="append").key == []
0026
0027
0028  def test_invalid_config_ranges():
0029      with pytest.raises(ValidationError):
0030          ChecksSettings(max_null_pct={"email": 1.2})
0031      with pytest.raises(ValidationError):
0032          RowCountSettings(min=9, max=2)
0033      with pytest.raises(ValidationError):
0034          SourceSettings(type="csv")
0035      with pytest.raises(ValidationError):
0036          SourceSettings(type="api", url="https://example.com", max_retries=20)
0037      with pytest.raises(ValidationError):
0038          ForeignKeySettings(values=["1"], path="file.csv", column="id")
0039
0040
0041  def test_build_tables_contain_target_and_staging_metadata():
0042      target = TargetSettings(table="customers", mode="scd2", key=["customer_id"])
0043      dim, stage = build_tables(Customer, target)
0044      assert dim.name == "customers"
0045      assert stage.name == "stg_customers"
0046      assert "effective_from" in dim.c
0047      assert "__etl_batch_id" in stage.c
0048
0049
0050  def test_model_target_columns_exist():
0051      with pytest.raises(ValueError, match="must exist"):
0052          build_tables(Customer, TargetSettings(table="bad_model", mode="upsert",
      key=["absent"]))
0053
0054
0055  def test_sqlalchemy_types_unsupported():
0056      with pytest.raises(ValueError, match="Unsupported"):
0057          sqlalchemy_type(list[str])
```

tests/test_coverage_edges.py

57 source lines.

```python
"""Edge cases for API contract, model typing and dataset checks."""

from __future__ import annotations

from decimal import Decimal


from etlguard.cleaning import Cleaner
from etlguard.config import ChecksSettings, CleaningSettings, ForeignKeySettings
from etlguard.core import ValidationEngine
from etlguard.examples.models import Customer
from etlguard.loaders.mysql import batches, sqlalchemy_type
from etlguard.validation import RowValidator


def test_batching_splits_work():
    assert batches([1, 2, 3, 4, 5], 2) == [[1, 2], [3, 4], [5]]
    assert batches([], 2) == []


def test_decimal_types():
    assert sqlalchemy_type(Decimal).scale == 2
    assert sqlalchemy_type(str | None).length == 255
    assert "DATETIME" in str(sqlalchemy_type(__import__("datetime").datetime)).upper()


def test_dataset_reference_values_are_strings():
    checks = ChecksSettings(foreign_keys={"customer_id": ForeignKeySettings(values=[1,
2])})
    from etlguard.checks import DatasetChecker

    checker = DatasetChecker(checks, __import__("pathlib").Path("."))
    assert checker.references["customer_id"] == {"1", "2"}


def test_core_no_duplicate_drop_can_fail_uniqueness(record_factory, customer_raw):
    first = record_factory(customer_raw, 2)
    second = record_factory(customer_raw, 3)
    engine = ValidationEngine(Customer, CleaningSettings(drop_duplicates=False),
                              ChecksSettings(unique=["customer_id"]))
    result = engine.run([first, second])
    assert result.rows_valid == 1
    assert result.rows_rejected == 1
    assert result.duplicates_dropped == 0


def test_raw_unsupported_type_preserved_as_error(record_factory, customer_raw):
    raw = {**customer_raw, "credit_limit": {"unexpected": "object"}}
    rec = record_factory(raw)
    outcome = RowValidator(Customer).validate(raw, raw, rec.lineage)
    assert outcome.candidate is None
    assert outcome.rejects[0].field == "credit_limit"


def test_cleaner_custom_null_token():
    cleaner = Cleaner(CleaningSettings(null_tokens=["MISSING"]))
    assert cleaner.clean({"name": "Missing"})["name"] is None
    assert cleaner.clean({"name": "NA"})["name"] == "NA"
```

tests/test_mysql_ddl_compile.py

38 source lines.

```python
"""Compile critical MySQL DDL offline; NOT a substitute for MySQL 8 execution."""
from sqlalchemy.schema import CreateTable
from sqlalchemy.dialects.mysql import dialect

from etlguard.audit import audit_table, commits_table
from etlguard.config import TargetSettings
from etlguard.examples.models import Customer
from etlguard.loaders.mysql import build_tables


def test_scd2_generated_active_marker_compiles_for_mysql8():
    target, stage = build_tables(
        Customer, TargetSettings(table="pilot_customer", mode="scd2",
key=["customer_id"])
    )
    ddl = str(CreateTable(target).compile(dialect=dialect()))
    assert "__etl_current_marker SMALLINT GENERATED ALWAYS AS" in ddl
    assert "UNIQUE (customer_id, __etl_current_marker)" in ddl
```

A
B
C
D
E
F
G
H

```
0018      assert "ENGINE=InnoDB" in ddl
0019      assert "DATETIME(6)" in ddl
0020      assert "__etl_batch_id VARCHAR(36) NOT NULL" in str(
0021          CreateTable(stage).compile(dialect=dialect())
0022      )
0023
0024
0025  def test_upsert_requires_mysql_unique_business_key_ddl():
0026      target, _stage = build_tables(
0027          Customer, TargetSettings(table="pilot_customer", mode="upsert",
      key=["customer_id"])
0028      )
0029      ddl = str(CreateTable(target).compile(dialect=dialect()))
0030      assert "UNIQUE (customer_id)" in ddl
0031      assert "ENGINE=InnoDB" in ddl
0032
0033
0034  def test_audit_and_commit_ledger_require_innodb():
0035      for table in (audit_table(), commits_table()):
0036          ddl = str(CreateTable(table).compile(dialect=dialect()))
0037          assert "ENGINE=InnoDB" in ddl
0038          assert "batch_id VARCHAR(36) NOT NULL" in ddl
```

tests/test_mysql_integration.py

345 source lines.

```
0001  """Real MySQL 8 integration tests; CI provisions a service container."""
0002
0003  from __future__ import annotations
0004
0005  import os
0006  from datetime import datetime, timezone
0007  from uuid import uuid4
0008
0009  import pytest
0010  from sqlalchemy import MetaData, Table, create_engine, select
0011
0012  from etlguard.audit import AuditWriter
0013  from etlguard.config import TargetSettings
0014  from etlguard.examples.models import Customer
0015  from etlguard.loaders.mysql import MysqlLoader
0016  from etlguard.pipeline import Pipeline, RunResult
0017  from etlguard.reconcile import LoadCounts, ReconciliationError
0018  from etlguard.sources.base import Lineage
0019  from etlguard.validation import Candidate
0020
0021  pytestmark = pytest.mark.integration
0022
0023
0024  @pytest.fixture
0025  def mysql_engine():
0026      url = os.getenv("ETLGUARD_TEST_MYSQL_URL")
0027      if not url:
0028          pytest.skip("Set ETLGUARD_TEST_MYSQL_URL to run MySQL 8 integration tests")
0029      engine = create_engine(url, pool_pre_ping=True)
0030      yield engine
0031      engine.dispose()
0032
0033
0034  @pytest.fixture
0035  def table_name(mysql_engine):
0036      name = "it_" + uuid4().hex[:16]
0037      yield name
0038      metadata = MetaData()
0039      with mysql_engine.begin() as conn:
0040          for table_name in [f"stg_{name}", name]:
0041              table = Table(table_name, metadata, autoload_with=conn)
0042              table.drop(conn, checkfirst=True)
0043
0044
0045  def candidate(pk="C001", name="Alice Doe", limit="100.00", row_number=2):
0046      item = Customer.model_validate({
0047          "customer_id": pk, "full_name": name, "email": f"{pk.lower()}@example.com",
0048          "signup_date": "2026-01-01", "status": "active", "credit_limit": limit,
0049      })
0050      return Candidate(
0051          data=item.model_dump(mode="python"), cleaned=item.model_dump(mode="python"),
      raw={},
0052          lineage=Lineage("integration", row_number, datetime.now(timezone.utc), "test"),
0053      )
```

A B C D E F G H

```
0054
0055
0056  def target_rows(engine, table_name):
0057      table = Table(table_name, MetaData(), autoload_with=engine)
0058      with engine.connect() as conn:
0059          return conn.execute(select(table)).mappings().all()
0060
0061
0062  def test_upsert_insert_and_update(mysql_engine, table_name):
0063      loader = MysqlLoader(engine=mysql_engine)
0064      target = TargetSettings(table=table_name, mode="upsert", key=["customer_id"],
      batch_size=1)
0065      first = loader.load([candidate("C001"), candidate("C002")], str(uuid4()), Customer,
      target)
0066      assert first == LoadCounts(2, 0)
0067      second = loader.load([candidate("C001", "Updated Name"), candidate("C003")],
0068                           str(uuid4()), Customer, target)
0069      assert second == LoadCounts(1, 1)
0070      rows = target_rows(mysql_engine, table_name)
0071      assert len(rows) == 3
0072      assert next(row for row in rows if row["customer_id"] == "C001")["full_name"] ==
      "Updated Name"
0073
0074
0075  def test_scd2_tracks_history_and_unchanged(mysql_engine, table_name):
0076      loader = MysqlLoader(engine=mysql_engine)
0077      target = TargetSettings(table=table_name, mode="scd2", key=["customer_id"],
0078                              tracked_columns=["full_name"], batch_size=1)
0079      assert loader.load([candidate()], str(uuid4()), Customer, target).inserted == 1
0080      update = loader.load([candidate(name="Alice Updated")], str(uuid4()), Customer,
      target)
0081      assert (update.updated, update.versions_created) == (1, 1)
0082      unchanged = loader.load([candidate(name="Alice Updated")], str(uuid4()), Customer,
      target)
0083      assert (unchanged.updated, unchanged.unchanged) == (1, 1)
0084      rows = target_rows(mysql_engine, table_name)
0085      assert len(rows) == 2
0086      assert len([row for row in rows if row["is_current"]]) == 1
0087      old = next(row for row in rows if not row["is_current"])
0088      assert old["effective_to"] is not None
0089      current = next(row for row in rows if row["is_current"])
0090      assert current["effective_to"] is None
0091      assert old["effective_to"] <= current["effective_from"]
0092
0093
0094  def test_truncate_insert_is_transactional_delete(mysql_engine, table_name):
0095      loader = MysqlLoader(engine=mysql_engine)
0096      target = TargetSettings(table=table_name, mode="truncate_insert")
0097      loader.load([candidate("C001"), candidate("C002")], str(uuid4()), Customer, target)
0098      assert len(target_rows(mysql_engine, table_name)) == 2
0099      loader.load([candidate("C003")], str(uuid4()), Customer, target)
0100      assert [row["customer_id"] for row in target_rows(mysql_engine, table_name)] ==
      ["C003"]
0101
0102
0103  def test_append_inserts_and_reconciles(mysql_engine, table_name):
0104      loader = MysqlLoader(engine=mysql_engine)
0105      target = TargetSettings(table=table_name, mode="append")
0106      assert loader.load([candidate()], str(uuid4()), Customer, target).loaded == 1
0107      assert loader.load([candidate("C002")], str(uuid4()), Customer, target).loaded == 1
0108      assert len(target_rows(mysql_engine, table_name)) == 2
0109
0110
0111  def test_reconciliation_mismatch_rolls_back_stage_and_target(mysql_engine, table_name,
      monkeypatch):
0112      loader = MysqlLoader(engine=mysql_engine)
0113      target = TargetSettings(table=table_name, mode="append")
0114      original = loader._merge
0115
0116      def faulty_merge(conn, table, records, config):
0117          original(conn, table, records, config)
0118          return LoadCounts(0, 0)  # Deliberate mismatch after target writes.
0119
0120      monkeypatch.setattr(loader, "_merge", faulty_merge)
0121      with pytest.raises(ReconciliationError):
0122          loader.load([candidate()], str(uuid4()), Customer, target)
0123      assert target_rows(mysql_engine, table_name) == []
0124      assert target_rows(mysql_engine, f"stg_{table_name}") == []
0125
0126
0127  def test_transaction_rollback_on_merge_failure(mysql_engine, table_name, monkeypatch):
0128      loader = MysqlLoader(engine=mysql_engine)
0129      target = TargetSettings(table=table_name, mode="append")
0130      original = loader._merge
0131
```

```
0132      def broken_merge(conn, table, records, config):
0133          original(conn, table, records, config)
0134          raise RuntimeError("injected crash after insert")
0135
0136      monkeypatch.setattr(loader, "_merge", broken_merge)
0137      with pytest.raises(RuntimeError, match="injected"):
0138          loader.load([candidate()], str(uuid4()), Customer, target)
0139      assert target_rows(mysql_engine, table_name) == []
0140      assert target_rows(mysql_engine, f"stg_{table_name}") == []
0141
0142
0143  def test_audit_table_rows_persist(mysql_engine):
0144      audit = AuditWriter(mysql_engine)
0145      audit.init_db()
0146      batch_id = str(uuid4())
0147      now = datetime.now(timezone.utc)
0148      result = RunResult(
0149          batch_id=batch_id, pipeline="integration", source="synthetic.csv",
0150          started_at=now, finished_at=now, status="success",
0151          rows_read=2, rows_valid=2, rows_rejected=0, rows_loaded=2,
0152      )
0153      audit.write(result, "synthetic.csv")
0154      with mysql_engine.connect() as conn:
0155          row = conn.execute(select(audit.table).where(audit.table.c.batch_id ==
      batch_id)).first()
0156      assert row is not None
0157      assert row.status == "success"
0158      assert row.rows_loaded == 2
0159
0160
0161  def test_pipeline_creates_success_audit(mysql_engine, table_name, monkeypatch, tmp_path):
0162      import yaml
0163
0164      url = os.environ["ETLGUARD_TEST_MYSQL_URL"]
0165      monkeypatch.setenv("ETLGUARD_DATABASE_URL", url)
0166      (tmp_path / "input.csv").write_text(
0167          "customer_id,full_name,email,signup_date,status,credit_limit\n"
0168          "C001,Alice Doe,alice@example.com,2026-01-01,active,100\n",
0169          encoding="utf-8",
0170      )
0171      config = {
0172          "pipeline": "integration_job",
0173          "source": {"type": "csv", "path": "input.csv"},
0174          "model": "etlguard.examples.models:Customer",
0175          "cleaning": {"date_columns": ["signup_date"], "number_columns":
      ["credit_limit"]},
0176          "target": {"table": table_name, "mode": "upsert", "key": ["customer_id"]},
0177      }
0178      file = tmp_path / "pipeline.yaml"
0179      file.write_text(yaml.safe_dump(config))
0180      result = Pipeline.from_yaml(file).run()
0181      assert result.status == "success", result.failures
0182      assert result.rows_loaded == 1
0183      audit = AuditWriter(mysql_engine)
0184      with mysql_engine.connect() as conn:
0185          entry = conn.execute(select(audit.table).where(audit.table.c.batch_id ==
      result.batch_id)).first()
0186      assert entry.status == "success"
0187
0188
0189  def test_atomic_batch_commit_and_append_retry_on_mysql8(mysql_engine, table_name):
0190      from etlguard.audit import BatchConflictError
0191
0192      loader = MysqlLoader(engine=mysql_engine)
0193      target = TargetSettings(table=table_name, mode="append")
0194      batch_id = str(uuid4())
0195      first = loader.load([candidate()], batch_id, Customer, target)
0196      replay = loader.load([candidate()], batch_id, Customer, target)
0197      assert first.loaded == replay.loaded == 1
0198      assert replay.replayed
0199      assert len(target_rows(mysql_engine, table_name)) == 1
0200      audit = AuditWriter(mysql_engine)
0201      with mysql_engine.connect() as conn:
0202          entry = conn.execute(select(audit.table).where(audit.table.c.batch_id ==
      batch_id)).one()
0203          commit = conn.execute(select(audit.commits).where(
0204              audit.commits.c.batch_id == batch_id
0205          )).one()
0206      assert entry.status == "success" and commit.rows_inserted == 1
0207      with pytest.raises(BatchConflictError):
0208          loader.load([candidate(name="changed")], batch_id, Customer, target)
0209      assert len(target_rows(mysql_engine, table_name)) == 1
0210
0211
0212  def test_mysql8_audit_failure_rolls_back_data_and_ledger(mysql_engine, table_name,
```

A
B
C
D
E
F
G
H

```
    monkeypatch):
0213        loader = MysqlLoader(engine=mysql_engine)
0214        target = TargetSettings(table=table_name, mode="append")
0215        batch = str(uuid4())
0216        original = AuditWriter.record_success
0217
0218        def fail_audit(self, conn, result, source, counts):
0219            original(self, conn, result, source, counts)
0220            raise RuntimeError("mysql8 injected audit failure")
0221
0222        with monkeypatch.context() as patch:
0223            patch.setattr(AuditWriter, "record_success", fail_audit)
0224            with pytest.raises(RuntimeError, match="injected audit"):
0225                loader.load([candidate()], batch, Customer, target)
0226        assert target_rows(mysql_engine, table_name) == []
0227        audit = AuditWriter(mysql_engine)
0228        with mysql_engine.connect() as conn:
0229            assert conn.execute(select(audit.commits).where(
0230                audit.commits.c.batch_id == batch
0231            )).first() is None
0232            assert conn.execute(select(audit.table).where(
0233                audit.table.c.batch_id == batch
0234            )).first() is None
0235
0236
0237    def test_mysql8_concurrent_scd2_updates_have_one_current(mysql_engine, table_name):
0238        from concurrent.futures import ThreadPoolExecutor
0239        from threading import Barrier
0240
0241        loader = MysqlLoader(engine=mysql_engine)
0242        target = TargetSettings(table=table_name, mode="scd2", key=["customer_id"],
0243                                tracked_columns=["full_name"], lock_timeout_seconds=30)
0244        loader.load([candidate()], str(uuid4()), Customer, target)
0245        start = Barrier(2)
0246
0247        def worker(name):
0248            start.wait(timeout=20)
0249            return loader.load([candidate(name=name)], str(uuid4()), Customer, target)
0250
0251        with ThreadPoolExecutor(max_workers=2) as executor:
0252            results = list(executor.map(worker, ["Alice Second", "Alice Third"]))
0253        assert all(count.loaded == 1 for count in results)
0254        assert sum(count.versions_created for count in results) == 2
0255        all_versions = target_rows(mysql_engine, table_name)
0256        assert len(all_versions) == 3
0257        assert sum(bool(row["is_current"]) for row in all_versions) == 1
0258
0259
0260    def test_mysql8_scd2_unique_active_index_blocks_external_duplicate(mysql_engine,
    table_name):
0261        from sqlalchemy.exc import IntegrityError
0262
0263        loader = MysqlLoader(engine=mysql_engine)
0264        target = TargetSettings(table=table_name, mode="scd2", key=["customer_id"])
0265        loader.load([candidate()], str(uuid4()), Customer, target)
0266        table = Table(table_name, MetaData(), autoload_with=mysql_engine)
0267        with pytest.raises(IntegrityError):
0268            with mysql_engine.begin() as conn:
0269                conn.execute(table.insert().values(
0270                    **candidate(name="Duplicate").data,
0271                    __etl_id=str(uuid4()),
    effective_from=datetime.now(timezone.utc).replace(tzinfo=None),
0272                    effective_to=None, is_current=True,
0273                ))
0274        assert len(target_rows(mysql_engine, table_name)) == 1
0275
0276
0277    def test_mysql8_upgrade_legacy_scd2_requires_explicit_migration(mysql_engine,
    table_name):
0278        from sqlalchemy import Boolean, Column, String
0279        from sqlalchemy.dialects.mysql import DATETIME
0280        from etlguard.loaders.mysql import _model_columns
0281
0282        meta = MetaData()
0283        Table(table_name, meta,
0284                      Column("__etl_id", String(36), primary_key=True),
    *_model_columns(Customer),
0285                      Column("effective_from", DATETIME(fsp=6), nullable=False),
0286                      Column("effective_to", DATETIME(fsp=6)),
0287                      Column("is_current", Boolean(), nullable=False),
    mysql_engine="InnoDB")
0288        meta.create_all(mysql_engine)
0289        loader = MysqlLoader(engine=mysql_engine)
0290        target = TargetSettings(table=table_name, mode="scd2", key=["customer_id"])
0291        with pytest.raises(ValueError, match="UNIQUE active-row guard"):
```

```
        loader.load([candidate()], str(uuid4()), Customer, target)
    loader.upgrade_scd2(Customer, target)
    assert loader.load([candidate()], str(uuid4()), Customer, target).inserted == 1
    assert len(target_rows(mysql_engine, table_name)) == 1


def test_distinct_target_writers_do_not_deadlock_shared_metadata(mysql_engine):
    """Synchronize missing metadata reads to expose REPEATABLE READ gap locks."""
    from concurrent.futures import ThreadPoolExecutor
    from threading import Barrier, local
    from sqlalchemy import event
    from etlguard.loaders.mysql import build_tables

    audit = AuditWriter(mysql_engine)
    audit.init_db()
    targets = [TargetSettings(table='it_parallel_' + uuid4().hex[:10], mode='append')
               for _ in range(3)]
    for target in targets:
        table, _ = build_tables(Customer, target)
        table.metadata.create_all(mysql_engine)
    from uuid import UUID

    base_id = uuid4().int & ~0xffff
    batch_ids = [str(UUID(int=base_id + i)) for i in range(3)]
    ledger_barrier = Barrier(3)
    audit_barrier = Barrier(3)
    state = local()

    def synchronize(conn, cursor, statement, params, context, many):
        if statement.startswith('SELECT etl_batch_commits.') and 'WHERE' in statement:
            if not getattr(state, 'ledger_seen', False):
                state.ledger_seen = True
                ledger_barrier.wait(timeout=10)
        elif statement.startswith('SELECT etl_run_audit.') and 'WHERE' in statement:
            if not getattr(state, 'audit_seen', False):
                state.audit_seen = True
                audit_barrier.wait(timeout=10)

    event.listen(mysql_engine, 'after_cursor_execute', synchronize)
    try:
        def worker(item):
            target, batch_id = item
            return MysqlLoader(engine=mysql_engine).load(
                [candidate()], batch_id, Customer, target)
        with ThreadPoolExecutor(max_workers=3) as executor:
            results = list(executor.map(worker, zip(targets, batch_ids, strict=True)))
        assert all(result.loaded == 1 for result in results)
        assert all(len(target_rows(mysql_engine, target.table)) == 1 for target in
targets)
    finally:
        event.remove(mysql_engine, 'after_cursor_execute', synchronize)
        with mysql_engine.begin() as conn:
            for target in targets:
                for name in [f'stg_{target.table}', target.table]:
                    Table(name, MetaData(), autoload_with=conn).drop(conn)
```

tests/test_pipeline.py

171 source lines.

```
from __future__ import annotations

import csv
import json
from pathlib import Path

import pytest
import yaml
from typer.testing import CliRunner

from etlguard.cli import app
from etlguard.config import ConfigError, load_config
from etlguard.pipeline import Pipeline, RunResult, resolve_model
from etlguard.profile import profile_csv
from etlguard.reconcile import LoadCounts, ReconciliationError, reconcile
from etlguard.rejects import RejectWriter, RejectedRow

RUNNER = CliRunner()


def write_pipeline(tmp_path: Path, csv_body: str, overrides=None) -> Path:
```

```
    (tmp_path / "input.csv").write_text(csv_body, encoding="utf-8")
    settings = {
        "pipeline": "test_fixture",
        "source": {"type": "csv", "path": "input.csv"},
        "model": "etlguard.examples.models:Customer",
        "cleaning": {"date_columns": ["signup_date", "end_date"], "number_columns":
["credit_limit"]},
        "checks": {"unique": ["customer_id"]},
        "target": {"table": "test_fixture", "mode": "upsert", "key": ["customer_id"]},
        "policy": {"fail_if_reject_rate_above": 0.05},
    }
    if overrides:
        settings.update(overrides)
    config_file = tmp_path / "pipeline.yaml"
    config_file.write_text(yaml.safe_dump(settings), encoding="utf-8")
    return config_file


VALID_CSV = (
    "customer_id,full_name,email,signup_date,end_date,status,credit_limit\n"
    "C001,Alice Doe,alice@example.com,2026-01-01,,active,1200.00\n"
)
INVALID_CSV = VALID_CSV + "C002,Invalid,email-bad,not-a-date,,active,-20\n"


def test_python_api_returns_typed_result_and_rejects(tmp_path):
    path = write_pipeline(tmp_path, INVALID_CSV)
    result = Pipeline.from_yaml(path).run(dry_run=True)
    assert isinstance(result, RunResult)
    assert result.status == "failed"
    assert result.rows_read == 2
    assert result.rows_valid == 1
    assert result.rows_rejected == 1
    assert result.rows_loaded == 0
    assert result.error_counts_by_field == {"credit_limit": 1, "email": 1, "signup_date":
 1}
    rows = list(csv.DictReader(Path(result.reject_csv).open(encoding="utf-8")))
    assert len(rows) == 3
    assert rows[0]["batch_id"] == result.batch_id
    assert rows[0]["raw_row_json"]
    assert json.loads(Path(result.summary_json).read_text())["rows_read"] == 2


def test_good_dry_run_no_database_writes(monkeypatch, tmp_path):
    monkeypatch.delenv("ETLGUARD_DATABASE_URL", raising=False)
    config = write_pipeline(tmp_path, VALID_CSV)
    result = Pipeline.from_yaml(config).run(dry_run=True)
    assert result.status == "dry_run"
    assert result.ok
    assert result.rows_valid == 1
    assert result.rows_loaded == 0
    assert Path(result.reject_csv).read_text().splitlines()[0].startswith("batch_id,")
    assert json.loads(Path(result.reject_json).read_text()) == []


def test_dataset_failure_causes_nonzero(tmp_path):
    config = write_pipeline(tmp_path, VALID_CSV, overrides={
        "checks": {"expected_row_count": {"min": 2}}
    })
    result = Pipeline.from_yaml(config).run(dry_run=True)
    assert result.status == "failed"
    assert result.rows_rejected == 0
    assert result.dataset_issues


def test_cli_validate_exit_codes_and_report(tmp_path):
    valid = write_pipeline(tmp_path, VALID_CSV)
    assert RUNNER.invoke(app, ["validate", str(valid)]).exit_code == 0
    invalid = write_pipeline(tmp_path, INVALID_CSV)
    failed = RUNNER.invoke(app, ["validate", str(invalid)])
    assert failed.exit_code == 1
    assert "Rejected" in failed.output


def test_cli_config_error_is_2(tmp_path):
    file = tmp_path / "bad.yaml"
    file.write_text("source: {type: bogus}")
    assert RUNNER.invoke(app, ["validate", str(file)]).exit_code == 2
    assert RUNNER.invoke(app, ["validate", str(tmp_path / "missing.yaml")]).exit_code ==
2
    assert RUNNER.invoke(app, ["run", str(file)]).exit_code == 2


def test_cli_run_missing_environment_is_config_2(tmp_path, monkeypatch):
    monkeypatch.delenv("ETLGUARD_DATABASE_URL", raising=False)
```

```
    config = write_pipeline(tmp_path, VALID_CSV)
    result = RUNNER.invoke(app, ["run", str(config)])
    assert result.exit_code == 2


def test_cli_profile(tmp_path):
    input_file = tmp_path / "profile.csv"
    input_file.write_text("amount,email\n1,a@example.com\n2,NA\n3,b@example.com\n")
    data = profile_csv(input_file)
    assert data[0]["inferred_type"] == "number"
    assert data[0]["distinct"] == 3
    assert data[1]["null_pct"] == pytest.approx(1 / 3, abs=0.0001)
    output = RUNNER.invoke(app, ["profile", str(input_file)])
    assert output.exit_code == 0
    assert "amount" in output.output


def test_resolve_model_invalid_and_config_validation(tmp_path):
    with pytest.raises(ConfigError):
        resolve_model("wrong")
    with pytest.raises(ConfigError):
        resolve_model("etlguard.examples.models:NOTAMODEL")
    file = write_pipeline(tmp_path, VALID_CSV, overrides={"unknown": 123})
    with pytest.raises(ConfigError, match="Invalid config"):
        load_config(file)


def test_empty_config_yaml_rejected(tmp_path):
    file = tmp_path / "empty.yaml"
    file.write_text("\n")
    with pytest.raises(ConfigError):
        Pipeline.from_yaml(file)


def test_reconciliation_counts():
    reconcile(3, LoadCounts(2, 1))
    with pytest.raises(ReconciliationError, match="mismatch"):
        reconcile(3, LoadCounts(1, 1))


def test_reject_writer(tmp_path):
    rows = [RejectedRow(
        batch_id="batch", source_row_number=3, field="amount",
error_type="decimal_parsing",
        error_message="bad amount", raw_value="abc", raw_row_json='{"amount": "abc"}',
    )]
    csv_path, json_path = RejectWriter().write(tmp_path / "report", rows)
    assert csv_path.exists() and json_path.exists()
    assert list(csv.DictReader(csv_path.open(encoding="utf-8")))[0]["raw_value"] == "abc"
    assert json.loads(json_path.read_text())[0]["field"] == "amount"


def test_sample_messy_feed_has_expected_counts(example_dir):
    path = example_dir / "pipelines" / "customers.yaml"
    result = Pipeline.from_yaml(path).run(dry_run=True)
    assert (result.rows_read, result.rows_valid, result.rows_rejected,
result.duplicates_dropped) == (
        8, 4, 3, 1,
    )
    assert result.status == "failed"  # 37.50% exceeds the example 5% policy.


def test_reject_csv_escapes_spreadsheet_formulas_but_json_keeps_raw(tmp_path):
    row = RejectedRow(
        batch_id="batch", source_row_number=2, field="amount", error_type="invalid",
        error_message="Unsafe", raw_value="=1+1", raw_row_json='{"amount": "=1+1"}',
    )
    csv_file, json_file = RejectWriter().write(tmp_path / "formulas", [row])
    assert next(csv.DictReader(csv_file.open(encoding="utf-8")))["raw_value"] == "'=1+1"
    assert json.loads(json_file.read_text())[0]["raw_value"] == "=1+1"
```

tests/test_production_safety.py

354 source lines.

```
"""Regression checks for atomic auditing, retry safety and SCD2 uniqueness."""

from __future__ import annotations

from contextlib import contextmanager
from datetime import datetime, timezone
```

A B C D E F G H

```
0007  from uuid import uuid4
0008
0009  import pytest
0010  from sqlalchemy import (Boolean, Column, DateTime, MetaData, String, Table,
0011                          create_engine, select)
0012  from sqlalchemy.exc import IntegrityError
0013
0014  from etlguard.audit import AuditWriter, BatchConflictError
0015  from etlguard.config import TargetSettings
0016  from etlguard.examples.models import Customer
0017  from etlguard.loaders.mysql import MysqlLoader, _model_columns
0018  from etlguard.pipeline import RunResult
0019  from etlguard.sources.base import Lineage
0020  from etlguard.validation import Candidate
0021
0022
0023  def item(pk="C001", name="Alice"):
0024      record = Customer.model_validate({
0025          "customer_id": pk, "full_name": name, "email": f"{pk}@example.com",
0026          "signup_date": "2026-01-01", "status": "active", "credit_limit": 100,
0027      })
0028      return Candidate(data=record.model_dump(mode="python"), cleaned={}, raw={},
0029                       lineage=Lineage("csv", 2, datetime.now(timezone.utc), "source"))
0030
0031
0032  @pytest.fixture
0033  def loader():
0034      engine = create_engine("sqlite:///:memory:")
0035      try:
0036          yield MysqlLoader(engine=engine)
0037      finally:
0038          engine.dispose()
0039
0040
0041  def table_rows(loader, name):
0042      table = Table(name, MetaData(), autoload_with=loader.engine)
0043      with loader.engine.connect() as conn:
0044          return conn.execute(select(table)).mappings().all()
0045
0046
0047  def test_direct_append_is_idempotent_and_audited(loader):
0048      target = TargetSettings(table="safe_append", mode="append")
0049      batch_id = str(uuid4())
0050      first = loader.load([item()], batch_id, Customer, target)
0051      second = loader.load([item()], batch_id, Customer, target)
0052      assert first.loaded == second.loaded == 1
0053      assert not first.replayed and second.replayed
0054      assert len(table_rows(loader, target.table)) == 1
0055      assert len(table_rows(loader, "etl_batch_commits")) == 1
0056      assert table_rows(loader, "etl_run_audit")[0]["status"] == "success"
0057
0058
0059  def test_batch_conflict_rejects_changed_input(loader):
0060      target = TargetSettings(table="safe_conflict", mode="append")
0061      batch_id = str(uuid4())
0062      loader.load([item()], batch_id, Customer, target)
0063      with pytest.raises(BatchConflictError, match="different input"):
0064          loader.load([item(name="Mallory")], batch_id, Customer, target)
0065      assert table_rows(loader, target.table)[0]["full_name"] == "Alice"
0066
0067
0068  def test_audit_exception_rolls_back_target_and_ledger(loader, monkeypatch):
0069      batch = str(uuid4())
0070      target = TargetSettings(table="safe_audit_failure", mode="append")
0071      original = AuditWriter.record_success
0072
0073      def outage(self, conn, result, source, counts):
0074          original(self, conn, result, source, counts)
0075          raise RuntimeError("simulated audit failure before commit")
0076
0077      with monkeypatch.context() as patch:
0078          patch.setattr(AuditWriter, "record_success", outage)
0079          with pytest.raises(RuntimeError, match="audit failure"):
0080              loader.load([item()], batch, Customer, target)
0081      assert table_rows(loader, target.table) == []
0082      assert table_rows(loader, "etl_batch_commits") == []
0083      assert table_rows(loader, "etl_run_audit") == []
0084      assert loader.load([item()], batch, Customer, target).loaded == 1
0085
0086
0087  def test_lost_commit_response_is_recovered_from_durable_record(loader, monkeypatch):
0088      target = TargetSettings(table="safe_lost_reply", mode="append")
0089      original = loader._locked_connection
0090
0091      @contextmanager
```

```
    def lose_reply(config):
        with original(config) as conn:
            yield conn
        raise ConnectionError("response lost after commit")

    monkeypatch.setattr(loader, "_locked_connection", lose_reply)
    result = loader.load([item()], str(uuid4()), Customer, target)
    assert result.loaded == 1 and result.replayed
    assert len(table_rows(loader, target.table)) == 1
    assert table_rows(loader, "etl_run_audit")[0]["status"] == "success"


def test_failure_audit_then_success_same_id(loader):
    batch_id = str(uuid4())
    now = datetime.now(timezone.utc)
    run = RunResult(batch_id=batch_id, pipeline="direct_api", source="direct_api",
                    started_at=now, finished_at=now, status="failed")
    auditor = AuditWriter(loader.engine)
    auditor.record_failure(run, run.source)
    assert table_rows(loader, "etl_run_audit")[0]["status"] == "failed"
    loader.load([item()], batch_id, Customer, TargetSettings(table="recovered",
mode="append"))
    assert table_rows(loader, "etl_run_audit")[0]["status"] == "success"
    run.status = "failed"
    auditor.record_failure(run, run.source)
    assert table_rows(loader, "etl_run_audit")[0]["status"] == "success"


def test_safe_replace_minimum_size_enforced(loader):
    batch_id = str(uuid4())
    loader.load([item("C001"), item("C002")], batch_id, Customer,
                TargetSettings(table="safe_replace", mode="append"))
    target = TargetSettings(table="safe_replace", mode="truncate_insert",
                            min_replace_ratio=0.75)
    with pytest.raises(ValueError, match="min_replace_ratio"):
        loader.load([item("C003")], str(uuid4()), Customer, target)
    assert len(table_rows(loader, "safe_replace")) == 2
    assert len(table_rows(loader, "etl_batch_commits")) == 1


def test_generated_scd2_active_unique_constraint(loader):
    target = TargetSettings(table="safe_scd2", mode="scd2", key=["customer_id"],
                            tracked_columns=["full_name"])
    loader.load([item()], str(uuid4()), Customer, target)
    table = Table(target.table, MetaData(), autoload_with=loader.engine)
    assert "__etl_current_marker" in table.c
    assert table.c["__etl_current_marker"].computed is not None
    with pytest.raises(IntegrityError):
        with loader.engine.begin() as conn:
            conn.execute(table.insert().values(
                __etl_id=str(uuid4()), **item(name="Bob").data,
                effective_from=datetime.now(timezone.utc).replace(tzinfo=None),
                effective_to=None, is_current=True,
            ))
    assert len(table_rows(loader, target.table)) == 1


def test_legacy_scd2_without_db_guard_fails_closed(loader):
    target = TargetSettings(table="safe_old_scd2", mode="scd2", key=["customer_id"])
    meta = MetaData()
    Table(target.table, meta,
          Column("__etl_id", String(36), primary_key=True), *_model_columns(Customer),
          Column("effective_from", DateTime(), nullable=False),
          Column("effective_to", DateTime()), Column("is_current", Boolean(),
nullable=False))
    meta.create_all(loader.engine)
    with pytest.raises(ValueError, match="UNIQUE active-row guard"):
        loader.load([item()], str(uuid4()), Customer, target)
    assert table_rows(loader, target.table) == []


def test_invalid_batch_id_does_not_write(loader):
    with pytest.raises(ValueError, match="canonical UUID"):
        loader.load([item()], "some id", Customer, TargetSettings(table="unsafe",
mode="append"))


def test_partial_source_failure_preserves_counts_without_loading():
    from etlguard.config import ChecksSettings, CleaningSettings
    from etlguard.core import ValidationEngine
    from etlguard.sources.base import SourceRecord

    def partial_source():
        yield SourceRecord(data=item().data, lineage=Lineage(
            "csv", 2, datetime.now(timezone.utc), "partial"
```

```
        ))
        raise OSError("midstream feed disconnect")

    outcome = ValidationEngine(Customer, CleaningSettings(),
ChecksSettings()).run(partial_source())
    assert (outcome.rows_read, outcome.rows_valid, outcome.rows_rejected) == (1, 1, 0)
    assert "midstream feed disconnect" in outcome.source_error


def test_full_pipeline_replays_identical_batch_and_preserves_success_audit(
    tmp_path, monkeypatch,
):
    import yaml
    from etlguard.pipeline import Pipeline

    db_engine = create_engine(f"sqlite:///{tmp_path / 'test.sqlite'}")
    from etlguard.loaders.mysql import MysqlLoader as RealLoader

    def sqlite_init(self, database_url=None, engine=None):
        self.engine = db_engine

    monkeypatch.setattr(RealLoader, "__init__", sqlite_init)
    monkeypatch.setenv("ETLGUARD_DATABASE_URL", "mysql+pymysql://not-used")
    file = tmp_path / "feed.csv"
    file.write_text(
        "customer_id,full_name,email,signup_date,status,credit_limit\n"
        "C001,Alice,C001@example.com,2026-01-01,active,100\n", encoding="utf-8",
    )
    config = tmp_path / "config.yaml"
    config.write_text(yaml.safe_dump({
        "pipeline": "once_only", "source": {"type": "csv", "path": "feed.csv"},
        "model": "etlguard.examples.models:Customer",
        "target": {"table": "once_only", "mode": "append"},
    }), encoding="utf-8")
    batch = str(uuid4())
    first = Pipeline.from_yaml(config).run(batch_id=batch)
    second = Pipeline.from_yaml(config).run(batch_id=batch)
    assert (first.status, second.status) == ("success", "success")
    assert not first.replayed and second.replayed
    assert first.durable_commit and second.durable_commit
    assert first.request_hash == second.request_hash
    assert len(table_rows(RealLoader(engine=db_engine), "once_only")) == 1
    file.write_text(
        "customer_id,full_name,email,signup_date,status,credit_limit\n"
        "C002,Bob,C002@example.com,2026-01-01,active,100\n", encoding="utf-8",
    )
    changed = Pipeline.from_yaml(config).run(batch_id=batch)
    assert changed.status == "failed"
    assert any("different input" in message for message in changed.failures)
    assert table_rows(RealLoader(engine=db_engine), "etl_run_audit")[0]["status"] ==
"success"
    assert len(table_rows(RealLoader(engine=db_engine), "once_only")) == 1
    db_engine.dispose()


def test_pipeline_run_tracks_partial_source_and_refuses_to_load(monkeypatch, tmp_path):
    import yaml
    from etlguard.pipeline import Pipeline
    from etlguard.sources.base import SourceRecord
    from etlguard.sources.csv import CsvSource

    def broken_source(self, batch_id):
        yield SourceRecord(item().data, Lineage("csv", 2, datetime.now(timezone.utc),
batch_id))
        raise OSError("connection failed during source read")

    monkeypatch.setattr(CsvSource, "read", broken_source)
    configuration = tmp_path / "pipe.yaml"
    configuration.write_text(yaml.safe_dump({
        "pipeline": "partial_job", "source": {"type": "csv", "path": "unused.csv"},
        "model": "etlguard.examples.models:Customer",
    }), encoding="utf-8")
    result = Pipeline.from_yaml(configuration).run(dry_run=True)
    assert result.status == "failed"
    assert result.rows_read == 1 and result.rows_valid == 1
    assert any("connection failed during source read" in m for m in result.failures)


def test_cli_exposes_stable_batch_retry_and_status_commands(tmp_path):
    from typer.testing import CliRunner
    from etlguard.cli import app

    from rich.text import Text

    cli = CliRunner()
```

```
0256      assert "--batch-id" in Text.from_ansi(cli.invoke(app, ["run",
    "--help"]).output).plain
0257      assert "batch-status" in Text.from_ansi(cli.invoke(app, ["--help"]).output).plain
0258      assert cli.invoke(app, ["batch-status", "not-a-uuid"]).exit_code == 2
0259      assert cli.invoke(app, ["upgrade-scd2", str(tmp_path / "missing.yaml")]).exit_code ==
     2
0260
0261
0262  def test_cli_db_status_and_upgrade_guard_with_sqlite(monkeypatch, tmp_path):
0263      import yaml
0264      from typer.testing import CliRunner
0265      from etlguard.cli import app
0266      from etlguard.loaders.mysql import MysqlLoader as RealLoader
0267
0268      engine = create_engine(f"sqlite:///{tmp_path / 'cli.sqlite'}")
0269      batch_id = str(uuid4())
0270      target = TargetSettings(table="cli_test", mode="append")
0271      MysqlLoader(engine=engine).load([item()], batch_id, Customer, target)
0272
0273      def inject_engine(self, database_url=None, engine=None):
0274          self.engine = engine or engine_obj
0275
0276      engine_obj = engine
0277      monkeypatch.setattr(RealLoader, "__init__", inject_engine)
0278      monkeypatch.setenv("ETLGUARD_DATABASE_URL", "mysql+pymysql://stub")
0279      runner = CliRunner()
0280      found = runner.invoke(app, ["batch-status", batch_id])
0281      assert found.exit_code == 0, found.output
0282      assert "rows_inserted" in found.output
0283      missing = runner.invoke(app, ["batch-status", str(uuid4())])
0284      assert missing.exit_code == 1
0285      setup = runner.invoke(app, ["init-db"])
0286      assert setup.exit_code == 0 and "etl_batch_commits" in setup.output
0287      config = tmp_path / "scd2.yaml"
0288      config.write_text(yaml.safe_dump({
0289          "pipeline": "legacy", "source": {"type": "csv", "path": "unused.csv"},
0290          "model": "etlguard.examples.models:Customer",
0291          "target": {"table": "cli_test", "mode": "scd2", "key": ["customer_id"]},
0292      }), encoding="utf-8")
0293      unsupported = runner.invoke(app, ["upgrade-scd2", str(config), "--yes"])
0294      assert unsupported.exit_code == 2
0295      assert "requires MySQL 8" in unsupported.output
0296      engine.dispose()
0297
0298
0299  def test_post_commit_summary_io_problem_is_warning_not_failed_load(tmp_path,
     monkeypatch):
0300      import yaml
0301      from etlguard.pipeline import Pipeline
0302      from etlguard.loaders.mysql import MysqlLoader as RealLoader
0303
0304      db_engine = create_engine(f"sqlite:///{tmp_path / 'io_test.sqlite'}")
0305
0306      def inject_engine(self, database_url=None, engine=None):
0307          self.engine = db_engine
0308
0309      monkeypatch.setattr(RealLoader, "__init__", inject_engine)
0310      monkeypatch.setenv("ETLGUARD_DATABASE_URL", "mysql+pymysql://stub")
0311      (tmp_path / "input.csv").write_text(
0312          "customer_id,full_name,email,signup_date,status,credit_limit\n"
0313          "C001,Alice,C001@example.com,2026-01-01,active,100\n", encoding="utf-8",
0314      )
0315      config = tmp_path / "pipeline.yaml"
0316      config.write_text(yaml.safe_dump({
0317          "pipeline": "report_io", "source": {"type": "csv", "path": "input.csv"},
0318          "model": "etlguard.examples.models:Customer",
0319          "target": {"table": "report_io", "mode": "append"},
0320      }), encoding="utf-8")
0321      from pathlib import Path
0322      original_write = Path.write_text
0323
0324      def reject_summary_write(self, *args, **kwargs):
0325          if self.name.endswith("_summary.json"):
0326              raise OSError("report disk full")
0327          return original_write(self, *args, **kwargs)
0328
0329      monkeypatch.setattr(Path, "write_text", reject_summary_write)
0330      result = Pipeline.from_yaml(config).run(batch_id=str(uuid4()))
0331      assert result.status == "success" and result.durable_commit
0332      assert result.summary_json is None
0333      assert any("report disk full" in issue for issue in result.warnings)
0334      assert len(table_rows(RealLoader(engine=db_engine), "report_io")) == 1
0335      db_engine.dispose()
0336
0337
```

```
def test_mysql_key_with_nullable_column_is_rejected_even_with_unique_index(loader):
    target = TargetSettings(table="unsafe_null_key", mode="upsert", key=["email"])
    with pytest.raises(ValueError, match="NOT NULL"):
        loader.load([item()], str(uuid4()), Customer, target)
    assert table_rows(loader, target.table) == []


def test_target_rejects_repeated_business_key_fields():
    with pytest.raises(ValueError, match="duplicate"):
        TargetSettings(table="unsafe_duplicate_key", mode="upsert",
                       key=["customer_id", "customer_id"])


def test_direct_loader_rejects_duplicate_business_keys_before_writing(loader):
    target = TargetSettings(table="duplicate_upsert", mode="upsert", key=["customer_id"])
    with pytest.raises(ValueError, match="Duplicate business keys"):
        loader.load([item(), item(name="Bob")], str(uuid4()), Customer, target)
```

tests/test_property_cleaning.py

24 source lines.

```
"""Property-based cleaning invariant: f(f(x)) == f(x) across varied strings."""

from hypothesis import given, settings, strategies as st

from etlguard.cleaning import Cleaner
from etlguard.config import CleaningSettings


@given(
    name=st.text(max_size=70),
    amount=st.one_of(st.text(max_size=40), st.integers(-10_000, 10_000)),
    state=st.text(max_size=20),
)
@settings(max_examples=200)
def test_cleaning_is_idempotent(name, amount, state):
    cleaner = Cleaner(CleaningSettings(
        date_columns=["signup_date"],
        number_columns=["amount"],
        lowercase=["name"], uppercase=["state"],
    ))
    row = {" Name ": name, " Amount ": amount, "State": state}
    once = cleaner.clean(row)
    twice = cleaner.clean(once)
    assert once == twice
```

tests/test_safety_regressions.py

142 source lines.

```
"""Regression tests for source-row accuracy and destructive load safety."""

from __future__ import annotations

from pathlib import Path
from uuid import uuid4

import pytest
import yaml
from sqlalchemy import MetaData, Table, UniqueConstraint, create_engine, select

from etlguard.config import ChecksSettings, CleaningSettings, TargetSettings
from etlguard.core import ValidationEngine
from etlguard.examples.models import Customer
from etlguard.loaders.mysql import MysqlLoader, build_tables
from etlguard.pipeline import Pipeline
from etlguard.sources.base import SourceRecord, lineage
from etlguard.validation import Candidate


def config_file(tmp_path: Path, csv_data: str, policy=None, mode="truncate_insert",
name="safe_job"):
    (tmp_path / "input.csv").write_text(csv_data, encoding="utf-8")
    cfg = {
        "pipeline": name,
        "source": {"type": "csv", "path": "input.csv"},
        "model": "etlguard.examples.models:Customer",
```

```
        "cleaning": {"date_columns": ["signup_date"], "number_columns":
["credit_limit"]},
        "target": {"table": "target_customers", "mode": mode},
        "policy": policy or {},
    }
    path = tmp_path / "pipeline.yaml"
    path.write_text(yaml.safe_dump(cfg), encoding="utf-8")
    return path


HEADER = "customer_id,full_name,email,signup_date,status,credit_limit\n"
BAD_ROW = "C001,X,bad!,invalid-date,active,-20\n"
GOOD_ROW = "C001,Alice Doe,alice@example.com,2026-01-01,active,120.00\n"


def test_identical_invalid_records_are_all_rejected(tmp_path):
    path = config_file(tmp_path, HEADER + BAD_ROW * 100,
                       policy={"fail_if_reject_rate_above": 0.05}, mode="append")
    outcome = Pipeline.from_yaml(path).run(dry_run=True)
    assert outcome.rows_read == 100
    assert outcome.rows_rejected == 100
    assert outcome.duplicates_dropped == 0
    assert outcome.rows_valid == 0
    assert outcome.status == "failed"
    assert any("reject rate 100.00%" in issue for issue in outcome.failures)


def test_only_identical_valid_rows_are_dropped():
    first = dict(customer_id="C001", full_name="Alice Doe", email="alice@example.com",
                 signup_date="2026-01-01", status="active", credit_limit="120.00")
    rows = [SourceRecord(first, lineage("synthetic", i, "batch")) for i in range(2, 5)]
    result = ValidationEngine(Customer, CleaningSettings(), ChecksSettings()).run(rows)
    assert (result.rows_valid, result.rows_rejected, result.duplicates_dropped) == (1, 0,
 2)


def test_empty_target_replacement_is_blocked_by_default(tmp_path):
    path = config_file(tmp_path, HEADER)
    outcome = Pipeline.from_yaml(path).run(dry_run=True)
    assert outcome.status == "failed"
    assert any("source is empty" in issue for issue in outcome.failures)


def test_empty_target_replacement_requires_explicit_opt_in(tmp_path):
    path = config_file(tmp_path, HEADER, policy={"allow_empty_replace": True})
    assert Pipeline.from_yaml(path).run(dry_run=True).status == "dry_run"


def test_partial_target_replacement_blocked_unless_explicitly_allowed(tmp_path):
    path = config_file(tmp_path, HEADER + GOOD_ROW + BAD_ROW)
    outcome = Pipeline.from_yaml(path).run(dry_run=True)
    assert outcome.status == "failed"
    assert outcome.rows_valid == 1 and outcome.rows_rejected == 1
    assert any("rejected source records" in issue for issue in outcome.failures)
    path = config_file(tmp_path, HEADER + GOOD_ROW + BAD_ROW,
                       policy={"allow_partial_replace": True})
    assert Pipeline.from_yaml(path).run(dry_run=True).status == "dry_run"


def test_rejects_path_traversal_in_pipeline_identifier(tmp_path):
    path = config_file(tmp_path, HEADER, name="../escape")
    with pytest.raises(ValueError, match="pipeline"):
        Pipeline.from_yaml(path)


def test_upsert_rejects_competing_unique_constraint_before_load():
    engine = create_engine("sqlite:///:memory:")
    try:
        target = TargetSettings(table="business_customers", mode="upsert",
key=["customer_id"])
        target_def, stage_def = build_tables(Customer, target)
        target_def.append_constraint(UniqueConstraint("email", name="uq_email_also"))
        target_def.metadata.create_all(engine)
        model = Customer.model_validate({
            "customer_id": "C001", "full_name": "Alice Doe", "signup_date": "2026-01-01",
            "status": "active", "email": "alice@example.com", "credit_limit": 100,
        })
        candidate = Candidate(model.model_dump(mode="python"), {}, {},
                              lineage("synthetic", 2, str(uuid4())))
        with pytest.raises(ValueError, match="competing UNIQUE keys"):
            MysqlLoader(engine=engine).load([candidate], str(uuid4()), Customer, target)
        current = Table(target.table, MetaData(), autoload_with=engine)
        with engine.connect() as conn:
            assert conn.execute(select(current)).all() == []
    finally:
```

A
B
C
D
E
F
G
H

```
0109          engine.dispose()
0110
0111
0112  def test_identical_invalid_foreign_keys_are_not_deduplicated():
0113      from etlguard.config import ForeignKeySettings
0114
0115      raw = dict(customer_id="C999", full_name="Alice Doe", signup_date="2026-01-01",
0116                 status="active", credit_limit="100", email="alice@example.com")
0117      records = [SourceRecord(raw, lineage("synthetic", i, "batch")) for i in range(2, 5)]
0118      checks = ChecksSettings(foreign_keys={"customer_id":
      ForeignKeySettings(values=["C001"])})
0119      result = ValidationEngine(Customer, CleaningSettings(), checks).run(records)
0120      assert (result.rows_valid, result.rows_rejected, result.duplicates_dropped) == (0, 3,
       0)
0121      assert result.error_counts_by_type["foreign_key"] == 3
0122
0123
0124  def test_repeated_business_key_violations_are_all_rejected():
0125      base = dict(customer_id="C001", full_name="Alice Doe", signup_date="2026-01-01",
0126                  status="active", credit_limit="100", email="alice@example.com")
0127      differing = {**base, "full_name": "Bob Smith"}
0128      records = [SourceRecord(row, lineage("synthetic", i, "batch"))
0129                 for i, row in enumerate([base, differing, differing], start=2)]
0130      result = ValidationEngine(Customer, CleaningSettings(),
0131                                ChecksSettings(unique=["customer_id"])).run(records)
0132      assert (result.rows_valid, result.rows_rejected, result.duplicates_dropped) == (1, 2,
       0)
0133      assert result.error_counts_by_type["unique"] == 2
0134
0135
0136  def test_rename_map_without_column_normalization():
0137      from etlguard.cleaning import Cleaner
0138
0139      cleaner = Cleaner(CleaningSettings(
0140          snake_case_columns=False, rename_map={"Input ID": "customer_id"}
0141      ))
0142      assert cleaner.clean({"Input ID": " C001 "}) == {"customer_id": "C001"}
```

tests/test_sources.py

72 source lines.

```
0001  from __future__ import annotations
0002
0003  import codecs
0004
0005  from openpyxl import Workbook
0006
0007  from etlguard.config import SourceSettings
0008  from etlguard.sources.csv import CsvSource
0009  from etlguard.sources.excel import ExcelSource
0010
0011
0012  def test_csv_bom_chunking_and_physical_lineage(tmp_path):
0013      path = tmp_path / "bom.csv"
0014      path.write_bytes(codecs.BOM_UTF8 + b"a,b\n1, 2\n3,4\n5,6\n")
0015      cfg = SourceSettings(type="csv", path="bom.csv", chunksize=1)
0016      records = list(CsvSource(cfg, tmp_path).read("batch-id"))
0017      assert [row.data for row in records] == [
0018          {"a": "1", "b": " 2"}, {"a": "3", "b": "4"}, {"a": "5", "b": "6"},
0019      ]
0020      assert [row.lineage.source_row_number for row in records] == [2, 3, 4]
0021      assert all(row.lineage.batch_id == "batch-id" for row in records)
0022      assert records[0].lineage.ingested_at.tzinfo is not None
0023
0024
0025  def test_csv_non_utf8_encoding_and_delimiter(tmp_path):
0026      path = tmp_path / "latin.csv"
0027      path.write_bytes("Name;City\nJosé;Paris\n".encode("cp1252"))
0028      cfg = SourceSettings(type="csv", path=path.name, delimiter=";", encoding="cp1252")
0029      records = list(CsvSource(cfg, tmp_path).read("b"))
0030      assert records[0].data == {"Name": "José", "City": "Paris"}
0031
0032
0033  def test_csv_blank_record_is_counted(tmp_path):
0034      path = tmp_path / "blank.csv"
0035      path.write_text("a,b\n1,2\n\n3,4\n")
0036      records = list(CsvSource(SourceSettings(type="csv", path="blank.csv"),
      tmp_path).read("b"))
0037      assert len(records) == 3
0038      assert records[1].lineage.source_row_number == 3
```

A
B
C
D
E
F
G
H

```
0039
0040
0041  def test_csv_header_offset(tmp_path):
0042      path = tmp_path / "header.csv"
0043      path.write_text("ignorable,text\na,b\n1,2\n")
0044      cfg = SourceSettings(type="csv", path="header.csv", header_row=2, chunksize=1)
0045      rows = list(CsvSource(cfg, tmp_path).read("b"))
0046      assert rows[0].data == {"a": "1", "b": "2"}
0047      assert rows[0].lineage.source_row_number == 3
0048
0049
0050  def test_excel_sheet_name_skip_rows_and_header(tmp_path):
0051      book = Workbook()
0052      ws = book.active
0053      ws.title = "Wrong"
0054      ws.append(["x", "y"])
0055      ws = book.create_sheet("Data")
0056      for row in [["Report note"], ["ID", "Name"], ["C001", "Alice"], ["C002", "Bob"]]:
0057          ws.append(row)
0058      book.save(tmp_path / "file.xlsx")
0059      settings = SourceSettings(type="excel", path="file.xlsx", sheet="Data", skip_rows=1)
0060      records = list(ExcelSource(settings, tmp_path).read("b"))
0061      assert [row.data["ID"] for row in records] == ["C001", "C002"]
0062      assert records[0].lineage.source_row_number == 3
0063
0064
0065  def test_excel_sheet_index(tmp_path):
0066      book = Workbook()
0067      ws = book.active
0068      ws.append(["Number"])
0069      ws.append([1])
0070      book.save(tmp_path / "num.xlsx")
0071      records = list(ExcelSource(SourceSettings(type="excel", path="num.xlsx", sheet=0),
      tmp_path).read("b"))
0072      assert records[0].data["Number"] == "1"
```

tests/test_sqlite_loader.py

149 source lines.

```
0001  """Fast SQLAlchemy transactional smoke tests (CI also runs against real MySQL 8)."""
0002
0003  from __future__ import annotations
0004
0005  from datetime import date, datetime, timezone
0006  from decimal import Decimal
0007  from typing import Literal, Optional
0008  from uuid import uuid4
0009
0010  import pytest
0011  from pydantic import BaseModel
0012  from sqlalchemy import MetaData, Table, create_engine, select
0013
0014  from etlguard.audit import AuditWriter
0015  from etlguard.config import TargetSettings
0016  from etlguard.examples.models import Customer
0017  from etlguard.loaders.mysql import MysqlLoader, build_tables, sqlalchemy_type
0018  from etlguard.pipeline import RunResult
0019  from etlguard.reconcile import LoadCounts, ReconciliationError
0020  from etlguard.sources.base import Lineage
0021  from etlguard.validation import Candidate
0022
0023
0024  def candidate(pk="C001", name="Alice Doe", limit="100.00"):
0025      model = Customer.model_validate({
0026          "customer_id": pk, "full_name": name,
0027          "signup_date": "2026-01-01", "status": "active", "credit_limit": limit,
0028      })
0029      return Candidate(data=model.model_dump(mode="python"), cleaned={}, raw={},
0030                       lineage=Lineage("synthetic", 2, datetime.now(timezone.utc),
      "unit-batch"))
0031
0032
0033  def rows(engine, name):
0034      table = Table(name, MetaData(), autoload_with=engine)
0035      with engine.connect() as conn:
0036          return conn.execute(select(table)).mappings().all()
0037
0038
0039  @pytest.fixture
0040  def sqlite_loader():
```

A B C D E F G H

```
    engine = create_engine("sqlite:///:memory:")
    try:
        yield MysqlLoader(engine=engine)
    finally:
        engine.dispose()


def test_mysql_loader_append_stage_and_replace_smoke(sqlite_loader):
    target = TargetSettings(table="unit_customers", mode="append", batch_size=1)
    first = sqlite_loader.load([candidate(), candidate("C002")], str(uuid4()), Customer, target)
    assert first == LoadCounts(2, 0)
    assert len(rows(sqlite_loader.engine, "unit_customers")) == 2
    assert rows(sqlite_loader.engine, "stg_unit_customers") == []
    replacement = TargetSettings(table="unit_customers", mode="truncate_insert")
    count = sqlite_loader.load([candidate("C003")], str(uuid4()), Customer, replacement)
    assert count.loaded == 1
    assert [row["customer_id"] for row in rows(sqlite_loader.engine, "unit_customers")] == ["C003"]


def test_scd2_versions_and_unchanged_smoke(sqlite_loader):
    target = TargetSettings(table="unit_scd2", mode="scd2", key=["customer_id"],
                            tracked_columns=["full_name"])
    first = sqlite_loader.load([candidate()], str(uuid4()), Customer, target)
    second = sqlite_loader.load([candidate(name="Alice Updated")], str(uuid4()), Customer, target)
    third = sqlite_loader.load([candidate(name="Alice Updated")], str(uuid4()), Customer, target)
    assert (first.inserted, second.versions_created, third.unchanged) == (1, 1, 1)
    history = rows(sqlite_loader.engine, "unit_scd2")
    assert len(history) == 2
    assert sum(record["is_current"] for record in history) == 1
    old = next(record for record in history if not record["is_current"])
    current = next(record for record in history if record["is_current"])
    assert old["effective_to"] == current["effective_from"]


def test_reconciliation_rollback_smoke(sqlite_loader, monkeypatch):
    target = TargetSettings(table="unit_rollback", mode="append")
    original = sqlite_loader._merge

    def mismatch(conn, table, data, settings):
        original(conn, table, data, settings)
        return LoadCounts(0, 0)

    monkeypatch.setattr(sqlite_loader, "_merge", mismatch)
    with pytest.raises(ReconciliationError, match="mismatch"):
        sqlite_loader.load([candidate()], str(uuid4()), Customer, target)
    assert rows(sqlite_loader.engine, "unit_rollback") == []
    assert rows(sqlite_loader.engine, "stg_unit_rollback") == []


def test_missing_business_unique_constraint_fails_without_writing(sqlite_loader):
    sqlite_loader.load([candidate()], str(uuid4()), Customer,
                       TargetSettings(table="unit_no_unique", mode="append"))
    with pytest.raises(ValueError, match="no UNIQUE key"):
        sqlite_loader.load([candidate()], str(uuid4()), Customer,
                           TargetSettings(table="unit_no_unique", mode="upsert",
                                          key=["customer_id"]))
    assert len(rows(sqlite_loader.engine, "unit_no_unique")) == 1


def test_model_type_mapping_and_safe_table_validation():
    for annotation in (str, int, float, bool, Decimal, date, datetime, Optional[int], Literal["a", "b"]):
        assert sqlalchemy_type(annotation) is not None
    with pytest.raises(ValueError, match="Unsupported"):
        sqlalchemy_type(dict)

    target, stage = build_tables(Customer, TargetSettings(table="unit_model", mode="upsert",
                                                          key=["customer_id"]))
    assert "customer_id" in stage.c
    assert "__etl_batch_id" in stage.c
    assert any(set(item.columns.keys()) == {"customer_id"} for item in target.constraints
               if item.__class__.__name__ == "UniqueConstraint")
    assert target.c["full_name"].type.length == 80

    with pytest.raises(ValueError, match="target.key"):
        build_tables(Customer, TargetSettings(table="unit_model", mode="upsert", key=["bogus"]))

    class ReservedName(BaseModel):
        effective_from: str
```

A
B
C
D
E
F
G
H

```
0119
0120      with pytest.raises(ValueError, match="reserved"):
0121          build_tables(ReservedName, TargetSettings(table="unit_model", mode="append"))
0122
0123      with pytest.raises(ValueError, match="mysql\\+pymysql"):
0124          MysqlLoader(database_url="sqlite:///:memory:")
0125
0126
0127  def test_audit_writer_smoke(sqlite_loader):
0128      engine = sqlite_loader.engine
0129      now = datetime.now(timezone.utc)
0130      result = RunResult(batch_id=str(uuid4()), pipeline="unit", source="synthetic",
0131                         started_at=now, finished_at=now, status="success",
0132                         rows_read=2, rows_valid=2, rows_rejected=0, rows_loaded=2)
0133      AuditWriter(engine).write(result, "synthetic")
0134      assert rows(engine, "etl_run_audit")[0]["status"] == "success"
0135
0136
0137  def test_empty_truncate_insert_clears_target(sqlite_loader):
0138      table = "unit_empty_replace"
0139      initial = TargetSettings(table=table, mode="append")
0140      sqlite_loader.load([candidate()], str(uuid4()), Customer, initial)
0141      with pytest.raises(ValueError, match="no valid records"):
0142          sqlite_loader.load([], str(uuid4()), Customer,
0143                             TargetSettings(table=table, mode="truncate_insert"))
0144      assert len(rows(sqlite_loader.engine, table)) == 1
0145      replaced = sqlite_loader.load([], str(uuid4()), Customer,
0146                                    TargetSettings(table=table, mode="truncate_insert"),
0147                                    allow_empty_replace=True)
0148      assert replaced.loaded == 0
0149      assert rows(sqlite_loader.engine, table) == []
```

tests/test_validation.py

121 source lines.

```
0001  from __future__ import annotations
0002
0003
0004  from etlguard.checks import DatasetChecker
0005  from etlguard.config import ChecksSettings, CleaningSettings, ForeignKeySettings,
      RowCountSettings
0006  from etlguard.core import ValidationEngine
0007  from etlguard.examples.models import Customer, Transaction
0008  from etlguard.validation import RowValidator
0009
0010
0011  def test_valid_customer_model_and_python_types(record_factory, customer_raw):
0012      item = record_factory(customer_raw)
0013      outcome = RowValidator(Customer).validate(item.data, item.data, item.lineage)
0014      assert outcome.rejects == []
0015      assert outcome.candidate is not None
0016      assert outcome.candidate.data["signup_date"].year == 2026
0017
0018
0019  def test_multiple_independent_field_errors_are_collected(record_factory, customer_raw):
0020      bad = {**customer_raw, "customer_id": "bad", "email": "bad!", "credit_limit": "abc"}
0021      item = record_factory(bad)
0022      outcome = RowValidator(Customer).validate(bad, bad, item.lineage)
0023      assert outcome.candidate is None
0024      assert {issue.field for issue in outcome.rejects} == {"customer_id", "email",
      "credit_limit"}
0025      assert all(issue.raw_row_json and issue.batch_id == "batch-123" for issue in
      outcome.rejects)
0026
0027
0028  def test_customer_row_level_end_date_rule(record_factory, customer_raw):
0029      bad = {**customer_raw, "end_date": "2025-01-01"}
0030      item = record_factory(bad)
0031      outcome = RowValidator(Customer).validate(bad, bad, item.lineage)
0032      assert len(outcome.rejects) == 1
0033      assert outcome.rejects[0].field == "__row__"
0034      assert "end_date" in outcome.rejects[0].error_message
0035
0036
0037  def test_debit_row_level_positive_rule(record_factory):
0038      bad = dict(transaction_id="T001", customer_id="C001", transaction_date="2026-01-01",
0039                 type="debit", amount="0", description=None)
0040      item = record_factory(bad)
0041      outcome = RowValidator(Transaction).validate(bad, bad, item.lineage)
0042      assert len(outcome.rejects) == 1
```

A B C D E F G H

```
    assert "debit amount" in outcome.rejects[0].error_message


def test_dataset_uniqueness_and_foreign_keys(record_factory, customer_raw):
    rows = [
        record_factory(customer_raw, 2),
        record_factory({**customer_raw, "full_name": "Alice New"}, 3),
        record_factory({**customer_raw, "customer_id": "C099", "full_name": "Other Name"}, 4),
    ]
    checks = ChecksSettings(
        unique=["customer_id"],
        foreign_keys={"customer_id": ForeignKeySettings(values=["C001"])},
    )
    core = ValidationEngine(Customer, CleaningSettings(), checks).run(rows)
    assert core.rows_valid == 1
    assert core.rows_rejected == 2
    assert set(core.error_counts_by_type) == {"unique", "foreign_key"}
    assert core.rows_read == 3


def test_dataset_null_pct_and_row_count_threshold(record_factory, customer_raw):
    row = record_factory({**customer_raw, "email": None})
    core = ValidationEngine(
        Customer, CleaningSettings(), ChecksSettings(
            expected_row_count=RowCountSettings(min=2),
            max_null_pct={"email": 0.2},
        ),
    ).run([row])
    assert core.rows_valid == 1
    assert {issue.error_type for issue in core.dataset_issues} == {"row_count", "null_pct"}
    assert core.error_counts_by_field["email"] == 1


def test_empty_feed_and_upper_count_limit(record_factory, customer_raw):
    engine = ValidationEngine(Customer, CleaningSettings(), ChecksSettings(
        expected_row_count=RowCountSettings(min=1, max=1)))
    empty = engine.run([])
    assert empty.rows_valid == 0
    assert len(empty.dataset_issues) == 1
    two = engine.run([
        record_factory(customer_raw, 2),
        record_factory({**customer_raw, "customer_id": "C002"}, 3),
    ])
    assert len(two.dataset_issues) == 1
    assert two.dataset_issues[0].error_type == "row_count"


def test_duplicate_rows_drop_after_cleaning(record_factory, customer_raw):
    one = record_factory(customer_raw, 2)
    two = record_factory({**customer_raw, "full_name": " Alice Doe "}, 3)
    engine = ValidationEngine(Customer, CleaningSettings(), ChecksSettings(unique=["customer_id"]))
    core = engine.run([one, two])
    assert core.rows_read == 2
    assert core.rows_cleaned == 2
    assert core.duplicates_dropped == 1
    assert core.rows_rejected == 0
    assert core.rows_valid == 1


def test_column_collision_does_not_stop_feed(record_factory, customer_raw):
    first = record_factory({**customer_raw, "Customer ID": "C002"}, 2)
    second = record_factory(customer_raw, 3)
    core = ValidationEngine(Customer, CleaningSettings(), ChecksSettings()).run([first, second])
    assert core.rows_valid == 1
    assert core.rows_rejected == 1
    assert core.rejects[0].error_type == "cleaning"


def test_reference_csv_support(tmp_path, record_factory, customer_raw):
    ref = tmp_path / "refs.csv"
    ref.write_text("customer_id\nC001\nC002\n", encoding="utf-8")
    checker = DatasetChecker(ChecksSettings(foreign_keys={
        "customer_id": ForeignKeySettings(path="refs.csv", column="customer_id")
    }), tmp_path)
    assert checker.references["customer_id"] == {"C001", "C002"}
    core = ValidationEngine(Customer, CleaningSettings(), checker.settings, tmp_path).run([
        record_factory({**customer_raw, "customer_id": "C003"})
    ])
    assert core.error_counts_by_type["foreign_key"] == 1
```

A B C D E F G H

43. Appendix D - Operator automation implementation

The following is the operator-side code that provisions and exercises disposable instances, performs acceptance and backup/restore tests, and builds evidence packages. The environment-specific code is not automatically safe on a real host: read every target and destructive operation before execution.

operations/assemble_delivery.py

396 source lines.

```
"""Build a secret-free, checksummed review packet from authentic local results."""

from pathlib import Path
from zipfile import ZipFile, ZIP_DEFLATED
import difflib
import hashlib
import json
import shutil
import subprocess
import sys

root = Path(__file__).resolve().parents[1]
project = root / "source/etl-validation-toolkit"
delivery = root.parent / "ETLGuard_v0.1.3_Production_Readiness_20261111"
delivery.mkdir()


def write(name, value):
    p = delivery / name
    p.parent.mkdir(parents=True, exist_ok=True)
    p.write_text(value)


def read(name):
    return json.loads((root / "fresh-evidence" / name).read_text())


perf = read("operations/performance.json")["details"]
backup = read("operations/backup_restore.json")["details"]
soak = read("operations/soak.json")
crash = read("operations/server-crash-recovery.json")
coverage = read("qa/coverage.json")["totals"]["percent_covered"]
promotion = read("promotion-review.json")
assert promotion["decision"] == "blocked" and len(promotion["reasons"]) == 4
assert read("operations/summary.json")["status"] == "passed"
assert read("browser/browser-report.json")["status"] == "passed"
assert read("package/package-installation.json")["status"] == "passed"
assert (
    read("staging/one-shot-staging-summary.json")["status"]
    == "staging_and_shadow_passed_not_production"
)
assert soak["status"] == crash["status"] == "passed"
shutil.copytree(root / "fresh-evidence", delivery / "evidence")
shutil.copytree(root / "previous-v012-evidence", delivery / "historical/v0.1.2-evidence")
shutil.copytree(root / "controlled-pilot", delivery / "controlled-pilot")
shutil.copytree(
    root / "operations",
    delivery / "operations",
    ignore=shutil.ignore_patterns("__pycache__", ".ruff_cache"),
)
# Package the source without environments, caches, generated reports or private state.
exclude = {
    ".venv",
    "venv",
    "build",
    "dist",
    "reports",
    "__pycache__",
    ".git",
    ".pytest_cache",
    ".mypy_cache",
    ".ruff_cache",
    ".hypothesis",
}
source_files = []
for p in sorted(project.rglob("*")):
    rel = p.relative_to(project)
    if (
        not p.is_file()
```

```
        or p.is_symlink()
        or any(x in exclude or x.endswith(".egg-info") for x in rel.parts)
    ):
        continue
    if p.name in {".coverage", "coverage.xml", "coverage-summary.json", ".DS_Store"}:
        continue
    dest = delivery / "source/etl-validation-toolkit" / rel
    dest.parent.mkdir(parents=True, exist_ok=True)
    shutil.copy2(p, dest)
    source_files.append(rel.as_posix())
(delivery / "artifacts").mkdir()
for name in (
    "etl_validation_toolkit-0.1.3-py3-none-any.whl",
    "etl_validation_toolkit-0.1.3.tar.gz",
):
    shutil.copy2(project / "dist" / name, delivery / "artifacts" / name)
with ZipFile(
    delivery / "artifacts/etl-validation-toolkit-v0.1.3-source.zip", "w", ZIP_DEFLATED
) as z:
    for name in source_files:
        z.write(project / name, "etl-validation-toolkit/" + name)
shutil.copy2(root / "local_staging_one_shot.py", delivery / "local_staging_one_shot.py")
shutil.copy2(root / "offline_acceptance.py", delivery /
"historical/offline_acceptance_v012.py")
shutil.copy2(
    root.parent / "ETLGuard_v0.1.2_Final_Local_Handoff.zip",
    delivery / "historical/ETLGuard_v0.1.2_Final_Local_Handoff.zip",
)
# A reviewable textual patch against the original clean source.
patch = []
with ZipFile(root / "etl-validation-toolkit-v0.1.2-clean-source.zip") as old:
    for name in source_files:
        original = "etl-validation-toolkit/" + name
        if name.endswith((".py", ".toml", ".md", ".html", ".json")) and original in
old.namelist():
            before = old.read(original).decode()
            after = (project / name).read_text()
            if before != after:
                patch.extend(
                    difflib.unified_diff(
                        before.splitlines(True),
                        after.splitlines(True),
                        fromfile="v0.1.2/" + name,
                        tofile="v0.1.3/" + name,
                    )
                )
write("changes/v0.1.2-to-v0.1.3.patch", "".join(patch))
freeze = subprocess.check_output(
    [sys.executable, "-m", "pip", "freeze", "--exclude-editable"], text=True
)
write("requirements-qa-lock.txt", freeze)
write("requirements-operations.txt", "psutil==7.2.2\n")
rows = "\n".join(
    f"| {x['rows']:,} | {x['end_to_end_seconds']:.3f} | {x['rows_per_second']:,.0f} |
{x['peak_combined_rss_mib']:.1f} | {x['probe_p95_ms']:.2f} |"
    for x in perf["measurements"]
)
write(
    "FINAL_STATUS.md",
    f"""# Final local status — ETL Guard v0.1.3

**Technical disposition: local validation completed; ready for formal controlled-pilot
review.**
**Production disposition: NO-GO until exact-target authorization and genuine owner
approvals.**

All fresh results below were executed locally on 11 November 2026 with Python 3.13.16 and
 MySQL 8.4.13 on a disposable loopback-only instance. Historical v0.1.2 evidence is kept
separately and is not counted as v0.1.3 verification.

## Confirmed defect and correction

The v0.1.2 contention soak exposed MySQL deadlock 1213 across distinct targets sharing
metadata tables. A deterministic regression synchronized missing UUID reads and
reproduced the failure. v0.1.3 sets READ COMMITTED on MySQL load connections to avoid
missing-row gap locks, while retaining per-target advisory locks, existing-row locks,
unique batch reservations and atomic target/ledger/audit transactions. The regression and
 full suite pass. A probe verified READ COMMITTED inside the load connection and reset to
 REPEATABLE READ after returning it to the pool. Only `etlguard/__init__.py` and
`etlguard/loaders/mysql.py` changed among the 33 runtime files.

## Fresh QA and staging

- 137 pytest tests passed, zero skips: Hypothesis, mocked HTTP APIs, CLI, validation and
14 real MySQL integration tests.
```

```
- 39 operator/safety unittest checks passed; Ruff and mypy passed; branch coverage: {coverage:.2f}%.
- Seven browser cases passed, including both schemas, empty/oversized/unsupported/malformed inputs and actual CSV download formula escaping. Screenshot and download are retained.
- Nine CLI checks passed from a clean wheel installation outside the source checkout; dependencies resolved and passed compatibility checks.
- The supplied one-shot completed first load, atomic ledger/audit checks, independent reconciliation and identical-UUID replay with unchanged physical target fingerprint.
- Source, wheel and sdist runtime files match; new artifacts were rebuilt for v0.1.3. Original handoff checksums passed 32/32 entries.

## Recovery and concurrency

- Injected audit failure rolled back target, stage, audit and ledger; the same UUID subsequently loaded once.
- A real DBAPI commit followed by connection invalidation simulated a lost acknowledgement; durable ledger lookup resolved it without duplicate loading.
- Killing the application process with uncommitted writes rolled back safely; the same UUID was recovered after checking the ledger.
- A real mysqld SIGKILL with an in-flight transaction preserved a prior committed fingerprint, rolled back uncommitted state and allowed safe same-UUID recovery/replay. Intervention-inclusive recovery: {crash["observed_intervention_inclusive_recovery_seconds"]:.2f} seconds.
- Eight simultaneous SCD2 writers over 50 keys produced 450 contiguous historical versions with exactly 50 active versions.
- Empty input, rejected input and insufficient replacement ratio all failed closed without changing target rows. Bounded lock timeout and recovery passed.
- Logical backup and actual restore into a separate empty schema verified **{backup["rows_verified"]:,} rows across {backup["tables_verified"]} tables**, including physical IDs and audits/ledger. Backup: {backup["backup_seconds"]:.3f}s; restore: {backup["restore_seconds"]:.3f}s; independent verification: {backup["verification_seconds"]:.3f}s. Restore plus verification: **{backup["observed_recovery_seconds"]:.3f}s**.
- The restore intentionally excluded one post-backup transaction, proving recovery to the backup snapshot. PITR and hardware power-loss testing are not claimed.

## Local capacity envelope

Synthetic Customer CSV, upsert, 1,000-row SQL chunks; 8 CPU cores, 8 GiB RAM, MySQL buffer pool 256 MiB. Timing covers CSV read, cleaning, validation, staging, merge, reconciliation and audit/ledger commit. Memory is sampled Python plus MySQL RSS during the initial load.

| Rows | End-to-end seconds | Rows/second | Peak combined MiB | Probe p95 ms |
|---:|---:|---:|---:|---:|
{rows}

All proposed local acceptance criteria passed: at least 500 rows/s, batches within 180s, combined RSS below 2,048 MiB, probe p95 below 250ms, at least 1 GiB disk free, zero added InnoDB log waits and indexed business-key lookup. These are provisional engineering criteria, not owner-approved production SLOs.

A {soak["observed_seconds"]:.2f}-second soak completed **{soak["completed_batches"]}** concurrent 2,000-row batches across three writers with zero failed batches. Probe p95 was {soak["metrics"]["probe_p95_ms"]:.2f}ms. The local alert drill delivered an actual HTTP failure notification to a local receiver. External production paging was not contacted.

## Delivery and environment

The final release includes source, wheel, sdist, configuration examples, reproducible operational scripts, all current evidence, the v0.1.2 regression proof and SHA-256 manifests. Generated database passwords, private client files, virtual environments and raw data directories are excluded. Both disposable MySQL instances were stopped; their data remains in private working folders. The browser preview remains available while its local server runs.

Read `GO_NO_GO_DECISION.md`, `TECHNICAL_REPORT.md`, `ROLLBACK_PLAN.md`, `MONITORING_PROCEDURE.md` and `DEPLOYMENT_CHECKLIST.md` before formal review.
""",
)
write(
    "GO_NO_GO_DECISION.md",
    """# Production decision: NO-GO pending authorization

**Local technical review packet: complete for the tested envelope.**
**Production deployment, production writes and scheduler enablement: not authorized.**

The authentic promotion assessor returned `blocked`, with exactly these unmet gates:

1. Approved, supervised real-target pilot.
2. Genuine data-owner approval.
3. Genuine database-owner approval.
4. Genuine operations-owner approval.
```

A
B
C
D
E
F
G
H

```
0189
0190  The nine local operational evidence entries are backed by actual files. No signatures or
      approvals were generated. The assessor input and output are
      `evidence/pilot-evidence.json` and `evidence/promotion-review.json`.
0191
0192  ## Actions requiring the user's authorization or external access
0193
0194  - Identify and explicitly approve the exact host/schema/table, source snapshot, mode,
      batch UUID, change ticket, maintenance window and named rollback owner/plan for one
      supervised pilot. The template intentionally has no production URL or approved target.
0195  - Provide an approved representative source and workload envelope; owners must accept
      data-quality thresholds, throughput/latency/capacity limits, backup policy, retention and
       RPO/RTO. The synthetic 100,000-row benchmark cannot establish a production SLA.
0196  - Authorize the actual monitoring and paging destination and escalation owners so
      external alert delivery and the real environment's observation window can be tested.
0197  - Supply authentic data, DBA and operations sign-offs after reviewing the evidence and
      exact target plan. Scheduler enablement requires separate go-live authorization.
0198
0199  No production credentials were used. Local logical restore proves the recorded backup
      snapshot; it does not establish production PITR, replication, infrastructure failover or
      an approved production RPO. Those target-specific controls belong in the exact-target
      change review.
0200  """,
0201  )
0202  write(
0203      "TECHNICAL_REPORT.md",
0204      """# Technical report — v0.1.3
0205
0206  ## Architecture and transaction boundary
0207
0208  CSV/Excel/REST readers attach batch lineage. Deterministic cleaning precedes Pydantic row
       checks and global dataset checks. Rejected rows produce CSV/JSON reports; accepted
      candidates go to a MySQL staging table. Append, transactional replacement, upsert and
      SCD2 merge share the target transaction with count reconciliation, staging cleanup,
      success audit and durable commit marker. DDL is prepared before that transaction because
      MySQL DDL can implicitly commit. Replacement uses DELETE rather than TRUNCATE.
0209
0210  A committed UUID plus source/config/model fingerprint supports identical replay. Changed
      input under the same UUID is rejected. Per-target GET_LOCK serialization and an
      active-row unique constraint protect SCD2. The browser shares the validation engine and
      writes only in-memory SQLite.
0211
0212  ## Defect analysis
0213
0214  The default REPEATABLE READ isolation allowed locking reads for missing audit/ledger keys
       to take gap locks. Distinct target locks do not serialize writers to shared metadata
      indexes. Concurrent insert intentions could therefore form a deadlock despite different
      batch UUIDs. The baseline deterministic test failed with MySQL 1213; the corrected test
      synchronizes both ledger and audit reads and passes under READ COMMITTED.
0215
0216  The change applies only to MySQL load connections. SQLAlchemy restores the pool
      connection's default isolation afterward. Existing-row locking, primary-key reservations
      and the per-target lock remain. No automatic blind load retry was added; uncertain
      outcomes still require durable ledger evidence. See `changes/v0.1.2-to-v0.1.3.patch`, the
       before/after regression logs and `evidence/operations/connection-isolation.json`.
0217
0218  MySQL documents reduced gap locking at READ COMMITTED, with remaining locking for
      duplicate-key and foreign-key checks: [MySQL 8.4 isolation documentation](https://dev.mys
      ql.com/doc/refman/8.4/en/innodb-transaction-isolation-levels.html). Unique reservations
      and target locks remain necessary; this change is not a claim that arbitrary external
      writers cannot cause contention.
0219
0220  ## Evidence provenance
0221
0222  The input checksum manifests verified all 32 entries. Original wheel/source/sdist runtime
       bytes matched 33/33. Current v0.1.3 wheel/source/sdist runtime matches are recorded
      separately. Historical GitHub run 43061543916 was queried through the authenticated
      connector; its four completed successful job records are retained. Those jobs validate
      v0.1.2, not this changed runtime. Fresh v0.1.3 evidence comes from this local Python
      3.13/MySQL 8.4 instance; current v0.1.3 MySQL 8.0 and Python 3.11 runs are not claimed.
0223
0224  ## Scope of fault and recovery testing
0225
0226  Lost-acknowledgement injection performs a real DBAPI commit, invalidates the connection
      and raises before acknowledgement reaches the loader. Application-process termination and
       actual database-process SIGKILL are separate exercises. The database restart recovered
      prior durable data and rejected uncommitted work. Physical power loss, storage-device
      corruption and infrastructure failover are outside the exercised local fault model.
0227
0228  The logical dump used a consistent transaction, omitted tablespace/global objects and was
       restored into an initially empty separate schema. Every row fingerprint matched the
      pre-backup snapshot. One deliberately later transaction was absent, as expected. Logical
      snapshot recovery does not prove PITR or an owner-approved production recovery point.
0229
0230  ## Capacity and operational limits
```

A
B
C
D
E
F
G
H

```
0231
0232  The validator materializes records for global checks and is not an out-of-core engine.
      Measurements cover synthetic Customer upsert batches through 100,000 records and a
      two-minute local contention soak. Larger sources, complex schemas, production
      storage/replication, long-running history, external writers and production alert routing
      require the approved target and workload. Query plans, CPU/RSS samples, disk growth,
      probe distributions and MySQL status counters are included in the raw JSON evidence.
0233
0234  The local alert drill exercised failure detection and an actual HTTP receiver; it does
      not authenticate an on-call recipient or external paging route. No production target,
      change ticket or human approval was supplied. The promotion tool correctly remains
      blocked.
0235  """,
0236  )
0237  write(
0238      "ROLLBACK_PLAN.md",
0239      """# Reviewed rollback plan template — approval required for exact target
0240
0241  1. Before the pilot, pin the exact source/config hashes and stable UUID; record target
      snapshot, existing ledger/audit, DDL inventory, approved backup/restore location and an
      exclusive rollback owner. Confirm the real backup's restore and agreed RPO/RTO; the
      included synthetic dump must never be used on a real target.
0242  2. Keep the scheduler disabled. Stop the specific job and coordinate other target writers
       on any unexplained mismatch, failure, lock-budget breach, missing audit/ledger or alert.
       Preserve logs and batch artifacts before intervention.
0243  3. For an uncertain commit, query `etlguard batch-status UUID --database-url-env
      APPROVED_ENV`. Reconcile the durable ledger and target before any retry. Never change the
       UUID merely to escape uncertainty. If the database is unavailable or state cannot be
      resolved, keep the job stopped and restore connectivity or escalate to the DBA.
0244  4. A verified pre-commit failure rolls back target, staging, audit and ledger. Retry the
      exact immutable batch only after confirming rollback/absence and resolving the fault. An
      identical already-committed UUID replays counts without a second load.
0245  5. A committed transaction cannot be undone with a later SQL ROLLBACK. An owner-approved
      compensating change or backup/PITR restore requires the exact target change ticket,
      stopped writers and independent before/after reconciliation. Restore first to a separate
      schema for verification. Preserve newer legitimate transactions according to the approved
       recovery plan.
0246  6. SCD2 schema changes require maintenance-window approval and backup. Do not
      automatically drop history or roll back generated active uniqueness guards.
0247  7. If reverting software to v0.1.2 is proposed, account for its confirmed cross-target
      metadata deadlock: it cannot serve as a concurrent-writer fallback. Any temporary
      globally serialized fallback requires explicit owner approval and fresh reconciliation.
0248
0249  Local evidence: `operations/audit_rollback.json`, `process_kill.json`,
      `lost_commit_ack.json`, `server-crash-recovery.json` and `backup_restore.json` under
      `evidence/`. Exact production commands are intentionally pending exact-target approval.
0250  """,
0251  )
0252  write(
0253      "MONITORING_PROCEDURE.md",
0254      """# Monitoring and escalation procedure
0255
0256  For the approved pilot, name an operations owner, DBA and data owner; approve the
      receiver, retention, sampling cadence and observation window. Do not embed passwords or
      raw customer rows in alerts. A monitoring identity needs read access to the approved
      audit/ledger and necessary target metrics; it does not need target DML or DDL privileges.
0257
0258  For each stable batch, record start/end UTC, source/config hashes,
      read/valid/rejected/loaded counts, duration, reject rate, lock wait, durable commit
      status and reconciliation result. Use batch-ID primary-key lookups in `etl_batch_commits`
       and `etl_run_audit`; investigate missing/mismatched records and leftover staging rows.
      Monitor CPU, process RSS, disk headroom, probe latency, InnoDB log waits/deadlocks,
      active transactions and connection limits. Plan metadata retention and indexed reporting
      for the real audit volume.
0259
0260  Proposed local test limits were throughput >=500 rows/s, duration <=180s per tested
      batch, Python+MySQL RSS <=2GiB, probe p95 <=250ms, free disk >=1GiB, zero reconciliation
      mismatches/duplicate loads and zero added InnoDB log waits. Owners must replace or
      approve these against the real workload; they are not production SLOs.
0261
0262  Page/escalate immediately for an unresolved commit, missing successful audit/ledger,
      unexpected target values/history, reconciliation mismatch, incomplete source or unsafe
      replacement attempt. Stop the job and preserve UUID/artifacts. Capacity/lock/latency
      breaches trigger the owner-approved circuit breaker and review before retry. Keep
      scheduler enablement a separate approved action.
0263
0264  Fresh evidence includes actual sampled metrics for all three volume runs and the
      two-minute soak. `evidence/operations/monitoring.json` records a real failed audit
      detected and delivered to a local HTTP receiver with a receipt. External paging and its
      recipient acknowledgement remain blocked pending the approved destination. No external
      notifications were sent.
0265  """,
0266  )
0267  write(
0268      "DEPLOYMENT_CHECKLIST.md",
```

```
0269      """# Deployment checklist — formal owner review
0270
0271  ## Completed locally
0272
0273  - [x] Input hashes and source/wheel/sdist integrity checked.
0274  - [x] Confirmed runtime defect reproduced, fixed, version bumped and regression added.
0275  - [x] Full fresh QA passed, zero MySQL skips, Ruff/mypy green.
0276  - [x] Wheel/sdist built and installed wheel exercised independently.
0277  - [x] Disposable MySQL with isolated scoped users, strict SQL mode and InnoDB.
0278  - [x] First load, audits, commit ledger, source/target values and unchanged retry
      fingerprint.
0279  - [x] Lost acknowledgement, process/database crashes, rollback, source tampering and lock
       timeout drills.
0280  - [x] Concurrent SCD2, replacement controls, measured capacity, actual logical restore
      and local alert delivery.
0281  - [x] Controlled-pilot template dry-validated; scheduler disabled.
0282  - [x] Authentic evidence submitted to promotion assessor; missing approvals remain false.
0283
0284  ## Required exact-target review
0285
0286  - [ ] Explicit host/schema/table/source/mode/change ticket/window/rollback plan
      authorization.
0287  - [ ] Data-owner acceptance of real input, quality thresholds and reconciliation.
0288  - [ ] DBA acceptance of actual schema, privileges, isolation/binlog/replication
      compatibility, backup/PITR, retention and RPO/RTO.
0289  - [ ] Operations acceptance of real workload SLOs, alert receiver, escalation, monitoring
       window and rollback owner.
0290  - [ ] Approved de-identified/realistic source and actual-environment capacity/restore
      checks where required.
0291  - [ ] Signed approvals and supervised limited pilot result.
0292  - [ ] Independent reconciliation and observation after pilot.
0293  - [ ] Separate explicit unattended scheduler/go-live authorization.
0294
0295  The production assessor's four remaining fields must be backed by authentic artifacts. Do
       not mark them complete from a test pass or from this checklist alone. No production
      deployment was performed.
0296  """,
0297  )
0298  write(
0299      "README_START_HERE.md",
0300      """# ETL Guard v0.1.3 production-readiness packet
0301
0302  Start with `FINAL_STATUS.md` and `GO_NO_GO_DECISION.md`. Current evidence is in
      `evidence/`; historical input and v0.1.2 records are in `historical/`. The corrected
      runtime is v0.1.3. This packet does not authorize production.
0303
0304  ## Contents
0305
0306  - `artifacts/`: tested v0.1.3 wheel, sdist and clean source ZIP.
0307  - `source/etl-validation-toolkit/`: complete editable source, tests, demo and runbooks.
0308  - `evidence/`: current staging, JUnit, coverage, browser screenshot/download, recovery
      dumps/logs, metrics, soak and assessor results.
0309  - `controlled-pilot/`: portable synthetic example and pinned prepared UUID/hashes, with
      execution approval and scheduler disabled.
0310  - `operations/`: reproducible local provisioning, QA, fault, capacity and packaging
      helpers.
0311  - `changes/`: source patch from v0.1.2.
0312  - `SHA256SUMS`: delivery integrity manifest, not a signature or owner approval.
0313
0314  ## Use the application
0315
0316  On macOS, double-click `source/etl-validation-toolkit/Launch ETL Guard.command`. It uses
      an available localhost port. Python 3.11+ is required, and browser startup downloads
      Pyodide/Pydantic/SQLite. CSV processing remains local to the browser. Alternatively run
      `python scripts/serve_demo.py` from the source project.
0317
0318  Install the wheel with a clean Python environment: `python -m pip install
      artifacts/etl_validation_toolkit-0.1.3-py3-none-any.whl`. Use `etlguard --help`. The
      included source runbooks describe CLI source/config and reports.
0319
0320  ## Reproduce local exercises
0321
0322  Use a NEW extracted working copy. macOS helpers expect the Homebrew `mysql@8.4` keg at
      `/opt/homebrew/opt/mysql@8.4`; they never use Homebrew services. Create the source
      `.venv` with Python 3.13, install `-e '.[dev]'`, `pip` and `-r
      requirements-operations.txt`. `requirements-qa-lock.txt` records this run's compatible QA
       dependency versions. Runtime compatibility and performance must still be validated on
      the intended platform.
0323
0324  Run `operations/bootstrap_mysql.py` in a persistent terminal. It creates `.private/` only
       if absent, initializes a new data directory with networking disabled, generates
      credentials, provisions four scoped schemas, then starts loopback-only MySQL. No
      production URL argument exists. Keep that session running.
0325
0326  In another terminal using the source environment, run `operations/run_staging.py`, then
```

```
`operations/run_operational_acceptance.py`, then `operations/run_soak.py`. They create
NEW `fresh-evidence/` directories and reject existing output directories where immutable
evidence would otherwise be overwritten. Do not blindly rerun uncertain writes. The crash
 drill's `stage`/`verify` modes are specifically for that generated private instance,
with `start_existing_mysql.py` between them. Review the code before the deliberately
destructive SIGKILL drill.

For wheel CLI QA, create `.private/wheel-venv`, install the supplied wheel with
dependencies, and run `operations/package_cli_acceptance.py`. For a prepared
pilot/assessor exercise, run `operations/prepare_approval_packet.py` only in a new output
 layout. It should remain blocked without owner artifacts.

Private generated credential/client files, raw database directories and virtual
environments are deliberately not included. `operations/assemble_delivery.py` describes
this exact local packaging run and includes fixed input/output names; adapt it to a new
destination rather than overwriting this packet. The generic source wheel/sdist build
uses `python -m build`.
""",
)
write(
    "evidence/INDEX.md",
    """# Current evidence index

- `integrity/handoff-verification.json`: original 32 manifest checks and 33 runtime
comparisons.
- `integrity/v013-runtime-match.json`: current wheel/source/sdist equality and changed
runtime file list.
- `integrity/authenticated-ci-jobs.json`: authenticated historical v0.1.2 GitHub matrix
job records.
- `qa/`: deterministic defect reproduction before/after, JUnit, coverage JSON/XML,
dependency/environment/build logs and operations lint.
- `staging/`: supplied one-shot's actual MySQL tests, immutable manifest, first/retry
loads and independent reconciliation.
- `operations/`: individual fault checks, backup dump plus before/after fingerprints,
performance inputs/configs/results, CPU/RSS/probe samples, two-minute soak, actual local
alert receipt, server crash/recovery and final lock/transaction health.
- `browser/`: seven real browser cases, screenshot, synthetic fixtures and actual escaped
 download.
- `package/`: nine installed-wheel CLI checks, logs and stable retry UUID.
- `pilot-evidence.json` and `promotion-review.json`: authentic local evidence and the
four remaining approval gates.

Reports retain their execution paths and UTC timestamps for provenance. Files in
`historical/` concern the original runtime or initial setup attempts, and must not be
counted as current v0.1.3 passes.
""",
)
# Verify current artifacts and exclude all generated secrets before hashing.
secret_values = []
for private in (root / ".private", root / ".private-v012"):
    cfg = json.loads((private / "runtime.json").read_text())
    secret_values.append(cfg["admin_password"])
    secret_values.extend(v["password"] for v in cfg["users"].values())
files = [p for p in delivery.rglob("*") if p.is_file()]
for p in files:
    data = p.read_bytes()
    assert not any(s.encode() in data for s in secret_values), f"Private secret found in
{p}"
    assert (
        ".private" not in p.relative_to(delivery).parts
        and ".venv" not in p.relative_to(delivery).parts
    )
manifest = []
for p in sorted(files):
    manifest.append(
        hashlib.sha256(p.read_bytes()).hexdigest() + "  " +
p.relative_to(delivery).as_posix()
    )
write("SHA256SUMS", "\n".join(manifest) + "\n")
archive = delivery.parent / (delivery.name + ".zip")
assert not archive.exists()
with ZipFile(archive, "w", ZIP_DEFLATED, compresslevel=6) as z:
    for p in sorted(delivery.rglob("*")):
        if p.is_file():
            z.write(p, delivery.name + "/" + p.relative_to(delivery).as_posix())
with ZipFile(archive) as z:
    assert z.testzip() is None
    for line in (delivery / "SHA256SUMS").read_text().splitlines():
        sha, name = line.split("  ", 1)
        assert hashlib.sha256(z.read(delivery.name + "/" + name)).hexdigest() == sha
archive.with_suffix(".zip.sha256").write_text(
    hashlib.sha256(archive.read_bytes()).hexdigest() + "  " + archive.name + "\n"
)
print(
    json.dumps(
```

```
        {
            "delivery": str(delivery),
            "archive": str(archive),
            "bytes": archive.stat().st_size,
            "files": len(files) + 1,
            "checksums_verified": len(manifest),
            "secret_scan": "passed",
        },
        indent=2,
    )
)
```

operations/bootstrap_mysql.py

138 source lines.

```
"""Provision a new loopback-only disposable instance, without Homebrew services."""

import json
import secrets
import socket
import subprocess
import time
from pathlib import Path

import pymysql

ROOT = Path(__file__).resolve().parents[1]
PRIVATE = ROOT / ".private"
PRIVATE.mkdir(mode=0o700)
DATA = PRIVATE / "mysql-data"
DATA.mkdir()
BIN = Path("/opt/homebrew/opt/mysql@8.4/bin")
SOCK = "/tmp/etlguard-" + secrets.token_hex(5) + ".sock"
with socket.socket() as s:
    s.bind(("127.0.0.1", 0))
    PORT = s.getsockname()[1]
log = (PRIVATE / "mysql-console.log").open("ab")
subprocess.run(
    [
        str(BIN / "mysqld"),
        "--no-defaults",
        "--initialize-insecure",
        "--datadir=" + str(DATA),
    ],
    stdout=log,
    stderr=log,
    check=True,
)
base = [
    str(BIN / "mysqld"),
    "--no-defaults",
    "--datadir=" + str(DATA),
    "--socket=" + SOCK,
    "--port=" + str(PORT),
    "--bind-address=127.0.0.1",
    "--mysqlx=OFF",
    "--innodb-buffer-pool-size=268435456",
    "--max-connections=64",
    "--log-error=" + str(PRIVATE / "mysql-error.log"),
    "--log-bin=" + str(DATA / "binlog"),
    "--server-id=7651",
    "--innodb-flush-log-at-trx-commit=1",
    "--sync-binlog=1",
]


def start(extra, password):
    proc = subprocess.Popen(base + extra, stdout=log, stderr=log)
    for _ in range(120):
        if proc.poll() is not None:
            raise RuntimeError("Disposable MySQL exited; inspect private error log")
        try:
            conn = pymysql.connect(
                unix_socket=SOCK, user="root", password=password, autocommit=True
            )
            return proc, conn
        except pymysql.Error:
            time.sleep(0.25)
    raise TimeoutError("MySQL startup")

```

A
B
C
D
E
F
G
H

```
0066
0067  proc, conn = start(["--skip-networking"], "")
0068  admin_password = secrets.token_hex(24)
0069  users = {}
0070  with conn.cursor() as cur:
0071      cur.execute("ALTER USER %s@%s IDENTIFIED BY %s", ("root", "localhost",
      admin_password))
0072      for schema, user in [
0073          ("etlguard_ci", "etlguard_ci"),
0074          ("etlguard_shadow_ci", "etlguard_shadow"),
0075          ("etlguard_ops_ci", "etlguard_ops"),
0076          ("etlguard_restore_ci", "etlguard_restore"),
0077      ]:
0078          password = secrets.token_hex(24)
0079          cur.execute(
0080              "CREATE DATABASE `" + schema + "` CHARACTER SET utf8mb4 COLLATE
      utf8mb4_0900_ai_ci"
0081          )
0082          cur.execute("CREATE USER %s@%s IDENTIFIED BY %s", (user, "127.0.0.1", password))
0083          cur.execute("GRANT ALL PRIVILEGES ON `" + schema + "`.* TO %s@%s", (user,
      "127.0.0.1"))
0084          users[schema] = {
0085              "user": user,
0086              "password": password,
0087              "url": f"mysql+pymysql://{user}:{password}@127.0.0.1:{PORT}/{schema}",
0088          }
0089      cur.execute("SHUTDOWN")
0090  conn.close()
0091  proc.wait(timeout=30)
0092  proc, conn = start(
0093      [
0094          "--slow-query-log=ON",
0095          "--long-query-time=0.1",
0096          "--slow-query-log-file=" + str(PRIVATE / "slow.log"),
0097      ],
0098      admin_password,
0099  )
0100  with conn.cursor() as cur:
0101      cur.execute(
0102          "SELECT VERSION(), @@port, @@bind_address, @@innodb_flush_log_at_trx_commit,
      @@sync_binlog"
0103      )
0104      settings = cur.fetchone()
0105  conn.close()
0106  record = {
0107      "port": PORT,
0108      "socket": SOCK,
0109      "pid": proc.pid,
0110      "admin_password": admin_password,
0111      "datadir": str(DATA),
0112      "bin": str(BIN),
0113      "users": users,
0114  }
0115  p = PRIVATE / "runtime.json"
0116  p.write_text(json.dumps(record, indent=2))
0117  p.chmod(0o600)
0118  (ROOT / "fresh-evidence/mysql").mkdir(parents=True)
0119  (ROOT / "fresh-evidence/mysql/instance.json").write_text(
0120      json.dumps(
0121          {
0122              "version": settings[0],
0123              "port": settings[1],
0124              "bind_address": settings[2],
0125              "innodb_flush_log_at_trx_commit": settings[3],
0126              "sync_binlog": settings[4],
0127              "datadir_is_new": True,
0128              "homebrew_service_used": False,
0129              "credentials": "generated; excluded from delivery",
0130              "admin_bootstrap": "Unix socket with networking disabled",
0131          },
0132          indent=2,
0133      )
0134  )
0135  print("Disposable MySQL ready:", settings[0], "on loopback port", PORT)
0136
0137  # Keep the managed child attached to its execution session.
0138  proc.wait()
```

operations/package_cli_acceptance.py

92 source lines.

A B C D E F G H

```
"""Exercise the installed wheel from outside the source checkout."""

import json
import os
import subprocess
from pathlib import Path
from uuid import uuid4

import yaml

root = Path(__file__).resolve().parents[1]
out = root / "fresh-evidence/package"
out.mkdir()
cli = root / ".private/wheel-venv/bin/etlguard"
python = root / ".private/wheel-venv/bin/python"
env = {k: v for k, v in os.environ.items() if not k.startswith("ETLGUARD_") and k !=
"PYTHONPATH"}
cfg = json.loads((root / ".private/runtime.json").read_text())
env["ETLGUARD_OPS_MYSQL_URL"] = cfg["users"]["etlguard_ops_ci"]["url"]
env["NO_COLOR"] = "1"
project = root / "source/etl-validation-toolkit"
package = subprocess.check_output(
    [
        str(python),
        "-c",
        'from importlib.resources import files; print(files("etlguard.examples"))',
    ],
    cwd="/tmp",
    env=env,
    text=True,
).strip()
assert ".private/wheel-venv/" in package
examples = Path(package)
checks = []


def run(name, args, expected=0, contains=None):
    r = subprocess.run(
        [str(cli), *args],
        cwd=out,
        env=env,
        text=True,
        stdout=subprocess.PIPE,
        stderr=subprocess.STDOUT,
    )
    output = r.stdout
    for user in cfg["users"].values():
        output = output.replace(user["password"], "[REDACTED]")
    (out / (name + ".log")).write_text(output)
    assert r.returncode == expected, (name, r.returncode, output)
    if contains:
        assert contains in output, (name, output)
    checks.append({"name": name, "exit_code": r.returncode, "status": "passed"})


run("help", ["--help"], contains="batch-status")
run("clean", ["validate", str(examples / "pipelines/customers_clean.yaml")])
run("dirty", ["validate", str(examples / "pipelines/customers.yaml")], 1)
run("invalid_config", ["validate", str(out / "missing.yaml")], 2)
run("profile", ["profile", str(examples / "data/customers_clean.csv")])
run("init_db", ["init-db", "--database-url-env", "ETLGUARD_OPS_MYSQL_URL"])
config = yaml.safe_load((examples / "pipelines/customers_clean.yaml").read_text())
config["source"]["path"] = str(examples / "data/customers_clean.csv")
config["target"]["table"] = "op_wheel_" + uuid4().hex[:8]
config["database_url_env"] = "ETLGUARD_OPS_MYSQL_URL"
config["output"] = {"directory": str(out / "reports")}
file = out / "wheel-pipeline.yaml"
file.write_text(yaml.safe_dump(config))
batch = str(uuid4())
run("first_load", ["run", str(file), "--batch-id", batch])
run(
    "identical_retry",
    ["run", str(file), "--batch-id", batch],
    contains="Already committed",
)
run(
    "batch_status",
    ["batch-status", batch, "--database-url-env", "ETLGUARD_OPS_MYSQL_URL"],
    contains="Committed batch",
)
(out / "package-installation.json").write_text(
    json.dumps(
        {
            "status": "passed",
            "installed_package_path": package,
```

```
            "working_directory_outside_checkout": True,
            "stable_batch_id": batch,
            "checks": checks,
        },
        indent=2,
    )
)
print("Installed wheel CLI checks passed:", len(checks))
```

operations/prepare_approval_packet.py

126 source lines.

```
"""Prepare a portable, non-executing pilot template and an honest evidence manifest."""

from pathlib import Path
import hashlib
import json
import shutil
import subprocess
import sys
from uuid import uuid4
import yaml

root = Path(__file__).resolve().parents[1]
project = root / "source/etl-validation-toolkit"
evidence = root / "fresh-evidence"
pilot = root / "controlled-pilot"
pilot.mkdir()
(pilot / "data").mkdir()
shutil.copy2(
    project / "src/etlguard/examples/data/customers_clean.csv",
    pilot / "data/approved-snapshot-example.csv",
)
config = {
    "pipeline": "controlled_customer_pilot",
    "source": {"type": "csv", "path": "data/approved-snapshot-example.csv"},
    "model": "etlguard.examples.models:Customer",
    "cleaning": {
        "trim": True,
        "date_columns": ["signup_date", "end_date"],
        "number_columns": ["credit_limit"],
    },
    "checks": {"unique": ["customer_id"], "expected_row_count": {"min": 3, "max": 3}},
    "target": {
        "table": "approval_required_target",
        "mode": "upsert",
        "key": ["customer_id"],
        "batch_size": 500,
        "lock_timeout_seconds": 30,
    },
    "policy": {
        "fail_if_reject_rate_above": 0.0,
        "allow_empty_replace": False,
        "allow_partial_replace": False,
    },
    "database_url_env": "ETLGUARD_APPROVED_PILOT_URL",
    "output": {"directory": "reports"},
}
p = pilot / "pilot-template.yaml"
p.write_text(yaml.safe_dump(config, sort_keys=False))
batch = str(uuid4())
manifest = {
    "toolkit_version": "0.1.3",
    "prepared_batch_id": batch,
    "scheduler_enabled": False,
    "production_execution_authorized": False,
    "exact_target": None,
    "change_ticket": None,
    "rollback_owner": None,
    "source": "data/approved-snapshot-example.csv",
    "source_sha256": hashlib.sha256(
        (pilot / "data/approved-snapshot-example.csv").read_bytes()
    ).hexdigest(),
    "config_sha256": hashlib.sha256(p.read_bytes()).hexdigest(),
    "note": "Synthetic approval template only. Real target, real approved source,
thresholds and change require a new pinned manifest/UUID before first execution. Never
change an uncertain batch.",
}
(pilot / "prepared-manifest.json").write_text(json.dumps(manifest, indent=2))
with (evidence / "qa/pilot-template-validation.log").open("w") as f:
```

A B C D E F G H

```
    subprocess.run(
        [sys.executable, "-m", "etlguard.cli", "validate", str(p)],
        stdout=f,
        stderr=subprocess.STDOUT,
        check=True,
    )
# Use the public CLI entry point: the module intentionally has no __main__ hook.
with (evidence / "qa/pilot-template-validation.log").open("w") as f:
    subprocess.run(
        [str(Path(sys.executable).parent / "etlguard"), "validate", str(p)],
        stdout=f,
        stderr=subprocess.STDOUT,
        check=True,
    )
template = json.loads((project / "docs/pilot-evidence-template.json").read_text())
paths = {
    "immutable_source": "staging/immutable_pilot/shadow-manifest.json",
    "backup_restore_rehearsal": "operations/backup_restore.json",
    "shadow_row_reconciliation": "staging/shadow-reconciliation-first.json",
    "same_batch_retry": "staging/shadow-reconciliation-identical_retry.json",
    "changed_input_rejected": "operations/changed_input.json",
    "commit_outcome_recovery": "operations/server-crash-recovery.json",
    "scd2_concurrency": "operations/scd2_concurrency.json",
    "rollback_rehearsal": "operations/audit_rollback.json",
    "monitoring_observation": "operations/monitoring.json",
}
for name, relative in paths.items():
    assert (evidence / relative).is_file()
    template["operational"][name].update(
        verified=True,
        evidence_file=relative,
        scope="Fresh synthetic disposable local MySQL; independent owner review remains
pending",
    )
template["operational"]["supervised_real_target_pilot"].update(
    verified=False,
    evidence_file="",
    reason="Exact target, change ticket and rollback approval not supplied",
)
for record in template["approvals"].values():
    record.update(
        verified=False, evidence_file="", reason="No human approval supplied; not
invented"
    )
template["notes"] = (
    "Authentic local technical evidence only. Local alert receiver does not verify
production paging. Production thresholds, data shape, routing, backup policy and
approvals remain pending."
)
(evidence / "pilot-evidence.json").write_text(json.dumps(template, indent=2))
code = subprocess.call(
    [
        sys.executable,
        str(project / "scripts/assess_promotion.py"),
        "--staging",
        str(evidence / "staging/staging-gate-report.json"),
        "--evidence",
        str(evidence / "pilot-evidence.json"),
        "--report",
        str(evidence / "promotion-review.json"),
    ]
)
assert code == 2, "Production assessor should remain blocked without authentic owner
approvals"
print("Prepared dry-validated pilot template and authentic blocked promotion review")
```

operations/run_operational_acceptance.py

767 source lines.

```
"""Actual recovery, contention, capacity and local monitoring drills on private MySQL.

Requires bootstrap_mysql.py's NEW disposable instance. No production URL input.
"""

from __future__ import annotations

import hashlib
import json
import os
import statistics
```

```
import subprocess
import sys
import threading
import time
from concurrent.futures import ThreadPoolExecutor
from datetime import UTC, datetime
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from pathlib import Path
from unittest.mock import patch
from uuid import uuid4

import psutil
import yaml
from etlguard.audit import AuditWriter, BatchConflictError
from etlguard.config import TargetSettings
from etlguard.examples.models import Customer
from etlguard.loaders.mysql import MysqlLoader
from etlguard.pipeline import Pipeline
from etlguard.sources.base import Lineage
from etlguard.validation import Candidate
from sqlalchemy import MetaData, Table, create_engine, event, inspect, select, text

ROOT = Path(__file__).resolve().parents[1]
CFG = json.loads((ROOT / ".private/runtime.json").read_text())
OUT = ROOT / "fresh-evidence/operations"
OUT.mkdir(parents=True, exist_ok=True)
URL = CFG["users"]["etlguard_ops_ci"]["url"]
ENGINE = create_engine(URL, pool_pre_ping=True, pool_size=12, max_overflow=4)
assert ENGINE.url.host == "127.0.0.1" and ENGINE.url.database == "etlguard_ops_ci"
LOADER = MysqlLoader(engine=ENGINE)
RESULTS = []


def now():
    return datetime.now(UTC).isoformat()


def save(name, data):
    (OUT / (name + ".json")).write_text(json.dumps(data, indent=2, default=str) + "\n")


def candidate(pk="C000001", name="Synthetic Person"):
    data = Customer.model_validate(
        dict(
            customer_id=pk,
            full_name=name,
            email=pk.lower() + "@example.test",
            signup_date="2026-01-01",
            status="active",
            credit_limit="100.00",
        )
    ).model_dump()
    return Candidate(
        data,
        data,
        {},
        Lineage("synthetic-operations", 2, datetime.now(UTC), "synthetic"),
    )


def table_name(label):
    return "op_" + label + "_" + uuid4().hex[:8]


def rows(name):
    t = Table(name, MetaData(), autoload_with=ENGINE)
    with ENGINE.connect() as c:
        return [dict(x) for x in c.execute(select(t)).mappings()]


def fingerprint(data):
    serialized = sorted(json.dumps(x, sort_keys=True, default=str) for x in data)
    return hashlib.sha256(json.dumps(serialized).encode()).hexdigest()


def committed(batch):
    a = AuditWriter(ENGINE)
    with ENGINE.connect() as c:
        return c.execute(select(a.commits).where(a.commits.c.batch_id ==
batch)).mappings().first()


def check(name, fn):
    start = time.perf_counter()
    try:
```

A B C D E F G H

```
        details = fn()
        result = {"check": name, "status": "passed", "details": details}
    except Exception as exc:
        import traceback

        error = str(exc)
        for user in CFG["users"].values():
            error = error.replace(user["password"], "[REDACTED]")
        result = {
            "check": name,
            "status": "failed",
            "error_type": type(exc).__name__,
            "error": error,
            "frames": [
                f"{Path(f.filename).name}:{f.lineno}"
                for f in traceback.extract_tb(exc.__traceback__)
            ],
        }
    result.update(timestamp_utc=now(), elapsed_seconds=time.perf_counter() - start)
    save(name, result)
    RESULTS.append(result)
    print(name, result["status"], round(result["elapsed_seconds"], 3), flush=True)


def preflight():
    with ENGINE.connect() as c:
        version = c.execute(text("SELECT VERSION()")).scalar_one()
        grants = list(c.execute(text("SHOW GRANTS")).scalars())
        sqlmode = c.execute(text("SELECT @@sql_mode")).scalar_one()
    assert version.startswith("8.") and "mariadb" not in version.lower()
    assert all("`etlguard_ops_ci`.*" in g or "USAGE ON *.*" in g for g in grants)
    AuditWriter(ENGINE).init_db()
    return dict(
        version=version,
        grants=grants,
        strict_sql_mode=sqlmode,
        source="fresh local execution",
        synthetic_only=True,
    )


def audit_rollback():
    name = table_name("rollback")
    batch = str(uuid4())
    target = TargetSettings(table=name, mode="append")
    original = AuditWriter.record_success

    def fail(self, conn, result, source, counts):
        original(self, conn, result, source, counts)
        raise RuntimeError("injected audit failure after target and audit inserts")

    with patch.object(AuditWriter, "record_success", fail):
        try:
            LOADER.load([candidate()], batch, Customer, target)
        except RuntimeError:
            pass
        else:
            raise AssertionError("Fault did not trigger")
    assert rows(name) == [] and rows("stg_" + name) == [] and committed(batch) is None
    with ENGINE.connect() as c:
        assert (
            c.execute(
                text("SELECT COUNT(*) FROM etl_run_audit WHERE batch_id=:b"),
                {"b": batch},
            ).scalar_one()
            == 0
        )
    recovered = LOADER.load([candidate()], batch, Customer, target)
    assert recovered.loaded == 1 and not recovered.replayed
    return dict(
        batch_id=batch,
        all_four_tables_rolled_back=True,
        same_uuid_retry_loaded_once=True,
    )


def lost_commit_ack():
    name = table_name("lostack")
    batch = str(uuid4())
    target = TargetSettings(table=name, mode="append")
    original = AuditWriter.record_success

    def install(self, conn, result, source, counts):
        original(self, conn, result, source, counts)

```

```
        def lost_ack(connection):
            connection.connection.driver_connection.commit()
            connection.invalidate()
            raise ConnectionError(
                "injected disconnect after real DBAPI commit before caller acknowledgement"
            )

        event.listen(conn, "commit", lost_ack, once=True)

    with patch.object(AuditWriter, "record_success", install):
        first = LOADER.load([candidate()], batch, Customer, target)
    assert first.replayed and first.loaded == 1 and committed(batch) is not None
    before = fingerprint(rows(name))
    retry = LOADER.load([candidate()], batch, Customer, target)
    assert retry.replayed and fingerprint(rows(name)) == before
    return dict(
        batch_id=batch,
        physical_sha256=before,
        ledger_resolved_success=True,
        duplicate_rows=0,
        method="Actual DBAPI COMMIT followed by socket invalidation and injected lost acknowledgement; not a server power failure",
    )


def crash_worker(name, batch, signal):
    original = LOADER._merge

    def pause(conn, table, records, target):
        result = original(conn, table, records, target)
        Path(signal).write_text("uncommitted target writes completed")
        time.sleep(90)
        return result

    LOADER._merge = pause
    LOADER.load([candidate()], batch, Customer, TargetSettings(table=name, mode="append"))


def process_kill():
    name = table_name("kill")
    batch = str(uuid4())
    signal = OUT / (name + ".signal")
    log = (OUT / (name + ".log")).open("w")
    proc = subprocess.Popen(
        [sys.executable, __file__, "--crash-worker", name, batch, str(signal)],
        stdout=log,
        stderr=log,
    )
    deadline = time.monotonic() + 30
    while not signal.exists() and proc.poll() is None and time.monotonic() < deadline:
        time.sleep(0.1)
    assert signal.exists(), "Worker never reached uncommitted writes"
    proc.kill()
    proc.wait(timeout=10)
    log.close()
    for _ in range(100):
        if not rows(name) and committed(batch) is None:
            break
        time.sleep(0.1)
    assert not rows(name) and not rows("stg_" + name) and committed(batch) is None
    status = committed(batch)
    retry = LOADER.load([candidate()], batch, Customer, TargetSettings(table=name, mode="append"))
    assert retry.loaded == 1 and len(rows(name)) == 1
    return dict(
        batch_id=batch,
        killed_process_returncode=proc.returncode,
        ledger_before_retry=status,
        uncommitted_rows_rolled_back=True,
        stable_uuid_retry_succeeded=True,
    )


def changed_input():
    name = table_name("changed")
    batch = str(uuid4())
    target = TargetSettings(table=name, mode="append")
    LOADER.load([candidate()], batch, Customer, target)
    before = fingerprint(rows(name))
    try:
        LOADER.load([candidate(name="Changed Synthetic")], batch, Customer, target)
    except BatchConflictError:
        pass
```

A
B
C
D
E
F
G
H

```
    else:
        raise AssertionError("Changed input accepted")
    assert fingerprint(rows(name)) == before
    return dict(batch_id=batch, changed_input_refused=True,
physical_sha256_unchanged=True)


def lock_timeout():
    name = table_name("lock")
    batch = str(uuid4())
    target = TargetSettings(table=name, mode="append", lock_timeout_seconds=1)
    with ENGINE.connect() as c:
        assert (
            c.execute(text("SELECT GET_LOCK(:k,0)"), {"k": "etlguard:table:" +
name}).scalar_one()
            == 1
        )
        c.commit()
        start = time.perf_counter()
        try:
            LOADER.load([candidate()], batch, Customer, target)
        except TimeoutError:
            pass
        else:
            raise AssertionError("Lock timeout not enforced")
        elapsed = time.perf_counter() - start
        c.execute(text("SELECT RELEASE_LOCK(:k)"), {"k": "etlguard:table:" + name})
        c.commit()
    assert committed(batch) is None
    assert LOADER.load([candidate()], batch, Customer, target).loaded == 1
    return dict(
        batch_id=batch,
        bounded_lock_timeout_seconds=elapsed,
        recovery_after_release=True,
    )


def scd_concurrency():
    name = table_name("scd")
    target = TargetSettings(
        table=name,
        mode="scd2",
        key=["customer_id"],
        tracked_columns=["full_name"],
        lock_timeout_seconds=30,
    )
    initial = [candidate(f"C{i:06d}") for i in range(50)]
    LOADER.load(initial, str(uuid4()), Customer, target)
    barrier = threading.Barrier(8)

    def worker(i):
        batch = str(uuid4())
        barrier.wait(timeout=20)
        start = time.perf_counter()
        counts = LOADER.load(
            [candidate(f"C{k:06d}", f"Worker {i} Person") for k in range(50)],
            batch,
            Customer,
            target,
        )
        return dict(
            batch_id=batch,
            elapsed_seconds=time.perf_counter() - start,
            versions_created=counts.versions_created,
        )

    with ThreadPoolExecutor(max_workers=8) as ex:
        results = list(ex.map(worker, range(8)))
    data = rows(name)
    active = [x for x in data if x["is_current"]]
    assert len(data) == 450 and len(active) == 50 and len({x["customer_id"] for x in
active}) == 50
    assert all(x["versions_created"] == 50 for x in results)
    for key in {x["customer_id"] for x in data}:
        versions = sorted(
            [x for x in data if x["customer_id"] == key],
            key=lambda x: x["effective_from"],
        )
        assert all(
            a["effective_to"] == b["effective_from"]
            for a, b in zip(versions, versions[1:], strict=False)
        )
    return dict(
        writers=8,
        keys=50,
```

A B C D E F G H

```
        total_versions=len(data),
        active_versions=len(active),
        worker_results=results,
        history_intervals_contiguous=True,
    )


def replacement():
    directory = OUT / "replacement"
    directory.mkdir()
    csvpath = directory / "source.csv"
    cfgpath = directory / "pipeline.yaml"
    name = table_name("replace")
    target = TargetSettings(table=name, mode="truncate_insert", min_replace_ratio=0.8)
    LOADER.load([candidate(f"C{i:06d}") for i in range(10)], str(uuid4()), Customer, target)
    before = fingerprint(rows(name))
    os.environ["ETLGUARD_OPS_MYSQL_URL"] = URL
    cfg = {
        "pipeline": "replacement_guard",
        "source": {"type": "csv", "path": str(csvpath)},
        "model": "etlguard.examples.models:Customer",
        "cleaning": {
            "date_columns": ["signup_date"],
            "number_columns": ["credit_limit"],
        },
        "target": target.model_dump(),
        "database_url_env": "ETLGUARD_OPS_MYSQL_URL",
        "policy": {"fail_if_reject_rate_above": 1.0},
        "output": {"directory": str(directory / "reports")},
    }
    cfgpath.write_text(yaml.safe_dump(cfg))
    header = "customer_id,full_name,email,signup_date,status,credit_limit\n"
    cases = {
        "empty": header,
        "partial": header + "C000001,Alice Person,bad,2026-01-01,active,100\n",
        "small_ratio": header + "C000001,Alice Person,a@example.test,2026-01-01,active,100\n",
    }
    results = {}
    for label, data in cases.items():
        csvpath.write_text(data)
        batch = str(uuid4())
        result = Pipeline.from_yaml(cfgpath).run(batch_id=batch)
        assert not result.ok and fingerprint(rows(name)) == before and committed(batch) is None
        results[label] = {
            "batch_id": batch,
            "status": result.status,
            "failures": result.failures,
            "target_preserved": True,
        }
    return results


class Monitor:
    def __init__(self, label):
        self.label = label
        self.done = threading.Event()
        self.samples = []
        self.thread = threading.Thread(target=self.run)

    def run(self):
        me = psutil.Process()
        db = psutil.Process(CFG["pid"])
        me.cpu_percent()
        db.cpu_percent()
        while not self.done.wait(0.2):
            t = time.perf_counter()
            with ENGINE.connect() as c:
                c.execute(text("SELECT COUNT(*) FROM etl_batch_commits")).scalar_one()
            self.samples.append(
                dict(
                    timestamp_utc=now(),
                    python_rss=me.memory_info().rss,
                    mysql_rss=db.memory_info().rss,
                    python_cpu_percent=me.cpu_percent(),
                    mysql_cpu_percent=db.cpu_percent(),
                    probe_ms=(time.perf_counter() - t) * 1000,
                )
            )

    def __enter__(self):
        self.thread.start()
        return self
```

```
0426
0427      def __exit__(self, *exc):
0428          self.done.set()
0429          self.thread.join()
0430          save("metrics_" + self.label, self.samples)
0431
0432      def summary(self):
0433          assert self.samples, "No monitoring samples collected"
0434          lat = sorted(x["probe_ms"] for x in self.samples)
0435          return dict(
0436              samples=len(self.samples),
0437              peak_combined_rss_mib=max(
0438                  (x["python_rss"] + x["mysql_rss"]) / 2**20 for x in self.samples
0439              ),
0440              peak_python_cpu_percent=max(x["python_cpu_percent"] for x in self.samples),
0441              peak_mysql_cpu_percent=max(x["mysql_cpu_percent"] for x in self.samples),
0442              probe_p50_ms=statistics.median(lat),
0443              probe_p95_ms=lat[min(len(lat) - 1, int(len(lat) * 0.95))],
0444          )
0445
0446
0447  def directory_bytes(path):
0448      return sum(p.stat().st_size for p in Path(path).rglob("*") if p.is_file())
0449
0450
0451  def performance():
0452      directory = OUT / "performance"
0453      directory.mkdir()
0454      os.environ["ETLGUARD_OPS_MYSQL_URL"] = URL
0455      measurements = []
0456      criteria = {
0457          "min_rows_per_second": 500,
0458          "max_batch_seconds": 180,
0459          "max_combined_rss_mib": 2048,
0460          "max_probe_p95_ms": 250,
0461          "min_free_disk_gib": 1,
0462          "owner_approved": False,
0463          "scope": "proposed local synthetic acceptance",
0464      }
0465      for n in (10000, 50000, 100000):
0466          data = directory / f"input-{n}.csv"
0467          config = directory / f"pipeline-{n}.yaml"
0468          name = table_name("perf" + str(n))
0469          batch = str(uuid4())
0470          with data.open("w") as f:
0471              f.write("customer_id,full_name,email,signup_date,status,credit_limit\n")
0472              for i in range(n):
0473                  f.write(
0474                      f"C{i:07d},Synthetic Person
      {i},c{i}@example.test,2026-01-01,active,100.00\n"
0475                  )
0476          cfg = {
0477              "pipeline": "performance_" + str(n),
0478              "source": {"type": "csv", "path": str(data)},
0479              "model": "etlguard.examples.models:Customer",
0480              "cleaning": {
0481                  "date_columns": ["signup_date"],
0482                  "number_columns": ["credit_limit"],
0483              },
0484              "checks": {"unique": ["customer_id"]},
0485              "target": {
0486                  "table": name,
0487                  "mode": "upsert",
0488                  "key": ["customer_id"],
0489                  "batch_size": 1000,
0490              },
0491              "database_url_env": "ETLGUARD_OPS_MYSQL_URL",
0492              "output": {"directory": str(directory / "reports")},
0493          }
0494          config.write_text(yaml.safe_dump(cfg))
0495          disk_before = directory_bytes(CFG["datadir"])
0496          with ENGINE.connect() as c:
0497              waits_before = int(
0498                  c.execute(text("SHOW GLOBAL STATUS LIKE 'Innodb_log_waits'")).one()[1]
0499              )
0500          with Monitor(str(n)) as monitor:
0501              start = time.perf_counter()
0502              result = Pipeline.from_yaml(config).run(batch_id=batch)
0503              elapsed = time.perf_counter() - start
0504          assert result.ok and result.rows_loaded == n, result.failures
0505          first_hash = fingerprint(rows(name))
0506          start = time.perf_counter()
0507          retry = Pipeline.from_yaml(config).run(batch_id=batch)
0508          retry_seconds = time.perf_counter() - start
0509          assert retry.ok and retry.replayed and first_hash == fingerprint(rows(name))
```

```
        with ENGINE.connect() as c:
            plan = [
                dict(x)
                for x in c.execute(
                    text(f"EXPLAIN SELECT * FROM `{name}` WHERE customer_id='C0000001'")
                ).mappings()
            ]
            waits_after = int(
                c.execute(text("SHOW GLOBAL STATUS LIKE 'Innodb_log_waits'")).one()[1]
            )
            stats = dict(
                c.execute(
                    text(
                        "SHOW GLOBAL STATUS WHERE Variable_name IN ('Threads_connected','Threads_running','Innodb_buffer_pool_reads','Innodb_row_lock_waits','Innodb_deadlocks')"
                    )
                ).all()
            )
        metrics = monitor.summary()
        free = psutil.disk_usage(CFG["datadir"]).free / 2**30
        record = dict(
            rows=n,
            batch_id=batch,
            end_to_end_seconds=elapsed,
            rows_per_second=n / elapsed,
            retry_seconds=retry_seconds,
            input_bytes=data.stat().st_size,
            input_sha256=hashlib.sha256(data.read_bytes()).hexdigest(),
            target_physical_sha256=first_hash,
            database_disk_growth_bytes=directory_bytes(CFG["datadir"]) - disk_before,
            free_disk_gib=free,
            innodb_log_waits_delta=waits_after - waits_before,
            explain=plan,
            mysql_status=stats,
            **metrics,
        )
        record["criteria_passed"] = (
            n / elapsed >= criteria["min_rows_per_second"]
            and elapsed <= criteria["max_batch_seconds"]
            and metrics["peak_combined_rss_mib"] <= criteria["max_combined_rss_mib"]
            and metrics["probe_p95_ms"] <= criteria["max_probe_p95_ms"]
            and free >= criteria["min_free_disk_gib"]
            and waits_after == waits_before
            and all(x.get("key") for x in plan)
        )
        measurements.append(record)
        save("performance_progress", {"criteria": criteria, "measurements": measurements})
        assert record["criteria_passed"], record
    return dict(
        criteria=criteria,
        measurements=measurements,
        hardware={
            "logical_cpus": psutil.cpu_count(),
            "physical_cpus": psutil.cpu_count(logical=False),
            "ram_gib": psutil.virtual_memory().total / 2**30,
        },
        limitation="Synthetic Customer/upsert capacity only; production shape, volume and owner SLO approval outstanding",
    )


def monitoring():
    received = []

    class Handler(BaseHTTPRequestHandler):
        def do_POST(self):
            received.append(
                {
                    "received_at_utc": now(),
                    "payload":
json.loads(self.rfile.read(int(self.headers["Content-Length"]))),
                }
            )
            self.send_response(204)
            self.end_headers()

        def log_message(self, *args):
            pass

    server = ThreadingHTTPServer(("127.0.0.1", 0), Handler)
    thread = threading.Thread(target=server.serve_forever)
    thread.start()
    import urllib.request

```

A
B
C
D
E
F
G
H

```
    batch = str(uuid4())
    audit = AuditWriter(ENGINE)
    from etlguard.pipeline import RunResult

    at = datetime.now(UTC)
    failure = RunResult(
        batch_id=batch,
        pipeline="alert_drill",
        source="synthetic",
        started_at=at,
        finished_at=at,
        status="failed",
        rows_read=1,
        rows_rejected=1,
        failures=["synthetic alert drill"],
    )
    audit.write(failure, "synthetic")
    start = time.perf_counter()
    try:
        with ENGINE.connect() as c:
            status = c.execute(
                text("SELECT status FROM etl_run_audit WHERE batch_id=:b"), {"b": batch}
            ).scalar_one()
        assert status == "failed"
        payload = {
            "alert": "etl_job_failed",
            "batch_id": batch,
            "pipeline": "alert_drill",
            "timestamp_utc": now(),
        }
        req = urllib.request.Request(
            f"http://127.0.0.1:{server.server_port}/alerts",
            json.dumps(payload).encode(),
            {"Content-Type": "application/json"},
        )
        with urllib.request.urlopen(req, timeout=5) as response:
            assert response.status == 204
        assert received and received[0]["payload"]["batch_id"] == batch
        return dict(
            batch_id=batch,
            alert_receipts=received,
            detect_and_deliver_seconds=time.perf_counter() - start,
            receiver="actual local HTTP receiver",
            production_notification_route_tested=False,
        )
    finally:
        server.shutdown()
        server.server_close()
        thread.join()


def snapshot(engine):
    result = {}
    for name in inspect(engine).get_table_names():
        table = Table(name, MetaData(), autoload_with=engine)
        with engine.connect() as c:
            data = [dict(x) for x in c.execute(select(table)).mappings()]
        result[name] = {"rows": len(data), "sha256": fingerprint(data)}
    return result


def backup_restore():
    directory = OUT / "backup-restore"
    directory.mkdir()
    before = snapshot(ENGINE)
    dump = directory / "synthetic-ops.sql"

    def options(schema):
        u = CFG["users"][schema]
        p = ROOT / ".private" / ("client-" + schema + ".cnf")
        p.write_text(
            f"[client]\nuser={u['user']}\npassword={u['password']}\nhost=127.0.0.1\nport=
{CFG['port']}\nprotocol=TCP\n"
        )
        p.chmod(0o600)
        return p

    t0 = now()
    start = time.perf_counter()
    with dump.open("wb") as f:
        subprocess.run(
            [
                str(Path(CFG["bin"]) / "mysqldump"),
                "--defaults-extra-file=" + str(options("etlguard_ops_ci")),
                "--single-transaction",
```

```
                "--skip-lock-tables",
                "--no-tablespaces",
                "--set-gtid-purged=OFF",
                "etlguard_ops_ci",
            ],
            stdout=f,
            stderr=subprocess.PIPE,
            check=True,
        )
    backup_seconds = time.perf_counter() - start
    restore_engine = create_engine(CFG["users"]["etlguard_restore_ci"]["url"])
    assert not inspect(restore_engine).get_table_names(), "Restore schema is not empty"
    # One deliberate post-backup transaction proves recovery to the dump point.
    late = table_name("afterbackup")
    late_batch = str(uuid4())
    LOADER.load([candidate()], late_batch, Customer, TargetSettings(table=late,
mode="append"))
    start = time.perf_counter()
    with dump.open("rb") as f:
        subprocess.run(
            [
                str(Path(CFG["bin"]) / "mysql"),
                "--defaults-extra-file=" + str(options("etlguard_restore_ci")),
                "etlguard_restore_ci",
            ],
            stdin=f,
            stdout=subprocess.PIPE,
            stderr=subprocess.PIPE,
            check=True,
        )
    restore_seconds = time.perf_counter() - start
    start = time.perf_counter()
    after = snapshot(restore_engine)
    verify_seconds = time.perf_counter() - start
    restore_engine.dispose()
    assert before == after, "Restored row fingerprints differ"
    assert late not in after and restore_seconds + verify_seconds <= 120
    result = dict(
        backup_started_at_utc=t0,
        backup_seconds=backup_seconds,
        restore_seconds=restore_seconds,
        verification_seconds=verify_seconds,
        observed_recovery_seconds=restore_seconds + verify_seconds,
        proposed_rto_seconds=120,
        backup_bytes=dump.stat().st_size,
        backup_sha256=hashlib.sha256(dump.read_bytes()).hexdigest(),
        tables_verified=len(before),
        rows_verified=sum(x["rows"] for x in before.values()),
        before=before,
        after=after,
        all_rows_and_physical_ids_match=True,
        recovery_point="consistent logical backup snapshot; deliberate post-backup
transaction absent",
        post_backup_batch_id=late_batch,
        post_backup_target_absent_from_restore=True,
        pitr_exercised=False,
        production_rpo_approved=False,
    )
    save("backup_manifest", result)
    return result


def main():
    if len(sys.argv) > 1 and sys.argv[1] == "--crash-worker":
        crash_worker(*sys.argv[2:])
        return
    for name, fn in [
        ("preflight", preflight),
        ("audit_rollback", audit_rollback),
        ("lost_commit_ack", lost_commit_ack),
        ("process_kill", process_kill),
        ("changed_input", changed_input),
        ("lock_timeout", lock_timeout),
        ("scd2_concurrency", scd_concurrency),
        ("replacement_guards", replacement),
        ("performance", performance),
        ("monitoring", monitoring),
        ("backup_restore", backup_restore),
    ]:
        check(name, fn)
    save(
        "summary",
        {
            "timestamp_utc": now(),
            "status": "passed" if all(x["status"] == "passed" for x in RESULTS) else
```

```
    "failed",
0758            "checks": RESULTS,
0759            "production_approved": False,
0760        },
0761    )
0762    ENGINE.dispose()
0763    raise SystemExit(0 if all(x["status"] == "passed" for x in RESULTS) else 1)
0764
0765
0766 if __name__ == "__main__":
0767     main()
```

operations/run_soak.py

91 source lines.

```
0001 """Two-minute synthetic local contention and monitoring observation."""
0002
0003 import json
0004 import threading
0005 import time
0006 from concurrent.futures import ThreadPoolExecutor
0007 from uuid import uuid4
0008
0009 from run_operational_acceptance import (
0010     ENGINE,
0011     LOADER,
0012     OUT,
0013     Customer,
0014     Monitor,
0015     TargetSettings,
0016     candidate,
0017     now,
0018     rows,
0019     table_name,
0020 )
0021
0022 duration = 120
0023 names = [table_name("soak") for _ in range(3)]
0024 targets = [
0025     TargetSettings(
0026         table=n,
0027         mode="upsert",
0028         key=["customer_id"],
0029         batch_size=1000,
0030         lock_timeout_seconds=10,
0031     )
0032     for n in names
0033 ]
0034 for t in targets:
0035     LOADER.load([candidate()], str(uuid4()), Customer, t)
0036 barrier = threading.Barrier(3)
0037 stop = time.monotonic() + duration
0038 counts = []
0039
0040
0041 def worker(index):
0042     completed = []
0043     barrier.wait(timeout=10)
0044     cycle = 0
0045     while time.monotonic() < stop:
0046         data = [candidate(f"C{i:06d}", f"Soak worker {index} cycle {cycle}") for i in
     range(2000)]
0047         batch = str(uuid4())
0048         start = time.perf_counter()
0049         r = LOADER.load(data, batch, Customer, targets[index])
0050         assert r.loaded == 2000
0051         completed.append({"batch_id": batch, "seconds": time.perf_counter() - start,
     "rows": 2000})
0052         cycle += 1
0053         time.sleep(0.5)
0054     return {"worker": index, "batches": completed, "target": names[index]}
0055
0056
0057 with Monitor("soak120") as monitor:
0058     start = time.perf_counter()
0059     with ThreadPoolExecutor(max_workers=3) as ex:
0060         counts = list(ex.map(worker, range(3)))
0061     elapsed = time.perf_counter() - start
0062 assert all(len(rows(name)) == 2000 for name in names)
0063 metrics = monitor.summary()
```

A
B
C
D
E
F
G
H

```
report = {
    "status": "passed",
    "timestamp_utc": now(),
    "observed_seconds": elapsed,
    "workers": 3,
    "rows_per_batch": 2000,
    "completed_batches": sum(len(x["batches"]) for x in counts),
    "worker_results": counts,
    "metrics": metrics,
    "criteria": {
        "zero_failed_batches": True,
        "combined_rss_cap_mib": 2048,
        "probe_p95_cap_ms": 250,
        "local_window_seconds": duration,
        "production_soak": False,
    },
}
assert metrics["peak_combined_rss_mib"] <= 2048 and metrics["probe_p95_ms"] <= 250
(OUT / "soak.json").write_text(json.dumps(report, indent=2))
print(
    "Soak passed",
    report["completed_batches"],
    "batches",
    round(elapsed, 2),
    "seconds",
    flush=True,
)
ENGINE.dispose()
```

operations/run_staging.py

45 source lines.

```
"""Run the supplied one-shot using only this run's generated disposable identities."""

from pathlib import Path
import json
import os
import subprocess
import sys

root = Path(__file__).resolve().parents[1]
cfg = json.loads((root / ".private/runtime.json").read_text())
assert Path(cfg["datadir"]).resolve() == root / ".private/mysql-data"
project = root / "source/etl-validation-toolkit"
output = root / "fresh-evidence/staging"
qa = root / "fresh-evidence/qa"
qa.mkdir(exist_ok=True)
env = {k: v for k, v in os.environ.items() if not k.startswith("ETLGUARD_")}
env["ETLGUARD_TEST_MYSQL_URL"] = cfg["users"]["etlguard_ci"]["url"]
env["ETLGUARD_SHADOW_MYSQL_URL"] = cfg["users"]["etlguard_shadow_ci"]["url"]
env["PYTEST_ADDOPTS"] = "--junitxml=" + str(qa / "pytest.xml")
env["COVERAGE_FILE"] = str(qa / ".coverage")
code = subprocess.call(
    [
        sys.executable,
        str(root / "local_staging_one_shot.py"),
        "--project",
        str(project),
        "--output",
        str(output),
        "--confirm-disposable-schema",
        "etlguard_ci",
        "--confirm-shadow-schema",
        "etlguard_shadow_ci",
    ],
    cwd=project,
    env=env,
)
if code == 0:
    for fmt in ("json", "xml"):
        subprocess.run(
            [sys.executable, "-m", "coverage", fmt, "-o", str(qa / ("coverage." + fmt))],
            cwd=project,
            env=env,
            check=True,
        )
raise SystemExit(code)
```

operations/server_crash_drill.py

117 source lines.

```
"""Stage or verify SIGKILL recovery of this run's private MySQL instance."""

import json
import os
import signal
import subprocess
import sys
import time
from pathlib import Path
from uuid import uuid4

import psutil
from run_operational_acceptance import (
    CFG,
    ENGINE,
    LOADER,
    OUT,
    ROOT,
    Customer,
    TargetSettings,
    candidate,
    committed,
    fingerprint,
    now,
    rows,
    table_name,
)

file = OUT / "server-crash-pending.json"
if sys.argv[1] == "stage":
    assert not file.exists()
    ack = json.loads((OUT / "lost_commit_ack.json").read_text())["details"]
    previous = committed(ack["batch_id"])
    name = previous["target_table"]
    before = fingerprint(rows(name))
    target = table_name("dbcrash")
    batch = str(uuid4())
    marker = OUT / (target + ".signal")
    worker = subprocess.Popen(
        [
            sys.executable,
            str(Path(__file__).with_name("run_operational_acceptance.py")),
            "--crash-worker",
            target,
            batch,
            str(marker),
        ],
        stdout=subprocess.DEVNULL,
        stderr=subprocess.DEVNULL,
    )
    deadline = time.monotonic() + 30
    while not marker.exists() and worker.poll() is None and time.monotonic() < deadline:
        time.sleep(0.05)
    assert marker.exists(), "Uncommitted worker not ready"
    db = psutil.Process(CFG["pid"])
    assert db.name() == "mysqld" and "--datadir=" + CFG["datadir"] in db.cmdline()
    record = {
        "staged_at_utc": now(),
        "committed_batch_id": ack["batch_id"],
        "committed_table": name,
        "before_physical_sha256": before,
        "uncommitted_table": target,
        "uncommitted_batch_id": batch,
        "mysql_pid": CFG["pid"],
        "log_offset": (ROOT / ".private/mysql-error.log").stat().st_size,
        "recovery_clock_start": time.time(),
    }
    file.write_text(json.dumps(record, indent=2))
    os.kill(CFG["pid"], signal.SIGKILL)
    worker.kill()
    worker.wait(timeout=10)
    print("Stopped only the new disposable mysqld with SIGKILL; pending transaction
recorded.")
else:
    d = json.loads(file.read_text())
    for _ in range(120):
        try:
            with ENGINE.connect() as c:
                c.exec_driver_sql("SELECT 1")
            break
```

A
B
C
D
E
F
G
H

SECTION MAP

```
        except Exception:
            time.sleep(0.25)
    assert committed(d["committed_batch_id"]) is not None
    assert fingerprint(rows(d["committed_table"])) == d["before_physical_sha256"]
    assert committed(d["uncommitted_batch_id"]) is None
    assert rows(d["uncommitted_table"]) == [] and rows("stg_" + d["uncommitted_table"])
== []
    recovered = LOADER.load(
        [candidate()],
        d["uncommitted_batch_id"],
        Customer,
        TargetSettings(table=d["uncommitted_table"], mode="append"),
    )
    assert recovered.loaded == 1 and len(rows(d["uncommitted_table"])) == 1
    retry = LOADER.load(
        [candidate()],
        d["uncommitted_batch_id"],
        Customer,
        TargetSettings(table=d["uncommitted_table"], mode="append"),
    )
    assert retry.replayed and len(rows(d["uncommitted_table"])) == 1
    log = (
        (ROOT / ".private/mysql-error.log").read_bytes()[d["log_offset"]
:].decode(errors="replace")
    )
    (OUT / "server-crash-recovery.log").write_text(log)
    d.update(
        status="passed",
        verified_at_utc=now(),
        committed_data_survived=True,
        uncommitted_data_rolled_back=True,
        stable_uuid_recovered_then_replayed=True,
        observed_intervention_inclusive_recovery_seconds=time.time() -
d["recovery_clock_start"],
        physical_power_loss_tested=False,
    )
    (OUT / "server-crash-recovery.json").write_text(json.dumps(d, indent=2))
    print(
        "Server crash recovery passed; committed state preserved; uncommitted batch
safely retried."
    )
ENGINE.dispose()
```

operations/start_existing_mysql.py

34 source lines.

```
"""Restart only the private disposable instance provisioned by this handoff."""

from pathlib import Path
import json
import os

root = Path(__file__).resolve().parents[1]
p = root / ".private/runtime.json"
cfg = json.loads(p.read_text())
assert Path(cfg["datadir"]).resolve() == root / ".private/mysql-data"
assert cfg["socket"].startswith("/tmp/etlguard-")
cfg["pid"] = os.getpid()
p.write_text(json.dumps(cfg, indent=2))
p.chmod(0o600)
args = [
    str(Path(cfg["bin"]) / "mysqld"),
    "--no-defaults",
    "--datadir=" + cfg["datadir"],
    "--socket=" + cfg["socket"],
    "--port=" + str(cfg["port"]),
    "--bind-address=127.0.0.1",
    "--mysqlx=OFF",
    "--innodb-buffer-pool-size=268435456",
    "--max-connections=64",
    "--log-error=" + str(root / ".private/mysql-error.log"),
    "--log-bin=" + str(Path(cfg["datadir"]) / "binlog"),
    "--server-id=7651",
    "--innodb-flush-log-at-trx-commit=1",
    "--sync-binlog=1",
    "--slow-query-log=ON",
    "--long-query-time=0.1",
    "--slow-query-log-file=" + str(root / ".private/slow.log"),
]
os.execv(args[0], args)
```

A
B
C
D
E
F
G
H

local_staging_one_shot.py

125 source lines.

```
0001  #!/usr/bin/env python3
0002  """One-shot ISOLATED staging gate and same-batch shadow reconciliation only.
0003
0004  No production mode exists. Never removes containers, existing tables or volumes.
0005  Run only after installing the source's [dev] dependencies and reviewing the
0006  local disposable MySQL Docker Compose configuration.
0007  """
0008  from __future__ import annotations
0009
0010  import argparse
0011  from datetime import datetime, timezone
0012  import json
0013  import os
0014  from pathlib import Path
0015  import subprocess
0016  import sys
0017  from uuid import uuid4
0018
0019
0020  def run_step(name: str, args: list[str], cwd: Path, env: dict[str, str], output: Path) ->
       None:
0021      print(f"[STAGING] {name}", flush=True)
0022      logfile = output / (name + ".log")
0023      with logfile.open("w", encoding="utf-8") as sink:
0024          result = subprocess.run(args, cwd=cwd, env=env, stdout=sink,
      stderr=subprocess.STDOUT,
0025                                  text=True, check=False, timeout=1800)
0026      if result.returncode:
0027          raise RuntimeError(f"{name} exited {result.returncode}; see {logfile} (redact
      secrets before sharing)")
0028
0029
0030  def main() -> int:
0031      p = argparse.ArgumentParser(description=__doc__)
0032      p.add_argument("--project", type=Path, required=True)
0033      p.add_argument("--output", type=Path, required=True)
0034      p.add_argument("--confirm-disposable-schema", required=True)
0035      p.add_argument("--confirm-shadow-schema", required=True)
0036      p.add_argument("--start-docker", action="store_true", help="Start the supplied
      disposable MySQL 8.4 Compose service")
0037      args = p.parse_args()
0038      project = args.project.resolve()
0039      output = args.output.resolve()
0040      report = {"generated_at_utc": datetime.now(timezone.utc).isoformat(),
0041                "scope": "local_disposable_mysql_staging_only", "status": "blocked",
0042                "production_approved": False, "actions": []}
0043      try:
0044          if args.confirm_disposable_schema != "etlguard_ci" or args.confirm_shadow_schema
      != "etlguard_shadow_ci":
0045              raise ValueError("Only the provided disposable test and shadow schemas are
      supported")
0046          if not (project / "scripts/verify_mysql8_staging.py").exists():
0047              raise ValueError("--project must be the extracted v0.1.2 source project")
0048          if output.exists() and any(output.iterdir()):
0049              raise FileExistsError("Output directory must be new or empty; never overwrite
       pilot evidence")
0050          output.mkdir(parents=True, exist_ok=True)
0051          env = os.environ.copy()
0052          if args.start_docker:
0053              run_step("docker_start", ["docker", "compose", "-f",
      "docker-compose.mysql8-staging.yml",
0054                                        "up", "-d", "--wait"], project, env, output)
0055              env["ETLGUARD_TEST_MYSQL_URL"] = (
0056
      "mysql+pymysql://etlguard_ci:etlguard_ci_local_only@127.0.0.1:3308/etlguard_ci"
0057              )
0058              env["ETLGUARD_SHADOW_MYSQL_URL"] = (
0059                  "mysql+pymysql://etlguard_shadow:etlguard_shadow_local_only@127.0.0.1:330
      8/etlguard_shadow_ci"
0060              )
0061          # All URLs are vetted again by the toolkit's separate privilege and schema
      checks.
0062          if not env.get("ETLGUARD_TEST_MYSQL_URL") or not
      env.get("ETLGUARD_SHADOW_MYSQL_URL"):
0063              raise ValueError("Provide both ETLGUARD_TEST_MYSQL_URL and
      ETLGUARD_SHADOW_MYSQL_URL or use --start-docker")
```

A B C D E F G H

SECTION MAP

```
0064          python = sys.executable
0065          run_step("operator_safety_tests", [python, "-m", "unittest", "discover", "-s",
      "scripts",
0066                                             "-p", "test_*.py", "-v"], project, env, output)
0067          stage_file = output / "staging-gate-report.json"
0068          run_step("mysql8_staging_gate", [python, "scripts/verify_mysql8_staging.py",
0069                                           "--confirm-disposable-schema",
      args.confirm_disposable_schema,
0070                                           "--execute-tests", "--report", str(stage_file)],
       project, env, output)
0071          report["actions"].append("mysql8_staging_gate_passed")
0072          pilot = output / "immutable_pilot"
0073          sample = project / "src/etlguard/examples/pipelines/customers_clean.yaml"
0074          run_step("shadow_plan", [python, "scripts/prepare_shadow_pilot.py", "plan",
0075                                   "--config", str(sample), "--output", str(pilot)],
      project, env, output)
0076          manifest = pilot / "shadow-manifest.json"
0077          run_step("shadow_manifest_verify", [python, "scripts/prepare_shadow_pilot.py",
      "verify",
0078                                              "--manifest", str(manifest)], project, env,
      output)
0079          run_step("shadow_validate_no_writes", [python, "scripts/run_shadow_pilot.py",
      "--manifest",
0080                                                 str(manifest), "--validate", "--report",
0081                                                 str(output / "shadow-validation.json")],
      project, env, output)
0082          def load_and_check(tag: str) -> None:
0083              run_step("shadow_load_" + tag, [python, "scripts/run_shadow_pilot.py",
      "--manifest", str(manifest),
0084                                              "--execute", "--confirm-shadow-schema",
      args.confirm_shadow_schema,
0085                                              "--staging-report", str(stage_file),
      "--report",
0086                                              str(output / ("shadow-load-" + tag +
      ".json"))], project, env, output)
0087              run_step("shadow_reconciliation_" + tag, [python,
      "scripts/verify_shadow_reconciliation.py",
0088                                                        "--manifest", str(manifest),
      "--staging-report",
0089                                                        str(stage_file),
      "--confirm-shadow-schema", args.confirm_shadow_schema,
0090                                                        "--report", str(output /
      ("shadow-reconciliation-" + tag + ".json"))],
0091                       project, env, output)
0092          load_and_check("first")
0093          # Never rerun after an uncertain or failed commit; only after a confirmed
      successful load.
0094          load_and_check("identical_retry")
0095          first_load = json.loads((output /
      "shadow-load-first.json").read_text(encoding="utf-8"))
0096          retry_load = json.loads((output /
      "shadow-load-identical_retry.json").read_text(encoding="utf-8"))
0097          before = json.loads((output /
      "shadow-reconciliation-first.json").read_text(encoding="utf-8"))
0098          after = json.loads((output /
      "shadow-reconciliation-identical_retry.json").read_text(encoding="utf-8"))
0099          assert first_load["status"] == "shadow_committed_not_production" and
      first_load["replayed"] is False
0100          assert retry_load["status"] == "shadow_committed_not_production" and
      retry_load["replayed"] is True
0101          assert retry_load["batch_id"] == first_load["batch_id"]
0102          assert before["status"] == after["status"] == "passed"
0103          assert before["target_physical_sha256"] == after["target_physical_sha256"]
0104          report.update({"status": "staging_and_shadow_passed_not_production",
0105                         "batch_id": first_load["batch_id"],
0106                         "rows_loaded": first_load["rows_loaded"],
0107                         "idempotent_retry_verified": True,
0108                         "physical_target_unchanged": True,
0109                         "checks": ["mysql8_integration", "complete_suite", "ruff", "mypy",
0110                                    "source_shadow_reconciliation", "identical_retry"]})
0111      except Exception as exc:
0112          report.update({"status": "blocked", "error_type": type(exc).__name__,
0113                         "error": str(exc).split(" (redact secrets")[0]})
0114          print(f"NO-GO: {type(exc).__name__} (see report/evidence directory)",
      file=sys.stderr)
0115      finally:
0116          output.mkdir(parents=True, exist_ok=True)
0117          (output / "one-shot-staging-summary.json").write_text(
0118              json.dumps(report, sort_keys=True, indent=2) + "\n", encoding="utf-8"
0119          )
0120          print(f"Status: {report['status']}. Evidence: {output}")
0121      return 0 if report["status"] == "staging_and_shadow_passed_not_production" else 2
0122
0123
```

```
if __name__ == "__main__":
    raise SystemExit(main())
```

44. Appendix E - Configuration, CI and browser assets

Exact configuration examples and browser entrypoints are shown below. Local-only development credentials, if present in these examples, must not be reused in production. The primary release artifact preserves original formatting.

source/etl-validation-toolkit/pyproject.toml

```
[build-system]
requires = ["setuptools>=75", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "etl-validation-toolkit"
version = "0.1.3"
description = "Typed, config-driven ETL data quality gates and MySQL load reconciliation."
readme = "README.md"
requires-python = ">=3.11"
license = "MIT"
authors = [{name = "ETL Guard Contributors"}]
keywords = ["etl", "data-quality", "validation", "pydantic", "mysql", "scd2"]
classifiers = ["Development Status :: 3 - Alpha", "Programming Language :: Python :: 3", "Intended Audience :: Developers"]
dependencies = [
  "pydantic>=2.10,<3",
  "pandas>=2.2,<3",
  "numpy>=1.26,<2.4",
  "openpyxl>=3.1,<4",
  "httpx>=0.27,<1",
  "SQLAlchemy>=2.0,<3",
  "PyMySQL>=1.1,<2",
  "typer>=0.12,<1",
  "PyYAML>=6,<7",
  "rich>=13,<16",
]

[project.optional-dependencies]
dev = ["pytest>=8,<10", "pytest-cov>=5,<8", "hypothesis>=6,<7", "respx>=0.21,<1", "ruff>=0.11,<1", "mypy>=1.15,<2", "pandas-stubs>=2.2,<3", "types-PyYAML>=6", "build>=1.2"]

[project.scripts]
etlguard = "etlguard.cli:app"

[project.urls]
Repository = "https://github.com/theraghavraman/etl-validation-toolkit"

[tool.setuptools]
package-dir = {"" = "src"}
include-package-data = true

[tool.setuptools.packages.find]
where = ["src"]

[tool.setuptools.package-data]
"etlguard.examples" = ["pipelines/*.yaml", "data/*.csv", "data/*.xlsx"]

[tool.pytest.ini_options]
testpaths = ["tests"]
markers = ["integration: needs a real MySQL 8 server"]
addopts = "-ra"

[tool.coverage.run]
source = ["etlguard"]
branch = true
omit = ["src/etlguard/examples/*", "src/etlguard/browser_demo.py"]

[tool.coverage.report]
show_missing = true
exclude_lines = ["pragma: no cover", "if TYPE_CHECKING:"]

[tool.ruff]
line-length = 100
```

A
B
C
D
E
F
G
H

```
target-version = "py311"

[tool.ruff.lint]
select = ["E4", "E7", "E9", "F", "B"]

[tool.mypy]
python_version = "3.11"
check_untyped_defs = true
warn_unused_ignores = true
warn_redundant_casts = true
no_implicit_optional = true
ignore_missing_imports = true
exclude = ["tests/", "scripts/"]
```

source/etl-validation-toolkit/.env.example

```
# Docker Compose local-only credentials (development; change for real deployments).
# Docker Compose exposes MySQL only on 127.0.0.1:3307.
ETLGUARD_DATABASE_URL=mysql+pymysql://etlguard:etlguard_local_only@127.0.0.1:3307/etlguard
# Example: API_SOURCE_TOKEN=replace-with-a-real-credential
```

source/etl-validation-toolkit/docker-compose.yml

```
services:
  mysql:
    image: mysql:8.0
    container_name: etlguard-mysql
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: root_local_only
      MYSQL_DATABASE: etlguard
      MYSQL_USER: etlguard
      MYSQL_PASSWORD: etlguard_local_only
    ports:
      - "127.0.0.1:3307:3306"
    healthcheck:
      test: ["CMD-SHELL", "mysqladmin ping -h 127.0.0.1 -u root -p$$MYSQL_ROOT_PASSWORD --silent"]
      interval: 5s
      timeout: 3s
      retries: 20
    volumes:
      - mysql-data:/var/lib/mysql
volumes:
  mysql-data:
```

source/etl-validation-toolkit/docker-compose.mysql8-staging.yml

```
# Disposable LOCAL test database only. Never attach real production volumes or data.
services:
  mysql8-staging:
    image: mysql:8.4
    environment:
      MYSQL_ROOT_PASSWORD: root_for_disposable_tests_only
      MYSQL_DATABASE: etlguard_ci
      MYSQL_USER: etlguard_ci
      MYSQL_PASSWORD: etlguard_ci_local_only
    ports:
      - "127.0.0.1:3308:3306"
    healthcheck:
      test: ["CMD-SHELL", "mysqladmin ping -h 127.0.0.1 -u root -p$$MYSQL_ROOT_PASSWORD --silent"]
      interval: 5s
      timeout: 5s
      retries: 40
      start_period: 30s
    volumes:
      - ./scripts/mysql8_init_shadow.sql:/docker-entrypoint-initdb.d/10-shadow.sql:ro
    tmpfs:
      - /var/lib/mysql:rw,size=512m
```

source/etl-validation-toolkit/src/etlguard/examples/pipelines/customers.yaml

```yaml
pipeline: customers_daily
source:
  type: csv
  path: ../data/customers_messy.csv
  chunksize: 1000
model: etlguard.examples.models:Customer
cleaning:
  trim: true
  null_tokens: ["", NA, N/A, "null", "-"]
  date_columns: [signup_date, end_date]
  number_columns: [credit_limit]
  lowercase: [email, status]
  uppercase: [customer_id]
  drop_duplicates: true
checks:
  unique: [customer_id]
  max_null_pct: {email: 0.5}
  expected_row_count: {min: 1, max: 100000}
target:
  table: customers
  mode: upsert
  key: [customer_id]
  batch_size: 500
policy:
  fail_if_reject_rate_above: 0.05
output: {directory: ../../../../reports}
```

source/etl-validation-toolkit/src/etlguard/examples/pipelines/customers_clean.yaml

```yaml
pipeline: customers_clean
source: {type: csv, path: ../data/customers_clean.csv}
model: etlguard.examples.models:Customer
cleaning:
  date_columns: [signup_date, end_date]
  number_columns: [credit_limit]
checks: {unique: [customer_id], max_null_pct: {email: 0.10}}
target: {table: customers_clean, mode: upsert, key: [customer_id]}
policy: {fail_if_reject_rate_above: 0.05}
output: {directory: ../../../../reports}
```

source/etl-validation-toolkit/src/etlguard/examples/pipelines/customers_excel.yaml

```yaml
pipeline: customers_excel
source:
  type: excel
  path: ../data/customers_messy.xlsx
  sheet: 0
  header_row: 1
model: etlguard.examples.models:Customer
cleaning:
  date_columns: [signup_date, end_date]
  number_columns: [credit_limit]
  lowercase: [email, status]
  uppercase: [customer_id]
checks: {unique: [customer_id]}
target: {table: customers_from_excel, mode: upsert, key: [customer_id]}
policy: {fail_if_reject_rate_above: 0.05}
output: {directory: ../../../../reports}
```

source/etl-validation-toolkit/src/etlguard/examples/pipelines/customers_scd2.yaml

```yaml
pipeline: customers_history
source: {type: csv, path: ../data/customers_clean.csv}
model: etlguard.examples.models:Customer
cleaning:
  date_columns: [signup_date, end_date]
  number_columns: [credit_limit]
  lowercase: [email, status]
  uppercase: [customer_id]
checks: {unique: [customer_id]}
target:
  table: customers_history
```

A
B
C
D
E
F
G
H

```
  mode: scd2
  key: [customer_id]
  tracked_columns: [full_name, email, status, credit_limit]
  batch_size: 500
policy: {fail_if_reject_rate_above: 0.05}
output: {directory: ../../../../reports}
```

source/etl-validation-toolkit/src/etlguard/examples/pipelines/transactions.yaml

```
pipeline: transactions_daily
source: {type: csv, path: ../data/transactions_messy.csv}
model: etlguard.examples.models:Transaction
cleaning:
  date_columns: [transaction_date]
  number_columns: [amount]
  lowercase: [type]
  uppercase: [customer_id, transaction_id]
checks:
  unique: [transaction_id]
  foreign_keys:
    customer_id: {values: [C001, C002, C003]}
target: {table: transactions, mode: append, key: [transaction_id]}
policy: {fail_if_reject_rate_above: 0.05}
output: {directory: ../../../../reports}
```

source/etl-validation-toolkit/.github/workflows/ci.yml

```
name: CI and GitHub Pages demo

on:
  push:
    branches: [main]
    tags: ["v*"]
  pull_request:
  workflow_dispatch:

permissions:
  contents: read

jobs:
  quality:
    name: Ruff, mypy, pytest, MySQL 8
    runs-on: ubuntu-latest
    services:
      mysql:
        image: mysql:8.0
        env:
          MYSQL_ROOT_PASSWORD: root_local_only
          MYSQL_DATABASE: etlguard
          MYSQL_USER: etlguard
          MYSQL_PASSWORD: etlguard_local_only
        ports:
          - 3306:3306
        options: >-
          --health-cmd="mysqladmin ping -h 127.0.0.1 -u root -proot_local_only --silent"
          --health-interval=5s --health-timeout=3s --health-retries=25
    env:
      ETLGUARD_TEST_MYSQL_URL:
mysql+pymysql://etlguard:etlguard_local_only@127.0.0.1:3306/etlguard
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
          cache: pip
      - name: Install package with development dependencies
        run: python -m pip install -e ".[dev]"
      - name: Ruff lint
        run: ruff check .
      - name: Mypy types
        run: mypy src/etlguard
      - name: Unit, property, mocked API, CLI and real MySQL 8 tests
        run: >-
          pytest -q --cov=etlguard --cov-branch
          --cov-report=term-missing --cov-report=xml
          --cov-fail-under=85
      - name: Generate live coverage badge
        run: |
```

```
          python -m coverage json -o coverage-summary.json
          python scripts/build_coverage_badge.py coverage-summary.json demo/coverage.json
      - uses: actions/upload-artifact@v4
        continue-on-error: true
        with:
          name: coverage-report
          path: |
            coverage.xml
            demo/coverage.json
          retention-days: 14

  pages:
    name: Publish browser demo
    if: github.event_name == 'push' && github.ref == 'refs/heads/main' &&
vars.ENABLE_PUBLIC_DEMO_PAGES == 'true'
    needs: quality
    runs-on: ubuntu-latest
    permissions:
      contents: read
      pages: write
      id-token: write
    concurrency:
      group: github-pages
      cancel-in-progress: true
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - name: Bundle production Python validation modules for Pyodide
        run: python scripts/build_demo_bundle.py
      - uses: actions/download-artifact@v4
        with:
          name: coverage-report
          path: badge-artifact
      - name: Put current coverage endpoint in deployed website
        run: cp badge-artifact/demo/coverage.json demo/coverage.json
      - uses: actions/configure-pages@v5
      - uses: actions/upload-pages-artifact@v3
        with:
          path: demo
      - id: deployment
        uses: actions/deploy-pages@v4
```

source/etl-validation-toolkit/.github/workflows/mysql8-staging-gate.yml

```
name: MySQL 8 Staging Verification

on:
  workflow_dispatch:

permissions:
  contents: read

jobs:
  mysql8-gate:
    name: MySQL ${{ matrix.mysql }} / Python ${{ matrix.python }}
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        mysql: ["8.0", "8.4"]
        python: ["3.11", "3.13"]
    services:
      mysql:
        image: mysql:${{ matrix.mysql }}
        env:
          MYSQL_ROOT_PASSWORD: root_for_disposable_tests_only
          MYSQL_DATABASE: etlguard_ci
          MYSQL_USER: etlguard_ci
          MYSQL_PASSWORD: etlguard_ci_local_only
        ports:
          - 3306:3306
        options: >-
          --health-cmd="mysqladmin ping -h 127.0.0.1 -uroot
-proot_for_disposable_tests_only --silent"
          --health-interval=5s --health-timeout=5s --health-retries=40
    env:
      ETLGUARD_TEST_MYSQL_URL:
```

A B C D E F G H

```
      mysql+pymysql://etlguard_ci:etlguard_ci_local_only@127.0.0.1:3306/etlguard_ci
0033      steps:
0034        - uses: actions/checkout@v4
0035        - uses: actions/setup-python@v5
0036          with:
0037            python-version: ${{ matrix.python }}
0038        - name: Install complete development dependencies
0039          run: python -m pip install -e ".[dev]"
0040        - name: Test safety-guard and pilot logic without a database
0041          run: python -m unittest discover -s scripts -p 'test_*.py' -v
0042        - name: Run disposable MySQL 8 staging and quality gates
0043          run: >-
0044            python scripts/verify_mysql8_staging.py
0045            --confirm-disposable-schema etlguard_ci
0046            --execute-tests
0047            --report staging-gate-report.json
0048        - name: CI-only isolated shadow smoke and idempotent replay
0049          env:
0050            ETLGUARD_CI_MYSQL_ROOT_URL:
      mysql+pymysql://root:root_for_disposable_tests_only@127.0.0.1:3306/etlguard_ci
0051            ETLGUARD_CI_ALLOW_BOOTSTRAP: DISPOSABLE_MYSQL_ONLY
0052          run: python scripts/ci_shadow_smoke.py --staging-report staging-gate-report.json
      --report shadow-smoke-report.json
0053        - name: Preserve evidence in job logs
0054          if: always()
0055          run: python scripts/archive_ci_evidence.py
0056        - name: Preserve verification evidence
0057          continue-on-error: true
0058          if: always()
0059          uses: actions/upload-artifact@v4
0060          with:
0061            name: mysql-${{ matrix.mysql }}-python-${{ matrix.python }}-verification
0062            path: |
0063              staging-gate-report.json
0064              mysql8-gate-logs/
0065              shadow-smoke-report.json
0066            retention-days: 30
0067            if-no-files-found: warn
```

source/etl-validation-toolkit/.github/workflows/release.yml

```
0001  name: Publish tagged GitHub release
0002
0003  on:
0004    push:
0005      tags:
0006        - "v*"
0007
0008  permissions:
0009    contents: write
0010
0011  jobs:
0012    github-release:
0013      runs-on: ubuntu-latest
0014      steps:
0015        - uses: actions/checkout@v4
0016        - uses: actions/setup-python@v5
0017          with:
0018            python-version: "3.11"
0019        - name: Build sdist and wheel
0020          run: |
0021            python -m pip install --upgrade build
0022            python -m build
0023        - name: Publish GitHub release assets
0024          uses: softprops/action-gh-release@v2
0025          with:
0026            generate_release_notes: true
0027            files: dist/*
```

source/etl-validation-toolkit/demo/app.js

```
0001  import { loadPyodide } from "https://cdn.jsdelivr.net/pyodide/v0.29.5/full/pyodide.mjs";
0002
0003  const $ = (id) => document.getElementById(id);
0004  let pyodide;
0005  let currentText = "";
0006  let userFileSelected = false;
0007  let result = null;
```

A B C D E F G H

```
let currentTab = "reject";

const setStatus = (status) => { $("status").textContent = status; };
const setValue = (id, value) => { $(id).textContent = String(value); };

function presentTable(rows, keys, errorColumn) {
  const container = $("table-container");
  container.replaceChildren();
  if (!rows.length) {
    const empty = document.createElement("div");
    empty.className = "empty";
    empty.textContent = "No records in this category.";
    container.append(empty);
    return;
  }
  const table = document.createElement("table");
  const thead = document.createElement("thead");
  const header = document.createElement("tr");
  for (const key of keys) {
    const th = document.createElement("th"); th.textContent = key; header.append(th);
  }
  thead.append(header); table.append(thead);
  const tbody = document.createElement("tbody");
  for (const row of rows.slice(0, 250)) {
    const tr = document.createElement("tr");
    for (const key of keys) {
      const cell = document.createElement("td");
      cell.textContent = row[key] == null ? "—" : String(row[key]);
      if (key === errorColumn) cell.className = "error-cell";
      cell.title = cell.textContent;
      tr.append(cell);
    }
    tbody.append(tr);
  }
  table.append(tbody); container.append(table);
}

function clearResult(message = "Load a sample or select your own CSV to see the data
quality report.") {
  result = null;
  for (const id of ["rows-read", "rows-valid", "rows-rejected", "rows-loaded"])
setValue(id, "—");
  setValue("deduped", "Duplicates dropped: —");
  setValue("errors", "Errors: —");
  setValue("result-label", "READY TO TEST");
  setValue("row-caption", "Row-level errors include the field and reason.");
  $("download").disabled = true;
  const empty = document.createElement("div");
  empty.className = "empty";
  empty.textContent = message;
  $("table-container").replaceChildren(empty);
}

function failInput(message) {
  currentText = "";
  userFileSelected = false;
  setValue("filename", "No feed loaded");
  clearResult(message);
  setStatus(message);
}

function renderTab(tab) {
  currentTab = tab;
  for (const name of ["reject", "valid", "json"]) {
    $(name + "-tab").classList.toggle("active", name === tab);
    $(name + "-tab").setAttribute("aria-selected", name === tab ? "true" : "false");
  }
  if (!result) return;
  if (tab === "reject") {
    presentTable(result.rejected_rows, ["source_row_number", "field", "error_type",
"error_message", "raw_value"], "error_message");
    setValue("row-caption", `${result.rejected_rows.length} detailed error record(s), one
 per error`);
  } else if (tab === "valid") {
    presentTable(result.valid_rows, Object.keys(result.valid_rows[0] || {}));
    setValue("row-caption", `${result.rows_valid} rows accepted by Pydantic`);
  } else {
    const pre = document.createElement("pre");
    const { valid_rows, rejected_rows, rejects_csv, ...summary } = result;
    pre.textContent = JSON.stringify(summary, null, 2);
    $("table-container").replaceChildren(pre);
    setValue("row-caption", "Machine-readable batch metrics");
  }
}

```

A
B
C
D
E
F
G
H

```
async function validate() {
  if (!pyodide || !currentText) return;
  clearResult("Validation is running…");
  setValue("result-label", "VALIDATING");
  $("validate").disabled = true;
  setStatus("Running Pydantic checks…");
  // Yield one paint frame before synchronous Python processing.
  await new Promise((resolve) => requestAnimationFrame(() => setTimeout(resolve, 0)));
  try {
    pyodide.globals.set("csv_text", currentText);
    pyodide.globals.set("selected_model", $("model").value);
    const json = pyodide.runPython("from etlguard.browser_demo import validate_csv;
validate_csv(csv_text, selected_model)");
    result = JSON.parse(json);
    setValue("rows-read", result.rows_read);
    setValue("rows-valid", result.rows_valid);
    setValue("rows-rejected", result.rows_rejected);
    setValue("rows-loaded", result.rows_loaded_sqlite);
    setValue("deduped", `Duplicates dropped: ${result.duplicates_dropped}`);
    setValue("errors", `Errors: ${result.rejected_rows.length}`);
    setValue("result-label", "VALIDATED LOCALLY");
    $("download").disabled = false;
    renderTab("reject");
    setStatus("Validation complete");
  } catch (error) {
    console.error(error);
    clearResult("Validation failed. Check the selected CSV and schema, then try again.");
    setValue("result-label", "VALIDATION ERROR");
    setStatus(`Validation error: ${(String(error.message ||
error).match(/^[A-Za-z]+(?:Error|Exception):[^\n]*/m)?.[0] || String(error.message ||
error)).slice(0, 180)}`);
  } finally {
    pyodide.globals.delete("csv_text");
    pyodide.globals.delete("selected_model");
    $("validate").disabled = false;
  }
}

async function sample() {
  const model = $("model").value.toLowerCase();
  try {
    const response = await fetch(`samples/${model}s_messy.csv`);
    if (!response.ok) throw new Error("Sample missing");
    currentText = await response.text();
    userFileSelected = false;
    setValue("filename", `${model}s_messy.csv (synthetic sample)`);
    await validate();
  } catch (error) { failInput(`Could not load sample: ${String(error.message ||
error)}`); }
}

async function loadFile(file) {
  if (!file) return;
  if (!file.name.toLowerCase().endsWith(".csv")) { failInput("Select a .csv file");
return; }
  if (file.size > 5_000_000) { failInput("Browser demo accepts CSV files up to 5 MB");
return; }
  setStatus("Reading CSV…");
  try {
    currentText = await file.text();
    if (!currentText.trim()) throw new Error("The selected CSV is empty");
    userFileSelected = true;
    setValue("filename", file.name);
    await validate();
  } catch (error) {
    failInput(`Could not read CSV: ${String(error.message || error)}`);
  }
}

$("dropzone").addEventListener("click", () => $("file").click());
$("dropzone").addEventListener("keydown", (e) => { if (e.key === "Enter" || e.key === "
") { e.preventDefault(); $("file").click(); } });
$("file").addEventListener("change", (e) => loadFile(e.target.files[0]));
$("dropzone").addEventListener("dragover", (e) => { e.preventDefault();
$("dropzone").classList.add("dragging"); });
$("dropzone").addEventListener("dragleave", () =>
$("dropzone").classList.remove("dragging"));
$("dropzone").addEventListener("drop", (e) => { e.preventDefault();
$("dropzone").classList.remove("dragging"); loadFile(e.dataTransfer.files[0]); });
$("sample").addEventListener("click", sample);
$("validate").addEventListener("click", validate);
$("model").addEventListener("change", () => userFileSelected ? validate() : sample());
for (const tab of ["reject", "valid", "json"]) $(tab + "-tab").addEventListener("click",
() => renderTab(tab));
$("download").addEventListener("click", () => {
```

```
  if (!result) return;
  const blob = new Blob(["\ufeff", result.rejects_csv], { type: "text/csv;charset=utf-8" });
  const url = URL.createObjectURL(blob);
  const anchor = document.createElement("a");
  anchor.href = url; anchor.download = `etlguard_${result.model.toLowerCase()}_rejects.csv`;
  anchor.click();
  setTimeout(() => URL.revokeObjectURL(url), 1000);
});

try {
  setStatus("Loading Pyodide runtime…");
  pyodide = await loadPyodide({ indexURL: "https://cdn.jsdelivr.net/pyodide/v0.29.5/full/" });
  setStatus("Loading Pydantic v2 and SQLite…");
  await pyodide.loadPackage(["pydantic", "sqlite3"]);
  const response = await fetch("etlguard_core.zip");
  if (!response.ok) throw new Error("Python core bundle unavailable");
  pyodide.FS.writeFile("/tmp/etlguard_core.zip", new Uint8Array(await response.arrayBuffer()));
  pyodide.runPython("import sys; sys.path.insert(0, '/tmp/etlguard_core.zip')");
  $("validate").disabled = false;
  setStatus("Python engine ready");
  await sample();
} catch (error) {
  setStatus(`Could not start browser Python: ${String(error).slice(0, 140)}`);
}
```

source/etl-validation-toolkit/demo/index.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta name="description" content="Run the real ETL Guard Python cleaning and Pydantic validation engine in your browser using Pyodide.">
  <title>ETL Guard — Interactive Data Quality Lab</title>
  <link rel="stylesheet" href="styles.css">
  <script type="module" src="app.js"></script>
</head>
<body>
  <div class="page">
    <header class="topbar"><a class="brand" href="./"><span class="shield">✓</span>ETL<span>GUARD</span></a><span class="topright"><span class="dot"></span> CLIENT-SIDE PYTHON · PYODIDE</span></header>
    <main>
      <section class="hero">
        <div class="eyebrow">DATA QUALITY ENGINE / LIVE DEMO</div>
        <h1>Trust the data.<br><em>Before it lands.</em></h1>
        <p class="lead">A reusable validation gate for CSV, Excel, and API feeds. Try the actual Pydantic-powered Python core on your own CSV—nothing leaves your browser.</p>
        <div class="hero-chips"><span>PYTHON 3.11+</span><span>PYDANTIC V2</span><span>SQLITE SANDBOX</span><span>ZERO BACKEND</span></div>
      </section>
      <div class="workspace">
        <section class="panel inputs" aria-label="Choose dataset">
          <div class="section-heading"><span class="step">01</span><h2>Input feed</h2></div>
          <label for="model">Validation schema</label>
          <select id="model"><option value="Customer">Customer dimension</option><option value="Transaction">Transaction fact</option></select>
          <div id="dropzone" class="dropzone" role="button" tabindex="0" aria-label="Drop or browse a CSV file">
            <div class="upload-icon">⇪</div>
            <strong>Drop your CSV here</strong><p>or click to browse · UTF-8 CSV</p>
            <input id="file" type="file" accept=".csv,text/csv" aria-label="Choose CSV file" hidden>
          </div>
          <div class="input-actions"><button id="sample" class="secondary">↻ Load messy sample</button><button id="validate" class="primary" disabled>Run validation <span>→</span></button></div>
          <div class="meta"><span id="filename">No feed loaded</span><span id="status" role="status">Initializing Python engine…</span></div>
          <div class="note"><strong>Same Python core as the CLI.</strong> The browser uses Pyodide + Pydantic and loads passing records into an in-memory SQLite table. Production MySQL writes are intentionally disabled.</div>
        </section>
        <section class="panel outputs" aria-label="Validation results">
          <div class="section-heading"><span class="step">02</span><h2>Quality report</h2><span class="result-label" id="result-label">READY TO TEST</span></div>
```

```
0037          <div class="metrics">
0038            <div class="metric"><span>ROWS READ</span><strong
      id="rows-read">—</strong></div>
0039            <div class="metric"><span>VALID</span><strong id="rows-valid"
      class="good">—</strong></div>
0040            <div class="metric"><span>REJECTED</span><strong id="rows-rejected"
      class="bad">—</strong></div>
0041            <div class="metric"><span>SQLITE LOADED</span><strong
      id="rows-loaded">—</strong></div>
0042          </div>
0043          <div class="submetrics"><span id="deduped">Duplicates dropped: —</span><span
      id="errors">Errors: —</span></div>
0044          <div class="tabs" role="tablist" aria-label="Result view"><button
      id="reject-tab" role="tab" aria-selected="true" class="active">Rejected
      rows</button><button id="valid-tab" role="tab" aria-selected="false">Valid
      rows</button><button id="json-tab" role="tab" aria-selected="false">Summary
      JSON</button></div>
0045          <div class="table-frame"><div id="table-container" class="table-scroll"><div
      class="empty">Load a sample or select your own CSV to see the data quality
      report.</div></div></div>
0046          <div class="result-footer"><span id="row-caption">Row-level errors include the
      field and reason.</span><button id="download" class="secondary" disabled>↓ Download
      rejects CSV</button></div>
0047        </section>
0048      </div>
0049      <footer><span>ETL GUARD / v0.1.3</span><span>LOCAL DEMO · NO FILE UPLOAD · MIT
      LICENCE</span></footer>
0050    </main>
0051  </div>
0052 </body>
0053 </html>
```

source/etl-validation-toolkit/demo/styles.css

```
0001  :root {color-scheme:dark;--bg:#0b1019;--panel:#141c29;--muted:#9caabb;--line:#263342;--te
      xt:#f3f6f9;--lime:#a9eb8e;--red:#ff8c91;--blue:#88baff}
0002  *{box-sizing:border-box}body{background:radial-gradient(ellipse 85% 70% at 70%
      -10%,#1b3144 0%,transparent 70%),var(--bg);color:var(--text);font:15px/1.6
      -apple-system,BlinkMacSystemFont,"Segoe
      UI",sans-serif;margin:0}.page{max-width:1370px;margin:auto;padding:0
      4%}a{color:inherit;text-decoration:none}.topbar{height:82px;border-bottom:1px solid var(-
      -line);display:flex;align-items:center;justify-content:space-between}.brand{font-weight:9
      00;letter-spacing:1.3px;font-size:17px}.brand span:last-child{color:var(--lime)}.shield{b
      ackground:var(--lime);color:#0b1611;display:inline-grid;place-items:center;width:28px;hei
      ght:28px;border-radius:8px;margin-right:8px}.topright,.eyebrow,.hero-chips,.meta,.result-
      label,.step,footer,.metric span{font-size:11px;font-weight:700;letter-spacing:1.5px}.topr
      ight{color:var(--muted)}.dot{display:inline-block;width:8px;height:8px;border-radius:50%;
      background:var(--lime);margin-right:8px;box-shadow:0 0 14px
      var(--lime)}.hero{padding:62px 0 42px}.eyebrow{color:var(--lime)}h1{font-size:clamp(43px,
      5.8vw,80px);letter-spacing:-3.5px;line-height:1.04;margin:21px 0 18px;font-weight:800}h1
      em{color:var(--lime);font-style:normal}.lead{color:#b0bbcc;max-width:720px;font-size:17px
      ;line-height:1.75}.hero-chips{display:flex;flex-wrap:wrap;gap:11px;margin-top:23px}.hero-
      chips span{border:1px solid #354253;border-radius:5px;padding:6px
      10px;color:#aeb9c9}.workspace{display:grid;grid-template-columns:minmax(300px,0.86fr)
      minmax(0,1.65fr);gap:17px}.panel{background:rgba(20,28,41,.94);border:1px solid
      var(--line);border-radius:13px;padding:23px;min-width:0;box-shadow:0 12px 35px #060a1355}
      .section-heading{display:flex;align-items:center;gap:12px;margin-bottom:26px}.step{backgr
      ound:#263749;color:var(--lime);padding:4px 9px;border-radius:5px}.section-heading h2{font
      -size:19px;letter-spacing:-.3px;margin:0}label{display:block;font-weight:650;font-size:13
      px;margin-bottom:8px}select{width:100%;appearance:none;padding:14px
      14px;color:var(--text);border:1px solid #354456;background:#1d2a39;border-radius:7px;font
      :inherit;outline:none}select:focus,.dropzone:focus{outline:2px solid
      var(--lime)}.dropzone{margin:20px 0;border:1.5px dashed
      #566374;border-radius:10px;text-align:center;padding:38px 10px;cursor:pointer;transition:
      .2s}.dropzone:hover,.dropzone.dragging{border-color:var(--lime);background:#1a342d55}.dro
      pzone strong{display:block;font-size:17px}.dropzone
      p{color:var(--muted);font-size:13px;margin:5px 0}.upload-icon{font-size:39px;color:var(--
      lime);line-height:1.3}.input-actions,.result-footer{display:flex;gap:10px;justify-content
      :space-between;align-items:center;flex-wrap:wrap}button{cursor:pointer;font:inherit;font-
      weight:700;border-radius:7px;border:1px solid transparent;padding:11px 13px}.primary{back
      ground:var(--lime);color:#101a12}.primary:hover{background:#baf6a2}.primary:disabled{opac
      ity:.5;cursor:not-allowed}.secondary{color:#d8e3ee;border:1px solid #435367;background:#2
      23044}.secondary:hover:not(:disabled){border-color:#8eabbc}.secondary:disabled{opacity:.4
      5;cursor:not-allowed}.meta{color:var(--muted);display:flex;flex-wrap:wrap;justify-content
      :space-between;gap:8px;margin-top:23px;line-height:1.6;letter-spacing:0;text-transform:no
      ne}.note{background:#1c2e34;border-left:3px solid var(--lime);padding:14px
      15px;border-radius:4px;font-size:12px;color:#b6c6cd;margin-top:27px}.note strong{color:#e
      1eee6}.result-label{margin-left:auto;color:var(--lime);white-space:nowrap}.metrics{displa
      y:grid;grid-template-columns:repeat(4,1fr);gap:10px}.metric{border:1px solid
      #304052;background:#1b2736;padding:14px 12px;border-radius:8px;min-width:0}.metric span{c
      olor:#aab6c5;display:block;white-space:nowrap;letter-spacing:.6px;font-size:9px}.metric
      strong{display:block;font-size:32px;line-height:1.5}.metric
```

A B C D E F G H

```
.good{color:var(--lime)}.metric .bad{color:var(--red)}.submetrics{display:flex;justify-co
ntent:space-between;gap:8px;font-size:12px;color:var(--muted);padding:14px 1px
21px}.tabs{display:flex;gap:19px;border-bottom:1px solid var(--line)}.tabs
button{background:transparent;border:0;border-bottom:2px solid
transparent;border-radius:0;color:#9baabd;padding:11px 2px;font-size:13px}.tabs button.ac
tive{color:var(--lime);border-color:var(--lime)}.table-frame{height:275px;border-bottom:1
px solid var(--line)}.table-scroll{width:100%;height:100%;overflow:auto}.table-scroll
table{border-collapse:collapse;width:100%;text-align:left;font-size:12px}.table-scroll th
{position:sticky;top:0;background:#243143;color:#d4e2eb;z-index:1;font-size:10px;letter-s
pacing:.4px}.table-scroll td,.table-scroll th{padding:11px 10px;border-bottom:1px solid #
2d3947;white-space:nowrap;max-width:270px;overflow:hidden;text-overflow:ellipsis}.table-s
croll td{color:#b8c5d1}.error-cell{color:var(--red)!important}.empty{display:grid;place-i
tems:center;height:100%;text-align:center;color:var(--muted);padding:24px}.result-footer{
padding-top:19px}.result-footer span{font-size:11px;color:var(--muted)}pre{font:12px/1.5
ui-monospace,Consolas,monospace;color:#c3e9cf;margin:0;padding:14px;white-space:pre-wrap;
overflow-wrap:anywhere}footer{margin:47px 0 24px;border-top:1px solid var(--line);padding
-top:22px;color:var(--muted);display:flex;justify-content:space-between;gap:15px;flex-wra
p:wrap}@media(max-width:920px){.workspace{grid-template-columns:1fr}.hero{padding-top:38p
x}.topright{font-size:9px}}@media(max-width:510px){.page{padding:0 4%}.panel{padding:15px
}.metrics{grid-template-columns:repeat(2,1fr)}h1{letter-spacing:-2px}.hero-chips{gap:6px}
.topright{max-width:135px;text-align:right}}
```

controlled-pilot/pilot-template.yaml

```yaml
pipeline: controlled_customer_pilot
source:
  type: csv
  path: data/approved-snapshot-example.csv
model: etlguard.examples.models:Customer
cleaning:
  trim: true
  date_columns:
  - signup_date
  - end_date
  number_columns:
  - credit_limit
checks:
  unique:
  - customer_id
  expected_row_count:
    min: 3
    max: 3
target:
  table: approval_required_target
  mode: upsert
  key:
  - customer_id
  batch_size: 500
  lock_timeout_seconds: 30
policy:
  fail_if_reject_rate_above: 0.0
  allow_empty_replace: false
  allow_partial_replace: false
database_url_env: ETLGUARD_APPROVED_PILOT_URL
output:
  directory: reports
```

45. Appendix F - Current operator runbooks and release decisions

These texts are the canonical narrative documents from the supplied v0.1.3 packet. Their status statements are source-evidence claims, not substitutions for independently authorized target operations.

README_START_HERE.md

ETL Guard v0.1.3 production-readiness packet

Start with FINAL_STATUS.md and GO_NO_GO_DECISION.md. Current evidence is in evidence/; historical input and v0.1.2 records are in historical/. The corrected runtime is v0.1.3. This packet does not authorize production.

Contents

- artifacts/: tested v0.1.3 wheel, sdist and clean source ZIP.
- source/etl-validation-toolkit/: complete editable source, tests, demo and runbooks.
- evidence/: current staging, JUnit, coverage, browser screenshot/download, recovery dumps/logs, metrics, soak and assessor results.
- controlled-pilot/: portable synthetic example and pinned prepared UUID/hashes, with execution approval and scheduler disabled.
- operations/: reproducible local provisioning, QA, fault, capacity and packaging helpers.
- changes/: source patch from v0.1.2.
- SHA256SUMS: delivery integrity manifest, not a signature or owner approval.

Use the application

On macOS, double-click source/etl-validation-toolkit/Launch ETL Guard.command. It uses an available localhost port. Python 3.11+ is required, and browser startup downloads Pyodide/Pydantic/SQLite. CSV processing remains local to the browser. Alternatively run python scripts/serve_demo.py from the source project.

Install the wheel with a clean Python environment: python -m pip install artifacts/etl_validation_toolkit-0.1.3-py3-none-any.whl. Use etlguard --help. The included source runbooks describe CLI source/config and reports.

Reproduce local exercises

Use a NEW extracted working copy. macOS helpers expect the Homebrew mysql@8.4 keg at /opt/homebrew/opt/mysql@8.4; they never use Homebrew services. Create the source .venv with Python 3.13, install -e '.[dev]', pip and -r requirements-operations.txt. requirements-qa-lock.txt records this run's compatible QA dependency versions. Runtime compatibility and performance must still be validated on the intended platform.

Run operations/bootstrap_mysql.py in a persistent terminal. It creates .private/ only if absent, initializes a new data directory with networking disabled, generates credentials, provisions four scoped schemas, then starts loopback-only MySQL. No production URL argument exists. Keep that session running.

In another terminal using the source environment, run operations/run_staging.py, then operations/run_operational_acceptance.py, then operations/run_soak.py. They create NEW fresh-evidence/ directories and reject existing output directories where immutable evidence would otherwise be overwritten. Do not blindly rerun uncertain writes. The crash drill's stage/verify modes are specifically for that generated private instance, with start_existing_mysql.py between them. Review the code before the deliberately destructive SIGKILL drill.

For wheel CLI QA, create .private/wheel-venv, install the supplied wheel with dependencies, and run operations/package_cli_acceptance.py. For a prepared pilot/assessor exercise, run operations/prepare_approval_packet.py only in a new output layout. It should remain **blocked** without owner artifacts.

Private generated credential/client files, raw database directories and virtual environments are deliberately not included. operations/assemble_delivery.py describes this exact local packaging run and includes fixed input/output names; adapt it to a new destination rather than overwriting this packet. The generic source wheel/sdist build uses python -m build.

FINAL_STATUS.md

Final local status — ETL Guard v0.1.3

Technical disposition: local validation completed; ready for formal controlled-pilot review.

Production disposition: **NO-GO** until exact-target authorization and genuine owner approvals.

All fresh results below were executed locally on 11 November 2026 with Python 3.13.16 and **MySQL 8**.4.13 on a disposable loopback-only instance. Historical v0.1.2 evidence is kept separately and is not counted as v0.1.3 verification.

Confirmed defect and correction

The v0.1.2 contention soak exposed MySQL deadlock 1213 across distinct targets sharing metadata tables. A deterministic regression synchronized missing UUID reads and reproduced the failure. v0.1.3 sets READ COMMITTED on MySQL load connections to avoid missing-row gap locks, while retaining per-target advisory locks, existing-row locks, unique batch reservations and atomic target/ledger/audit transactions. The regression and full suite **pass**. A probe verified READ COMMITTED inside the load connection and reset to REPEATABLE READ after returning it to the pool. Only etlguard/__init__.py and etlguard/loaders/mysql.py changed among the 33 runtime files.

Fresh QA and staging

- 137 pytest tests **passed**, zero skips: Hypothesis, mocked HTTP APIs, CLI, validation and 14 real MySQL integration tests.
- 39 operator/safety unittest checks **passed**; Ruff and mypy **passed**; branch coverage: 91.19%.
- Seven browser cases **passed**, including both schemas, empty/oversized/unsupported/malformed inputs and actual CSV download formula escaping. Screenshot and download are retained.
- Nine CLI checks **passed** from a clean wheel installation outside the source checkout; dependencies resolved and **passed** compatibility checks.
- The supplied one-shot completed first load, atomic ledger/audit checks, independent reconciliation and identical-UUID replay with unchanged physical target fingerprint.
- Source, wheel and sdist runtime files match; new artifacts were rebuilt for v0.1.3. Original handoff checksums **passed** 32/32 entries.

Recovery and concurrency

- Injected audit failure rolled back target, stage, audit and ledger; the same UUID subsequently loaded once.
- A real DBAPI commit followed by connection invalidation simulated a lost acknowledgement; durable ledger lookup resolved it without duplicate loading.
- Killing the application process with uncommitted writes rolled back safely; the same UUID was recovered after checking the ledger.
- A real mysqld SIGKILL with an in-flight transaction preserved a prior committed fingerprint, rolled back uncommitted state and allowed safe same-UUID recovery/replay. Intervention-inclusive recovery: 15.19 seconds.
- Eight simultaneous **SCD2** writers over 50 keys produced 450 contiguous historical versions with exactly 50 active versions.
- Empty input, rejected input and insufficient replacement ratio all **failed** closed without changing target rows. Bounded lock timeout and recovery **passed**.
- Logical backup and actual restore into a separate empty schema verified 160,510 rows across 24 tables, including physical IDs and audits/ledger. Backup: 0.308s; restore: 1.787s; independent verification: 1.614s. Restore plus verification: 3.401s.
- The restore intentionally excluded one post-backup transaction, proving recovery to the backup snapshot. PITR and hardware power-loss testing are not claimed.

Local capacity envelope

Synthetic Customer CSV, upsert, 1,000-row SQL chunks; 8 CPU cores, 8 GiB RAM, MySQL buffer pool 256 MiB. Timing covers CSV read, cleaning, validation, staging, merge, reconciliation and audit/ledger commit. Memory is sampled Python plus MySQL RSS during the initial load.

Rows	End-to-end seconds	Rows/second	Peak combined MiB	Probe p95 ms
10,000	1.327	7,534	249.5	31.31
50,000	6.999	7,144	383.1	35.40
100,000	15.731	6,357	563.7	38.63

All proposed local acceptance criteria **passed**: at least 500 rows/s, batches within 180s, combined RSS below 2,048 MiB, probe p95 below 250ms, at least 1 GiB disk free, zero added InnoDB log waits and indexed business-key lookup. These are provisional engineering criteria, not owner-approved production SLOs.

A 120.31-second soak completed 405 concurrent 2,000-row batches across three writers with zero **failed** batches. Probe p95 was 45.68ms. The local alert drill delivered an actual HTTP failure notification to a local receiver. External production paging was not contacted.

Delivery and environment

The final release includes source, wheel, sdist, configuration examples, reproducible operational scripts, all current evidence, the v0.1.2 regression proof and **SHA-256** manifests. Generated database passwords, private client files, virtual environments and raw data directories are excluded. Both disposable MySQL instances were stopped; their data remains in private working folders. The browser preview remains available while its local server runs.

Read GO_NO_GO_DECISION.md, TECHNICAL_REPORT.md, ROLLBACK_PLAN.md, MONITORING_PROCEDURE.md and DEPLOYMENT_CHECKLIST.md before formal review.

TECHNICAL_REPORT.md

Technical report — v0.1.3

Architecture and transaction boundary

CSV/Excel/REST readers attach batch lineage. Deterministic cleaning precedes Pydantic row checks and global dataset checks. Rejected rows produce CSV/JSON reports; accepted candidates go to a MySQL staging table. Append, transactional replacement, upsert and **SCD2** merge share the target transaction with count reconciliation, staging cleanup, success audit and durable commit marker. DDL is prepared before that transaction because MySQL DDL can implicitly commit. Replacement uses DELETE rather than TRUNCATE.

A committed UUID plus source/config/model fingerprint supports identical replay. Changed input under the same UUID is rejected. Per-target GET_LOCK serialization and an active-row unique constraint protect **SCD2**. The browser shares the validation engine and writes only in-memory SQLite.

Defect analysis

The default REPEATABLE READ isolation allowed locking reads for missing audit/ledger keys to take gap locks. Distinct target locks do not serialize writers to shared metadata indexes. Concurrent insert intentions could therefore form a deadlock despite different batch UUIDs. The baseline deterministic test **failed** with MySQL 1213; the corrected test synchronizes both ledger and audit reads and passes under READ COMMITTED.

The change applies only to MySQL load connections. SQLAlchemy restores the pool connection's default isolation afterward. Existing-row locking, primary-key reservations and the per-target lock remain. No automatic blind load retry was added; uncertain outcomes still require durable ledger evidence. See changes/v0.1.2-to-v0.1.3.patch, the before/after regression logs and evidence/operations/connection-isolation.json.

MySQL documents reduced gap locking at READ COMMITTED, with remaining locking for duplicate-key and foreign-key checks: [**MySQL 8**.4 isolation documentation](https://dev.mysql.com/doc/refman/8.4/en/innodb-transaction-isolation-levels.html). Unique reservations and target locks remain necessary; this change is not a claim that arbitrary external writers cannot cause contention.

Evidence provenance

The input checksum manifests verified all 32 entries. Original wheel/source/sdist runtime bytes matched 33/33. Current v0.1.3 wheel/source/sdist runtime matches are recorded separately. Historical GitHub run 43061543916 was queried through the authenticated connector; its four completed successful job records are retained. Those jobs validate v0.1.2, not this changed runtime. Fresh v0.1.3 evidence comes from this local Python 3.13/**MySQL 8**.4 instance; current v0.1.3 **MySQL 8**.0 and Python 3.11 runs are not claimed.

Scope of fault and recovery testing

Lost-acknowledgement injection performs a real DBAPI commit, invalidates the connection and raises before acknowledgement reaches the loader. Application-process termination and actual database-process SIGKILL are separate exercises. The database restart recovered prior durable data and rejected uncommitted work. Physical power loss, storage-device corruption and infrastructure failover are outside the exercised local fault model.

The logical dump used a consistent transaction, omitted tablespace/global objects and was restored into an initially empty separate schema. Every row fingerprint matched the pre-backup snapshot. One deliberately later transaction was absent, as expected. Logical snapshot recovery does not prove PITR or an owner-approved production recovery point.

Capacity and operational limits

The validator materializes records for global checks and is not an out-of-core engine. Measurements cover synthetic Customer upsert batches through 100,000 records and a two-minute local contention soak. Larger sources, complex schemas, production storage/replication, long-running history, external writers and production alert routing require the approved target and workload. Query plans, CPU/RSS samples, disk growth, probe distributions and MySQL status counters are included in the raw JSON evidence.

The local alert drill exercised failure detection and an actual HTTP receiver; it does not authenticate an on-call recipient or external paging route. No production target, change ticket or human approval was supplied. The promotion tool correctly remains **blocked**.

GO_NO_GO_DECISION.md

Production decision: NO-GO pending authorization

Local technical review packet: complete for the tested envelope.

Production deployment, production writes and scheduler enablement: not authorized.

The authentic promotion assessor returned **blocked**, with exactly these unmet gates:

1. Approved, supervised real-target pilot.
2. Genuine data-owner approval.
3. Genuine database-owner approval.
4. Genuine operations-owner approval.

The nine local operational evidence entries are backed by actual files. No signatures or approvals were generated. The assessor input and output are evidence/pilot-evidence.json and evidence/promotion-review.json.

Actions requiring the user's authorization or external access

- Identify and explicitly approve the exact host/schema/table, source snapshot, mode, batch UUID, change ticket, maintenance window and named rollback owner/plan for one supervised pilot. The template intentionally has no production URL or approved target.
- Provide an approved representative source and workload envelope; owners must accept data-quality thresholds, throughput/latency/capacity limits, backup policy, retention and RPO/RTO. The synthetic 100,000-row benchmark cannot establish a production SLA.
- Authorize the actual monitoring and paging destination and escalation owners so external alert delivery and the real environment's observation window can be tested.
- Supply authentic data, DBA and operations sign-offs after reviewing the evidence and exact target plan. Scheduler enablement requires separate go-live authorization.

No production credentials were used. Local logical restore proves the recorded backup snapshot; it does not establish production PITR, replication, infrastructure failover or an approved production RPO. Those target-specific controls belong in the exact-target change review.

DEPLOYMENT_CHECKLIST.md

Deployment checklist — formal owner review

Completed locally

- [x] Input hashes and source/wheel/sdist integrity checked.
- [x] Confirmed runtime defect reproduced, fixed, version bumped and regression added.
- [x] Full fresh QA **passed**, zero MySQL skips, Ruff/mypy green.
- [x] Wheel/sdist built and installed wheel exercised independently.
- [x] Disposable MySQL with isolated scoped users, strict SQL mode and InnoDB.
- [x] First load, audits, commit ledger, source/target values and unchanged retry fingerprint.
- [x] Lost acknowledgement, process/database crashes, rollback, source tampering and lock timeout drills.
- [x] Concurrent **SCD2**, replacement controls, measured capacity, actual logical restore and local alert delivery.
- [x] Controlled-pilot template dry-validated; scheduler disabled.
- [x] Authentic evidence submitted to promotion assessor; missing approvals remain false.

Required exact-target review

- [ ] Explicit host/schema/table/source/mode/change ticket/window/rollback plan authorization.
- [ ] Data-owner acceptance of real input, quality thresholds and reconciliation.
- [ ] DBA acceptance of actual schema, privileges, isolation/binlog/replication compatibility, backup/PITR, retention and RPO/RTO.
- [ ] Operations acceptance of real workload SLOs, alert receiver, escalation, monitoring window and rollback owner.
- [ ] Approved de-identified/realistic source and actual-environment capacity/restore checks where required.
- [ ] Signed approvals and supervised limited pilot result.
- [ ] Independent reconciliation and observation after pilot.
- [ ] Separate explicit unattended scheduler/go-live authorization.

The production assessor's four remaining fields must be backed by authentic artifacts. Do not mark them complete from a test **pass** or from this checklist alone. No production deployment was performed.

ROLLBACK_PLAN.md

Reviewed rollback plan template — approval required for exact target

1. Before the pilot, pin the exact source/config hashes and stable UUID; record target snapshot, existing ledger/audit, DDL inventory, approved backup/restore location and an exclusive rollback owner. Confirm the real backup's restore and agreed RPO/RTO; the included synthetic dump must never be used on a real target.
2. Keep the scheduler disabled. Stop the specific job and coordinate other target writers on any unexplained mismatch, failure, lock-budget breach, missing audit/ledger or alert. Preserve logs and batch artifacts before intervention.
3. For an uncertain commit, query **`etlguard batch-status UUID --database-url-env APPROVED_ENV`**. Reconcile the durable ledger and target before any retry. Never change the UUID merely to escape uncertainty. If the database is unavailable or state cannot be resolved, keep the job stopped and restore connectivity or escalate to the DBA.
4. A verified pre-commit failure rolls back target, staging, audit and ledger. Retry the exact immutable batch only after confirming rollback/absence and resolving the fault. An identical already-committed UUID replays counts without a second load.
5. A committed transaction cannot be undone with a later SQL ROLLBACK. An owner-approved compensating change or backup/PITR restore requires the exact target change ticket, stopped writers and independent before/after reconciliation. Restore first to a separate schema for verification. Preserve newer legitimate transactions according to the approved recovery plan.

6. **SCD2** schema changes require maintenance-window approval and backup. Do not automatically drop history or roll back generated active uniqueness guards.
7. If reverting software to v0.1.2 is proposed, account for its confirmed cross-target metadata deadlock: it cannot serve as a concurrent-writer fallback. Any temporary globally serialized fallback requires explicit owner approval and fresh reconciliation.

Local evidence: operations/audit_rollback.json, process_kill.json, lost_commit_ack.json, server-crash-recovery.json and backup_restore.json under evidence/. Exact production commands are intentionally **pending** exact-target approval.

MONITORING_PROCEDURE.md

Monitoring and escalation procedure

For the approved pilot, name an operations owner, DBA and data owner; approve the receiver, retention, sampling cadence and observation window. Do not embed passwords or raw customer rows in alerts. A monitoring identity needs read access to the approved audit/ledger and necessary target metrics; it does not need target DML or DDL privileges.

For each stable batch, record start/end UTC, source/config hashes, read/valid/rejected/loaded counts, duration, reject rate, lock wait, durable commit status and reconciliation result. Use batch-ID primary-key lookups in etl_batch_commits and etl_run_audit; investigate missing/mismatched records and leftover staging rows. Monitor CPU, process RSS, disk headroom, probe latency, InnoDB log waits/deadlocks, active transactions and connection limits. Plan metadata retention and indexed reporting for the real audit volume.

Proposed local test limits were throughput >=500 rows/s, duration <=180s per tested batch, Python+MySQL RSS <=2GiB, probe p95 <=250ms, free disk >=1GiB, zero reconciliation mismatches/duplicate loads and zero added InnoDB log waits. Owners must replace or approve these against the real workload; they are not production SLOs.

Page/escalate immediately for an unresolved commit, missing successful audit/ledger, unexpected target values/history, reconciliation mismatch, incomplete source or unsafe replacement attempt. Stop the job and preserve UUID/artifacts. Capacity/lock/latency breaches trigger the owner-approved circuit breaker and review before retry. Keep scheduler enablement a separate approved action.

Fresh evidence includes actual sampled metrics for all three volume runs and the two-minute soak. evidence/operations/monitoring.json records a real **failed** audit detected and delivered to a local HTTP receiver with a receipt. External paging and its recipient acknowledgement remain **blocked pending** the approved destination. No external notifications were sent.

46. Appendix G - Existing project design notes and historical documentation

The existing source repository includes v0.1.1/v0.1.2 retrospectives and project docs. They are included for engineering history, but current runtime and test conclusions must be drawn from v0.1.3 files and evidence.

configuration.md

Configuration reference

Each YAML pipeline is independently validated using strict **Pydantic v2** configuration models. Unknown keys, missing required source locations and invalid SQL identifiers produce a configuration error (CLI exit code 2). Relative paths are resolved from the YAML file directory.

Source

YAML property	Meaning	Default
`source.type`	`csv`, `excel`, `api`	required
`source.name`	Optional lineage/display name	file name or URL
`source.path`	CSV or Excel file	required for file sources
`source.delimiter`, `encoding`	CSV separators and codec	`,`, `utf-8-sig` (accepts BOM)
`source.header_row`	1-based header row; CSV counts header lines, Excel relative to `skip_rows`	`1`
`source.chunksize`	pandas CSV input rows per chunk	`10000`
`source.sheet`	Excel sheet name or 0-based index	`0`
`source.skip_rows`	Number of leading worksheet rows to skip	`0`
`source.url`, `records_path`	REST GET URL and nested list location, e.g. `data.items`	URL required for API
`source.bearer_token_env`	Name of an environment variable containing a bearer token	none
`source.timeout_seconds`	httpx per-request timeout	`15`
`source.max_retries`, `backoff_seconds`	Exponential backoff on transient network/429/5xx errors	`3`, `0.5`
`source.pagination`	See below	`{mode: none}`

Page/offset pagination

```yaml
source:
  type: api
  name: synthetic_transactions
  url: https://api.example.test/v1/transactions
  records_path: data.items
  bearer_token_env: DATA_FEED_API_TOKEN
  max_retries: 3
  backoff_seconds: 0.5
  pagination:
    mode: page                 # or offset
    page_param: page
    offset_param: offset
    limit_param: limit
    page_size: 100
    start: 1                   # normally 0 for offsets
    max_pages: 100
```

Page and offset pagination stop when a response contains fewer records than page_size. Next-link mode reads links.next by default; links must retain the same URL origin as the initial API to prevent bearer tokens from being forwarded to a different server. Repeated next links are rejected. max_pages limits accidental endless feeds. Endpoint and token values in this documentation are placeholders, not live services or valid credentials.

```yaml
source:
  type: api
  url: https://api.example.test/v1/customers
  records_path: data.items
  pagination: {mode: next, next_path: links.next, max_pages: 100}
```

Cleaning

```yaml
cleaning:
  trim: true
  snake_case_columns: true
  rename_map: {"Client Number": customer_id}
  null_tokens: ["", "NA", "N/A", "null", "-"]
  date_columns: [signup_date, end_date]
  date_formats: ["%Y-%m-%d", "%Y/%m/%d", "%d/%m/%Y"]
  number_columns: [credit_limit]
  lowercase: [email, status]
  uppercase: [customer_id]
  drop_duplicates: true
```

Cleaning is deterministic and idempotent: applying the same configuration again should not change the data. Number parsing removes common $, €, £, ₹, comma and spacing separators, parses accounting negatives like (1,234.00), and preserves Decimal. Unparseable values are **passed** to Pydantic to produce a reject rather than silently erased. Date parsing honors the format list order (ambiguous 01/02/2026 is intentionally interpreted according to your configuration). The duplicate fingerprint is the complete normalized row, and duplicates are suppressed only after both row-level and reference/uniqueness checks **pass**; repeated business keys with differing data are not silently dropped and are caught by checks.unique.

Pydantic model and dataset checks

```
model: etlguard.examples.models:Customer
checks:
  unique: [customer_id]
  foreign_keys:
    customer_id: {values: [C001, C002, C003]}
    # Or: {path: ../data/reference_customers.csv, column: customer_id}
  expected_row_count: {min: 1, max: 50000}
  max_null_pct: {email: 0.10}
```

unique checks accepted candidate rows; a second record with the same business key is rejected. foreign_keys checks records against inline reference values or an independent CSV reference list, not against live database tables. expected_row_count and max_null_pct run over the cleaned (pre-validation) rows and put dataset failures in the run summary. A schema field may be optional, yet a dataset null-threshold can still **fail** the batch.

Models live in importable Python modules with BaseModel, Field constraints and optional @model_validator(mode="after") methods. See [Customer and Transaction](../src/etlguard/examples/models.py). The model import string is dotted.python.module:ClassName, and the package containing a custom model must be installed or on PYTHONPATH.

Target and policy

```
target:
  table: customers
  mode: upsert          # append | truncate_insert | upsert | scd2
  key: [customer_id]    # required for upsert or scd2
  tracked_columns: [full_name, email, status]  # for scd2; defaults to all non-key fields
  batch_size: 500
  lock_timeout_seconds: 30
  min_replace_ratio: null  # e.g. 0.90 for truncate_insert minimum-vs-current count
policy:
  fail_if_reject_rate_above: 0.05
  allow_partial_replace: false  # recommended; applies to truncate_insert only
  allow_empty_replace: false    # recommended; applies to truncate_insert only
output:
  directory: ../../../../reports
database_url_env: ETLGUARD_DATABASE_URL
```

Important: Use only a local or trusted MySQL database; a real run creates missing tables and writes to stg_<table>, the target table and etl_run_audit. Business-key SQL identifiers are validated. A **SCD2** table stores historical versions. No auto-migration is performed for pre-existing tables. The configured failure policy prevents the load when rejects exceed the threshold; dataset-level failures also prevent loading. For truncate_insert, the toolkit *additionally* refuses any rejected source records or a genuinely empty source by default because replacement can erase existing data. Set policy.allow_partial_replace: true or policy.allow_empty_replace: true only after explicitly confirming that loss of target rows is acceptable. Identical invalid rows are never silently de-duplicated; each counts toward quality thresholds. Pipeline IDs must be filename-safe identifiers (1–128 characters).

CLI and Python API

```
etlguard validate path/to/pipeline.yaml # reports only, no database access
etlguard run path/to/pipeline.yaml      # validate + stage + merge + reconcile + audit
etlguard profile path/to/feed.csv       # column overview
etlguard init-db                        # create audit + atomic-commit ledger (requires URL env)
```

0 = success; 1 = quality/job/load error; 2 = configuration error. Both dry runs and load runs write a <pipeline>_<uuid>_summary.json, _rejects.csv and _rejects.json in the output directory. The CLI produces a Rich table; Pipeline.from_yaml(...).run(dry_run=True) returns a Pydantic RunResult.

v0.1.2 concurrency, audit and retries

- Both etl_run_audit and etl_batch_commits are required; etlguard init-db creates them outside the load transaction. Data, a success audit and its batch ledger entry commit together on InnoDB.
- Business key columns for upsert and **scd2** must be NOT NULL. **scd2** targets require a generated active-row uniqueness guard; migrate existing tables explicitly using etlguard upgrade-**scd2** <pipeline.yaml> --yes in a maintenance window.

A B C D E F G H

- For recoverable runs, assign and persist a batch UUID before the first attempt and use etlguard run <config.yaml> --batch-id <uuid>. After interruption, call etlguard batch-status <uuid> and retry with the same UUID and immutable source/config. Different input with the same UUID is rejected.
- target.lock_timeout_seconds (0–300; default 30) sets the MySQL advisory lock wait limit. target.min_replace_ratio (0–1, optional) protects against drastic shrinkage during truncate_insert. All four ETL tables must use InnoDB.

See [the production-hardening runbook](production-v0.1.2.md) for migration and staging test commands.

design-decisions.md

Design and ETL operations decisions

Separation of concerns

Source adapters return SourceRecord objects with immutable lineage. The cleaner operates on dictionaries without pandas, HTTP or SQL dependencies. The validator uses **Pydantic v2** models. Dataset checks and reject serialization are also database-independent. This allows the same code to run under CPython in the CLI and under Pyodide inside the browser.

Auditability

Every batch gets a UUID. A record stores source name, logical row number, UTC ingestion timestamp and batch UUID. Before merge, these are persisted in the staging table. The reject CSV/JSON stores the batch, source row number, field, type, message, original value and original source record. The run summary stores rows read/cleaned/valid/rejected/loaded, duplicate counts, error counts by field and type, duration, output file paths, policy failures, commit state, request hash and replay status. Rows rejected counts distinct records; multiple field errors from a record produce multiple reject-file lines.

Transaction design

MySQL DDL can implicitly commit, so table creation and schema verification are deliberately performed before the load transaction. One SQLAlchemy transaction encloses staging insert, target merge, row-count reconciliation, staging cleanup, durable batch-commit marker and successful audit. MySQL target writers are serialized through a connection-scoped advisory lock, and all participating tables must use InnoDB. truncate_insert uses transactional DELETE rather than DDL TRUNCATE. A reconciliation mismatch throws and rolls back the DML operations; the audit writer uses a separate connection/transaction to record a **failed** job when possible.

MySQL upsert rowcount can be 0, 1 or 2 per row depending on flags and whether values changed, so the loader counts existing business keys *before* the merge instead. rows_updated includes matched but unchanged rows, and rows_unchanged is separately disclosed for **SCD2**. Thus rows_valid = rows_inserted + rows_updated has one consistent meaning. The loader refuses upsert without a business-key UNIQUE constraint (which is created for new tables).

SCD Type 2

A new business key creates the first current version. An existing key whose tracked attributes changed causes the previous current row to be closed and a new current version inserted, with a common UTC boundary. Unchanged rows cause no historical insert. effective_to is an exclusive boundary and is null for current rows. MySQL writers using ETL Guard's loader are serialized with a bounded advisory lock. A stored computed active marker and UNIQUE constraint provide a database-level single-active-version guarantee; external writers should still observe the same lock protocol for correct ordering. Legacy **SCD2** schemas require an explicit migration. For dimensions with enormous history, a set-based merge and appropriately indexed columns are recommended.

Scale and data safety boundaries

CSV is read in chunks using pandas; validation currently materializes validated candidates and cleaned rows in memory to support global uniqueness/null-percentage rules. This is not an out-of-core, distributed ETL engine. Very

large feeds need streaming-to-disk or a staging-backed validation design. The REST connector accepts a base GET URL with page, offset or next-link pagination, bounded retries and max-pages safeguards. The browser demo accepts up to 5 MB CSV files and stores validated rows in memory-only SQLite. The frontend never sends the user CSV to a backend, though the Python runtime and Pydantic are downloaded from a public CDN.

Security hygiene

Never commit API tokens or database passwords. Use bearer_token_env and database_url_env names in YAML; actual secrets are provided as environment variables. The Docker Compose passwords and loopback port are local-only demo credentials, not production defaults. Using a privileged MySQL account or pointing a pipeline at an existing target can be destructive, especially with truncate_insert: review configs before running and use least-privilege credentials.

Explicit non-goals

- Event orchestration, CDC, automatic retries for **failed** loads, distributed state management.
- Automatic schema migration on ETL load (an operator-invoked guarded **SCD2** upgrade is available).
- Cross-database transaction support, unmanaged external **SCD2** writers, or deterministic **SCD2** effective timestamps supplied by source systems.
- Running SQL writes to MySQL from Pyodide; the browser uses SQLite only.

deployment.md

Publishing to GitHub and deploying the demo

The v0.1.2 archive contains source code without internal .git history. Initialize a repository before publishing (git init -b main && git add . && git commit -m "Release reviewed v0.1.2"). A GitHub repository, live Pages URL, CI badges and a public release are only created after you push it into an account you control.

1. Publish the repository

Create an empty public GitHub repository named etl-validation-toolkit (do not initialize it with another README). From this directory:

```
git remote add origin https://github.com/theraghavraman/etl-validation-toolkit.git
git push -u origin main
```

Replace theraghavraman with the actual GitHub account before running. Edit pyproject.toml [project.urls] and README badges/demo link to match that account and commit the changes before pushing. You can alternatively use GitHub CLI (gh repo create etl-validation-toolkit --public --source=. --remote=origin --push) if it is installed and authenticated.

2. GitHub Pages

Open Settings → Pages → Build and deployment → Source, choose GitHub Actions. Push to main. The ci.yml workflow runs checks against a real **MySQL 8** service container, generates an up-to-date coverage badge JSON, builds the small browser-safe Python ZIP, and deploys demo/. Once it succeeds, the site lives at:

https://theraghavraman.github.io/etl-validation-toolkit/

The coverage badge JSON is at /etl-validation-toolkit/coverage.json and is consumed by the README Shields endpoint badge. You do not need to manually host a server, database, or API; Pyodide runs locally inside visitor browsers. If Pages actions are restricted by organization policies, enable Pages deployment permissions for the workflow.

3. Release v0.1.2

After confirming the full CI suite, commit changes and create the release tag locally (the reviewed archive has no embedded tags):

```
git tag v0.1.2
git push origin v0.1.2
```

The release.yml workflow then builds Python wheel and source distribution with python -m build and creates a GitHub Release with downloadable assets. Tags are immutable release milestones by convention: if you make functionality changes after the v0.1.2 tag, use a later patch release instead of rewriting the published tag. A GitHub Release does not exist before this tag is pushed and the action succeeds.

4. Optional PyPI trusted publishing

PyPI publication is intentionally not enabled by default. If you want it later, create a PyPI project with a trusted publisher tied to your GitHub repository, workflow and environment, then add an explicitly approved publish job using pypa/gh-action-pypi-publish. Keep publishing independent of CI and avoid API tokens in repository secrets whenever a trusted publisher is available.

5. Local quality check

```
python -m pip install -e ".[dev]"
ruff check .
mypy src/etlguard
pytest --cov=etlguard --cov-branch --cov-report=xml --cov-fail-under=85
python scripts/build_demo_bundle.py
```

The integration tests require a live **MySQL 8** database. With Docker Compose, point ETLGUARD_TEST_MYSQL_URL to the loopback address and port 3307, using the local-only demo credentials in .env.example; the tests create disposable it_ tables and clean up afterward. CI uses the MySQL service on port 3306.

Promotion gate: Before tagging or publishing, run and sign off the **MySQL 8** integration checks in [the v0.1.2 runbook](production-v0.1.2.md). A wheel build alone does not establish production readiness.

mysql8-staging-and-pilot-gate.md

ETL Validation Toolkit v0.1.2: MySQL 8 Staging Gate and Controlled Pilot

Release decision

NO-GO for unattended production until real **MySQL 8** tests, operational recovery rehearsal, controlled pilot, and approval are complete. This kit does not make production changes and does not turn a local unit-test **pass** into production certification.

What this staging gate verifies

- Connects to an expressly confirmed, disposable etlguard_ci, etlguard_test, or etlguard_staging_ci schema. It will not run integration tests on a schema with unrelated tables.
- Refuses MySQL root/admin account names, MySQL 5.7/MariaDB servers, non-InnoDB default engine, read-only servers, wrong schema, and remote hosts unless --allow-remote is explicitly provided.
- Runs the repository's 13 **MySQL 8** integration tests, complete pytest suite with at least 85% branch coverage, Ruff, and mypy.
- Records command exits and skipped tests in staging-gate-report.json and detailed logs. A nonzero status or ANY skipped test prevents a **pass**.
- MySQL integration tests exercise upsert, **SCD Type 2**, concurrency, migration, uniqueness, rollback, audit and retry logic.

Important: These integration tests CREATE and DROP tables; never point ETLGUARD_TEST_MYSQL_URL at customer/staging application schemas or any production schema. This is not a security assessment, performance certification, or proof of real-world RPO/RTO.

Option A: Local disposable MySQL 8.4 with Docker

From the extracted source directory:

```
python -m venv .venv
# On Linux/macOS: source .venv/bin/activate
# On Windows PowerShell: .venv\Scripts\Activate.ps1
python -m pip install -e '.[dev]'

docker compose -f docker-compose.mysql8-staging.yml up -d
# The example is ONLY available on localhost port 3308 and uses ephemeral tmpfs storage.
export ETLGUARD_TEST_MYSQL_URL='mysql+pymysql://etlguard_ci:etlguard_ci_local_only@127.0.0.1:330
  8/etlguard_ci'
python scripts/verify_mysql8_staging.py --confirm-disposable-schema etlguard_ci --execute-tests
  --report staging-gate-report.json
```

For Windows PowerShell, set the test URL with:

```
$env:ETLGUARD_TEST_MYSQL_URL =
  'mysql+pymysql://etlguard_ci:etlguard_ci_local_only@127.0.0.1:3308/etlguard_ci'
python scripts/verify_mysql8_staging.py --confirm-disposable-schema etlguard_ci --execute-tests
  --report staging-gate-report.json
```

Exit 0 plus "status": "**passed**" is a staging test **pass**, not a production deployment approval. Exit 1 means at least one test/quality gate **failed**; exit 2 means preflight was **blocked**. For local cleanup, docker compose -f docker-compose.mysql8-staging.yml down; the database uses a tmpfs and is never to contain real data. Do not reuse the example passwords anywhere beyond this local test environment.

Option B: Your own disposable MySQL 8 server

1. Create a **new, empty** schema named `etlguard_staging_ci` and a dedicated NON-ROOT account that can create/drop tables only inside that schema.
2. Set `ETLGUARD_TEST_MYSQL_URL` using environment/secret management; do not commit credentials to a repository or send them by email/chat.
3. Run `python scripts/verify_mysql8_staging.py --confirm-disposable-schema etlguard_staging_ci --execute-tests --allow-remote` on the trusted test runner. Omit `--allow-remote` when connecting through a localhost SSH tunnel.
4. Save the JSON report and logs from `mysql8-gate-logs/` in restricted CI artifacts.
5. Run the same suite on the precise MySQL major/minor version, SQL mode, charset, and resource limits used by the intended staging environment. Docker default settings are not necessarily representative.

Option C: GitHub Actions, with no external database credentials

Copy this project into your own private GitHub repository. Run the included .github/workflows/mysql8-staging-gate.yml via Actions > **MySQL 8** Staging Verification > Run workflow. It provisions disposable **MySQL 8**.0 and **MySQL 8**.4 service containers, runs Python 3.11 and 3.13 jobs, and preserves test reports as artifacts. This is a CI compatibility gate only; it is not your application staging database.

Controlled pilot protocol (manual approval required)

Phase 0: Preconditions / written sign-off

- [ ] **MySQL 8** staging gates GREEN (zero skipped checks); staging schema, charset, SQL mode, and data volumes are representative.
- [ ] Input source is immutable or snapshotted; scheduler stores the UUID *before* the first attempt.
- [ ] Data owner and DB owner have approved the exact pipeline YAML, target schema, writes, and observation window.
- [ ] Verified backup and a timed restore exercise have **passed**; RPO/RTO are documented.
- [ ] **SCD2** older tables are explicitly upgraded during maintenance; no duplicate active rows; all participating tables use InnoDB.
- [ ] Privileged DDL is disabled for regular runtime identities; reject reports and logs have retention/PII access controls.
- [ ] Running jobs and external writers are paused or coordinated, especially when testing **SCD2**.

Phase 1: Shadow pilot, isolated target

1. Clone the target schema to a dedicated staging/shadow table; never run `truncate_insert` against the real production table using a sample subset. It would replace the entire target.
2. Run `etlguard validate <pipeline.yaml>` first; inspect row counts, reject reasons, source hash and dataset-level checks.
3. Generate ONE UUID, persist it with the immutable source, and execute `etlguard run <pilot-pipeline.yaml> --batch-id <uuid>` only against the shadow target.
4. Run `etlguard batch-status <uuid>`, compare target counts and business keys, and inspect the `etl_batch_commits` and `etl_run_audit` rows.
5. Retry the SAME input and UUID; verify the committed target does not change. Retry with changed input and SAME UUID; verify it is rejected.
6. Rehearse connection loss before and around COMMIT in staging; verify unknown outcomes are resolved using batch-status, not a newly generated UUID.
7. Re-run concurrent **SCD2** updates and verify exactly one active row for every business key, including alongside any non-ETL Guard writers.

Phase 2: Approved, limited real-target pilot

1. Obtain written change authorization specifying environment, pipeline, exact target, permitted load mode, batch/time limits and a rollback owner.
2. Start with one supervised run during a designated observation window, not unattended cron. For replacement jobs, execute only the approved complete snapshot with independent pre-run row-count comparison.
3. Monitor process exit code, MySQL sessions/locks, validation failures, row deltas, rejection percentage, audit/commit ledger consistency and query performance.
4. Compare downstream business totals to baseline; keep a snapshot of pre-pilot and post-pilot metrics. Hold the scheduler disabled until sign-off.
5. If the result is ambiguous, do not create a fresh batch UUID. Query `batch-status` after DB recovery; reconcile target state. A committed load cannot be undone by simply issuing ROLLBACK.
6. If a committed change must be reverted, use the tested backup/PITR or explicitly approved compensating SQL; have the DB owner lead restoration.

Phase 3: Go / no-go

GO only after all test gates **pass**, audit/ledger and target reconcile, backup/restore is proved, repeat/recovery is safe, performance is acceptable, and the data owner, DB owner, and operations owner sign off. Otherwise **NO-GO**; keep unattended jobs disabled.

Gate results template

```
| Gate | Evidence | Status |
| --- | --- | --- |
| MySQL 8.0 integration | `staging-gate-report.json` + logs | NOT RUN |
| MySQL 8.4 integration | `staging-gate-report.json` + logs | NOT RUN |
| Ruff / mypy | CI logs | NOT RUN |
| Complete pytest suite / coverage | CI logs | NOT RUN |
| SCD2 active-row uniqueness and concurrency | MySQL test logs + verification SQL | NOT RUN |
| Immutable input and durable batch ID | Scheduler configuration evidence | NOT RUN |
| Backup restore rehearsal | Restore log and elapsed time | NOT RUN |
| Shadow pilot and reconciliation | Signed pilot evidence | NOT RUN |
| Live supervised pilot | Change record / monitoring | NOT RUN |
| Production approval | Data owner + DB owner + Operations | NOT APPROVED |
```

Never set a gate to **PASS** from documentation alone.

phase-3-shadow-reconciliation.md

Phase 3: Read-only shadow-table reconciliation and retry integrity

Release: ETL Validation Toolkit v0.1.2, source-only verification extension.

Status: MySQL 8.0/8.4 staging and independent shadow reconciliation **passed** on Python 3.11/3.13. See [completion evidence](completion-report.md).

Safety: This workflow is ONLY for an approved, disposable **MySQL 8** shadow schema.

Purpose and evidence

A successful ETL exit status or matched row count cannot prove that the target

contains the correct values. Phase 3 adds scripts/verify_shadow_reconciliation.py,

which independently re-reads the immutable CSV/Excel snapshot, repeats validation,

and compares all modeled values of every record with the persisted target.

The verifier checks:

- The source/config/checksum manifest still matches the originally prepared pilot.
- A real passing **MySQL 8** staging report is available and current.
- Only a localhost/SSH-tunneled, dedicated non-root, disposable shadow schema is used.
- A durable committed batch, corresponding success audit, and zero uncommitted staging rows exist.
- Source record counts, valid/rejected counts, source request hash, batch ledger counts and audit totals agree.
- No missing, extra, modified or schema-incompatible model rows exist in target.
- For a first-load **SCD2** table, exactly one active record per key and no unexpected historical versions exist.
- A stable **SHA-256** fingerprint of the *physical target rows*, including ETL IDs, remains unchanged across an identical retry.

The JSON report contains counts, mismatch categories, identifiers and hashes,

not raw customer rows, email addresses or database passwords. Protect the report

as operational metadata; hashes do not make the report a digital signature.

Run on a separate, disposable MySQL 8 staging instance

1. Use the earlier **`docs/shadow-pilot-and-promotion.md`** to run the staging gate successfully and prepare a pilot-demo/shadow-manifest.json pinned to an immutable source snapshot.
2. Set **`ETLGUARD_SHADOW_MYSQL_URL`** to a dedicated **`etlguard_shadow_ci`** user/schema on localhost (or a localhost SSH tunnel); never set production credentials.
3. Run one explicitly authorized shadow load:

```
python scripts/run_shadow_pilot.py --manifest pilot-demo/shadow-manifest.json \
  --execute --confirm-shadow-schema etlguard_shadow_ci \
  --staging-report staging-gate-report.json \
  --report pilot-demo/first-run.json
```

4. Verify the **actual persisted rows** with this new read-only command:

```
python scripts/verify_shadow_reconciliation.py \
  --manifest pilot-demo/shadow-manifest.json \
  --staging-report staging-gate-report.json \
  --confirm-shadow-schema etlguard_shadow_ci \
  --report pilot-demo/reconciliation-before-retry.json
```

5. Repeat the load with the same `--manifest` and save to pilot-demo/identical-retry.json. The load result must say "replayed": true. Run verify_shadow_reconciliation.py again with output pilot-demo/reconciliation-after-retry.json.
6. Compare the two reports: both must have `"status": "passed"`, zero mismatches and exactly the same target_physical_sha256. Retain the source manifest, staging gate, first/retry load reports, and both reconciliation reports in access-controlled deployment evidence. These reports can support the shadow_row_reconciliation and same_batch_retry operational checks, only after an independent operator verifies they are genuine.

PowerShell example

```
$env:ETLGUARD_SHADOW_MYSQL_URL =
  'mysql+pymysql://etlguard_shadow:etlguard_shadow_local_only@127.0.0.1:3308/etlguard_shadow_ci'
python scripts/verify_shadow_reconciliation.py --manifest pilot-demo/shadow-manifest.json
  --staging-report staging-gate-report.json --confirm-shadow-schema etlguard_shadow_ci --report
  pilot-demo/reconciliation-before-retry.json
```

The password in this example is an intentional public fixture valid only for the local disposable Compose database. Never reuse it on shared or production hosts.

Automated GitHub Actions shadow smoke

The existing .github/workflows/mysql8-staging-gate.yml now also runs independent reconciliation before and after the same-UUID retry, via scripts/ci_shadow_smoke.py. The CI job fails if rows mismatch or the physical fingerprint changes; its report includes summarized reconciliation evidence. The workflow must be manually triggered; merely having the YAML file does not mean the MySQL tests have been executed.

Offline validation and outstanding gates

- Run PYTHONPATH=src python -m unittest discover -s scripts -p 'test_*.py' -v. The new tests use SQLite for deterministic fault injection, not as a replacement for **MySQL 8** service tests.
- Run PYTHONPATH=src python -m pytest -q --cov=etlguard --cov-branch --cov-**fail**-under=85.
- CI must run a genuine **MySQL 8**.0 and 8.4 service, the full integration suite, Ruff and mypy, and the strengthened shadow smoke; all reports must be archived.
- Backup + restore, real-world performance and RPO/RTO checks, rollback exercises, monitoring, and all human approvals remain separate gates.

Release decision: Neither a passing SQLite demo nor a passing shadow reconciliation is production authorization. No unattended production scheduler

may be enabled until the real **MySQL 8** staging/pilot evidence and independent

change-control approvals are complete.

shadow-pilot-and-promotion.md

ETL Validation Toolkit v0.1.2 — Phase 2: Shadow Pilot and Promotion Review

Status: NOT production-certified. These tools are for controlled verification. No production database writes are performed automatically. The original etlguard Python runtime and wheel stay at v0.1.2.

Prerequisites

- Python 3.11+ and python -m pip install -e '.[dev]' from the extracted source.
- A passing **MySQL 8** staging gate from scripts/verify_mysql8_staging.py on a disposable **MySQL 8** instance. The gate now also requires strict SQL mode, schema-scoped grants and all existing test tables InnoDB.
- An immutable local CSV/Excel snapshot. API feeds must be captured to a controlled snapshot first.
- Operator-approved source schema, row counts, target scope and changes. No live/customer data is needed to run the synthetic example below.

Step 1 — Start a disposable MySQL 8 and run the automated staging gate

```
python -m pip install -e '.[dev]'
docker compose -f docker-compose.mysql8-staging.yml up -d
export ETLGUARD_TEST_MYSQL_URL='mysql+pymysql://etlguard_ci:etlguard_ci_local_only@127.0.0.1:330
  8/etlguard_ci'
python scripts/verify_mysql8_staging.py --confirm-disposable-schema etlguard_ci --execute-tests
  --report staging-gate-report.json
```

On Windows PowerShell use $env:ETLGUARD_TEST_MYSQL_URL = 'mysql+pymysql://etlguard_ci:etlguard_ci_local_only@127.0.0.1:3308/etlguard_ci'. The Docker Compose setup is disposable, bound to loopback, and now provisions a second shadow-only schema etlguard_shadow_ci with user etlguard_shadow. Passwords in these examples are intentionally public local-test credentials and must never be used for production or in a shared network.

The staging script exits 0 only when all integration, full pytest, Ruff and mypy checks **pass** with zero skipped tests. Record its JSON report and logs. Do not proceed to a database write when this gate fails.

Without local Docker: Upload this source to an access-controlled GitHub repository, open Actions → **MySQL 8** Staging Verification → Run workflow. The included manually dispatched workflow runs **MySQL 8**.0 and 8.4 with Python 3.11/3.13. After passing the staging gate it runs a disposable shadow smoke test that exercises the first load and the same-UUID retry; it exports shadow-smoke-report.json alongside the gate logs. The workflow does not connect to a production database. No claim of a CI **pass** is made until it actually executes and publishes its logs. The separate public-demo Pages deployment is disabled by default unless repository variable ENABLE_PUBLIC_DEMO_PAGES is explicitly set to true.

Step 2 — Prepare a pinned synthetic shadow pilot (NO database writes)

```
python scripts/prepare_shadow_pilot.py plan \
  --config src/etlguard/examples/pipelines/customers_clean.yaml \
  --output pilot-demo
python scripts/prepare_shadow_pilot.py verify --manifest pilot-demo/shadow-manifest.json
python scripts/run_shadow_pilot.py --manifest pilot-demo/shadow-manifest.json --validate \
  --report pilot-demo/shadow-validation.json
```

This creates shadow-pipeline.yaml and shadow-manifest.json containing a single persistent batch UUID, **SHA-256** hashes of the original YAML, original CSV/Excel and generated shadow YAML, and a pilot_ prefixed shadow target. The generated YAML is constrained to ETLGUARD_SHADOW_MYSQL_URL and disables permissive empty/partial replacement options. Do not overwrite, regenerate or edit these artifacts on retries. For another input, prepare a new pilot output directory, with a new UUID. The planner refuses API feeds, zero-byte files, unsafe target identifiers, and truncate_insert unless explicitly authorized for an isolated shadow table.

A B C D E F G H

The --validate operation reads the snapshot and writes local validation reports without contacting any database. Investigate any rejection or source problem before doing Step 3.

Step 3 — Perform an approved, isolated shadow-only write

First verify the **MySQL 8** staging gate in Step 1 **passed**. Use a fresh local shadow schema; the included disposable Compose example initializes it automatically. In a staging environment that uses its own server, expose only the separately provisioned disposable shadow schema through a localhost SSH tunnel; give the dedicated shadow identity privileges *only* on that schema.

```
export ETLGUARD_SHADOW_MYSQL_URL='mysql+pymysql://etlguard_shadow:etlguard_shadow_local_only@127
  .0.0.1:3308/etlguard_shadow_ci'
python scripts/run_shadow_pilot.py --manifest pilot-demo/shadow-manifest.json \
  --execute --confirm-shadow-schema etlguard_shadow_ci \
  --staging-report staging-gate-report.json \
  --report pilot-demo/shadow-first-run.json
```

PowerShell:

```
$env:ETLGUARD_SHADOW_MYSQL_URL =
  'mysql+pymysql://etlguard_shadow:etlguard_shadow_local_only@127.0.0.1:3308/etlguard_shadow_ci'
python scripts/run_shadow_pilot.py --manifest pilot-demo/shadow-manifest.json --execute
  --confirm-shadow-schema etlguard_shadow_ci --staging-report staging-gate-report.json --report
  pilot-demo/shadow-first-run.json
```

run_shadow_pilot.py requires a real passing **MySQL 8** staging report, verifies the manifest, runs validation first, then refuses root/admin credentials, remote database endpoints, non-disposable schema names, out-of-schema grants, non-InnoDB tables, a pre-existing untracked target, unrelated rows in the commit ledger, leftover staging rows and missing success audits. It accepts only genuine **MySQL 8** with strict SQL mode. The generated shadow target is isolated from the original configured table. It does not itself certify that the server is production-independent — the operator must independently confirm that this is a disposable staging instance.

Do not bypass these safety checks by directly invoking etlguard run on a production connection. After a successful shadow run, repeat the same command with exactly the same manifest. The second result must have "replayed": true and must not change target rows. A changed CSV/Excel or YAML checksum fails verification before a database write. Preserve both run reports and etlguard batch-status <batch_uuid> --database-url-env ETLGUARD_SHADOW_MYSQL_URL output as evidence. This does not replace a separate simulated lost-commit test, **SCD2** concurrency test or verified backup/restore rehearsal.

Phase 3 — Independently verify persisted rows and unchanged retry fingerprint

After the initial --execute, run the new read-only verifier, then repeat

the same batch ID and run it again. Both reports must **pass** and have identical

target_physical_sha256 values. The checker detects same-count row corruption

that a simple rows_loaded check would miss. See

docs/phase-3-shadow-reconciliation.md for the exact commands.

Step 4 — Assemble evidence; assess readiness for human approval

Copy docs/pilot-evidence-template.json to an evidence folder as pilot-evidence.json. Attach the actual logs, reconciliations, backup-restore and incident-recovery evidence and signed owner approvals under that same directory. After independently verifying each artifact, set the relevant verified field to true and set evidence_file to the file name. Do not use placeholder documents to mark a check complete.

```
python scripts/assess_promotion.py \
  --staging staging-gate-report.json \
  --evidence ./pilot-evidence/pilot-evidence.json \
  --report promotion-review.json
```

The assessor requires all four automated checks to **pass** with zero skips, real **MySQL 8** preflight details, a staging report no older than 30 days, required operational evidence and data-owner, database-owner and operations-owner approval artifacts. It records **SHA-256** digests of available evidence files. The possible decisions are:

- **blocked**: required evidence/test results are missing, **failed**, stale or skipped. Exit code 2.
- review_ready_not_authorized: supplied evidence is complete enough to send to human change-control reviewers, but no production authorization or database write is issued. Exit code 0.

Crucial limitation: the tool cannot authenticate signatures, independently verify the truth of uploaded screenshots/reports, prove backup recovery or grant permission to operate production. People with operational responsibility must review these artifacts independently and grant explicit change authorization.

Step 5 — Supervised production pilot only after human sign-off

A separate authorized deployment team must confirm the actual production schema, table, write mode, source snapshot, schedule, tested backup/PITR restore, performance/lock budget, monitoring and rollback owner. Start with a single supervised job, record before-and-after counts and ledger state and leave the cron/scheduler disabled until owners approve. A pilot that fails must be investigated; never substitute a new UUID to retry an uncertain commit. For truncate_insert, never use a partial data sample against a live target.

Known boundaries and recovery

- This kit cannot run a real **MySQL 8** service in the current artifact-building environment. An external **MySQL 8** staging instance is needed for the 13 database integration tests and the shadow load.
- Local automated checks do not test infrastructure-specific timeouts, replication lag, cloud privileges, backup/PITR or actual workload performance.
- A successfully committed transaction cannot be undone with a later ROLLBACK. Use approved compensating actions or independently rehearsed restore/PITR.
- Source and config **SHA-256** checks detect changes between checks; protect shared source files against concurrent edits while the load is running.
- This kit deliberately never issues a production-ready or GO decision automatically.

production-v0.1.2.md

v0.1.2 production-hardening runbook (MySQL 8)

Release status: staging candidate until the real **MySQL 8** integration suite, schema checks, and a staging load have **passed** in your environment. Local SQLite tests do not validate MySQL transaction/advisory-lock/index behavior.

Safeguards added

1. **Atomic commit:** For each successful loader call, target rows, **`etl_batch_commits`** (**SHA-256** request fingerprint and logical row counts), and **`etl_run_audit`** success status are written on the same InnoDB transaction. Before doing DML, the code verifies that the target, staging, audit, and ledger tables all use InnoDB. For a direct **`MysqlLoader.load()`** call, an audit is also written with pipeline/source **`direct_api`**.
2. **Idempotent retries:** A canonical UUID batch ID is the retry identity; the complete incoming record stream plus validated pipeline settings and model schema form a deterministic request hash. Reusing the same UUID with the same input/configuration returns the committed logical counts without touching target rows. Using the UUID with changed input/settings is rejected. The low-level **`load`** method computes its own fingerprint when no **`request_hash`** is **passed**.
3. **Unknown commit recovery:** If an error occurs when a commit acknowledgement is lost, the loader checks the durable ledger. Only a matching committed record can prove success. If the database cannot be contacted, the commit outcome is **unknown**, not proven rolled back; query status after restoration and retry the **same UUID**, not a fresh UUID.
4. **Concurrent **SCD2**:** Every MySQL ETL Guard writer obtains a server-side **`GET_LOCK('etlguard:table:<table>', timeout)`** for the target before merging. New **SCD2** tables additionally

have a generated `__etl_current_marker` and a unique index on `(business_key_columns..., __etl_current_marker)`; the marker is `1` for current rows and `NULL` for historical rows. The database thus rejects a second active version of a business key even when external code bypasses the ETL advisory lock. The business key must be `NOT NULL`.

5. **Safer replacement:** `truncate_insert` continues to use **transactional DELETE, never MySQL TRUNCATE**. Direct loader calls refuse an empty input without `allow_empty_replace=True`; pipeline YAML also blocks empty and partially rejected replacements by default. Set `target.min_replace_ratio` (e.g. `0.90`) to reject sudden drops relative to the existing target row count **before** deletion. All input records are staged before replacing the target.
6. **Operational visibility:** `etlguard batch-status <uuid>` reads committed evidence; failures are audited separately and cannot overwrite an existing successful audit. JSON summaries disclose `request_hash`, `durable_commit`, `replayed`, and `warnings`. Report file failures after a committed transaction are warnings, not load failures. Source stream failures retain partial counters and block loading. Rejected CSV values beginning with spreadsheet formula characters are escaped; JSON retains the exact original value.

Fresh installation and staging verification

Requires Python 3.11+, a disposable **MySQL 8**.x InnoDB database and database credentials for creating tables/indexes during staging (use separate minimum-privilege credentials in production).

```
python -m pip install ./etl_validation_toolkit-0.1.2-py3-none-any.whl
etlguard --help

# Optional local MySQL 8 test database provided by the source ZIP.
docker compose up -d mysql
export
  ETLGUARD_TEST_MYSQL_URL='mysql+pymysql://etlguard:etlguard_local_only@127.0.0.1:3307/etlguard'
export ETLGUARD_DATABASE_URL="$ETLGUARD_TEST_MYSQL_URL"

# Initialize shared audit and commit tables outside the target transaction.
etlguard init-db

# From the extracted SOURCE ZIP, with development dependencies installed:
python -m pip install -e '.[dev]'
python -m pytest -q tests/test_mysql_integration.py -m integration
python -m pytest -q --cov=etlguard --cov-branch --cov-fail-under=85
ruff check .
mypy src/etlguard

# Validate without touching MySQL first.
etlguard validate src/etlguard/examples/pipelines/customers_clean.yaml
```

Do not run tests against real customer data or tables. The test suite creates and drops temporary it_* tables, and writes test audit/ledger rows. Always use a disposable test schema.

Using stable batch IDs in production

The scheduler must generate a unique UUID before starting the ETL attempt and persist the assignment alongside the immutable source snapshot. Reuse the same UUID and the same source/configuration for all retries of that logical attempt; use a different UUID for the next scheduled feed.

```
# Generate and retain this ID in your orchestrator/state database.
python -c 'import uuid; print(uuid.uuid4())'

etlguard run path/to/pipeline.yaml --batch-id 11111111-1111-4111-8111-111111111111
etlguard batch-status 11111111-1111-4111-8111-111111111111

# Retrying with the same UUID is a read-only replay if already committed:
etlguard run path/to/pipeline.yaml --batch-id 11111111-1111-4111-8111-111111111111
```

batch-status reads only v0.1.2 etl_batch_commits. Absence of a row does not prove that a pre-v0.1.2 load never committed, since v0.1.1 had no ledger. For legacy data, reconcile manually before replaying; do not blindly generate a new batch ID. A changed API response can change the fingerprint; use stable snapshots for API pipelines.

Replacement protection example

```
target:
  table: customers_daily
  mode: truncate_insert
  min_replace_ratio: 0.90 # reject input containing fewer than 90% of current rows
  lock_timeout_seconds: 30
policy:
  fail_if_reject_rate_above: 0.05
  allow_empty_replace: false
  allow_partial_replace: false
```

A ratio threshold is optional. Choose a realistic bound based on expected business changes; it does not replace independent row-count checks or source-versus-target reconciliation. To deliberately replace with an empty dataset you must explicitly set policy.allow_empty_replace: true, and the direct loader additionally requires allow_empty_replace=True.

Existing SCD2 target migration

New **SCD2** tables include the active-row generated-column uniqueness guard automatically. Existing v0.1.1 **SCD2** tables are not silently ALTERed. Regular loads reject them until an operator performs an explicit migration:

1. Back up the existing table and confirm all other writers are stopped (maintenance window).
2. Check that the business key is NOT NULL and no key has more than one `is_current=1` row. If duplicate active rows already exist, correct them before migration.
3. Run `etlguard upgrade-scd2 path/to/scd2-pipeline.yaml --yes` using a database principal with schema-change permissions. The command adds the stored generated marker and a unique index; MySQL DDL commits independently and cannot be rolled back together with an ETL job.
4. Run the disposable **MySQL 8** integration tests and perform a sample update, no-change replay and a concurrent-update check in staging.

Do not enable the upgrade command in an automatic scheduler. Its MySQL schema modification can lock a large table. Coordinate the maintenance window, measure lock impact, and take a verified backup first.

Observability and rollback procedure

- Alert on CLI exit codes 1 (quality/load failure), 2 (invalid configuration), persistent warnings, unknown commit outcomes, or unexpected row-count changes. Persist stdout/stderr, the batch UUID and JSON summary in restricted-access job storage.
- On a **failed** or interrupted job, run etlguard batch-status <uuid> before deciding how to retry. If committed, do not replay under a new UUID. If not committed and the input snapshot is identical, retry with the original UUID.
- If a run **failed** after an audit or database connection error, inspect etl_run_audit and etl_batch_commits, reconcile target row counts, and escalate any mismatch. A success ledger row means the target commit was acknowledged by InnoDB; the database is the source of truth rather than local report files.
- Transactional failures before commit roll back staged and target DML automatically. A committed replacement or **SCD2** change requires an approved restore/compensating migration; it cannot be undone by ROLLBACK after commit.
- Keep MySQL binary logs and backups per your RPO/RTO. Restrict database credentials, filesystem reports and rejection files, because they may contain sensitive source values. The library still holds the validated batch in memory and is not yet a bounded-memory streaming loader for massive feeds.

Go/no-go criteria

- [ ] All **MySQL 8** integration tests **pass** in CI and on the actual staging configuration.
- [ ] All business keys are NOT NULL, required indexes exist, and all participating tables use InnoDB.
- [ ] Existing **SCD2** targets are migrated after backup and duplicate-current checks.
- [ ] Concurrent **SCD2**, rollback, duplicate-retry, changed-input, replacement, and connection-failure scenarios are verified in staging.
- [ ] Scheduler persists batch IDs and immutable input snapshots; monitoring alerts and failure recovery procedure are in place.

A B C D E F G H

- [ ] Security and scale limits are reviewed for the real data volumes, report retention and access rights.

Until all checks are completed, v0.1.2 is a production-hardening *candidate*, not a certified production deployment.

Follow-on staging gate

Use the separate [**MySQL 8** staging gate and controlled-pilot checklist](mysql8-staging-and-pilot-gate.md) to execute the tests against a disposable **MySQL 8** database and collect auditable go/**no-go** evidence. No production certification is implied by shipping the verification kit.

review-v0.1.1.md

ETL Validation Toolkit — Code Review, Test Results & Patch Handover

Review date: 10 November 2026

Original artifacts: etl-validation-toolkit-v0.1.0.zip, etl_validation_toolkit-0.1.0-py3-none-any.whl

Reviewed outputs: etl-validation-toolkit-v0.1.1-reviewed.zip, etl_validation_toolkit-0.1.1-py3-none-any.whl

Release classification: Reviewed local candidate; not certified production-ready.

Executive summary

The project is a well-structured Python ETL data-quality framework with CSV, Excel and REST API sources; cleaning; Pydantic row validation; dataset checks; rejected-row CSV/JSON; a CLI; MySQL staging/upsert/transactional replacement; and **SCD Type 2** history. It also contains a static browser demo, test suite, example datasets, configuration reference and CI workflows.

The uploaded original wheel's 33 package and resource files were byte-for-byte identical to the corresponding original source files. The original wheel installed locally without fetching dependencies; its CLI launched and validate .../customers_clean.yaml **passed**.

An important false-**pass** condition was reproduced in the original: 100 identical invalid records gave rows_rejected=1, duplicates_dropped=99 and allowed a 5%-threshold quality gate to **pass**. A destructive replacement operation was also demonstrated in SQLite: loading zero validated records into truncate_insert deleted an existing target row. The reviewed 0.1.1 patch addresses both risks and three additional safety/correctness issues.

Patches included in v0.1.1

```
| ID | Severity | Finding | Reviewed fix and regression |
| --- | --- | --- | --- |
| F-01 | High | Identical invalid rows were deduplicated before validation, understating rejection rates. |
Deduplication is now limited to accepted rows after Pydantic and dataset-level checks. Tests cover invalid model
rows, invalid foreign keys and repeated unique-key violations. |
| F-02 | High | `truncate_insert` could replace the entire target with empty or partially rejected data. |
Replacement now fails on an empty input or any rejects unless the operator explicitly enables
`policy.allow_empty_replace` / `policy.allow_partial_replace`. |
| F-03 | High | MySQL `ON DUPLICATE KEY UPDATE` can update the wrong business record if the target has multiple
competing unique indexes. | Preflight rejects competing uniqueness constraints beyond the declared business key and
ETL primary key. |
| F-04 | Medium | Pipeline identifiers could contain directory separators and affect generated report paths. | Added
filename-safe pipeline identifier validation. |
| F-05 | Medium | `rename_map` was silently ignored when `snake_case_columns: false` used non-normalized source
headers. | The rename map now respects column-normalization mode. |
```

Replacement policy compatibility note: Existing pipelines using truncate_insert on empty or partially rejected feeds now **fail** closed by default. Opt-ins are available, but should be enabled only after checking the intended data-loss semantics. Other loading modes retain their original rejection-policy behavior.

Testing performed

```
| Check | Original v0.1.0 | Reviewed v0.1.1 |
| --- | --- | --- |
| Wheel vs matching source contents | 33 of 33 match | Verified after build |
| Automated tests run | 81 passed / 10 skipped | **91 passed / 10 skipped** |
| Statement/branch coverage (`pytest-cov`, aggregate) | 89% | **90%** |
| CLI startup / `--help` | Passed | Passed |
```

```
| Example clean CSV validation | Passed | Passed |
| Reproduce repeated-invalid false-pass | Reproduced | Fixed; regression passes |
| SQLite-backed loader smoke tests | Passed | Passed |
| Local wheel build | Original wheel provided | Built successfully offline |
| Python compile check | Not required | Passed |
| Actual MySQL 8 end-to-end integration | Not run | **Not run** |
```

Ten skipped test cases consist of eight MySQL integration tests (no MySQL test instance), plus two tests/modules requiring optional respx and hypothesis packages. The environment could not download additional packages, so these checks are not claimed as completed. The project includes workflows to run them in a properly provisioned CI environment.

Remaining production-readiness concerns

1. **Audit failure after committed data load (high):** MySQL target DML commits in `MysqlLoader.load()` before `AuditWriter.write()` executes. If auditing fails, `Pipeline.run()` marks the job **failed** even though data may already be committed. **Do not automatically retry a **failed** append load without confirming the batch's commit state**. Design an idempotent load/audit state machine or durable commit marker before unattended production retries.
2. **MySQL integration and schema safety (high):** Full tests for transaction behavior, MySQL-specific upsert, index inspection and **SCD2** still need a real **MySQL 8** server. Avoid using a privileged DB account. Validate schemas and indexes on representative existing tables.
3. **Concurrent **SCD2** writers (high):** This implementation deliberately assumes single-writer execution per target; coordinate jobs through a scheduler/lock to prevent competing current versions.
4. **Data scale (medium):** CSV reading is chunked, but validation stores candidate and clean rows in memory. Large datasets need bounded-memory/spill-to-disk processing.
5. **Reject-file privacy (medium):** Rejected-row reports include raw values and entire source rows, potentially including personal data. Configure restricted report directories, retention limits, encryption at rest and appropriate access controls. Treat reject CSV cells as untrusted when opening them in spreadsheet software.
6. **Report accuracy after mid-stream source failure (medium):** A source failure during iteration can prevent partial row statistics being copied into the run summary. Add a partial-batch accounting test and recovery pathway before relying on audit counts for **failed** streaming jobs.
7. **Release housekeeping (low):** Repository and demo URLs are placeholders (`YOUR_USERNAME`). Configure actual links, run the full CI workflow (Ruff, mypy, Hypothesis, respx, MySQL), and publish through an owned repository only after approval.

Install and run

```
# Requires Python 3.11+ and network/package access for dependencies
python -m pip install ./etl_validation_toolkit-0.1.1-py3-none-any.whl
etlguard --help

# In the extracted reviewed source folder:
etlguard validate src/etlguard/examples/pipelines/customers_clean.yaml
etlguard validate src/etlguard/examples/pipelines/customers.yaml

# Full verification once dependencies and a MySQL 8 test instance are available:
python -m pip install -e '.[dev]'
ruff check .
mypy src/etlguard
pytest -q --cov=etlguard --cov-branch --cov-report=term-missing --cov-fail-under=85
```

For MySQL integration tests, supply ETLGUARD_TEST_MYSQL_URL as documented in docs/deployment.md. Keep test credentials isolated and never run destructive replacement tests against a live customer database.

Conclusion

Use v0.1.1 in development/staging for further evaluation. The original v0.1.0 should not be used with untrusted/duplicate data quality gates or destructive truncate_insert loads. Promote the reviewed candidate only after the real MySQL, missing optional test, lint/type and operational audit/idempotency checks are addressed.

47. Appendix H - Glossary, engineering invariants and final checklist

Term	Definition
Atomic commit	Database unit of work where target DML and successful ledger/audit are committed together.
Batch UUID	Stable canonical identifier reused on retry of the identical immutable request.
Request hash	SHA-256-derived fingerprint of normalized config, model schema and source digest.
Replay	Response that returns earlier committed counts without loading a second time.
BatchConflictError	Error when a reused UUID points at changed input/config/model/target.
InnoDB	Transactional MySQL storage engine required for atomic target/audit/ledger guarantees.
Staging table	Per-target temporary working table for batched accepted candidates.
Append	Load mode inserting accepted candidates as new records.
Transactional replacement	DELETE plus INSERT inside transaction, with empty/partial-source guards.
Upsert	Business-key insert or update semantics.
SCD2	Slowly changing dimension history with version intervals and one active row per key.
Active marker	Computed SCD2 generated column used with a unique constraint.
GET_LOCK	MySQL named advisory lock protecting participating writers per target.
READ COMMITTED	Isolation used by v0.1.3 MySQL load connections to reduce missing-row gap-lock deadlocks.
Gap lock	InnoDB index-range locking implicated in the historical shared-metadata deadlock.
FOR UPDATE	Row-level locking read for current SCD2 versions or durable audit status.
Idempotency	Property that repeated identical approved batch does not create duplicate target effects.
PITR	Point-in-time recovery; not proved by local logical restore alone.
RPO	Recovery point objective; maximum acceptable data loss measured in time.
RTO	Recovery time objective; maximum acceptable restoration interval.
SLO	Service level objective approved for a real workload and infrastructure.
Shadow pilot	Isolated target load exercised before requesting real-target write authorization.
Reconciliation	Independent comparison of accepted records, affected counts, and persisted target/metadata.
Canonical source	Immutable full input whose digest and lineage define a batch.

A
B
C
D
E
F
G
H

SECTION MAP

Term	Definition
Report formula escaping	CSV export precaution that prefixes potentially executable spreadsheet cell values.
Dry run	Validate-only action that does not contact target database to write data.
Release assessor	Read-only evidence tool that can signal readiness for review, not production authorization.
Production GO	Owner-approved deployment decision after exact target pilot and operational controls.
NO-GO	Current disposition while real-target and human approval gates remain unmet.
Owner approval	Genuine independent sign-off by data, DBA and operations authorities.

Invariant summary

- Source must finish reading without an unresolved error before write execution.
- Invalid duplicate source rows count individually as rejects; only previously accepted duplicates may be dropped.
- Reject rate uses distinct rejected source identities divided by rows read.
- No target DML is permitted when a failure policy or replacement guard has **failed**.
- Committed accepted row counts reconcile with inserted + processed/updated counts.
- Successful target data, audit and UUID ledger commit atomically under InnoDB.
- A reused UUID with same request hash produces no second physical load.
- A reused UUID with changed input/configuration/target is rejected.
- **SCD2** has one active row per business key and contiguous UTC version boundaries under coordinated writers.
- Any uncertain commit outcome is resolved through durable lookup and reconciliation before retry.
- No local synthetic test, document or evidence assessor grants production write or scheduling authority.

Final readiness checklist

[ ] Confirm reviewed archive version and hashes

[ ] Confirm exact runtime and dependency compatibility

[ ] Confirm selected target and source ownership

[ ] Confirm validation/reject threshold business acceptance

[ ] Confirm InnoDB, unique constraints and intended lock/isolation behavior

[ ] Confirm stable UUID persisted before scheduling

[ ] Confirm independent source-target ledger reconciliation

[ ] Confirm backup and full restore/PITR where applicable

[ ] Confirm monitored supervised pilot with approved ticket

[ ] Obtain genuine data/DBA/operations approvals

[ ] Authorize unattended scheduler separately

48. Appendix I - Structured operational and release evidence

This appendix reproduces key small JSON artifacts from the supplied v0.1.3 evidence packet so the reasoning behind test, recovery and promotion status can be examined without switching to external logs. Absolute personal home paths and credential-like fields are masked in the printed excerpts. The original packet remains the exact-fidelity evidence archive. Raw large-volume performance samples, binary exports, and the logical SQL dump are indexed elsewhere rather than embedded in an engineering manual.

evidence/promotion-review.json


```
{
  "assessed_utc": "2026-11-11T18:07:35.424222+00:00",
  "database_writes_executed": false,
  "decision": "blocked",
  "disclaimer": "Artifact presence and user assertions are not cryptographic signatures. Human
  approval is still required.",
  "evidence_artifact_sha256": {
    "operational.backup_restore_rehearsal":
  "65098a0413393e70cc1348538a4f3d45fe6caa15080d0bb95f2c228b8bdf6c3b",
    "operational.changed_input_rejected":
  "25951451b530abf790c7070932a56cf2653106cbd3559c9f5a0a48d4a5d61e67",
    "operational.commit_outcome_recovery":
  "1f34203a3864dc92c5f2f5a0ac054c435b261b4efeb9b1918ea33750b6406ef3",
    "operational.immutable_source":
  "6d41beb815a19814a097427a149b881f34059256b4f3ccdba7bfc4c8c2343fd8",
    "operational.monitoring_observation":
  "55a28f969415e6358c60cb7e60fda19f16855ed4899243a526f9006b7a497382",
    "operational.rollback_rehearsal":
  "e21ee8fbfb5777b29fcfe4f3ad213106e4d73cb9132eef8ea0f1ffcfc965ae09",
    "operational.same_batch_retry":
  "7095124cf7ce33d439ca7d3d507fd605e86b630df9ff1addae23b1feeb460e28",
    "operational.scd2_concurrency":
  "f56ef39b53197a25a28a0780e52c4fb6590542762f91e13eaf2e53ead9d573aa",
    "operational.shadow_row_reconciliation":
  "615c259f0c9af45e9e9477910d1e07915a42f0e253aed80f90084d0d5750e078"
  },
  "evidence_manifest_sha256":
  "c6020193cdc14052a080d9c3885e089337f6f653934f994fc830c81d6d9aee7f",
  "production_approved": false,
  "reasons": [
    "Missing verified operational: supervised_real_target_pilot",
    "Missing verified approvals: data_owner",
    "Missing verified approvals: database_owner",
    "Missing verified approvals: operations_owner"
  ],
  "staging_report_sha256": "ffc63bf79d2440f8426e9f303cd0163df06603644f4cba4ee78f3ee57ec7fb5f",
  "toolkit_version": "0.1.3"
}
```


evidence/pilot-evidence.json


```
{
  "toolkit_version": "0.1.3",
  "notes": "Authentic local technical evidence only. Local alert receiver does not verify
  production paging. Production thresholds, data shape, routing, backup policy and approvals
  remain pending.",
  "operational": {
    "immutable_source": {
      "verified": true,
      "evidence_file": "staging/immutable_pilot/shadow-manifest.json",
      "description": "Source snapshot SHA-256, persisted UUID, and verification log",
      "scope": "Fresh synthetic disposable local MySQL; independent owner review remains
  pending"
    },
    "backup_restore_rehearsal": {
      "verified": true,
      "evidence_file": "operations/backup_restore.json",
      "description": "Signed backup verification + timed restore logs and RPO/RTO",
      "scope": "Fresh synthetic disposable local MySQL; independent owner review remains
  pending"
    },
    "shadow_row_reconciliation": {
      "verified": true,
      "evidence_file": "staging/shadow-reconciliation-first.json",
      "description": "Before/after target counts, source totals, rejected rows and sample keys",
```

```
      "scope": "Fresh synthetic disposable local MySQL; independent owner review remains
  pending"
    },
    "same_batch_retry": {
      "verified": true,
      "evidence_file": "staging/shadow-reconciliation-identical_retry.json",
      "description": "Repeated identical batch ID and unchanged target checksum/row count",
      "scope": "Fresh synthetic disposable local MySQL; independent owner review remains
  pending"
    },
    "changed_input_rejected": {
      "verified": true,
      "evidence_file": "operations/changed_input.json",
      "description": "Negative test with altered snapshot and same UUID refused",
      "scope": "Fresh synthetic disposable local MySQL; independent owner review remains
  pending"
    },
    "commit_outcome_recovery": {
      "verified": true,
      "evidence_file": "operations/server-crash-recovery.json",
      "description": "Connection loss/recovery or failure injection plus batch-status evidence",
      "scope": "Fresh synthetic disposable local MySQL; independent owner review remains
  pending"
    },
    "scd2_concurrency": {
      "verified": true,
      "evidence_file": "operations/scd2_concurrency.json",
      "description": "Concurrent SCD2 writers with exactly one active business key",
      "scope": "Fresh synthetic disposable local MySQL; independent owner review remains
  pending"
    },
    "rollback_rehearsal": {
      "verified": true,
      "evidence_file": "operations/audit_rollback.json",
      "description": "Injected failure and successful rollback/restore logs",
      "scope": "Fresh synthetic disposable local MySQL; independent owner review remains
  pending"
    },
    "monitoring_observation": {
      "verified": true,
      "evidence_file": "operations/monitoring.json",
      "description": "Alerting, query performance, metrics and observed stability window",
      "scope": "Fresh synthetic disposable local MySQL; independent owner review remains
  pending"
    },
    "supervised_real_target_pilot": {
      "verified": false,
      "evidence_file": "",
      "description": "Written change authorization and supervised limited pilot result",
      "reason": "Exact target, change ticket and rollback approval not supplied"
    }
  },
  "approvals": {
    "data_owner": {
      "verified": false,
      "evidence_file": "",
      "description": "Data quality and business reconciliation sign-off",
      "reason": "No human approval supplied; not invented"
    },
    "database_owner": {
      "verified": false,
      "evidence_file": "",
      "description": "Backup/restore, DDL, access control, and performance sign-off",
      "reason": "No human approval supplied; not invented"
    },
    "operations_owner": {
      "verified": false,
      "evidence_file": "",
      "description": "Monitoring, incident recovery, scheduling and rollback sign-off",
      "reason": "No human approval supplied; not invented"
    }
  }
}
```


evidence/staging/staging-gate-report.json


```
{
  "checks": [
    {
      "exit_code": 0,
      "log": "/Users/<user>/Downloads/ETLGuard-readiness-ux4v66ir/fresh-evidence/staging/mysql8-
  gate-logs/mysql8_integration.log",
```


A B C D E F G H

```
      "name": "mysql8_integration",
      "skipped": 0,
      "status": "passed"
    },
    {
      "exit_code": 0,
      "log": "/Users/<user>/Downloads/ETLGuard-readiness-ux4v66ir/fresh-evidence/staging/mysql8-
  gate-logs/complete_suite.log",
      "name": "complete_suite",
      "skipped": 0,
      "status": "passed"
    },
    {
      "exit_code": 0,
      "log": "/Users/<user>/Downloads/ETLGuard-readiness-ux4v66ir/fresh-evidence/staging/mysql8-
  gate-logs/ruff.log",
      "name": "ruff",
      "skipped": 0,
      "status": "passed"
    },
    {
      "exit_code": 0,
      "log": "/Users/<user>/Downloads/ETLGuard-readiness-ux4v66ir/fresh-evidence/staging/mysql8-
  gate-logs/mypy.log",
      "name": "mypy",
      "skipped": 0,
      "status": "passed"
    }
  ],
  "gate": "mysql8_staging",
  "mysql8_integration_executed": true,
  "mysql8_server_verified": true,
  "note": "MySQL8 + static/functional staging gates passed; a separate approved controlled pilot
   is still required.",
  "preflight": {
    "database": "etlguard_ci",
    "default_storage_engine": "InnoDB",
    "existing_table_count": 0,
    "non_innodb_tables": [],
    "read_only": false,
    "schema_only_privileges": true,
    "server_version": "8.4.13",
    "strict_sql_mode": true,
    "unexpected_tables": []
  },
  "production_pilot_executed": false,
  "status": "passed",
  "timestamp_utc": "2026-11-11T18:03:10.223010+00:00",
  "toolkit_version": "0.1.3"
}
```


evidence/staging/one-shot-staging-summary.json


```
{
  "actions": [
    "mysql8_staging_gate_passed"
  ],
  "batch_id": "fdd2e6ed-57d6-4a1e-a04e-a6107d701b3b",
  "checks": [
    "mysql8_integration",
    "complete_suite",
    "ruff",
    "mypy",
    "source_shadow_reconciliation",
    "identical_retry"
  ],
  "generated_at_utc": "2026-11-11T18:03:08.905954+00:00",
  "idempotent_retry_verified": true,
  "physical_target_unchanged": true,
  "production_approved": false,
  "rows_loaded": 3,
  "scope": "local_disposable_mysql_staging_only",
  "status": "staging_and_shadow_passed_not_production"
}
```

evidence/staging/shadow-load-first.json


```
{
  "batch_id": "fdd2e6ed-57d6-4a1e-a04e-a6107d701b3b",
  "replayed": false,
  "request_hash": "70f0c862ccf5c37e160208be7efe2abd172f4fae78db20af0e16dd5d670117ba",
  "rows_inserted": 3,
  "rows_loaded": 3,
  "rows_read": 3,
  "rows_updated": 0,
  "shadow_preflight": {
    "audit_or_ledger_tables": false,
    "database": "etlguard_shadow_ci",
    "replay": false,
    "server_version": "8.4.13",
    "target": "pilot_customers_clean"
  },
  "status": "shadow_committed_not_production",
  "summary_json": "/Users/<user>/Downloads/ETLGuard-readiness-ux4v66ir/fresh-evidence/staging/im
  mutable_pilot/reports/customers_clean_shadow_fdd2e6ed-57d6-4a1e-a04e-a6107d701b3b_summary.json"
}
```


evidence/staging/shadow-load-identical_retry.json


```
{
  "batch_id": "fdd2e6ed-57d6-4a1e-a04e-a6107d701b3b",
  "replayed": true,
  "request_hash": "70f0c862ccf5c37e160208be7efe2abd172f4fae78db20af0e16dd5d670117ba",
  "rows_inserted": 3,
  "rows_loaded": 3,
  "rows_read": 3,
  "rows_updated": 0,
  "shadow_preflight": {
    "audit_or_ledger_tables": true,
    "database": "etlguard_shadow_ci",
    "replay": true,
    "server_version": "8.4.13",
    "target": "pilot_customers_clean"
  },
  "status": "shadow_committed_not_production",
  "summary_json": "/Users/<user>/Downloads/ETLGuard-readiness-ux4v66ir/fresh-evidence/staging/im
  mutable_pilot/reports/customers_clean_shadow_fdd2e6ed-57d6-4a1e-a04e-a6107d701b3b_summary.json"
}
```


evidence/staging/shadow-reconciliation-first.json


```
{
  "audit_rows_observed": 1,
  "batch_id": "fdd2e6ed-57d6-4a1e-a04e-a6107d701b3b",
  "commit_rows_observed": 1,
  "database_writes_executed_by_verifier": false,
  "manifest_sha256": "6d41beb815a19814a097427a149b881f34059256b4f3ccdba7bfc4c8c2343fd8",
  "mismatch_types": [],
  "missing_source_rows": 0,
  "note": "Read-only reconciliation on disposable MySQL 8; not production authorization",
  "phase": "shadow_row_reconciliation",
  "production_approved": false,
  "schema": "etlguard_shadow_ci",
  "shadow_target": "pilot_customers_clean",
  "source_rows_read": 3,
  "source_rows_rejected": 0,
  "source_rows_valid": 3,
  "staging_report_sha256": "ffc63bf79d2440f8426e9f303cd0163df06603644f4cba4ee78f3ee57ec7fb5f",
  "staging_rows_observed": 0,
  "status": "passed",
  "target_physical_sha256": "48796ff1f6a51a60b2bedbe5b2aa821d42d42072aa8ca5737f8e3d6a70f3994b",
  "target_rows_observed": 3,
  "timestamp_utc": "2026-11-11T18:03:17.559960+00:00",
  "toolkit_version": "0.1.3",
  "unexpected_target_rows": 0
}
```

evidence/staging/shadow-reconciliation-identical_retry.json


```
{
  "audit_rows_observed": 1,
  "batch_id": "fdd2e6ed-57d6-4a1e-a04e-a6107d701b3b",
  "commit_rows_observed": 1,
  "database_writes_executed_by_verifier": false,
  "manifest_sha256": "6d41beb815a19814a097427a149b881f34059256b4f3ccdba7bfc4c8c2343fd8",
  "mismatch_types": [],
  "missing_source_rows": 0,
  "note": "Read-only reconciliation on disposable MySQL 8; not production authorization",
  "phase": "shadow_row_reconciliation",
  "production_approved": false,
  "schema": "etlguard_shadow_ci",
  "shadow_target": "pilot_customers_clean",
  "source_rows_read": 3,
  "source_rows_rejected": 0,
  "source_rows_valid": 3,
  "staging_report_sha256": "ffc63bf79d2440f8426e9f303cd0163df06603644f4cba4ee78f3ee57ec7fb5f",
  "staging_rows_observed": 0,
  "status": "passed",
  "target_physical_sha256": "48796ff1f6a51a60b2bedbe5b2aa821d42d42072aa8ca5737f8e3d6a70f3994b",
  "target_rows_observed": 3,
  "timestamp_utc": "2026-11-11T18:03:18.684336+00:00",
  "toolkit_version": "0.1.3",
  "unexpected_target_rows": 0
}
```


evidence/staging/shadow-validation.json


```
{
  "batch_id": "fdd2e6ed-57d6-4a1e-a04e-a6107d701b3b",
  "rows_read": 3,
  "rows_valid": 3,
  "shadow_target": "pilot_customers_clean",
  "status": "validated_only_no_database_writes"
}
```


evidence/browser/browser-report.json


```
{
  "runtime_version": "0.1.3",
  "executed_at": "2026-11-11T18:05:45.238Z",
  "cases": [
    {
      "case": "customer_sample",
      "state": {
        "loaded": "4",
        "read": "8",
        "rejected": "3",
        "status": "Validation complete",
        "valid": "4"
      }
    },
    {
      "case": "transaction_sample",
      "state": {
        "loaded": "3",
        "read": "6",
        "rejected": "2",
        "status": "Validation complete",
        "valid": "3"
      }
    },
    {
      "case": "empty.csv",
      "state": {
        "downloadDisabled": true,
        "read": "\u2014",
        "status": "Could not read CSV: The selected CSV is empty"
      }
    },
    {
      "case": "too-large.csv",
      "state": {
        "downloadDisabled": true,
        "read": "\u2014",
```


A B C D E F G H

```
        "status": "Browser demo accepts CSV files up to 5 MB"
      }
    },
    {
      "case": "unsupported.txt",
      "state": {
        "downloadDisabled": true,
        "read": "\u2014",
        "status": "Select a .csv file"
      }
    },
    {
      "case": "malformed.csv",
      "state": {
        "downloadDisabled": true,
        "read": "\u2014",
        "status": "Validation error: ValueError: source read failed: ValueError: A CSV row has
  more fields than its header"
      }
    },
    {
      "case": "formula.csv",
      "state": {
        "downloadDisabled": false,
        "read": "8",
        "status": "Validation complete"
      }
    }
  ],
  "downloaded_path": "/Users/<user>/Downloads/etlguard_customer_rejects (2).csv",
  "status": "passed",
  "download_formula_escaped": true
}
```


evidence/integrity/v013-runtime-match.json


```
{
  "version": "0.1.3",
  "wheel_source_match": [
    {
      "file": "etlguard/__init__.py",
      "passed": true
    },
    {
      "file": "etlguard/audit.py",
      "passed": true
    },
    {
      "file": "etlguard/browser_demo.py",
      "passed": true
    },
    {
      "file": "etlguard/checks.py",
      "passed": true
    },
    {
      "file": "etlguard/cleaning.py",
      "passed": true
    },
    {
      "file": "etlguard/cli.py",
      "passed": true
    },
    {
      "file": "etlguard/config.py",
      "passed": true
    },
    {
      "file": "etlguard/core.py",
      "passed": true
    },
    {
      "file": "etlguard/pipeline.py",
      "passed": true
    },
    {
      "file": "etlguard/profile.py",
      "passed": true
    },
    {
      "file": "etlguard/reconcile.py",
      "passed": true
```

```
  },
  {
    "file": "etlguard/rejects.py",
    "passed": true
  },
  {
    "file": "etlguard/scd2.py",
    "passed": true
  },
  {
    "file": "etlguard/validation.py",
    "passed": true
  },
  {
    "file": "etlguard/examples/__init__.py",
    "passed": true
  },
  {
    "file": "etlguard/examples/models.py",
    "passed": true
  },
  {
    "file": "etlguard/examples/data/customers_clean.csv",
    "passed": true
  },
  {
    "file": "etlguard/examples/data/customers_messy.csv",
    "passed": true
  },
  {
    "file": "etlguard/examples/data/customers_messy.xlsx",
    "passed": true
  },
  {
    "file": "etlguard/examples/data/transactions_messy.csv",
    "passed": true
  },
  {
    "file": "etlguard/examples/pipelines/customers.yaml",
    "passed": true
  },
  {
    "file": "etlguard/examples/pipelines/customers_clean.yaml",
    "passed": true
  },
  {
    "file": "etlguard/examples/pipelines/customers_excel.yaml",
    "passed": true
  },
  {
    "file": "etlguard/examples/pipelines/customers_scd2.yaml",
    "passed": true
  },
  {
    "file": "etlguard/examples/pipelines/transactions.yaml",
    "passed": true
  },
  {
    "file": "etlguard/loaders/__init__.py",
    "passed": true
  },
  {
    "file": "etlguard/loaders/base.py",
    "passed": true
  },
  {
    "file": "etlguard/loaders/mysql.py",
    "passed": true
  },
  {
    "file": "etlguard/sources/__init__.py",
    "passed": true
  },
  {
    "file": "etlguard/sources/api.py",
    "passed": true
  },
  {
    "file": "etlguard/sources/base.py",
    "passed": true
  },
  {
    "file": "etlguard/sources/csv.py",
    "passed": true
  },
```

```
    {
      "file": "etlguard/sources/excel.py",
      "passed": true
    }
  ],
  "runtime_files_changed_from_v012": [
    "etlguard/__init__.py",
    "etlguard/loaders/mysql.py"
  ],
  "sdist_source_match": [
    {
      "file": "etlguard/__init__.py",
      "passed": true
    },
    {
      "file": "etlguard/audit.py",
      "passed": true
    },
    {
      "file": "etlguard/browser_demo.py",
      "passed": true
    },
    {
      "file": "etlguard/checks.py",
      "passed": true
    },
    {
      "file": "etlguard/cleaning.py",
      "passed": true
    },
    {
      "file": "etlguard/cli.py",
      "passed": true
    },
    {
      "file": "etlguard/config.py",
      "passed": true
    },
    {
      "file": "etlguard/core.py",
      "passed": true
    },
    {
      "file": "etlguard/examples/__init__.py",
      "passed": true
    },
    {
      "file": "etlguard/examples/data/customers_clean.csv",
      "passed": true
    },
    {
      "file": "etlguard/examples/data/customers_messy.csv",
      "passed": true
    },
    {
      "file": "etlguard/examples/data/customers_messy.xlsx",
      "passed": true
    },
    {
      "file": "etlguard/examples/data/transactions_messy.csv",
      "passed": true
    },
    {
      "file": "etlguard/examples/models.py",
      "passed": true
    },
    {
      "file": "etlguard/examples/pipelines/customers.yaml",
      "passed": true
    },
    {
      "file": "etlguard/examples/pipelines/customers_clean.yaml",
      "passed": true
    },
    {
      "file": "etlguard/examples/pipelines/customers_excel.yaml",
      "passed": true
    },
    {
      "file": "etlguard/examples/pipelines/customers_scd2.yaml",
      "passed": true
    },
    {
      "file": "etlguard/examples/pipelines/transactions.yaml",
      "passed": true
```

```
    },
    {
      "file": "etlguard/loaders/__init__.py",
      "passed": true
    },
    {
      "file": "etlguard/loaders/base.py",
      "passed": true
    },
    {
      "file": "etlguard/loaders/mysql.py",
      "passed": true
    },
    {
      "file": "etlguard/pipeline.py",
      "passed": true
    },
    {
      "file": "etlguard/profile.py",
      "passed": true
    },
    {
      "file": "etlguard/reconcile.py",
      "passed": true
    },
    {
      "file": "etlguard/rejects.py",
      "passed": true
    },
    {
      "file": "etlguard/scd2.py",
      "passed": true
    },
    {
      "file": "etlguard/sources/__init__.py",
      "passed": true
    },
    {
      "file": "etlguard/sources/api.py",
      "passed": true
    },
    {
      "file": "etlguard/sources/base.py",
      "passed": true
    },
    {
      "file": "etlguard/sources/csv.py",
      "passed": true
    },
    {
      "file": "etlguard/sources/excel.py",
      "passed": true
    },
    {
      "file": "etlguard/validation.py",
      "passed": true
    }
  ]
}
```


evidence/operations/audit_rollback.json


```
{
  "check": "audit_rollback",
  "status": "passed",
  "details": {
    "batch_id": "3da44a3d-a4cc-4da7-b481-2539c30e49b2",
    "all_four_tables_rolled_back": true,
    "same_uuid_retry_loaded_once": true
  },
  "timestamp_utc": "2026-11-11T18:03:19.434915+00:00",
  "elapsed_seconds": 0.05351112500648014
}
```


evidence/operations/changed_input.json


```
{
  "check": "changed_input",
```

```
  "status": "passed",
  "details": {
    "batch_id": "4f20a584-c6b3-4e99-9c79-24a48d9cd25a",
    "changed_input_refused": true,
    "physical_sha256_unchanged": true
  },
  "timestamp_utc": "2026-11-11T18:03:19.850903+00:00",
  "elapsed_seconds": 0.02854166697943583
}
```


evidence/operations/connection-isolation.json


```
{
  "isolation_level": "READ-COMMITTED",
  "binlog_format": "ROW",
  "pool_return_isolation": "REPEATABLE-READ"
}
```


evidence/operations/final-instance-health.json


```
{
  "status": "passed",
  "mysql_status": {
    "Aborted_connects": "0",
    "Innodb_log_waits": "0",
    "Threads_connected": "1",
    "Threads_running": "2"
  },
  "innodb_deadlock_metric": [
    "lock_deadlocks",
    0,
    "enabled"
  ],
  "active_innodb_transactions": 0,
  "targets_checked_for_lock_release": 16,
  "held_etl_locks": 0,
  "disposable_mysql_shutdown": true,
  "data_retained_privately": true,
  "production_accessed": false
}
```


evidence/operations/lock_timeout.json


```
{
  "check": "lock_timeout",
  "status": "passed",
  "details": {
    "batch_id": "e805bb29-1b83-4444-8dc3-c509d28bc42a",
    "bounded_lock_timeout_seconds": 1.0185452919977251,
    "recovery_after_release": true
  },
  "timestamp_utc": "2026-11-11T18:03:20.907276+00:00",
  "elapsed_seconds": 1.0561657499929424
}
```


evidence/operations/lost_commit_ack.json


```
{
  "check": "lost_commit_ack",
  "status": "passed",
  "details": {
    "batch_id": "2d7f0965-fd58-4fc3-b201-6bdeddec08fb",
    "physical_sha256": "0ffa9b3607b79332456da7e32d4018e365a63ea5838e2d745d9a16e6f8cee150",
    "ledger_resolved_success": true,
    "duplicate_rows": 0,
    "method": "Actual DBAPI COMMIT followed by socket invalidation and injected lost
  acknowledgement; not a server power failure"
  },
  "timestamp_utc": "2026-11-11T18:03:19.484811+00:00",
  "elapsed_seconds": 0.049619542027357966
}
```

evidence/operations/metrics_10000.json


```
[
  {
    "timestamp_utc": "2026-11-11T18:03:21.862516+00:00",
    "python_rss": 145391616,
    "mysql_rss": 86163456,
    "python_cpu_percent": 99.8,
    "mysql_cpu_percent": 0.6,
    "probe_ms": 31.23129199957475
  },
  {
    "timestamp_utc": "2026-11-11T18:03:22.070005+00:00",
    "python_rss": 152403968,
    "mysql_rss": 85704704,
    "python_cpu_percent": 66.5,
    "mysql_cpu_percent": 34.8,
    "probe_ms": 0.7384579803328961
  },
  {
    "timestamp_utc": "2026-11-11T18:03:22.274873+00:00",
    "python_rss": 163938304,
    "mysql_rss": 93454336,
    "python_cpu_percent": 61.8,
    "mysql_cpu_percent": 51.6,
    "probe_ms": 0.6440000142902136
  },
  {
    "timestamp_utc": "2026-11-11T18:03:22.515794+00:00",
    "python_rss": 160776192,
    "mysql_rss": 99565568,
    "python_cpu_percent": 82.9,
    "mysql_cpu_percent": 19.2,
    "probe_ms": 31.30554099334404
  },
  {
    "timestamp_utc": "2026-11-11T18:03:22.753261+00:00",
    "python_rss": 160792576,
    "mysql_rss": 100876288,
    "python_cpu_percent": 85.2,
    "mysql_cpu_percent": 18.5,
    "probe_ms": 29.518249997636303
  }
]
```


evidence/operations/monitoring.json


```
{
  "check": "monitoring",
  "status": "passed",
  "details": {
    "batch_id": "6961e9c3-cad4-41be-8b04-98b4cc62a64e",
    "alert_receipts": [
      {
        "received_at_utc": "2026-11-11T18:03:53.758465+00:00",
        "payload": {
          "alert": "etl_job_failed",
          "batch_id": "6961e9c3-cad4-41be-8b04-98b4cc62a64e",
          "pipeline": "alert_drill",
          "timestamp_utc": "2026-11-11T18:03:53.706583+00:00"
        }
      }
    ],
    "detect_and_deliver_seconds": 0.05272916698595509,
    "receiver": "actual local HTTP receiver",
    "production_notification_route_tested": false
  },
  "timestamp_utc": "2026-11-11T18:03:54.259603+00:00",
  "elapsed_seconds": 0.6062468330201227
}
```

evidence/operations/preflight.json


```
{
  "check": "preflight",
  "status": "passed",
  "details": {
    "version": "8.4.13",
    "grants": [
      "GRANT USAGE ON *.* TO `etlguard_ops`@`127.0.0.1`",
      "GRANT ALL PRIVILEGES ON `etlguard_ops_ci`.* TO `etlguard_ops`@`127.0.0.1`"
    ],
    "strict_sql_mode": "ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERRO
R_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION",
    "source": "fresh local execution",
    "synthetic_only": true
  },
  "timestamp_utc": "2026-11-11T18:03:19.381185+00:00",
  "elapsed_seconds": 0.02099970798008144
}
```


evidence/operations/process_kill.json


```
{
  "check": "process_kill",
  "status": "passed",
  "details": {
    "batch_id": "7e74817a-06c1-4447-9998-ef3cb9d4d68e",
    "killed_process_returncode": -9,
    "ledger_before_retry": null,
    "uncommitted_rows_rolled_back": true,
    "stable_uuid_retry_succeeded": true
  },
  "timestamp_utc": "2026-11-11T18:03:19.822085+00:00",
  "elapsed_seconds": 0.3369695830042474
}
```


evidence/operations/replacement_guards.json


```
{
  "check": "replacement_guards",
  "status": "passed",
  "details": {
    "empty": {
      "batch_id": "def44a70-8d87-424e-ac4a-198642ce61be",
      "status": "failed",
      "failures": [
        "truncate_insert blocked: source is empty; set policy.allow_empty_replace=true to
explicitly allow this"
      ],
      "target_preserved": true
    },
    "partial": {
      "batch_id": "a205e9e8-8dfe-4f47-8080-b2f04db7fc19",
      "status": "failed",
      "failures": [
        "truncate_insert blocked: rejected source records; set policy.allow_partial_replace=true
 to explicitly allow this"
      ],
      "target_preserved": true
    },
    "small_ratio": {
      "batch_id": "e9f12f6f-5ee2-4d08-b365-5bc0a37dffdb",
      "status": "failed",
      "failures": [
        "ValueError: truncate_insert blocked: 1 valid rows vs 10 existing rows below
min_replace_ratio=0.80"
      ],
      "target_preserved": true
    }
  },
  "timestamp_utc": "2026-11-11T18:03:21.608501+00:00",
  "elapsed_seconds": 0.30913412498193793
}
```

evidence/operations/scd2_concurrency.json


```
{
  "check": "scd2_concurrency",
  "status": "passed",
  "details": {
    "writers": 8,
    "keys": 50,
    "total_versions": 450,
    "active_versions": 50,
    "worker_results": [
      {
        "batch_id": "ea7937bb-f324-467e-a4ab-f67baba8b45e",
        "elapsed_seconds": 0.13900591601850465,
        "versions_created": 50
      },
      {
        "batch_id": "26f3571b-4e08-4a3f-9a3b-29a779357d63",
        "elapsed_seconds": 0.0666115420171991,
        "versions_created": 50
      },
      {
        "batch_id": "d0d2b42a-0818-40dd-9496-c13e66cd8227",
        "elapsed_seconds": 0.1761636669980362,
        "versions_created": 50
      },
      {
        "batch_id": "64ee5488-cdd1-4f25-8899-042de495eb0d",
        "elapsed_seconds": 0.24842358301975764,
        "versions_created": 50
      },
      {
        "batch_id": "52f26ba6-0bcc-4a13-9be5-c99716735b77",
        "elapsed_seconds": 0.21209887499571778,
        "versions_created": 50
      },
      {
        "batch_id": "7ad557b6-f7ea-4029-9f33-4ae24877318c",
        "elapsed_seconds": 0.324678834003862,
        "versions_created": 50
      },
      {
        "batch_id": "ffbf3dd5-2f76-4a15-b008-47d3d2283548",
        "elapsed_seconds": 0.28471150001860224,
        "versions_created": 50
      },
      {
        "batch_id": "788a557d-a635-43d8-a8dd-5f795b5ae6e5",
        "elapsed_seconds": 0.10415879101492465,
        "versions_created": 50
      }
    ],
    "history_intervals_contiguous": true
  },
  "timestamp_utc": "2026-11-11T18:03:21.299097+00:00",
  "elapsed_seconds": 0.39151979199959897
}
```


evidence/operations/server-crash-pending.json


```
{
  "staged_at_utc": "2026-11-11T18:06:08.437136+00:00",
  "committed_batch_id": "2d7f0965-fd58-4fc3-b201-6bdeddec08fb",
  "committed_table": "op_lostack_e7e0d5b8",
  "before_physical_sha256": "0ffa9b3607b79332456da7e32d4018e365a63ea5838e2d745d9a16e6f8cee150",
  "uncommitted_table": "op_dbcrash_f128797f",
  "uncommitted_batch_id": "cd6ea42d-328c-4b82-94ac-239f793caca2",
  "mysql_pid": 19044,
  "log_offset": 2892,
  "recovery_clock_start": 1794420368.43722
}
```


evidence/operations/server-crash-recovery.json


```
{
  "staged_at_utc": "2026-11-11T18:06:08.437136+00:00",
  "committed_batch_id": "2d7f0965-fd58-4fc3-b201-6bdeddec08fb",
```

```
  "committed_table": "op_lostack_e7e0d5b8",
  "before_physical_sha256": "0ffa9b3607b79332456da7e32d4018e365a63ea5838e2d745d9a16e6f8cee150",
  "uncommitted_table": "op_dbcrash_f128797f",
  "uncommitted_batch_id": "cd6ea42d-328c-4b82-94ac-239f793caca2",
  "mysql_pid": 19044,
  "log_offset": 2892,
  "recovery_clock_start": 1794420368.43722,
  "status": "passed",
  "verified_at_utc": "2026-11-11T18:06:23.624963+00:00",
  "committed_data_survived": true,
  "uncommitted_data_rolled_back": true,
  "stable_uuid_recovered_then_replayed": true,
  "observed_intervention_inclusive_recovery_seconds": 15.187747955322266,
  "physical_power_loss_tested": false
}
```


evidence/operations/backup_restore.json


```
{
  "check": "backup_restore",
  "status": "passed",
  "timestamp_utc": "2026-11-11T18:03:59.704175+00:00",
  "elapsed_seconds": 5.4437704579904675,
  "details": {
    "backup_started_at_utc": "2026-11-11T18:03:55.936598+00:00",
    "backup_seconds": 0.308070790983038,
    "restore_seconds": 1.786908250011038,
    "verification_seconds": 1.6143606250116136,
    "observed_recovery_seconds": 3.4012688750226516,
    "proposed_rto_seconds": 120,
    "backup_bytes": 21336534,
    "backup_sha256": "bc0858c8522591885a0e441bceb794422b70f40a7ad5580593479fec4682c99b",
    "tables_verified": 24,
    "rows_verified": 160510,
    "all_rows_and_physical_ids_match": true,
    "recovery_point": "consistent logical backup snapshot; deliberate post-backup transaction
  absent",
    "post_backup_batch_id": "cba291da-9cf2-45d4-985c-571833116832",
    "post_backup_target_absent_from_restore": true,
    "pitr_exercised": false,
    "production_rpo_approved": false
  },
  "per_table_row_fingerprints": "See original evidence/operations/backup_restore.json for all
  table-level digests."
}
```


evidence/package/package-installation.json


```
{
  "status": "passed",
  "installed_package_path": "/Users/<user>/Downloads/ETLGuard-readiness-ux4v66ir/.private/wheel-
  venv/lib/python3.13/site-packages/etlguard/examples",
  "working_directory_outside_checkout": true,
  "stable_batch_id": "cbb77dce-a720-4e29-948a-3bc4278f29b4",
  "checks": [
    {
      "name": "help",
      "exit_code": 0,
      "status": "passed"
    },
    {
      "name": "clean",
      "exit_code": 0,
      "status": "passed"
    },
    {
      "name": "dirty",
      "exit_code": 1,
      "status": "passed"
    },
    {
      "name": "invalid_config",
      "exit_code": 2,
      "status": "passed"
    },
    {
      "name": "profile",
      "exit_code": 0,
```

```
      "status": "passed"
    },
    {
      "name": "init_db",
      "exit_code": 0,
      "status": "passed"
    },
    {
      "name": "first_load",
      "exit_code": 0,
      "status": "passed"
    },
    {
      "name": "identical_retry",
      "exit_code": 0,
      "status": "passed"
    },
    {
      "name": "batch_status",
      "exit_code": 0,
      "status": "passed"
    }
  ]
}
```


49. Appendix J - Release patch and changelog

The v0.1.2-to-v0.1.3 patch is the source-grounded record of the MySQL isolation change and version bump. The changelog preserves the incremental release history; refer to FINAL_STATUS.md for the fresh v0.1.3 evidence rather than counting historical checks as current.

changes/v0.1.2-to-v0.1.3.patch

```
0001  --- v0.1.2/CHANGELOG.md
0002  +++ v0.1.3/CHANGELOG.md
0003  @@ -1,3 +1,9 @@
0004  +## 0.1.3 — 2026-11-11
0005  +
0006  +- Use READ COMMITTED for MySQL load connections, preventing missing ledger/audit row gap
       locks from deadlocking independent target writers. Existing-row locks, unique
      reservations, atomic audits and per-target locks remain in force.
0007  +- Add a deterministic three-target ledger/audit concurrency regression.
0008  +- Make staging and promotion tooling report the installed runtime version.
0009  +
0010   ## Application completion — 2026-11-11
0011
0012   - Fixed Pyodide SQLite loading, stale browser reports and browser CSV formula escaping.
0013  --- v0.1.2/README.md
0014  +++ v0.1.3/README.md
0015  @@ -1,3 +1,5 @@
0016  +> Current runtime: **v0.1.3**. Local production-readiness work fixed a shared-metadata
      gap-lock deadlock across distinct targets. See the final handoff reports for current
      evidence; v0.1.2 reports below are historical.
0017  +
0018   # ETL Guard · `etl-validation-toolkit`
0019
0020   [![CI](https://github.com/theraghavraman/etl-validation-toolkit/actions/workflows/ci.yml
      /badge.svg)](https://github.com/theraghavraman/etl-validation-toolkit/actions/workflows/c
      i.yml)
0021  @@ -57,12 +59,12 @@
0022
0023   The `etlguard.core` / cleaning / checks / validation modules **do not import SQLAlchemy
      or MySQL**. Database code lives in `etlguard.loaders` and is imported lazily by the
      orchestration layer. The demo runs the shared validator in the browser instead of
      duplicating it in JavaScript.
0024
0025  -## Install this v0.1.2 production-hardening candidate
0026  +## Install this v0.1.3 production-hardening candidate
0027
0028   From the directory containing the supplied wheel:
0029
0030   ```bash
0031  -python -m pip install ./etl_validation_toolkit-0.1.2-py3-none-any.whl
0032  +python -m pip install ./etl_validation_toolkit-0.1.3-py3-none-any.whl
0033   etlguard --help
0034   ```
0035
```

```
--- v0.1.2/demo/index.html
+++ v0.1.3/demo/index.html
@@ -46,7 +46,7 @@
            <div class="result-footer"><span id="row-caption">Row-level errors include the field and reason.</span><button id="download" class="secondary" disabled>↓ Download rejects CSV</button></div>
          </section>
        </div>
-      <footer><span>ETL GUARD / v0.1.2</span><span>LOCAL DEMO · NO FILE UPLOAD · MIT LICENCE</span></footer>
+      <footer><span>ETL GUARD / v0.1.3</span><span>LOCAL DEMO · NO FILE UPLOAD · MIT LICENCE</span></footer>
     </main>
   </div>
 </body>
--- v0.1.2/docs/pilot-evidence-template.json
+++ v0.1.3/docs/pilot-evidence-template.json
@@ -1,5 +1,5 @@
 {
-  "toolkit_version": "0.1.2",
+  "toolkit_version": "0.1.3",
   "notes": "Fill only after completed tasks; evidence_file must point to an existing nonempty file inside the same evidence directory. Human reviewer still verifies authenticity.",
   "operational": {
     "immutable_source": {
--- v0.1.2/pyproject.toml
+++ v0.1.3/pyproject.toml
@@ -4,7 +4,7 @@

 [project]
 name = "etl-validation-toolkit"
-version = "0.1.2"
+version = "0.1.3"
 description = "Typed, config-driven ETL data quality gates and MySQL load reconciliation."
 readme = "README.md"
 requires-python = ">=3.11"
--- v0.1.2/scripts/assess_promotion.py
+++ v0.1.3/scripts/assess_promotion.py
@@ -7,6 +7,8 @@
 from __future__ import annotations

 import argparse
+from etlguard import __version__
+
 from datetime import datetime, timedelta, timezone
 from hashlib import sha256
 import json
@@ -35,8 +37,8 @@

 def check_staging(report: dict[str, Any], *, now: datetime | None = None) -> list[str]:
     reasons: list[str] = []
-    if report.get("toolkit_version") != "0.1.2":
-        reasons.append("Staging report version is not v0.1.2")
+    if report.get("toolkit_version") != __version__:
+        reasons.append(f"Staging report version is not v{__version__}")
     if (report.get("status") != "passed" or not report.get("mysql8_server_verified")
             or not report.get("mysql8_integration_executed")):
         reasons.append("A completed, passing real MySQL 8 staging gate is required")
@@ -80,8 +82,8 @@
 def check_operational(evidence: dict[str, Any], evidence_file: Path) -> tuple[list[str], dict[str, str]]:
     reasons: list[str] = []
     hashes: dict[str, str] = {}
-    if evidence.get("toolkit_version") != "0.1.2":
-        reasons.append("Pilot evidence version is not v0.1.2")
+    if evidence.get("toolkit_version") != __version__:
+        reasons.append(f"Pilot evidence version is not v{__version__}")
     for section, keys in [("operational", REQUIRED_OPERATIONAL), ("approvals", REQUIRED_APPROVERS)]:
         entries = evidence.get(section) or {}
         if not isinstance(entries, dict):
@@ -117,7 +119,7 @@
     reasons = staging_reasons + operational_reasons
     return {
         "assessed_utc": datetime.now(timezone.utc).isoformat(),
-        "toolkit_version": "0.1.2",
+        "toolkit_version": __version__,
         "decision": "blocked" if reasons else "review_ready_not_authorized",
         "production_approved": False,
         "database_writes_executed": False,
--- v0.1.2/scripts/prepare_shadow_pilot.py
+++ v0.1.3/scripts/prepare_shadow_pilot.py
@@ -7,6 +7,8 @@
```

A
B
C
D
E
F
G
H

```
 from __future__ import annotations

 import argparse
+from etlguard import __version__
+
 from datetime import datetime, timezone
 from hashlib import sha256
 import json
@@ -80,7 +82,7 @@
     batch_id = str(uuid4())
     manifest: dict[str, Any] = {
         "format_version": 1,
-        "toolkit_version": "0.1.2",
+        "toolkit_version": __version__,
         "created_at_utc": datetime.now(timezone.utc).isoformat(),
         "status": "prepared_not_executed",
         "batch_id": batch_id,
--- v0.1.2/scripts/test_pilot_release_gate.py
+++ v0.1.3/scripts/test_pilot_release_gate.py
@@ -45,7 +45,7 @@

     def staging_ok(self):
         return {
-            "toolkit_version": "0.1.2", "status": "passed", "mysql8_server_verified":
True,
+            "toolkit_version": "0.1.3", "status": "passed", "mysql8_server_verified":
True,
             "mysql8_integration_executed": True,
             "timestamp_utc": datetime.now(timezone.utc).isoformat(),
             "preflight": {"server_version": "8.4.7", "default_storage_engine": "InnoDB",
@@ -131,7 +131,7 @@
         self.assertTrue(check_staging(report))

     def test_missing_operational_evidence_is_blocked(self):
-        template = {"toolkit_version": "0.1.2", "operational": {}, "approvals": {}}
+        template = {"toolkit_version": "0.1.3", "operational": {}, "approvals": {}}
         reasons, hashes = check_operational(template, self.root / "evidence.json")
         self.assertEqual(len(reasons), len(REQUIRED_OPERATIONAL) +
len(REQUIRED_APPROVERS))
         self.assertEqual(hashes, {})
@@ -140,7 +140,7 @@
         outside = self.root.parent / "outside.log"
         outside.write_text("fake", encoding="utf-8")
         self.addCleanup(outside.unlink)
-        evidence = {"toolkit_version": "0.1.2", "operational": {"immutable_source": {
+        evidence = {"toolkit_version": "0.1.3", "operational": {"immutable_source": {
             "verified": True, "evidence_file": "../outside.log"}}, "approvals": {}}
         reasons, _ = check_operational(evidence, self.root / "evidence.json")
         self.assertTrue(any("outside directory" in reason for reason in reasons))
@@ -151,7 +151,7 @@
         artifact.write_text("dummy test fixture; independent verification still
required")
         data = {
-            "toolkit_version": "0.1.2",
+            "toolkit_version": "0.1.3",
             "operational": {name: {"verified": True, "evidence_file": "proof.log"}
                             for name in REQUIRED_OPERATIONAL},
             "approvals": {name: {"verified": True, "evidence_file": "proof.log"}
@@ -197,7 +197,7 @@
     def test_shadow_execute_refuses_failed_staging_gate(self):
         plan(self.yaml, self.out, None, False)
         staging = self.root / "staging.json"
-        staging.write_text(json.dumps({"status": "blocked", "toolkit_version":
"0.1.2"}),
+        staging.write_text(json.dumps({"status": "blocked", "toolkit_version":
"0.1.3"}),
                            encoding="utf-8")
         with self.assertRaisesRegex(ValueError, "Staging gate is not fully passed"):
             run_pilot(self.out / "shadow-manifest.json", True, "etlguard_shadow_ci",
--- v0.1.2/scripts/verify_mysql8_staging.py
+++ v0.1.3/scripts/verify_mysql8_staging.py
@@ -8,6 +8,8 @@
 from __future__ import annotations

 import argparse
+from etlguard import __version__
+
 from datetime import datetime, timezone
 import json
 import os
@@ -191,7 +193,7 @@

     report = {
         "timestamp_utc": datetime.now(timezone.utc).isoformat(),
```

```
-        "toolkit_version": "0.1.2",
+        "toolkit_version": __version__,
         "gate": "mysql8_staging",
         "status": "blocked",
         "mysql8_server_verified": False,
--- v0.1.2/scripts/verify_shadow_reconciliation.py
+++ v0.1.3/scripts/verify_shadow_reconciliation.py
@@ -11,6 +11,8 @@
 from __future__ import annotations

 import argparse
+from etlguard import __version__
+
 from collections import Counter
 from datetime import date, datetime, timezone
 from decimal import Decimal
@@ -233,7 +235,7 @@
     return {
         **results,
         "timestamp_utc": datetime.now(timezone.utc).isoformat(),
-        "toolkit_version": "0.1.2",
+        "toolkit_version": __version__,
         "phase": "shadow_row_reconciliation",
         "schema": confirmed_schema,
         "shadow_target": manifest["shadow_target"],
--- v0.1.2/src/etlguard/__init__.py
+++ v0.1.3/src/etlguard/__init__.py
@@ -2,7 +2,7 @@

 from typing import Any

-__version__ = "0.1.2"
+__version__ = "0.1.3"
 __all__ = ["Pipeline", "RunResult", "__version__"]


--- v0.1.2/src/etlguard/loaders/mysql.py
+++ v0.1.3/src/etlguard/loaders/mysql.py
@@ -144,6 +144,12 @@
             lock = f"etlguard:table:{target.table}"
             acquired = False
             if conn.dialect.name == "mysql":
+                # Missing ledger/audit rows must not take REPEATABLE READ gap
+                # locks: independent target writers would deadlock when they
+                # reserve different UUIDs in the same primary-index gap.
+                # Existing-row locks, unique batch reservations and the target
+                # advisory lock still protect retries and SCD2 ordering.
+                conn.execution_options(isolation_level="READ COMMITTED")
                 value = conn.execute(text("SELECT GET_LOCK(:name, :timeout)"), {
                     "name": lock, "timeout": target.lock_timeout_seconds,
                 }).scalar()
--- v0.1.2/tests/test_mysql_integration.py
+++ v0.1.3/tests/test_mysql_integration.py
@@ -293,3 +293,53 @@
     loader.upgrade_scd2(Customer, target)
     assert loader.load([candidate()], str(uuid4()), Customer, target).inserted == 1
     assert len(target_rows(mysql_engine, table_name)) == 1
+
+
+def test_distinct_target_writers_do_not_deadlock_shared_metadata(mysql_engine):
+    """Synchronize missing metadata reads to expose REPEATABLE READ gap locks."""
+    from concurrent.futures import ThreadPoolExecutor
+    from threading import Barrier, local
+    from sqlalchemy import event
+    from etlguard.loaders.mysql import build_tables
+
+    audit = AuditWriter(mysql_engine)
+    audit.init_db()
+    targets = [TargetSettings(table='it_parallel_' + uuid4().hex[:10], mode='append')
+               for _ in range(3)]
+    for target in targets:
+        table, _ = build_tables(Customer, target)
+        table.metadata.create_all(mysql_engine)
+    from uuid import UUID
+
+    base_id = uuid4().int & ~0xffff
+    batch_ids = [str(UUID(int=base_id + i)) for i in range(3)]
+    ledger_barrier = Barrier(3)
+    audit_barrier = Barrier(3)
+    state = local()
+
+    def synchronize(conn, cursor, statement, params, context, many):
+        if statement.startswith('SELECT etl_batch_commits.') and 'WHERE' in statement:
+            if not getattr(state, 'ledger_seen', False):
+                state.ledger_seen = True
```

```
+                ledger_barrier.wait(timeout=10)
+        elif statement.startswith('SELECT etl_run_audit.') and 'WHERE' in statement:
+            if not getattr(state, 'audit_seen', False):
+                state.audit_seen = True
+                audit_barrier.wait(timeout=10)
+
+    event.listen(mysql_engine, 'after_cursor_execute', synchronize)
+    try:
+        def worker(item):
+            target, batch_id = item
+            return MysqlLoader(engine=mysql_engine).load(
+                [candidate()], batch_id, Customer, target)
+        with ThreadPoolExecutor(max_workers=3) as executor:
+            results = list(executor.map(worker, zip(targets, batch_ids, strict=True)))
+        assert all(result.loaded == 1 for result in results)
+        assert all(len(target_rows(mysql_engine, target.table)) == 1 for target in
targets)
+    finally:
+        event.remove(mysql_engine, 'after_cursor_execute', synchronize)
+        with mysql_engine.begin() as conn:
+            for target in targets:
+                for name in [f'stg_{target.table}', target.table]:
+                    Table(name, MetaData(), autoload_with=conn).drop(conn)
```

source/etl-validation-toolkit/CHANGELOG.md

```
## 0.1.3 — 2026-11-11

- Use READ COMMITTED for MySQL load connections, preventing missing ledger/audit row gap
locks from deadlocking independent target writers. Existing-row locks, unique
reservations, atomic audits and per-target locks remain in force.
- Add a deterministic three-target ledger/audit concurrency regression.
- Make staging and promotion tooling report the installed runtime version.

## Application completion — 2026-11-11

- Fixed Pyodide SQLite loading, stale browser reports and browser CSV formula escaping.
- Added a local launcher, complete source packaging and current repository links.
- Fixed lint/types, Python 3.11 dependency compatibility, escaped MySQL grant parsing and
 metadata aliases.
- Verified all four MySQL 8.0/8.4 and Python 3.11/3.13 staging/shadow jobs; see
`docs/completion-report.md`.

# Changelog

## 0.1.2 — production hardening candidate (2026-11-10)

- Durable `etl_batch_commits` ledger and atomic target+success-audit commit using InnoDB
transactions.
- Stable UUID retries via CLI `--batch-id`; input/configuration fingerprints prevent
changed batches reusing an ID.
- Recovered outcomes after lost commit acknowledgements when the durable ledger confirms
success.
- `batch-status` for operator commit status and failed-audit context.
- Serialized per-target MySQL writers via bounded `GET_LOCK`/`RELEASE_LOCK`;
database-enforced single-active-version SCD2 uniqueness with explicit migration command
for older tables.
- NOT NULL business-key enforcement and InnoDB storage-engine preflight for transactional
 safety.
- Fail-closed empty replacement for low-level loaders; optional `min_replace_ratio`
against existing row counts.
- Partial source-read counts retained; incomplete feeds never load. Report write failures
 after commit are warnings.
- Rejected CSV formula escaping (JSON remains lossless), and expanded SQLite/MySQL 8
regression suite.
- Real MySQL 8 testing remains a promotion gate and is NOT certified by local tests.


All notable changes to this repository are documented here.

## [0.1.1] - 2026-11-10

### Fixed

- Count every invalid source record, even when many have identical cleaned values.
- Block unsafe empty and partially rejected `truncate_insert` batches by default;
overrides require explicit policy opt-in.
- Validate pipeline names before using them as report file basenames.
- Refuse MySQL upserts when competing UNIQUE indexes could update the wrong business
record.
- Honor rename_map when snake_case_columns is disabled.
- Add dedicated regression tests and update the packaged version.
```

A
B
C
D
E
F
G
H

```
## [0.1.0] - 2026-11-10

### Added

- Pluggable CSV, Excel and paginated REST API readers with lineage metadata.
- Configurable deterministic cleaning and typed Pydantic v2 row/dataset checks.
- Rich CLI with dry-run, rejects, JSON summaries and configurable failure policy.
- MySQL 8 staging-to-target append, transactional replacement, upsert and SCD2 modes.
- Count reconciliation, durable ETL run audit and reusable Python API.
- Pyodide + SQLite browser demo powered by the same Python validation engine.
- Unit, property, mocked API, CLI and MySQL service integration tests.

Historical v0.1.0 and v0.1.1 artifacts are preserved separately. Promote or tag v0.1.2
only after the MySQL 8 integration suite and production-hardening acceptance checklist
pass.

### v0.1.2 — Staging and controlled-pilot verification add-on (source ZIP only)

- Added a non-overwriting shadow pilot plan and immutable file/config checksums with
stable UUID.
- Added a fail-closed shadow-only runner: dry validation by default; explicitly confirmed
  disposable MySQL 8 shadow schema, scoped grants, strict SQL mode, no untracked tables.
- Added evidence-based promotion review which **never** authorizes production.
- Expanded MySQL8 staging preflight to check strict SQL mode, schema-only privileges and
  existing table engine; added MySQL DDL compilation and offline safety tests.
- Docker Compose config now provisions a second synthetic shadow-only test schema.
- This add-on does **not** change the v0.1.2 installable Python wheel/runtime.

### v0.1.2 — Phase 3 independent shadow reconciliation (source ZIP only)

- Added read-only source-vs-target field-level reconciliation on isolated MySQL 8
  shadow tables, along with audit/ledger/staging consistency checks.
- Added physical target SHA-256 row fingerprints across stable-ID retries.
- Updated disposable MySQL CI smoke to enforce checks before and after retry.
- Added SQLite fault-injection tests for values, audits, ledger, staging rows,
  SCD2 unexpected history, source checksum changes, and retry invariants.
- The v0.1.2 runtime/wheel is unchanged. Real MySQL integration, backups,
  production pilot and human sign-off remain outstanding.
```

A
B
C
D
E
F
G
H

50. Appendix K - Example input walkthrough and quality case studies

These synthetic source records are included in the project's public examples. They explain the relationship between raw input, deterministic cleaning, Pydantic schema errors, dataset-level checks and write gating. No example implies a real target has been authorized.

src/etlguard/examples/data/customers_clean.csv

```
customer_id,full_name,email,signup_date,end_date,status,credit_limit
C101,Amina Khan,amina@example.com,2026-01-01,,active,1200.00
C102,Rohan Das,rohan@example.com,2026-01-02,,active,500.00
C103,Paula Reed,paula@example.com,2026-01-03,,inactive,200.00
```

- Confirm header names, source row numbers and intended Pydantic model.
- Run validate before proposing a database write; check rejection rate and global issues.
- Inspect raw JSON errors, cleaned values and accepted business keys.
- Pin the final approved complete snapshot and its **SHA-256**, then persist a stable UUID for write attempts.

src/etlguard/examples/data/customers_messy.csv

```
Customer ID,Full Name,Email,Signup Date,End Date,Status,Credit Limit
 C001 , Alice Kumar ,Alice@EXAMPLE.com,2025/01/05,, ACTIVE ,"$1,200.00"
C002,Bob Patel,NA,13/01/2025,,active,900.00
C003,Claire Chen,claire@example.com,2025-02-01,2025-01-01,active,500
C004,Dina Roy,bad-at-email,2025-02-03,,inactive,100
C005,Ed Singh,ed@example.com,not-a-date,,active,1000
 C001 , Alice Kumar ,Alice@EXAMPLE.com,2025/01/05,, ACTIVE ,"$1,200.00"
```

```
C006,Frank Lee,frank@example.com,2025-03-05,,inactive,250
C007,Gee,NA,2025-03-10,,active,300
```

- Confirm header names, source row numbers and intended Pydantic model.
- Run validate before proposing a database write; check rejection rate and global issues.
- Inspect raw JSON errors, cleaned values and accepted business keys.
- Pin the final approved complete snapshot and its **SHA-256**, then persist a stable UUID for write attempts.

src/etlguard/examples/data/transactions_messy.csv

```
Transaction ID,Customer ID,Transaction Date,Type,Amount,Description
T001,C001,2026-02-01,debit,"$1,250.00",Order paid
T002,C002,02/02/2026,credit,500.00,Refund
T003,C003,2026-02-03,debit,0,Invalid zero debit
T004,C001,bad-date,credit,20.00,Wrong date
T005,C999,2026-02-04,credit,100,Unknown customer FK (CLI example)
T001,C001,2026-02-01,debit,"$1,250.00",Order paid
```

- Confirm header names, source row numbers and intended Pydantic model.
- Run validate before proposing a database write; check rejection rate and global issues.
- Inspect raw JSON errors, cleaned values and accepted business keys.
- Pin the final approved complete snapshot and its **SHA-256**, then persist a stable UUID for write attempts.

Specific data-quality scenarios

Input condition	Expected behavior	Operator decision
Exact repeat of an accepted row	May be removed by allowed duplicate suppression	Inspect duplicate count and accepted set
Repeated invalid reference	Each invalid source record contributes to rejection rate	Never hide invalid duplicates as harmless repeats
Bad date string	Cleaner leaves bad value for Pydantic to reject	Fix input or supported date format
Partially returned API pages	Source error fails the whole batch	Re-export or recover complete immutable feed
Null-rate bound exceeded	Dataset issue fails the batch	Data owner reviews missing values and thresholds
Same UUID with changed configuration	A committed mismatching request is refused	Investigate; do not evade gate by guessing UUID

51. Appendix L - Worked SCD Type 2 transitions and DBA queries

The scenarios below are conceptual examples derived from the versioning code, not measured outputs. **SCD2** compares only the configured tracked_columns, stores UTC naive DATETIME(6), closes the old version at T and begins the new one at T. The active unique marker is the database invariant; external writers require coordination.

Scenario	Physical action	Count treatment
First C001	Insert current version, effective_to NULL	inserted +1

Scenario	Physical action	Count treatment
C001 unchanged tracked values	No version write	updated +1; unchanged +1
C001 tracked field changed	Close old record at T, insert new at T	updated +1; versions_created +1
Multiple coordinated writers	Advisory lock and FOR UPDATE serialize	Verify one current row and contiguous history
Uncoordinated external write	Not covered by ETLGuard advisory protocol	DBA must design external writer coordination

Batch ledger state

```
SELECT batch_id, request_hash, pipeline, target_table, committed_at, rows_valid, rows_inserted,
  rows_updated FROM etl_batch_commits WHERE batch_id = :batch_uuid;
```

Use only a read-only connection and approved parameter/identifier bindings. Compare the result with immutable source records and target fingerprints before deciding to retry or compensate.

Audit row

```
SELECT batch_id, status, rows_read, rows_valid, rows_rejected, rows_loaded FROM etl_run_audit
  WHERE batch_id = :batch_uuid;
```

Use only a read-only connection and approved parameter/identifier bindings. Compare the result with immutable source records and target fingerprints before deciding to retry or compensate.

Unexpected multiple active versions

```
SELECT customer_id, COUNT(*) AS current_rows FROM <approved_scd2_table> WHERE is_current = 1
  GROUP BY customer_id HAVING COUNT(*) > 1;
```

Use only a read-only connection and approved parameter/identifier bindings. Compare the result with immutable source records and target fingerprints before deciding to retry or compensate.

Current transactional table engines

```
SELECT TABLE_NAME, ENGINE FROM information_schema.TABLES WHERE TABLE_SCHEMA = DATABASE() AND
  TABLE_NAME IN ('etl_run_audit','etl_batch_commits');
```

Use only a read-only connection and approved parameter/identifier bindings. Compare the result with immutable source records and target fingerprints before deciding to retry or compensate.

Index review

```
SELECT TABLE_NAME, INDEX_NAME, NON_UNIQUE, COLUMN_NAME, SEQ_IN_INDEX FROM
  information_schema.STATISTICS WHERE TABLE_SCHEMA = DATABASE() AND TABLE_NAME = :table_name ORDER
   BY INDEX_NAME, SEQ_IN_INDEX;
```

Use only a read-only connection and approved parameter/identifier bindings. Compare the result with immutable source records and target fingerprints before deciding to retry or compensate.

52. Appendix M - Detailed supervised-pilot evidence checklist

Record each review item against authentic evidence and the approved change. The checkboxes in this appendix are deliberately unmarked. No local test or document may substitute for human authorization.

Artifact identity

[ ] Confirm v0.1.3 wheel, source and sdist hashes

[ ] Check all release checksums and extraction

[ ] Verify package installed outside checkout

[ ] Rerun full test suite with no critical skips

Data correctness

[ ] Pin representative approved source and YAML hashes

[ ] Confirm Pydantic schema, cleaning and business keys

[ ] Review reject counts, thresholds and partial-source handling

[ ] Reconcile source against exact approved target

Database safety

[ ] Confirm InnoDB and unique constraints

[ ] Approve read committed behavior and external writer coordination

[ ] Verify batch ledger and success audit transaction

[ ] Perform original-UUID retry/conflict tests

Recovery

[ ] Verify exact target snapshot backup

[ ] Rehearse restore/PITR under accepted RPO/RTO

[ ] Approve lock timeout, alert and escalation procedure

[ ] Verify monitoring receiver and evidence retention

Approval

[ ] Record exact host/schema/table/UUID/change ticket/window

[ ] Obtain data-owner sign-off

[ ] Obtain DBA sign-off

[ ] Obtain operations-owner sign-off

[ ] Obtain separate unattended scheduler/go-live authorization